



CompTIA

DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

<https://www.DeepDumps.com>

About DeepDumps

At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.

With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://DeepDumps.com>

[Mail us on - support@deepdumps.com](mailto:support@deepdumps.com)



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John



Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Dana



Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You 🙏

Ahamod Shibly



Customer support is really fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Passed my exam today with 845 marks



Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

These questions are real and 100 % valid.



Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria



Simple, easy to understand explanations. To anyone who is writing the exam of AZ900,

Databricks

(Certified Data Engineer Professional)

Certified Data Engineer Professional

Total: **412 Questions**

Link: <https://deepdumps.com/papers/databricks/certified-data-engineer-professional>

Question: 1

An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code: `df = spark.read.format("parquet").load(f"/mnt/source/(date)")`

Which code block should be used to create the date Python variable used in the above code block?

- A. `date = spark.conf.get("date")`
- B. `input_dict = input()`
`date= input_dict["date"]`
- C. `import sys`
`date = sys.argv[1]`
- D. `date = dbutils.notebooks.getParam("date")`
- E. `dbutils.widgets.text("date", "null")`
`date = dbutils.widgets.get("date")`

Answer: E

Explanation:

The correct answer is **E**, which uses Databricks widgets to handle the date parameter passed from the Jobs API. Let's break down why:

1. **Databricks Widgets:** Widgets are UI elements that allow users to pass parameters to notebooks at runtime. This is crucial for scheduled jobs that need dynamic input. `dbutils.widgets.text("date", "null")` creates a text widget named "date" with a default value of "null". `dbutils.widgets.get("date")` retrieves the value of this widget. When the Databricks Jobs API calls the notebook, it can override the default value with the batch date.
2. **Why other options are incorrect:**
3. **A. `date = spark.conf.get("date")`:** `spark.conf.get()` retrieves Spark configuration properties, not parameters passed directly to a notebook job. Spark configuration settings are typically for Spark engine behaviors and resources.
4. **B. `input_dict = input()` `date= input_dict["date"]`:** This approach expects user input via the console, which is unsuitable for automated, scheduled jobs. Jobs API does not provide console input to scheduled tasks.
5. **C. `import sys` `date = sys.argv[1]`:** This is suitable for command-line scripts but not Databricks notebooks invoked by the Jobs API. While `sys.argv` can pass arguments to a Python script, the Databricks Jobs API is not designed to directly populate `sys.argv` when executing notebooks. The Jobs API parameter passing mechanism is integrated with Databricks `dbutils`.
6. **D. `date = dbutils.notebooks.getParam("date")`:** `dbutils.notebooks.getParam()` is intended for passing parameters between notebooks when using the `%run` magic command or `dbutils.notebook.run()`, not for receiving parameters from the Jobs API. It is designed for notebook workflows, not external API calls.
7. **Benefits of using widgets:**
8. **Parameterization:** Widgets enable you to parameterize your notebooks, making them reusable for different data batches or scenarios.
9. **Automation:** The Jobs API can set widget values when scheduling notebooks, enabling automated data processing pipelines.
10. **User Interface:** Widgets are visible and editable in the Databricks notebook UI, which is useful for debugging and interactive exploration.

In summary, using Databricks widgets is the correct and most efficient approach for retrieving parameters passed to a notebook by the Databricks Jobs API, allowing for automated, parameterized data processing workflows.

Authoritative Links:

1. **Databricks Widgets:** <https://docs.databricks.com/notebooks/widgets.html>
2. **Databricks Jobs API:** <https://docs.databricks.com/api/workspace/jobs>
3. **dbutils.notebook.run():** <https://docs.databricks.com/notebooks/notebook-workflows.html#run-a-notebook>

Question: 2

Deep Dumps

The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

- A. "Can Manage" privileges on the required cluster
- B. Workspace Admin privileges, cluster creation allowed, "Can Attach To" privileges on the required cluster
- C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
- D. "Can Restart" privileges on the required cluster
- E. Cluster creation allowed, "Can Restart" privileges on the required cluster

Answer: D

Explanation:

Here's a detailed justification for why option D ("Can Restart" privileges on the required cluster) is the most appropriate answer, along with supporting concepts and authoritative links:

The core requirement is to allow users to execute workloads against already configured clusters that might be terminated due to inactivity. The key here is that the clusters already exist. We are not creating new clusters. Therefore, any option requiring cluster creation privileges can be immediately eliminated (options B, C, and E).

"Can Manage" privileges (option A) are far too broad. "Can Manage" grants the user full control over the cluster, including editing its configuration, resizing it, deleting it, and managing permissions. The question asks for minimal permissions. "Can Manage" goes beyond the scope of simply starting and attaching.

The Databricks documentation defines specific cluster access control levels. A cluster terminated due to inactivity does not necessarily need to be completely re-created (requiring "Can Manage"). Instead, it can be restarted.

When a cluster terminates due to inactivity, it enters a terminated state, but the cluster configuration persists. A user with "Can Restart" privileges can bring the cluster back online using its existing configuration. Once the cluster is running, they can then attach to it to execute workloads.

"Can Attach To" is inherently required to execute workloads against a running cluster. However, if the cluster is terminated, "Can Attach To" alone is insufficient; the cluster must first be restarted. Since the cluster is configured to terminate after inactivity, the ability to restart becomes crucial.

Therefore, the combination of "Can Restart" allows the user to bring the stopped cluster back up, and implicitly the ability to "Can Attach To" allows the user to execute workloads when the cluster is running. The minimal permission required is "Can Restart" because, in the scenario described, the user needs to be able to bring the cluster from a terminated (inactive) state to a running state, then "Can Attach To" access is given automatically.

Authoritative Links:

Databricks Cluster Access Control: <https://docs.databricks.com/security/access-control/cluster-access-control.html>

Databricks Cluster Lifecycle: <https://docs.databricks.com/clusters/clusters-manage.html#cluster-states>

Question: 3

Deep Dumps

When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

- A.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
- B.Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1
- C.Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- D.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- E.Cluster: Existing All-Purpose Cluster;
Retries: None;
Maximum Concurrent Runs: 1

Answer: D

Explanation:

Here's a detailed justification for why option D (Cluster: New Job Cluster; Retries: Unlimited; Maximum Concurrent Runs: 1) is the most appropriate choice for scheduling production Structured Streaming jobs on Databricks, emphasizing fault tolerance and cost efficiency:

The key requirements for production Structured Streaming jobs are reliability (automatic recovery from failures) and cost optimization. A "New Job Cluster" is purpose-built for a specific task, which ensures resource isolation. This means that a failure in one streaming job will not impact other jobs. Creating a new cluster for each job provides a clean environment.

"Retries: Unlimited" provides the desired fault tolerance. If a streaming query fails due to transient issues (network glitch, temporary resource unavailability), Databricks will automatically restart it. Without retries, a failure would require manual intervention, leading to data loss or processing delays.

"Maximum Concurrent Runs: 1" enforces sequential execution. While running multiple concurrent jobs on the same cluster might seem appealing for resource utilization, it introduces complexity regarding resource contention and potential interference. Limiting to one run at a time simplifies management and minimizes the risk of failures due to resource exhaustion.

Option A is incorrect due to "Maximum Concurrent Runs: Unlimited." Running unlimited jobs concurrently could overload the cluster, leading to instability and increased failure rates. Option B is flawed because "Retries: None" means the job fails permanently upon encountering an error. Option C is not a good choice due to using an "Existing All-Purpose Cluster." All-Purpose clusters are designed for interactive use and data exploration and can lead to resource contention with other jobs if used for production jobs. Option E fails for the same reason as option C and also has "Retries: None," which makes it prone to permanent failures.

In summary, a new job cluster, unlimited retries, and one concurrent run provides the best balance between fault tolerance and manageable costs for production Structured Streaming jobs on Databricks. The cluster is spun up only when needed, minimizing idle time costs, and retries ensure resilience.

For further research, consult Databricks documentation on:

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html>

Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Cluster Management: <https://docs.databricks.com/clusters/index.html>

Question: 4

Deep Dumps

The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The `recent_sensor_recordings` table contains an identifying `sensor_id` alongside the timestamp and temperature for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when `mean(temperature) > 120`. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

- A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
- B. The `recent_sensor_recordings` table was unresponsive for three consecutive runs of the query
- C. The source query failed to update properly for three consecutive minutes and then restarted
- D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query
- E. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query

Answer: E

Explanation:

The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query.

Question: 5

Deep Dumps

A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named `dev-2.3.9` is not available from the branch selection dropdown.

Which approach will allow this developer to review the current logic for this notebook?

- A. Use Repos to make a pull request use the Databricks REST API to update the current branch to `dev-2.3.9`
- B. Use Repos to pull changes from the remote Git repository and select the `dev-2.3.9` branch.
- C. Use Repos to checkout the `dev-2.3.9` branch and auto-resolve conflicts with the current branch
- D. Merge all changes back to the main branch in the remote Git repository and clone the repo again
- E. Use Repos to merge the current branch and the `dev-2.3.9` branch, then make a pull request to sync with the remote repository

Answer: B

Explanation:

The correct approach is **B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.**

Here's a detailed justification:

The problem stems from the developer working with an outdated version of the code in their personal branch within Databricks Repos. The desired dev-2.3.9 branch, containing the correct logic, is not locally available. To rectify this, the developer needs to update their local repository with the latest changes from the remote repository where dev-2.3.9 resides.

Option B directly addresses this by using Repos' functionality to "pull" (fetch and merge) changes from the remote Git repository. This action downloads the latest commits and branches, including dev-2.3.9, to the developer's local repository. Once the pull is complete, the developer can then select (checkout) the dev-2.3.9 branch within Databricks Repos. This will switch the notebook to the version present in that branch, allowing the developer to review and use the correct logic.

Let's analyze why the other options are not suitable:

1. **A:** Making a pull request is intended for contributing changes, not for simply acquiring them. Also using the Databricks REST API to update the current branch feels overly complex for a simple git update.
2. **C:** Checking out the branch and auto-resolving conflicts could lead to unintended changes if the current branch has modifications the developer wants to discard or keep separate.
3. **D:** Merging to the main branch and recloning is a drastic measure and disrupts the established Git workflow. It's unnecessary to involve main for a localized branch update.
4. **E:** Merging the current branch with dev-2.3.9 before pulling could introduce unwanted conflicts if the developer's local branch contains errors.

Therefore, pulling changes from the remote repository and then selecting the correct branch is the most efficient and safest way for the junior developer to access the current logic.

Here are some links to further research Databricks Repos and Git operations:

1. **Databricks Repos Documentation:** <https://docs.databricks.com/repos/index.html>
2. **Git Basics - Getting a Git Repository:** <https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>
3. **Git Branching - Basic Branching and Merging:** <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>

Question: 6

Deep Dumps

The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.

After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
      .read
      .format("jdbc")
      .option("url", connection)
      .option("dbtable", tablename)
      .option("user", username)
      .option("password", password)
      )
```

Which statement describes what will happen when the above code is executed?

- A.The connection to the external table will fail; the string "REDACTED" will be printed.
- B.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
- C.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
- D.The connection to the external table will succeed; the string value of password will be printed in plain text.
- E.The connection to the external table will succeed; the string "REDACTED" will be printed.

Answer: E

Explanation:

The connection to the external table will succeed; the string "REDACTED" will be printed.

<https://learn.microsoft.com/en-us/azure/databricks/security/secrets/redaction>

Question: 7

Deep Dumps

The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named preds with the schema "customer_id LONG, predictions DOUBLE, date DATE".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
    "customer_id",
    model(*columns).alias("predictions"),
    current_date().alias("date")
    )
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A.preds.write.mode("append").saveAsTable("churn_preds")

B.preds.write.format("delta").save("/preds/churn_preds")

```
(preds.writeStream
  .outputMode("overwrite")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .start("/preds/churn_preds")
```

C.)

```
(preds.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("churn_preds")
```

D.)

```
(preds.writeStream
  .outputMode("append")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .table("churn_preds")
```

E.)

Answer: A

Explanation:

You need:- Batch operation since it is at most once a day- Append, since you need to keep track of past predictions A is the correct answer. You don't need to specify "format" when you use saveAsTable.

Question: 8

Deep Dumps

An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the date variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields customer_id and order_id serve as a composite key to uniquely identify each order. If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

A.Each write to the orders table will only contain unique records, and only those records without duplicates in the target table will be written.

B.Each write to the orders table will only contain unique records, but newly written records may have duplicates

already present in the target table.

C.Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.

D.Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, the operation will fail.

E.Each write to the orders table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

Answer: B

Explanation:

Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.

Question: 9

Deep Dumps

A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the geo_lookup table.

Before executing the code, running SHOW TABLES on the current database indicates the database contains only two tables: geo_lookup and sales.

Cmd 1

```
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]
```

Cmd 2

```
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

Which statement correctly describes the outcome of executing these command cells in order in an interactive notebook?

A.Both commands will succeed. Executing show tables will show that countries_af and sales_af have been registered as views.

B.Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named countries_af: if this entity exists, Cmd 2 will succeed.

C.Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable representing a PySpark DataFrame.

D.Both commands will fail. No new variables, tables, or views will be created.

E.Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Answer: E

Explanation:

Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Cmd 1 is a PySpark command that collects the list of countries from the 'geo_lookup' table where the continent is Africa ('AF'). This command will execute successfully, resulting in countries_af being a list of

country names (strings) in Python's local memory.

Cmd 2 is an SQL command intended to create a view named 'sales_af' from the 'sales' table, filtered by the cities in the countries_af list. However, this will fail because the countries_af variable exists in the Python environment and is not recognized in the SQL context. SQL does not have access to Python variables directly; they are two separate execution contexts within a Databricks notebook. There is no table or view named countries_af that SQL can reference; it is merely a Python list variable.

The other options are incorrect because they either assume cross-contextual operation between Python and SQL within a Databricks notebook (which is not possible in the way described in the commands), or they do not correctly interpret the outcome of running the commands.

Question: 10

Deep Dumps

A Delta table of weather records is partitioned by date and has the below schema: date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT

To find all the records from within the Arctic Circle, you execute a query with the below filter: latitude > 66.3
Which statement describes how the Delta engine identifies which files to load?

- A. All records are cached to an operational database and then the filter is applied
- B. The Parquet file footers are scanned for min and max statistics for the latitude column
- C. All records are cached to attached storage and then the filter is applied
- D. The Delta log is scanned for min and max statistics for the latitude column
- E. The Hive metastore is scanned for min and max statistics for the latitude column

Answer: D

Explanation:

The correct answer is D: The Delta log is scanned for min and max statistics for the latitude column.

Delta Lake enhances data lake reliability and performance by introducing a transaction log that meticulously records every change made to the data. This log contains metadata, including min/max statistics for each column within each data file. This enables Delta Lake to intelligently skip irrelevant files during query execution, a technique known as data skipping. When you execute a query with the filter latitude > 66.3, the Delta engine uses the transaction log to evaluate the min/max latitude values stored for each data file. If the minimum latitude value for a particular file exceeds 66.3, that file is guaranteed to contain relevant records and is included in the read set. Conversely, if the maximum latitude value for a file is less than or equal to 66.3, the file is skipped entirely, significantly reducing I/O operations and accelerating query performance. The Delta log eliminates the need to scan Parquet file footers directly (option B) or rely on external metastores like Hive for min/max statistics (option E). Options A and C are incorrect because they involve caching all records, which defeats the purpose of efficient data skipping. Delta Lake prioritizes reading only the data that satisfies the query condition.

For further research, consult these resources:

Delta Lake Documentation on Data Skipping: <https://docs.databricks.com/delta/data-skipping.html>

Delta Lake Under the Hood: <https://databricks.com/blog/2019/08/22/diving-into-delta-lake-unpacking-the-transaction-log.html>

Question: 11

Deep Dumps

The data engineering team has configured a job to process customer requests to be forgotten (have their data

deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings. The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of VACUUM commands on all Delta Lake tables throughout the organization. The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data. Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

- A. Because the VACUUM command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
- B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the VACUUM job is run the following day.
- C. Because Delta Lake time travel provides full access to the entire history of a table, deleted records can always be recreated by users with full admin privileges.
- D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.
- E. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the VACUUM job is run 8 days later.

Answer: E

Explanation:

The correct answer is E because it accurately reflects Delta Lake's default data retention behavior and how it interacts with the VACUUM command in relation to data deletion and time travel.

Here's the detailed justification:

Delta Lake's time travel feature allows users to query older versions of a table. This is achieved by retaining older data files. By default, Delta Lake retains historical data for 7 days. This means data files containing records deleted in the last week will still be present in the underlying storage (e.g., Azure Blob Storage or AWS S3).

The VACUUM command is used to remove files from the Delta Lake table's underlying storage that are no longer referenced in the current table version and are older than the retention period. Without VACUUM, Delta Lake tables can accumulate a significant number of files over time, impacting query performance and storage costs.

The compliance officer's concern is valid. Even though records are deleted from the Delta Lake table, the underlying data files containing those deleted records persist for the retention period. Because the deletion job runs on Sunday at 1 am and the VACUUM job runs on Monday at 3 am the following week, this is 8 days later. Therefore, the files with deleted records will stick around for the retention period of 7 days, plus a little more time until the VACUUM command executes to remove those files.

Let's analyze why other options are incorrect:

A: While VACUUM does permanently delete files, it only does so after the retention period. The retention is not 24 hours by default.

B: The default retention is not 24 hours.

C: While admins can access history, the point is to minimize access to deleted data after the retention period, which VACUUM facilitates. This statement suggests data can always be recreated, which contradicts the purpose of data deletion and the VACUUM command.

D: Delta Lake's delete operations are ACID compliant, ensuring data consistency. However, ACID compliance doesn't mean immediate and permanent deletion from the storage system. The data still persists within the older versions for a defined period (retention period).

Therefore, option E accurately describes how Delta Lake's default retention policy affects the accessibility of deleted data via time travel and how the VACUUM command, run after the retention period, eventually

removes those files.

Authoritative Links:

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake VACUUM: <https://docs.databricks.com/delta/vacuum.html>

Question: 12

Deep Dumps

A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint 2.0/jobs/create.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Assuming that all configurations and referenced resources are available, which statement describes the result of executing this workload three times?

- A. Three new jobs named "Ingest new data" will be defined in the workspace, and they will each run once daily.
- B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
- C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
- D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.
- E. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.

Answer: C

Explanation:

Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.

Question: 13

Deep Dumps

An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field pk_id.

For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system. For analytical purposes, only the most recent value for each record needs to be recorded. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

- A. Create a separate history table for each pk_id resolve the current state of the table by running a union all filtering the history tables for the most recent state.
- B. Use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a bronze table, then propagate all changes throughout the system.
- C. Iterate through an ordered set of changes to the table, applying each in turn; rely on Delta Lake's versioning ability to create an audit log.

D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.

E. Ingest all log information into a bronze table; use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state.

Answer: D

Explanation:

The correct answer is **E** (Ingest all log information into a bronze table; use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state.) because it provides a structured and scalable approach to handle CDC data while fulfilling both audit and analytical requirements.

Here's a detailed justification:

1. **Bronze Table for Raw Data (Audit):** The initial ingestion into a bronze table ensures that all CDC logs are captured exactly as they arrive from the upstream system. This fulfills the audit requirement of maintaining a full record of all values that have ever been valid. The bronze table acts as the immutable source of truth, preserving all changes for historical analysis and compliance.
2. **Silver Table for Current State (Analytical):** The silver table is designed to hold the most recent value for each record. This caters to the analytical needs of the data governance team.
3. **MERGE INTO for Upserts:** The use of MERGE INTO operation on the silver table efficiently handles inserts, updates, and deletes based on the pk_id. Within each hourly batch, the MERGE INTO operation effectively overwrites older values for a given pk_id with the latest one found in the CDC logs, ensuring the silver table always reflects the most current state.
4. **Scalability and Performance:** This approach is scalable because Delta Lake and MERGE INTO are optimized for large datasets. The incremental processing within each hourly batch allows for efficient updates without requiring a full table scan.

Why other options are less suitable:

A: Creating a separate history table for each pk_id is unscalable and makes querying the current state extremely complex due to the need for a UNION ALL operation and complex filtering.

B: Propagating all changes from a bronze table directly is not suitable as the system requires two different views of the data - the full history, and the current state.

C: While Delta Lake's versioning provides an audit log, iterating through an ordered set of changes and applying them manually is less efficient and harder to manage than using MERGE INTO.

D: Delta Lake Change Data Feed (CDF) is powerful, but doesn't directly address the requirement to maintain a full historical audit log in addition to the most recent state without additional processing steps. CDF captures changes from a Delta table, not changes to a target Delta table based on external CDC.

Relevant Cloud Computing Concepts:

Data Lakehouse: The overall architecture leverages the Data Lakehouse paradigm, combining the benefits of data lakes (scalability, flexibility) and data warehouses (structure, governance). Bronze and silver tables represent different layers in the Lakehouse architecture.

Change Data Capture (CDC): The solution is specifically designed to handle CDC data, which is a common pattern for integrating data from operational systems.

Upsert Operation: MERGE INTO is an upsert operation that efficiently combines inserts and updates into a single operation, crucial for handling CDC data.

Delta Lake: Delta Lake provides ACID properties and versioning to the data lake, enabling reliable and scalable data processing.

Authoritative Links:

Delta Lake MERGE INTO: <https://docs.databricks.com/en/delta/delta-update.html#use-merge-into>

Delta Lake Change Data Feed: <https://docs.databricks.com/en/delta/delta-change-data-feed.html>

Data Lakehouse: <https://databricks.com/glossary/data-lakehouse>

Question: 14

Deep Dumps

An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The user_id field represents a unique key for the data, which has the following schema: user_id BIGINT, username STRING, user_utc STRING, user_region STRING, last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT

New records are all ingested into a table named account_history which maintains a full record of all data in the same schema as the source. The next table in the system is named account_current and is implemented as a Type 1 table representing the most recent value for each unique user_id.

Assuming there are millions of user accounts and tens of thousands of records processed hourly, which implementation can be used to efficiently update the described account_current table as part of each hourly batch job?

- A. Use Auto Loader to subscribe to new files in the account_history directory; configure a Structured Streaming trigger once job to batch update newly detected files into the account_current table.
- B. Overwrite the account_current table with each batch using the results of a query against the account_history table grouping by user_id and filtering for the max value of last_updated.
- C. Filter records in account_history using the last_updated field and the most recent hour processed, as well as the max last_login by user_id write a merge statement to update or insert the most recent value for each user_id.
- D. Use Delta Lake version history to get the difference between the latest version of account_history and one version prior, then write these records to account_current.
- E. Filter records in account_history using the last_updated field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each username.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most efficient and appropriate solution for updating the account_current table, along with explanations and relevant links.

Justification for Option C: Filter and Merge

Option C offers the most efficient method for maintaining a Type 1 slowly changing dimension (SCD) table like account_current given the provided scenario. Here's why:

1. **Incremental Processing:** It avoids processing the entire account_history table every hour. By filtering account_history based on the last_updated field and the most recent hour, you significantly reduce the data volume needing processing. This is crucial for performance, especially with millions of user accounts and tens of thousands of hourly records.
2. **Efficient Deduplication:** By identifying the maximum last_login for each user_id within the hourly batch, the solution ensures that only the most recent update for each user is considered. This prevents older or redundant records from overwriting the most current data in account_current.
3. **MERGE Statement for UPSERT:** The MERGE statement is designed for efficiently updating existing records and inserting new records in a single operation (UPSERT - Update or Insert). This is more performant than separate update and insert operations. Delta Lake's optimized MERGE implementation further enhances this efficiency. The MERGE statement is preferred as it efficiently

combines the update and insert operations based on matching keys.

4. **Key-Based Update:** Using `user_id` as the key in the MERGE statement ensures that updates are applied to the correct user's record in the `account_current` table.

Why other options are less ideal:

1. **A (Auto Loader with Streaming):** While Auto Loader is great for ingestion, using Structured Streaming with a trigger once batch job adds unnecessary complexity and overhead. Batch processing is already in place, so using streaming is not the most direct approach. Also, it does not naturally solve the de-duplication needed.
2. **B (Overwrite):** Overwriting the entire `account_current` table every hour is highly inefficient, particularly with millions of users. It requires reprocessing all data, even if only a small subset has changed. This also loses the time travel capabilities that Delta lake brings to the table.
3. **D (Delta Lake Version History):** While Delta Lake version history is useful, calculating the difference between versions requires reading and comparing potentially large datasets. Using `last_updated` and MERGE is more direct and targeted. Also, this approach does not account for users who may have had multiple updates in a single batch.
4. **E (Deduplicate on username):** Deduplicating on username is incorrect, as the problem statement uses `user_id` as the unique key for accounts. The username might change.

Relevant Cloud Computing Concepts:

1. **Slowly Changing Dimensions (SCDs):** `account_current` is a Type 1 SCD, meaning it always reflects the most current data.
2. **UPSERT (Update or Insert):** The MERGE statement performs UPSERT operations efficiently.
3. **Delta Lake:** Delta Lake enhances data reliability and performance through ACID transactions, schema enforcement, and time travel.
4. **Incremental Data Processing:** Processing only the changed or new data, as opposed to the full dataset, is a core principle for efficient data pipelines.

Authoritative Links:

1. **Delta Lake MERGE:** <https://docs.databricks.com/delta/delta-update.html#upsert-into-a-delta-table-using-merge>
2. **Slowly Changing Dimensions:** https://en.wikipedia.org/wiki/Slowly_changing_dimension

In summary, option C provides the most efficient and scalable solution by leveraging incremental processing, deduplication, and the MERGE statement to update the `account_current` table based on hourly batches of data.

Question: 15

Deep Dumps

A table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

A. Apply the churn model to all rows in the `customer_churn_params` table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.

B. Convert the batch job to a Structured Streaming job using the complete output mode; configure a Structured Streaming job to read from the `customer_churn_params` table and incrementally predict against the churn model.

C. Calculate the difference between the previous model predictions and the current customer_churn_params on a key identifying unique customers before making new predictions; only make predictions on those customers not in the previous predictions.

D. Modify the overwrite logic to include a field populated by calling spark.sql.functions.current_timestamp() as data are being written; use this field to identify records written on a particular date.

E. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.

Answer: E

Explanation:

The correct answer is **E** because it offers the most efficient and reliable way to identify and process changed records for churn prediction. Let's break down why:

Change Data Capture (CDC) with MERGE: Option E leverages CDC via a MERGE statement. This is a core data warehousing concept specifically designed to track and propagate changes in data sources.

Efficiency: By using MERGE, you only update records that have actually changed. This avoids processing the entire table every night, significantly reducing computational resources and time.

Change Data Feed (CDF): Change Data Feed (CDF) provides a record of every change made to a Delta table, allowing you to efficiently identify which rows have been inserted, updated, or deleted. By writing logic to make predictions only on the changed records identified by the CDF, you dramatically reduce the scope of the prediction task.

Simplified Identification: MERGE inherently identifies changed records. You can easily extract these changes and flag them for prediction. No complex comparison logic is needed.

Why other options are less suitable:

A: Applying the model to all rows is wasteful and defeats the purpose of optimizing for changed records. Upserts can be complex and add unnecessary overhead.

B: Structured Streaming with complete output mode is not efficient for this use case. Complete mode requires reprocessing the entire table on each update, which negates the benefits of identifying changed records.

C: Calculating the difference between predictions introduces complexities in maintaining and comparing prediction histories. It's not a standard or efficient practice.

D: While current_timestamp() provides a timestamp, it doesn't inherently capture changes. Records might have the same timestamp but unchanged data. This is a workaround and not a robust CDC solution. It doesn't track specific columns that changed.

In Summary: Option E effectively implements CDC using MERGE and the change data feed to identify, isolate, and process changed records. It's the most performant, maintainable, and scalable solution for this scenario. Using CDC aligns with best practices for data warehousing and enables incremental processing, thus providing a simpler and more resource-efficient solution.

Authoritative Links:

Delta Lake Change Data Feed: <https://docs.databricks.com/delta/delta-change-data-feed.html>

Delta Lake MERGE: <https://docs.databricks.com/delta/delta-update.html#upsert-into-a-delta-table-using-merge>

Question: 16

Deep Dumps

A table is registered with the following code:


```
CREATE TABLE recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
     FROM users) a
    INNER JOIN
    (SELECT user_id, order_id, order_date
     FROM orders
     WHERE order_date >= (current_date() - 7)) b
    ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables. Which statement describes the results of querying recent_orders?

- A. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
- B. All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.
- C. Results will be computed and cached when the table is defined; these cached results will incrementally update as new records are inserted into source tables.
- D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.
- E. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.

Answer: D

Explanation:

All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Question: 17

Deep Dumps

A production workload incrementally applies updates from an external Change Data Capture feed to a Delta Lake table as an always-on Structured Stream job. When data was initially migrated for this table, OPTIMIZE was executed and most data files were resized to 1 GB. Auto Optimize and Auto Compaction were both turned on for the streaming production job. Recent review of data files shows that most data files are under 64 MB, although each partition in the table contains at least 1 GB of data and the total table size is over 10 TB.

Which of the following likely explains these smaller file sizes?

- A. Databricks has autotuned to a smaller target file size to reduce duration of MERGE operations
- B. Z-order indices calculated on the table are preventing file compaction
- C. Bloom filter indices calculated on the table are preventing file compaction
- D. Databricks has autotuned to a smaller target file size based on the overall size of data in the table
- E. Databricks has autotuned to a smaller target file size based on the amount of data in each partition

Answer: A

Explanation:

The most likely explanation for the observed smaller file sizes (under 64MB) in the Delta Lake table, despite initial optimization to 1GB files and Auto Optimize/Compaction being enabled, is that Databricks has autotuned to a smaller target file size to optimize MERGE operations.

Here's a detailed justification:

MERGE Operation Impact: MERGE operations are used to update Delta tables based on Change Data Capture (CDC) feeds. These operations can be I/O intensive and slow, especially with large files.

Auto Tuning for MERGE: Delta Lake's auto-tuning mechanism monitors the performance of MERGE operations. When it detects that MERGE operations are taking longer than expected, especially related to the size of the base files being updated, it can automatically adjust the target file size downwards.

Smaller File Size Benefits: Reducing the target file size results in more frequent but smaller compactions. This approach makes MERGE operations more efficient because they need to rewrite smaller amounts of data during updates.

Auto Optimize and Compaction: Auto Optimize generally takes the overall size of the delta table into account to optimize the number of files. Auto compaction consolidates small files. But since the data is constantly being updated through the stream job, autotuning likely took precedence.

Partition Size: While each partition contains at least 1 GB of data, the smaller file sizes suggest the optimization is happening at a finer granularity to specifically accelerate the MERGE operations.

Other Options:

Z-order and Bloom Filters: Z-order clustering and bloom filters can affect query performance, but they don't directly prevent file compaction.

Table Size as Primary Factor: Autotuning generally considers the table's overall size, but it primarily responds to the performance characteristics of ongoing operations, such as MERGE. Therefore, the size of the table itself isn't necessarily the primary trigger.

Therefore, the continuous CDC feed and resulting MERGE operations likely caused Databricks to automatically reduce the target file size to improve update performance, even though Auto Optimize and Compaction were initially configured.

Relevant Links:

[Delta Lake Auto Optimize](#)

[Delta Lake MERGE](#)

Question: 18

Deep Dumps

Which statement regarding stream-static joins and static Delta tables is correct?

- A. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch.
- B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
- C. The checkpoint directory will be used to track state information for the unique keys present in the join.
- D. Stream-static joins cannot use static Delta tables because of consistency issues.
- E. The checkpoint directory will be used to track updates to the static Delta table.

Answer: B

Explanation:

The correct statement regarding stream-static joins and static Delta tables is that each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization. This

behavior is crucial for ensuring consistency during the streaming process. When a streaming job starts, it reads the static Delta table. During each micro-batch, the streaming data is joined against the same snapshot of the static table that was available at the job's start.

This design choice avoids inconsistencies that could arise from using different versions of the static table across different micro-batches. It also improves performance, as the static table is only read once at the beginning, rather than for every microbatch. This contrasts with stream-stream joins, which need to handle updates on both sides. The checkpoint directory primarily stores state information related to the streaming data and aggregation/transformation logic, not updates to the static Delta table. While Delta Lake ensures ACID properties for the static table, the stream-static join mechanism uses a single snapshot for the entire streaming job's duration. Therefore, the statement that stream-static joins cannot use static Delta tables due to consistency issues is incorrect; they are designed to work together by utilizing a consistent snapshot of the static data.

For more information, please refer to the official Databricks documentation on structured streaming and Delta Lake. Specifically, consider these pages:

[Structured Streaming with Delta Lake](#): This is a general starting point.

[Delta Lake and Structured Streaming](#): Blog posts discussing how Delta Lake can be used in streaming pipelines.

[Structured Streaming Programming Guide](#): Covers the fundamentals of structured streaming.

These resources will provide deeper insights into stream-static joins and how they interact with Delta Lake.

Question: 19

Deep Dumps

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df.withWatermark("event_time", "10 minutes")
  .groupBy(
    _____
    "device_id"
  )
  .agg(
    avg("temp").alias("avg_temp"),
    avg("humidity").alias("avg_humidity")
  )
  .writeStream
  .format("delta")
  .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

- A.to_interval("event_time", "5 minutes").alias("time")
- B.window("event_time", "5 minutes").alias("time")
- C."event_time"
- D.window("event_time", "10 minutes").alias("time")
- E.lag("event_time", "10 minutes").alias("time")

Answer: B

Explanation:

```
window("event_time", "5 minutes").alias("time")
```

Question: 20

Deep Dumps

A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:

```
./bronze
├── __checkpoint
├── __delta_log
├── year_week=2020_01
├── year_week=2020_02
└── ...
```

Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

- A.No; Delta Lake manages streaming checkpoints in the transaction log.
- B.Yes; both of the streams can share a single checkpoint directory.
- C.No; only one stream can write to a Delta Lake table.
- D.Yes; Delta Lake supports infinite concurrent writers.
- E.No; each of the streams needs to have its own checkpoint directory.

Answer: E

Explanation:

No; each of the streams needs to have its own checkpoint directory.

Reference:

<https://docs.databricks.com/en/optimizations/isolation-level.html#:~:text=If%20a%20streaming%20query%20using%20the%20same%20checkpoint%20location%20is%20started%20multiple%20times%20concurrently%20and%20tries%20to%20write%20to%20the%20Delta%20table%20at%20the%20same%20time.%20You%20should%20never%20have%20two%20streaming%20queries%20use%20the%20same%20checkpoint%20location%20and%20run%20at%20the%20same%20time.>

Question: 21

Deep Dumps

A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.

Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

- A. Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
- B. Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped.
- C. The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
- D. Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.
- E. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

Answer: E

Explanation:

Here's a breakdown of why option E is the most appropriate solution and why the others are less suitable for addressing the streaming job delays:

The core problem is that during peak hours, micro-batch processing times increase significantly, exceeding the desired 10-second limit. The goal is to ensure records are processed within this time frame to prevent backlog and maintain near real-time processing.

Why Option E is Correct: Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

Preventing Backlog: By reducing the trigger interval (e.g., from 10 seconds to 5 seconds), the system ingests and processes data more frequently. This helps to avoid large accumulations of data waiting to be processed, especially during peak periods when the input rate increases. Smaller batches reduce the strain on executors.

Reducing Spill: When batches become excessively large (due to infrequent triggering and higher input rates), they might exceed available memory, leading to "spilling" data to disk. Disk I/O is significantly slower than memory access, drastically increasing processing time. Smaller batches (with more frequent triggers) reduce the likelihood of spilling.

Improved Resource Utilization: Frequent triggering allows executors to switch to new batches as soon as they're ready, thus preventing idle time.

Example: Imagine a conveyor belt carrying packages. If the belt only moves every 10 seconds, packages pile up if the arrival rate is high. If the belt moves every 5 seconds, the pile-up is less severe.

Why Other Options are Incorrect:

Option A: Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish. This answer contains a partial truth. While decreasing the trigger interval is a viable option, the reasoning for this is slightly misleading as the executors working on the longer running tasks of the previous batch cannot begin processing the new batch as they are still busy. Option E provides a much more thorough description of the benefits.

Option B: Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped.

Increasing the trigger interval is completely counterproductive. It exacerbates the problem by creating even larger batches, which will take even longer to process and increase the risk of spilling. Also, increasing the trigger to 30 seconds while the requirement is processing records in less than 10 seconds immediately makes this an unviable option.

Option C: The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism. This is factually incorrect. The trigger interval can be changed dynamically without modifying the checkpoint directory. This is because the trigger interval affects the frequency, not the state. Although increasing shuffle partitions can

improve parallelism, it does not directly address the issue of peak hour delays stemming from the stream configuration. Additionally, increasing shuffle partitions may not necessarily lead to a performance improvement if the data isn't properly partitioned.

Option D: Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch. trigger once is designed for one-time data processing rather than continuous streaming. While using Databricks jobs scheduled every 10 seconds sounds similar, it isn't the same as a structured stream. Trigger once will process all available data and then stop, requiring re-scheduling. Also, scheduling it with Databricks jobs loses the fault-tolerance and state management capabilities inherent in Structured Streaming. This also doesn't solve the root cause of long processing times.

Supporting Concepts:

Structured Streaming: A scalable and fault-tolerant stream processing engine built on Apache Spark.

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Micro-Batching: Structured Streaming processes data in small batches (micro-batches) at specified intervals.

Trigger Interval: Determines how frequently new micro-batches are created and processed.

Backpressure: Mechanism for handling situations where the rate of incoming data exceeds the processing capacity. While not explicitly mentioned in the answer options, this is closely related.

Resource Utilization: Efficiently using available compute resources (CPU, memory) to maximize throughput and minimize latency.

In summary, decreasing the trigger interval is the most direct and effective way to address the micro-batch processing delays during peak hours, preventing backlogs, reducing the risk of spills, and optimizing resource utilization.

Question: 22

Deep Dumps

Which statement describes Delta Lake Auto Compaction?

- A. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- B. Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.
- C. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
- D. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
- E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.

Answer: E

Explanation:

Delta Lake Auto Compaction is a crucial feature that optimizes the storage layout of Delta tables. The correct statement is **E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.**

Here's why this is the correct explanation:

Asynchronous Process: Auto Compaction runs independently and after write operations (inserts, updates, deletes). This means it doesn't block or delay the write transactions themselves.

File Size Monitoring: The auto compaction process monitors the size of the files in the Delta table. It identifies files that are smaller than a threshold size (small files).

OPTIMIZE Command Trigger: If the auto compaction detects an opportunity to compact files, it essentially runs the OPTIMIZE command on the table in the background. The OPTIMIZE command consolidates small files into larger ones.

Default Target Size: The key detail that distinguishes option E from option A is the target file size after compaction. Delta Lake auto compaction aims to produce files around 128 MB in size by default. This is much smaller than the 1GB in Option A which is likely to be for whole-table optimization jobs.

The goal of auto compaction is to reduce the number of small files in the Delta table. A large number of small files can negatively impact query performance. Reading numerous small files requires more metadata operations and can lead to increased I/O overhead. By consolidating these files into larger ones, auto compaction improves query speed and efficiency.

Options A, B, C, and D are incorrect for the following reasons:

Option A: The default target file size of 1 GB is incorrect. The correct default is 128 MB.

Option B: This describes a completely different and non-existent feature. OPTIMIZE is not automatically executed when a Jobs cluster terminates.

Option C: Optimized Writes and partition handling are different features that don't involve Auto Compaction. Optimized writes do help minimize small file creation from the start, rather than afterward.

Option D: Using a messaging bus like Kafka for direct data committing is not relevant to the function of Delta Lake Auto Compaction.

Authoritative Links:

Databricks Documentation on Auto Compaction: <https://docs.databricks.com/delta/optimizations/auto-optimize.html>

Databricks Blog on Delta Lake Optimizations: (search Databricks blog for "Delta Lake optimization techniques") - this contains relevant blog posts to explain the concepts.

Question: 23

Deep Dumps

Which statement characterizes the general programming model used by Spark Structured Streaming?

- A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
- B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
- C. Structured Streaming uses specialized hardware and I/O streams to achieve sub-second latency for data transfer.
- D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.
- E. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.

Answer: D

Explanation:

The correct answer is D because it accurately describes the fundamental programming model of Spark Structured Streaming. Spark Structured Streaming treats a live data stream as a continuously appending table. As new data arrives, it's considered as new rows being added to this unbounded table. This abstraction allows users to apply familiar batch-oriented SQL queries and DataFrame operations to streaming data, making it easier to process.

Option A is incorrect because while Spark can utilize GPUs for specific computations, Structured Streaming doesn't inherently depend on them for general parallel processing. Spark primarily uses its distributed

execution engine for parallel data processing, leveraging clusters of CPUs. Option B is incorrect as Structured Streaming is built on top of the Spark SQL engine, not derived from Apache Kafka. While Kafka is a common data source for streaming applications using Spark, it's not the foundation of Structured Streaming's implementation. Kafka acts as a messaging system, while Structured Streaming provides the processing framework. Option C is inaccurate as Structured Streaming focuses on fault-tolerant and scalable processing using Spark's distributed architecture, rather than relying on specialized hardware or I/O streams for low latency. Latency is addressed through micro-batching or continuous processing mode within Spark's fault-tolerant framework. Option E describes a general concept of caching and distributed state management in Spark, but it doesn't specifically define the programming model of Structured Streaming. Spark does maintain state information for certain operations (like windowing or aggregations), but the core programming model revolves around the unbounded table abstraction.

In essence, Structured Streaming aims to unify batch and streaming processing by providing a declarative API to define data pipelines, abstracting away many of the complexities of stream processing and allowing developers to focus on the data transformation logic. This unified approach is a key characteristic of modern data engineering practices.

Further research can be found at:

Apache Spark Structured Streaming Documentation: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Databricks Documentation on Structured Streaming: <https://docs.databricks.com/structured-streaming/index.html>

Question: 24

Deep Dumps

Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

- A.spark.sql.files.maxPartitionBytes
- B.spark.sql.autoBroadcastJoinThreshold
- C.spark.sql.files.openCostInBytes
- D.spark.sql.adaptive.coalescePartitions.minPartitionNum
- E.spark.sql.adaptive.advisoryPartitionSizeInBytes

Answer: A

Explanation:

Here's a detailed justification for why option A, spark.sql.files.maxPartitionBytes, is the configuration parameter that directly affects the size of a Spark partition during data ingestion:

When Spark reads data from files, it divides the data into partitions for parallel processing. The size of these partitions significantly impacts performance. If partitions are too small, the overhead of managing them overwhelms the processing time. If they are too large, it can lead to memory issues on executors and reduce parallelism.

spark.sql.files.maxPartitionBytes directly controls the maximum number of bytes Spark attempts to pack into a single partition when reading data from file-based sources like Parquet, Avro, or CSV. Spark aims to create partitions as close to this size as possible without splitting individual files. Increasing this value generally reduces the number of partitions created and increases the size of each partition, and vice versa.

Option B, spark.sql.autoBroadcastJoinThreshold, affects whether Spark uses a broadcast join strategy, which impacts how data is distributed for joins, not the initial partition size during data ingestion.

Option C, `spark.sql.files.openCostInBytes`, influences how Spark estimates the cost of opening a file. This value plays a role in determining the optimal number of files to read per task, affecting partitioning but less directly than `maxPartitionBytes`. It helps Spark avoid opening too many small files in one task but doesn't explicitly set a target partition size.

Option D, `spark.sql.adaptive.coalescePartitions.minPartitionNum`, governs the minimum number of partitions after adaptive query execution (AQE) coalesces partitions. It's relevant post-ingestion and during query optimization, not the initial partitioning.

Option E, `spark.sql.adaptive.advisoryPartitionSizeInBytes`, acts as a suggestion or hint to AQE during the query optimization phase. It doesn't directly determine the initial partition size at the time of data ingestion. AQE might use it to guide its repartitioning decisions, but the initial partitioning is handled separately.

Therefore, `spark.sql.files.maxPartitionBytes` is the most direct and influential parameter for controlling Spark partition size during data ingestion from file-based sources.

Authoritative Links:

Apache Spark Configuration: <https://spark.apache.org/docs/latest/configuration.html> (Search for the specific properties to find the documentation.)

Databricks Documentation on Spark Configuration: <https://docs.databricks.com/optimizations/tune-spark-jobs.html> (This provides broader guidance on Spark tuning within the Databricks environment.)

Question: 25

Deep Dumps

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum. Which situation is causing increased duration of the overall job?

- A. Task queueing resulting from improper thread pool assignment.
- B. Spill resulting from attached volume storage being too small.
- C. Network latency due to some cluster nodes being in different regions from the source data
- D. Skew caused by more data being assigned to a subset of spark-partitions.
- E. Credential validation errors while pulling data from an external system.

Answer: D

Explanation:

The correct answer is **D. Skew caused by more data being assigned to a subset of spark-partitions.**

Here's why: The Spark UI showing a vastly higher maximum task duration compared to the minimum and median durations within a stage strongly indicates **data skew**. Data skew occurs when some partitions in a Spark DataFrame/RDD contain significantly more data than others. This imbalance leads to some tasks (processing the larger partitions) taking much longer to complete than others. The "min" and "median" durations representing the tasks that process smaller, "normal" sized partitions will be similar. However, the "max" duration represents the task processing the disproportionately large partition, resulting in the observed 100x difference.

Let's eliminate the other options:

A. Task queueing resulting from improper thread pool assignment: While thread pool misconfiguration can cause delays, it generally affects all tasks fairly evenly. You wouldn't typically see a massive difference in the duration of individual tasks, but rather a delay before tasks start. Queueing would affect all tasks across

executors rather than certain partitions disproportionately.

B. Spill resulting from attached volume storage being too small: Spill can cause performance degradation, but it would likely lead to more uniformly slowed tasks rather than extreme variations. Also, spill is limited by the executor memory, not attached volume storage. The `spark.local.dir` configuration can direct the local dir to spill to attached volume storage. While the total task time would be affected, the difference between min/median/max wouldn't be as drastic as 100x if only spill was the problem. Also, it affects all tasks, not just a few.

C. Network latency due to some cluster nodes being in different regions from the source data: Network latency might increase overall job duration, but would again likely affect most tasks somewhat evenly. While certain partitions might have slight network latency, it is unlikely to cause specific task to take 100x longer than others.

E. Credential validation errors while pulling data from an external system: These errors would likely lead to task failures or potentially indefinite delays, but not consistently long task durations. It would be more of an all-or-nothing situation.

Data skew directly addresses the specific symptom of widely varying task durations. The partitions with very large data amounts cause individual tasks to take exponentially more time to process. Mitigating data skew involves techniques such as salting, bucketing, or repartitioning the data to distribute it more evenly across partitions before the stage in question.

Supporting Links:

Apache Spark Performance Tuning: Data Skew: <https://spark.apache.org/docs/latest/tuning.html#data-skew>

- This link provides detailed information on how to deal with data skew.

Databricks - Handling Skewed Data: <https://www.databricks.com/blog/2015/07/22/handling-skewed-data-in-spark.html>

Question: 26

Deep Dumps

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given a job with at least one wide transformation, which of the following cluster configurations will result in maximum performance?

- A. • Total VMs: 1
 - 400 GB per Executor
 - 160 Cores / Executor
- B. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- C. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores/Executor
- E. • Total VMs: 2
 - 200 GB per Executor
 - 80 Cores / Executor

Answer: A

Explanation:

The optimal cluster configuration for maximum performance in a Spark job with wide transformations, given the constraints, leans towards minimizing network overhead and maximizing resource locality. While all options have the same total RAM and cores, the distribution across VMs impacts performance differently.

Option A, with a single large VM, consolidates all data and processing within one node. Wide transformations, which require shuffling data across the cluster, benefit from this locality because all data transfer occurs internally within the VM's memory. This minimizes network latency, a significant bottleneck in distributed computing.

Options B, C, D, and E involve multiple VMs. Distributing the data across multiple machines introduces network overhead during the shuffle phase of wide transformations. The more VMs, the more data must traverse the network, increasing latency and decreasing performance. While having more executors can offer increased parallelism for certain operations, the penalty incurred from network shuffles outweighs the benefit in this scenario where network transfer is the major bottleneck, due to a wide transformation.

Furthermore, having only one executor per VM simplifies resource allocation and avoids potential resource contention within a single VM. This can improve predictability and stability of the job.

Therefore, option A, utilizing a single VM with all resources, offers the highest performance by eliminating network overhead during wide transformations, even though it might seemingly offer less parallelism at first glance. The benefits of local data access outweigh the reduced parallelism, which would have otherwise required data to be passed through the network.

For more information on Spark performance tuning, see:

[Spark Performance Tuning](#)

[Databricks Spark Optimization](#) (While specific to Databricks, the concepts apply broadly).

Question: 27

Deep Dumps

A junior data engineer on your team has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
    INSERT *
```

The view new_events contains a batch of records with the same schema as the events Delta table. The event_id field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same event_id as an existing record?

- A.They are merged.
- B.They are ignored.
- C.They are updated.
- D.They are inserted.
- E.They are deleted.

Answer: B

Explanation:

Correct answer is B:They are ignored.

Reference:

Question: 28

Deep Dumps

A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a bronze table created with the property `delta.enableChangeDataFeed = true`. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
 .option("readChangeFeed", "true")
 .option("startingVersion", 0)
 .table("bronze")
 .filter(col("_change_type").isin(["update_postimage", "insert"]))
 .write
 .mode("append")
 .table("bronze_history_type1")
)
```

Which statement describes the execution and results of running the above query multiple times?

- A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
- B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
- C. Each time the job is executed, the target table will be overwritten using the entire history of inserted or updated records, giving the desired result.
- D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.
- E. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.

Answer: B

Explanation:

Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.

Delta Lake's Change Data Feed (CDF) functionality allows tracking and capturing changes (inserts, updates, and deletes) in a table. When leveraging this functionality, it's important to ensure that only the new or changed records are processed to avoid duplicates.

The provided job appears to capture all changes from the bronze table and append them to the target table. However, if the job is executed daily without filtering out records that have already been processed, the entire history of changes will be appended each time, leading to multiple duplicate entries.

Question: 29

Deep Dumps

A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

- A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
- B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
- C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
- D. Data can never be permanently dropped or deleted from Delta Lake, so data loss is not possible under any circumstance.
- E. Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

Answer: E

Explanation:

The correct answer is E because ingesting raw Kafka data and its metadata into a bronze Delta table provides a complete and immutable record of the original data source. This approach ensures that even if downstream pipelines or transformations inadvertently omit a field, the original data is still available.

Here's a detailed justification:

Bronze Layer as Source of Truth: The bronze layer in a medallion architecture acts as the source of truth, containing the raw, unprocessed data. Storing Kafka data in this layer ensures that no data is lost during initial ingestion.

Immutability and Replayability: Delta Lake's append-only nature guarantees that data is not overwritten or deleted. This immutability, combined with Delta Lake's versioning capabilities, allows for replaying the Kafka stream from any point in time, even after the retention period on Kafka has expired, as the data is persisted in long-term storage.

Historical Data Access: By retaining the raw data, it becomes possible to retroactively correct errors or omissions in downstream pipelines. If the critical field was present in the original Kafka stream, it can be retrieved from the bronze table and propagated to silver and gold tables.

Metadata Preservation: Saving metadata alongside the data is crucial. This metadata can include timestamps, offsets, and other Kafka-specific information, which can be invaluable for debugging and auditing purposes.

Avoids Irreversible Data Loss: If the data engineering team realizes after three months that a critical field was missing, they can replay the Kafka stream from the bronze table to generate a new version of the silver and gold tables containing the critical field.

Why other options are incorrect:

A: Delta Lake and Structured Streaming checkpoints do not contain the full history of the Kafka producer's data; they are operational metadata for stream processing.

B: Delta Lake schema evolution is useful for adapting to changes in the data schema, but it cannot magically recreate missing data if it wasn't ingested in the first place. Schema evolution updates the table schema to accommodate new fields, but it doesn't populate historical data with values that were never present.

C: Delta Lake does not enforce that all source fields must be present. It can be configured to enforce schema, but it's not automatic.

D: While Delta Lake protects against accidental deletion and provides time travel, it doesn't prevent data loss caused by incorrect data ingestion or transformation logic. It doesn't guarantee data is never dropped under any circumstance. For instance, an improperly configured transformation could still lead to data exclusion.

Authoritative Links:

Delta Lake Bronze to Gold: <https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-transaction-log.html>

Medallion Architecture: <https://www.databricks.com/glossary/medallion-architecture>

Structured Streaming + Delta Lake: <https://spark.apache.org/docs/latest/structured-streaming-delta-lake-integration.html>

Question: 30

Deep Dumps

A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
     .format("parquet")
     .load(f"/mnt/daily_batch/{year}/{month}/{day}")
     .select("*",
             time_col.alias("ingest_time"),
             input_file_name().alias("source_file")
            )
     .write
     .mode("append")
     .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

def new_records():

A.return spark.readStream.table("bronze")

B.return spark.readStream.load("bronze")

```
    return (spark.read
            .table("bronze")
            .filter(col("ingest_time") == current_timestamp())
```

C.)

D.return spark.read.option("readChangeFeed", "true").table ("bronze")

E.

```
    return (spark.read
            .table("bronze")
            .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
            )
```

Answer: D

Explanation:

return spark.read.option("readChangeFeed", "true").table ("bronze")

Question: 31

Deep Dumps

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
- C. Human labor in writing code is the largest cost associated with data engineering workloads; as such, automating table declaration logic should be a priority in all migration workloads.
- D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.
- E. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.

Answer: D

Explanation:

The correct answer is D. Here's why:

Option D highlights a crucial aspect of data quality and control in a data lakehouse environment. While Databricks offers schema inference, it prioritizes processing data and automatically infers the schema based on the first few records. This inferred schema might use more generic data types to accommodate all observed values. For instance, if a column sometimes contains integers and sometimes strings, the inferred type might be string to avoid data loss. However, this can lead to unexpected behavior and inconsistencies downstream, potentially breaking production dashboards or models that expect specific data types. Manually defining the schema enforces strict data type constraints, ensuring that only valid data conforming to the expected schema is ingested. This promotes data quality, prevents errors, and ensures consistency for downstream applications. This is especially important when dealing with production systems where reliability and accuracy are paramount. By explicitly defining the schema, the data engineer gains greater control over the data quality and guarantees that the data aligns with the expectations of the production monitoring dashboards and the production model.

Let's analyze why the other options are incorrect:

A: While Databricks uses Tungsten for optimized data processing, relying solely on string types for all data, even with JSON support, is generally inefficient. String storage and parsing can consume more resources compared to using appropriate native data types (integer, boolean, etc.) for each field, especially when performing aggregations or calculations.

B: Delta Lake does use Parquet, but schema evolution in Delta Lake involves updating the Delta transaction log and potentially rewriting data files, not just modifying file footers. It is a more involved process for data consistency.

C: While automation is important, it should not come at the expense of careful design and data quality. In critical systems, manual intervention and verification may be necessary to ensure the accuracy and reliability of the data. The need to consider the specific data characteristics and the downstream application requirements outweighs the cost of manual code.

E: Schema inference in Databricks does not guarantee perfect type matching with downstream systems. Downstream systems might have different expectations or limitations, necessitating explicit schema definition and data type conversions.

Therefore, manually setting the schema provides more control over data quality and ensures compatibility with downstream applications, which is particularly vital for production systems.

Supporting links:

Delta Lake Schema Evolution: <https://docs.databricks.com/delta/delta-schema.html>

Databricks Data Types: <https://spark.apache.org/docs/latest/sql-ref.html#data-types>

Question: 32

Deep Dumps

The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
.select(
    orderDF.accountID,
    orderDF.itemID,

    itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
.select(
    orderWithItemDF["*"],

    accountDF.city))

(finalDF.write
    .mode("overwrite")
    .table("enriched_itemized_orders_by_account"))
```

Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A. A batch job will update the enriched_itemized_orders_by_account table, replacing only those rows that have different values than the current version of the table, using accountID as the primary key.

B. The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in

each of the three tables referenced in the join logic.

C.An incremental job will leverage information in the state store to identify unjoined rows in the source tables and write these rows to the enriched_itemized_orders_by_account table.

D.An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the enriched_itemized_orders_by_account table.

E.No computation will occur until enriched_itemized_orders_by_account is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.

Answer: B

Explanation:

The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.

Assuming that the code is logically correct and the source data is already de-duplicated and validated, the code likely includes a join operation to combine data from multiple tables (itemized_orders, account_info, and perhaps another source) to create or refresh the enriched_itemized_orders_by_account table.

Question: 33

Deep Dumps

The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels. The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

A.Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.

B.Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.

C.Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.

D.Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.

E.Because all tables must live in the same storage containers used for the database they're created in, organizations should be prepared to create between dozens and thousands of databases depending on their data isolation requirements.

Answer: A

Explanation:

The best practice for implementing the Lakehouse architecture with bronze, silver, and gold tables, considering security, collaboration, and data quality tiers, is isolating tables in separate databases based on data quality tiers. This approach aligns with the principle of least privilege, improving security by restricting access to specific data sets based on user roles and responsibilities.

Databases in Databricks (and more broadly in cloud data platforms) are not merely logical constructs; they provide a crucial level of namespace isolation and access control. By creating separate databases for bronze,

silver, and gold layers, administrators can grant granular permissions through database Access Control Lists (ACLs). This prevents unauthorized users from accessing sensitive raw data in the bronze layer while still granting access to pseudonymized or anonymized data in silver and gold layers.

Furthermore, separate databases allow for physical separation of storage locations for managed tables. This isolation can be crucial for compliance and regulatory requirements, allowing you to enforce different retention policies and encryption settings for each data tier. It's also beneficial for performance optimization; you can tailor storage settings based on the specific workload accessing each tier.

Option B is incorrect because databases significantly impact security and discoverability through ACLs and metadata organization. Option C, storing all production tables in a single database, undermines security by making it difficult to restrict access to sensitive raw data. Option D is incorrect; working in the default database is not secure and lacks proper organization. Finally, option E is impractical and inefficient. The number of databases should be based on logical data groupings and security requirements, not simply mirroring the number of tables. Database proliferation increases management overhead.

Therefore, option A offers the best balance of security, discoverability, and manageability by leveraging databases for logical and physical isolation of data tiers with varying sensitivity levels and use cases.

Relevant links:

Databricks documentation on Data Governance and Security: <https://www.databricks.com/glossary/data-governance>

Azure Data Lake Storage Gen2 Access Control Model: <https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-access-control-model>

Question: 34

Deep Dumps

The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. Whenever a database is being created, make sure that the LOCATION keyword is used
- B. When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
- C. Whenever a table is being created, make sure that the LOCATION keyword is used.
- D. When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.
- E. When the workspace is being configured, make sure that external cloud object storage has been mounted.

Answer: C

Explanation:

The correct answer is C: "Whenever a table is being created, make sure that the LOCATION keyword is used." This ensures that the Delta Lake table is external, meaning the data files are stored in a location outside the Delta Lake metastore's managed location.

Here's a detailed justification:

External vs. Managed Tables: Delta Lake supports both managed and external tables. Managed tables' data files are stored in a default location managed by the Delta Lake metastore (e.g., the Hive metastore or Unity Catalog). External tables, on the other hand, have their data files stored in a user-specified location.

The LOCATION Keyword: The LOCATION keyword in the CREATE TABLE statement explicitly specifies the storage location for the table's data files. This forces the creation of an external table. Without the LOCATION keyword, Delta Lake defaults to creating a managed table and storing the data within its own managed

directory structure.

Why Other Options are Incorrect:

A: While specifying a location during database creation can influence where managed tables within that database are stored, it doesn't inherently guarantee all tables are external. Tables can still be created without a location within that database, reverting to default managed behavior.

B: Configuring an external data warehouse and using Databricks for ELT doesn't directly enforce that all Delta Lake tables are external. The table creation process still needs to explicitly define the storage location.

D: There is no EXTERNAL keyword in standard Delta Lake CREATE TABLE syntax that automatically makes the table external. The LOCATION keyword achieves this.

E: Mounting external cloud object storage only provides access to the storage; it doesn't automatically create external tables. Tables still need to be created with a LOCATION referring to the mounted storage to be external.

Meeting the Data Architect's Mandate: By consistently using the LOCATION keyword in every CREATE TABLE statement, you ensure that all tables created are external Delta Lake tables, thus adhering to the data architect's requirement. This allows for greater control over data storage locations and independent management of data files.

For further research, refer to the following resources:

CREATE TABLE [USING]: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-using.html> - Pay attention to the LOCATION keyword.

Delta Lake Table Management: (Search for "managed vs. external tables" on the Databricks documentation) <https://docs.databricks.com/delta/index.html>

Question: 35

Deep Dumps

To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.

The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added. Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

- A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
- B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
- C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
- D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.
- E. Add a table comment warning all users that the table schema and field names will be changing on a given date; overwrite the table in place to the specifications of the customer-facing application.

Answer: B

Explanation:

The correct solution is B because it addresses the requirements with minimal disruption and without unnecessary duplication of data. Here's a detailed justification:

Minimizing disruption: Option B maintains the original schema for existing users through a view. A view is a virtual table based on a query, allowing users to continue querying the original table name and schema without modification. This avoids breaking existing dashboards, applications, and queries relying on the original table structure.

Meeting new requirements: The new table with the required fields and names serves the customer-facing application's specific needs. This ensures the application has the data it needs in the required format, preventing modification of the original table that would impact other users.

Avoiding data duplication: Using a view avoids creating a completely separate copy of the data. The view simply aliases and selects from the new table, which is the single source of truth for the underlying data. This minimizes storage costs and ensures data consistency.

Other options' drawbacks:

Option A: While notifying users is important, it shifts the burden of schema adaptation to all existing users. This is disruptive and requires them to modify their queries, leading to increased operational overhead.

Option C: Deep cloning is more appropriate for creating backups or isolated environments, not for schema evolution. It doubles the storage required, which is contrary to the goal of cost reduction.

Option D: Replacing the table with a view may impact performance, especially if the underlying query is complex. Also, creating another table specifically for one application introduces unnecessary complexity.

Option E: Overwriting the table in place will break all downstream workloads using the original schema and is therefore highly disruptive. A table comment isn't sufficient mitigation.

In summary, option B leverages the flexibility of views to provide schema compatibility for existing users while adapting the data structure to meet the new customer-facing application's requirements, minimizing disruption and cost.

Here are some authoritative links for further research:

Databricks Views: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html>

Delta Lake Deep Clone: <https://docs.databricks.com/delta/clone.html>

Question: 36

Deep Dumps

A Delta Lake table representing metadata about content posts from users has the following schema: user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE. This table is partitioned by the date column. A query is run with the following filter: longitude < 20 & longitude > -20

Which statement describes how data will be filtered?

- A. Statistics in the Delta Log will be used to identify partitions that might include files in the filtered range.
- B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
- C. The Delta Engine will use row-level statistics in the transaction log to identify the files that meet the filter criteria.
- D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.
- E. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.

Answer: D

Explanation:

The correct answer is D: Statistics in the Delta Log will be used to identify data files that might include

records in the filtered range. This is because Delta Lake maintains metadata, including statistics, about each data file in the Delta Log. These statistics include min/max values for each column within the data file.

When a query with a filter on longitude (longitude < 20 & longitude > -20) is executed, Delta Lake's query optimizer leverages these statistics. It checks the longitude min/max values for each data file recorded in the Delta Log. If the specified range (-20 to 20) overlaps with the data file's longitude range, the data file might contain relevant records.

This process enables data skipping. Delta Lake only reads data files that potentially contain records satisfying the longitude filter condition, significantly reducing the amount of data scanned and improving query performance. The partition column, date, is not relevant in this scenario because the filter is on the longitude column, not the date column. Row-level statistics aren't used; Delta Lake relies on aggregate statistics per file for skipping. Parquet footers are not directly scanned for skipping; metadata in the Delta Log is used.

Option A is partially correct but focuses on partitions, not data files. Option B is incorrect because file skipping will occur based on longitude statistics. Option C refers to row-level statistics, which are generally not used for skipping. Option E describes inefficient behavior, contradicting Delta Lake's optimization capabilities.

Therefore, Delta Lake's capability to use the Delta Log statistics to identify potentially relevant data files is the key to efficient data filtering.

Relevant documentation:

Delta Lake Statistics and Data Skipping: <https://docs.databricks.com/delta/data-skipping.html>

Delta Lake Optimization: <https://docs.databricks.com/delta/optimizations/index.html>

Question: 37

Deep Dumps

A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.

The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.

Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

A.Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.

B.Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.

C.Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.

D.Databricks leverages user workstations as the driver during interactive development; as such, users should always use a workspace deployed in a region they are physically near.

E.Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

Answer: C

Explanation:

The optimal Databricks workspace location hinges on minimizing latency and costs associated with data access. Option C correctly identifies this.

Here's why:

Data Locality: Placing compute resources (Databricks clusters) closer to the data storage reduces the physical distance data needs to travel. This minimizes latency, the delay in data transfer, which is crucial for performance-sensitive AI pipelines.

Network Costs: Cloud providers typically charge for data transfer between regions (cross-region egress). Consolidating compute and storage within the same region avoids these egress charges, leading to significant cost savings, especially with large datasets.

Performance Impact: High latency can significantly slow down data processing tasks, negatively impacting the performance of data engineering pipelines and downstream AI applications. Deploying the workspace in the same region ensures faster data reads and writes, optimizing performance.

Options A, B, D, and E are incorrect:

Option A: Databricks does not run HDFS directly on cloud volume storage in the way suggested. It uses cloud object storage like AWS S3, Azure Blob Storage, or Google Cloud Storage directly.

Option B: Databricks workspace location definitely matters due to data locality and associated costs.

Option D: While user workstation proximity can be relevant for interactive notebook responsiveness, it's secondary to the data locality consideration for production data pipelines. The driver is on the cluster, not the user's machine.

Option E: Code is not sent from the user's browser to the compute cluster over the open internet. Databricks uses secure connections and internal network infrastructure within the cloud provider.

In conclusion, for efficient data pipelines, aligning the Databricks workspace location with the data storage region is paramount, primarily due to reduced latency and cost optimization.

Further Reading:

Azure Databricks Regions: <https://azure.microsoft.com/en-us/explore/global-infrastructure/regions/> (Illustrates regional considerations in cloud deployments)

AWS Data Transfer Pricing: <https://aws.amazon.com/ec2/pricing/on-demand/> (Example of cost implications related to data transfer)

Google Cloud Data Transfer Pricing: <https://cloud.google.com/products/calculator/> (Example of cost implications related to data transfer)

Question: 38

Deep Dumps

The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid latitude and longitude values in the activity_details table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add CHECK constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

A. Because another team uses this table to support a frequently running application, two-phase locking is preventing the operation from committing.

- B.The activity_details table already exists; CHECK constraints can only be added during initial table creation.
- C.The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.
- D.The activity_details table already contains records; CHECK constraints can only be added prior to inserting values into a table.
- E.The current table schema does not contain the field valid_coordinates; schema evolution will need to be enabled before altering the table to add a constraint.

Answer: C

Explanation:

The correct option is C, with constraints, if added to an existing table the existing data in the table must be consistent with the constraint otherwise it fails.

<https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-alter-table.html#add-constraint>

Question: 39

Deep Dumps

Which of the following is true of Delta Lake and the Lakehouse?

- A.Because Parquet compresses data row by row. strings will only be compressed when a character is repeated multiple times.
- B.Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
- C.Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
- D.Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
- E.Z-order can only be applied to numeric values stored in Delta Lake tables.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with supporting information and links.

Justification for Option B: Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.

Delta Lake enhances data lake reliability and performance through several key features, and the automatic collection of statistics is a crucial aspect. Delta Lake automatically collects statistics on the first 32 columns of a Delta table when the table is created or updated. These statistics include min/max values, null counts, and other metadata for each column within each data file. This information is then used by the query optimizer for data skipping.

Data skipping significantly improves query performance. When a query has a filter condition, the query optimizer leverages the collected statistics to determine which data files do not contain relevant data based on the filter criteria. These irrelevant files are then skipped during query execution, resulting in a significant reduction in the amount of data read and processed. This reduces I/O operations, CPU usage, and overall query latency.

Why the other options are incorrect:

A. Because Parquet compresses data row by row. strings will only be compressed when a character is

repeated multiple times. Parquet uses columnar storage, not row-based storage. Columnar storage allows for better compression, especially for repeating values in the same column. String compression in Parquet is not limited to repeated characters within a row.

C. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.

Views are logical constructs defined by a query. They do not automatically maintain a cache of the most recent versions of source tables. Views are evaluated at query time.

D. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table. While constraints are being developed, Delta Lake, as of the current knowledge cutoff, does not fully enforce primary key and foreign key constraints in the same way that traditional relational databases do. You can define them, but they are not automatically enforced during write operations to prevent duplicates.

E. Z-order can only be applied to numeric values stored in Delta Lake tables. Z-ordering can be applied to string columns as well, not just numeric columns. Z-ordering is a technique to improve data locality and query performance by co-locating related data on disk based on multiple columns.

Supporting Links:

Databricks Delta Lake Documentation - Data Skipping: <https://docs.databricks.com/optimizations/data-skipping.html>

Databricks Delta Lake Documentation - Z-Ordering: <https://docs.databricks.com/optimizations/z-order.html>

Databricks Blog - Deep Dive into Delta Lake Uniqueness and Constraint Management:
<https://www.databricks.com/blog/2023/03/14/deep-dive-delta-lake-uniqueness-and-constraint-management.html>

These resources provide in-depth explanations of Delta Lake's data skipping, Z-ordering, and constraint management capabilities. Remember to always refer to the latest Databricks documentation for the most up-to-date information.

Question: 40

Deep Dumps

The view updates represents an incremental batch of all newly ingested data to be inserted or updated in the customers table.

The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_key, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
  null)
```

Which statement describes this implementation?

A. The customers table is implemented as a Type 3 table; old values are maintained as a new column alongside the current value.

B.The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

C.The customers table is implemented as a Type 0 table; all writes are append only with no changes to existing values.

D.The customers table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.

E.The customers table is implemented as a Type 2 table; old values are overwritten and new customers are appended.

Answer: B

Explanation:

The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

Question: 41

Deep Dumps

The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.

What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

A.Can Manage

B.Can Edit

C.No permissions

D.Can Read

E.Can Run

Answer: D

Explanation:

The correct answer is D, "Can Read." Here's why:

The core objective is to grant access for reviewing production logic without risking unintended modifications to the code or the resulting data. The principle of least privilege dictates granting only the minimum necessary permissions to achieve the task.

A. Can Manage: "Manage" permissions provide the user with complete control over the notebook. They can edit, run, delete, change permissions, and essentially do anything with the notebook. This clearly violates the requirement to prevent accidental changes to production code.

B. Can Edit: "Edit" permissions, as the name suggests, allow the user to modify the notebook's content. This directly contradicts the requirement to avoid accidental code changes. Granting "Edit" access presents a high risk of inadvertently altering the production logic, which is unacceptable.

C. No permissions: This would prevent the new hire from reviewing the code, making it impossible for them to onboard or understand the production logic, therefore, failing to meet the user's requirements.

D. Can Read: "Read" permissions allow the user to view the notebook's contents, including the code and any markdown explanations. They cannot modify the notebook in any way. This is the ideal permission level for code review as it satisfies the requirement to review the production logic while preventing any accidental changes to production notebooks and data.

E. Can Run: "Run" permissions allows the user to execute the notebook. While they cannot change the code, running the production notebook could inadvertently trigger data processing or modifications within the environment, which can lead to unintended side effects in the production system. Since the requirement is only to review the code, running the notebook is not necessary and adds unnecessary risk.

Therefore, "Can Read" is the most restrictive permission level that still allows the user to accomplish the requested task of reviewing the notebook content. This aligns with the principle of least privilege and safeguards the production environment.

Relevant Documentation:

Databricks Notebook Permissions: While specific documentation directly detailing the impacts of each permission level on Jobs is limited, general Databricks ACL and Permissions documentation can give insight. Searching the Databricks documentation for "Access Control Lists" or "ACLs" and "Notebook Permissions" will provide related information.

Question: 42

Deep Dumps

A table named `user_ltv` is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The `user_ltv` table has the following schema:

`email` STRING, `age` INT, `ltv` INT

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
    is_member('marketing') THEN email
    ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv -
```

Which statement describes the results returned by this query?

- A. Three columns will be returned, but one column will be named "REDACTED" and contain only null values.
- B. Only the email and ltv columns will be returned; the email column will contain all null values.
- C. The email and ltv columns will be returned with the values in `user_ltv`.
- D. The email, age, and ltv columns will be returned with the values in `user_ltv`.
- E. Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

Answer: E

Explanation:

Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each

row.

Question: 43

Deep Dumps

The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PII) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property "contains_pii" = true.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

- A.DESCRIBE EXTENDED dev.pii_test
- B.DESCRIBE DETAIL dev.pii_test
- C.SHOW TBLPROPERTIES dev.pii_test
- D.DESCRIBE HISTORY dev.pii_test
- E.SHOW TABLES dev

Answer: A

Explanation:

DESCRIBE EXTENDED dev.pii_test.

Question: 44

Deep Dumps

The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named users.

```
DELETE FROM users
WHERE user_id IN
(SELECT user_id FROM delete_requests)
```

Assuming that user_id is a unique identifying key and that delete_requests contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

- A.Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- B.No; the Delta cache may return records from previous versions of the table until the cluster is restarted.
- C.Yes; the Delta cache immediately updates to reflect the latest data files recorded to disk.
- D.No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- E.No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Answer: E

Explanation:

No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Question: 45**Deep Dumps**

An external object storage container has been mounted to the location /mnt/finance_eda_bucket. The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the finance group, which statement describes how the tx_sales table will be created?

- A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
- B. An external table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- C. A logical table will persist the physical plan to the Hive Metastore in the Databricks control plane.
- D. An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- E. A managed table will be created in the DBFS root storage container.

Answer: D**Explanation:**

An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

Reference:

<https://docs.databricks.com/en/data-governance/unity-catalog/create-schemas.html#language-SQL>

Question: 46**Deep Dumps**

Although the Databricks Utilities Secrets module provides tools to store sensitive credentials and avoid accidentally displaying them in plain text users should still be careful with which credentials are stored here and which users have access to using these secrets.

Which statement describes a limitation of Databricks Secrets?

- A. Because the SHA256 hash is used to obfuscate stored secrets, reversing this hash will display the value in plain text.
- B. Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console.
- C. Secrets are stored in an administrators-only table within the Hive Metastore; database administrators have permission to query this table by default.

D.Iterating through a stored secret and printing each character will display secret contents in plain text.

E.The Databricks REST API can be used to list secrets in plain text if the personal access token has proper credentials.

Answer: B

Explanation:

The correct answer is B: Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console. Here's why:

Databricks Secrets are designed to protect sensitive information by storing them securely in a backend system (e.g., Azure Key Vault, AWS Secrets Manager). While Databricks obscures these secrets within the Databricks workspace itself, making them difficult for regular users to access directly, a key limitation arises from the access granted to account administrators.

Account administrators typically possess elevated privileges to manage the entire Databricks account, including workspace settings, user permissions, and security configurations. This level of access can include the ability to view secrets in plaintext through the Databricks Accounts console or through API calls utilizing their administrative credentials. This is because, to effectively manage and troubleshoot the account, administrators sometimes require the ability to audit or verify security configurations related to secret usage.

Options A, C, D, and E are incorrect:

A is incorrect: Databricks Secrets use encryption, not hashing (like SHA256), to protect secrets. Encryption is reversible with the correct key, while a hash is not.

C is incorrect: Secrets are not stored in the Hive Metastore. They're stored in a secure secret store (like Azure Key Vault or AWS Secrets Manager). The Hive Metastore stores metadata about data tables, not sensitive credentials.

D is incorrect: Databricks is designed not to print secrets in plain text even when iterating. Trying to directly access the secret will return a redacted value or an error, preventing accidental exposure.

E is incorrect: The Databricks REST API will not return secrets in plain text, even with proper credentials. It only provides operations to manage the secret's existence and scope, not to reveal its content.

In summary, while Databricks Secrets offer a layer of security, the principle of least privilege applies. The fact that account administrators can view secrets in plaintext necessitates carefully managing who holds those administrative roles and implementing robust auditing practices.

Authoritative Links:

Databricks Secrets: <https://docs.databricks.com/security/secrets/index.html>

Azure Key Vault: <https://learn.microsoft.com/en-us/azure/key-vault/general/basic-concepts>

AWS Secrets Manager: <https://aws.amazon.com/secrets-manager/>

Question: 47

Deep Dumps

What statement is true regarding the retention of job run history?

A.It is retained until you export or delete job run logs

B.It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3

C.It is retained for 60 days, during which you can export notebook run results to HTML

D.It is retained for 60 days, after which logs are archived

E.It is retained for 90 days or until the run-id is re-used through custom run configuration

Answer: C

Explanation:

The correct answer is C: It is retained for 60 days, during which you can export notebook run results to HTML.

Here's a detailed justification:

Databricks retains the history of job runs for a specific period to enable monitoring, debugging, and auditing of past executions. Understanding this retention policy is crucial for managing job run data and archiving necessary information before it's purged. While logs are critical, Databricks provides options to export various artifacts, including notebook run results.

Option A is incorrect because Databricks automatically manages job run retention; it's not solely contingent on manual export or deletion. Option B is incorrect because the retention period exceeds 30 days. Option D is partially correct about the 60-day period but incorrect in that logs are archived. Instead they are purged. Option E is incorrect because the 90-day retention claim is inaccurate, and run-id re-use doesn't directly affect retention.

Option C is correct because Databricks indeed retains job run history for 60 days. During this period, users can export the results of notebook executions, often in HTML format, for archival or analysis. This allows for detailed inspection of the notebook's output, including visualizations and data, at a later time. The ability to export results is particularly beneficial for long-term monitoring or compliance purposes. This option is also the only one mentioning specifically notebook run results.

Further research can be conducted using the official Databricks documentation:

Databricks Job Monitoring: <https://docs.databricks.com/workflows/jobs/index.html#monitor-jobs>

Databricks Job API: <https://docs.databricks.com/api/workspace/jobs> (while not directly addressing retention period, it shows the API interface for interacting with job runs, implying the existence of a retention mechanism).

Question: 48

Deep Dumps

A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens. Which statement describes the contents of the workspace audit logs concerning these events?

- A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
- B. Because User B last configured the jobs, their identity will be associated with both the job creation events and the job run events.
- C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
- D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.
- E. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most accurate description of the Databricks audit logs in the given scenario:

The core concept here is that Databricks audit logs record actions based on the identity that initiated the action. In this case, two distinct actions are happening: job creation and job run triggering.

User A created the jobs programmatically using the REST API and their personal access token. Therefore, the audit logs will record User A's identity as the entity that performed the job creation actions. This is a direct consequence of the REST API using User A's personal access token to authenticate the requests. Databricks associates actions with the user associated with the token used.

Subsequently, User B configured an external orchestration tool (also using the REST API and their personal access token) to trigger job runs. When the orchestration tool executes and makes REST API calls to trigger the job runs, it's doing so under User B's identity. This is because User B's personal access token is being used for authentication.

Therefore, the audit logs will separately record User B's identity as the entity that triggered the job run events. It's crucial to remember that Databricks auditing aims to track the specific user or service principal responsible for each discrete action. Since job creation and run triggering are separate actions initiated by different authenticated entities, they are logged separately.

Option A is incorrect because a Service Principal would need to be explicitly configured and used for authentication; simply using the REST API does not automatically trigger Service Principal usage. Option B is incorrect because while User B configured the job runs, they did not create the jobs themselves. Option D is incorrect as user identity is captured in the audit logs when using personal access tokens. Option E is also incorrect because User A only created the jobs and did not trigger the runs. The audit logs track distinct actions separately.

Here are some relevant links for further research:

Databricks Audit Logging: <https://docs.databricks.com/administration-guide/account-settings/audit-logs.html>

Databricks REST API Authentication: <https://docs.databricks.com/dev-tools/api/latest/authentication.html>

Question: 49

Deep Dumps

A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using `display()` calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.

Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

A. Scala is the only language that can be accurately tested using interactive notebooks; because the best performance is achieved by using Scala code compiled to JARs, all PySpark and Spark SQL logic should be refactored.

B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.

C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.

D. Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

E. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.

Answer: B

Explanation:

The most accurate measurement of production performance is achieved by mimicking production conditions as closely as possible during testing. This includes using data volumes and cluster configurations that are representative of the production environment. Let's analyze why the other options are less optimal. Option A is incorrect because it unnecessarily limits the user to Scala. PySpark and Spark SQL are perfectly viable and performant options in Databricks. Option C proposes using a local build of open-source Spark and Delta Lake, which will likely have a different configuration and environment compared to the Databricks runtime, leading to inaccurate performance assessments. Option D correctly identifies the impact of `display()` and caching, but it doesn't offer a comprehensive solution for accurate performance testing. Option E suggests using the Jobs UI, but it falsely claims Photon can only be enabled for scheduled jobs. Photon is available for interactive clusters as well.

Therefore, option B is the best choice. "Run All" ensures the entire pipeline is executed, revealing the performance bottlenecks that cell-by-cell execution might mask. Utilizing production-sized data accurately reflects the system's behavior under realistic loads. Production-sized clusters ensure sufficient resources are allocated, avoiding artificial constraints that might occur on smaller development clusters. Mimicking production settings allows for reliable benchmarking.

For more information on Databricks performance tuning, refer to the official Databricks documentation:

[Databricks Performance Tuning](#)
[Monitor Databricks Workloads](#)

Question: 50

Deep Dumps

A production cluster has 3 executor nodes and uses the same virtual machine type for the driver and executor. When evaluating the Ganglia Metrics for this cluster, which indicator would signal a bottleneck caused by code executing on the driver?

- A. The five Minute Load Average remains consistent/flat
- B. Bytes Received never exceeds 80 million bytes per second
- C. Total Disk Space remains constant
- D. Network I/O never spikes
- E. Overall cluster CPU utilization is around 25%

Answer: D

Explanation:

The correct answer is **D. Network I/O never spikes**. Here's a detailed justification:

A bottleneck on the driver node, particularly due to code execution, manifests as increased network activity. The driver orchestrates tasks, manages state, and communicates with executors. Heavy driver-side computations often involve fetching data, aggregating results, or coordinating complex operations that necessitate frequent network interactions.

If the driver is struggling with computations, it may not be able to keep up with sending instructions or receiving results from the executors quickly enough. This causes the network I/O to plateau, failing to reflect the actual processing potential of the cluster. The driver essentially becomes a choke point, limiting overall cluster throughput. A lack of network I/O spikes, while executors are otherwise idle (as potentially inferred from low overall CPU), suggests the driver isn't sending or receiving data at its optimal rate.

Option A is incorrect because a consistent load average doesn't specifically point to a driver bottleneck. It could indicate overall system load, regardless of where the bottleneck resides. Option B is incorrect because a constant Bytes Received rate might indicate limitations on another part of the infrastructure rather than the

driver node. Option C about Total Disk Space remaining constant doesn't directly relate to a driver-side CPU bottleneck. Option E, low overall CPU utilization, is partially relevant because a driver bottleneck prevents executors from being fully utilized, but it's not the most direct indicator. Network I/O provides a more precise signal about the driver's ability to communicate with the executors.

Therefore, the lack of network I/O spikes is the strongest indicator of a driver-side computational bottleneck, specifically when other metrics (implicitly or explicitly) suggest the cluster has more compute capacity than is currently being leveraged. This indicates that the bottleneck is not in processing but data movement, which is usually managed by the driver.

For more information, consult the following resources:

Apache Spark Documentation on Monitoring: Look for details about monitoring metrics like network I/O and CPU utilization. While the question specifically talks about Ganglia, the principles are similar. You can find documentation about monitoring these metrics using Spark's built-in UI and external monitoring tools.<https://spark.apache.org/docs/latest/monitoring.html>

Databricks Documentation on Cluster Performance: Check the Databricks documentation on troubleshooting cluster performance issues, which often includes guidance on identifying driver bottlenecks and network limitations.<https://docs.databricks.com/en/clusters/>

Question: 51

Deep Dumps

Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

- A. In the Executor's log file, by grepping for "predicate push-down"
- B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
- C. In the Storage Detail screen, by noting which RDDs are not stored on disk
- D. In the Delta Lake transaction log, by noting the column statistics
- E. In the Query Detail screen, by interpreting the Physical Plan

Answer: E

Explanation:

The correct answer is **E. In the Query Detail screen, by interpreting the Physical Plan**. Here's why:

Predicate pushdown is a performance optimization technique in Spark (and other database systems) where filtering operations (predicates) are applied as early as possible in the query execution plan, ideally at the data source level. This reduces the amount of data that needs to be read and processed, leading to significant performance improvements.

The Spark UI's Query Detail screen is the most appropriate place to diagnose a lack of predicate pushdown because it displays the **Physical Plan** of your query. The Physical Plan visualizes how Spark intends to execute your query.

Why the Physical Plan matters: By examining the Physical Plan, you can see exactly where filtering operations are occurring. If predicate pushdown is working correctly, you'll see filters being applied before data is read from the data source (e.g., before a Parquet scan or a Delta table read). If it's not working, you'll see that Spark is reading a large amount of data and then applying the filter, indicating a performance bottleneck.

Example: Imagine you are querying a large table with a WHERE clause limiting the data to a specific date range. If predicate pushdown is enabled and effective, the Physical Plan will show the data source (e.g.,

Parquet file) being filtered before it's read into Spark. If it's not working, you'll see Spark reading the entire Parquet file and then applying the date filter, which is less efficient.

Let's address why the other options are less suitable:

A. Executor Logs: While executor logs contain valuable information, grepping for "predicate push-down" is unlikely to provide a clear indication of whether it's effectively working or not. You won't see the Physical Plan.

B. Stage Detail Screen: The Stage Detail screen shows metrics about stages of execution, including data read. A large input size might suggest a problem, but it doesn't pinpoint the reason (lack of predicate pushdown) as directly as the Physical Plan does. There may be other reasons to read a large input such as large schemas.

C. Storage Detail Screen: The Storage Detail screen focuses on RDD persistence. While caching can improve performance, it doesn't directly relate to predicate pushdown. The core issue is the amount of input to spark not caching behaviour.

D. Delta Lake Transaction Log: The Delta Lake transaction log primarily records metadata about changes to the Delta table. It does contain column statistics which enable predicate pushdown, but it doesn't directly show whether predicate pushdown actually occurred in a specific query. While the metadata supports predicate pushdown, seeing it active is different.

Therefore, the Query Detail screen and its Physical Plan provide the clearest and most direct way to diagnose performance problems caused by a lack of predicate pushdown. You can see visually whether filters are being applied early in the process, which is crucial for optimization.

Authoritative Links:

Apache Spark Documentation on Understanding the Query Plan: <https://spark.apache.org/docs/latest/sql-performance-tuning.html> (Look for sections explaining how to analyze the query plan.)

Databricks Documentation on Predicate Pushdown: (Search Databricks documentation for "predicate pushdown" for specific details on how it works with Databricks and Delta Lake; you'll often find it within Delta Lake performance tuning documentation.) Databricks documentation is often proprietary and requires an account, so be wary of linking directly to specific pages that might require authentication to view.

Question: 52

Deep Dumps

Review the following error traceback:

```

-----
AnalysisException                                Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
1693         return DataFrame(jdf, self.sql_ctx)
1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
1302
1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
1305             answer, self.gateway_client, self.target_id, self.name)
1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
121         # Hide where the exception came from that shows a non-Pythonic
122         # JVM exception message.
--> 123         raise converted from None
124     else:
125         raise

AnalysisException: cannot resolve '`heartrateheartrateheartrate`' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet

```

Which statement describes the error being raised?

- A.The code executed was PySpark but was executed in a Scala notebook.
- B.There is no column in the table named heartrateheartrateheartrate
- C.There is a type error because a column object cannot be multiplied.
- D.There is a type error because a DataFrame object cannot be multiplied.
- E.There is a syntax error because the heartrate column is not correctly identified as a column.

Answer: B

Explanation:

It's B, there is no column with that name.

Question: 53

Deep Dumps

Which distribution does Databricks support for installing custom Python code packages?

- A.sbt
- B.CRANC. npm
- D.Wheels
- E.jars

Answer: D

Explanation:

The correct answer is indeed D, Wheels. Here's a detailed justification:

Databricks, a leading cloud-based data engineering platform built on Apache Spark, leverages Python extensively. To extend the functionality of Databricks clusters with custom or third-party Python code, users need a mechanism to package and distribute these code packages. Databricks supports the installation of Python packages using the pip package installer, the standard package installer for Python. Pip relies on the Wheel format for distributing Python packages.

Wheels (extension .whl) are a built-package format for Python. They are designed to be easily installed without requiring compilation, making them faster and more reliable compared to source distributions. This is crucial in a cloud environment like Databricks where minimizing deployment time and ensuring consistency are paramount. Wheels contain all the necessary files and metadata for a Python package in a pre-built format, ready for installation.

Options A (sbt), B (CRAN), C (npm), and E (jars) are incorrect because they pertain to different package management ecosystems. sbt is a build tool for Scala projects, CRAN is the Comprehensive R Archive Network for R packages, npm is the Node Package Manager for JavaScript packages, and jars are Java Archive files used for distributing Java code. While Databricks supports Scala, R, Java, and JavaScript through various means, when it comes to Python, Wheels are the primary supported distribution format.

Therefore, to reliably install custom Python libraries and dependencies on Databricks clusters, using the Wheel format is the recommended and supported approach, ensuring compatibility and efficient deployment. Databricks provides mechanisms to install Wheels directly from DBFS (Databricks File System), cloud storage, or through a custom package index.

For further research, you can refer to the official Databricks documentation on libraries and package management:

Databricks documentation on Libraries: <https://docs.databricks.com/libraries/index.html>

Pip documentation: <https://pip.pypa.io/en/stable/>

Wheel documentation: <https://packaging.python.org/en/latest/specifications/binary-distribution-format/>

Question: 54**Deep Dumps**

Which Python variable contains a list of directories to be searched when trying to locate required modules?

- A.importlib.resource_path
- B.sys.path
- C.os.path
- D.pypi.path
- E.pylib.source

Answer: B**Explanation:**

The correct answer is B, sys.path. This variable, accessible through the sys module in Python, holds a list of directory paths that the Python interpreter searches when importing modules. When you execute an import statement, Python iterates through the paths specified in sys.path in the order they appear, looking for the module file (either a .py file, a .pyc file, or a compiled extension).

Option A, importlib.resource_path, is related to accessing resources packaged within a module, not the general

module search path. Option C, `os.path`, is a module containing functions for manipulating file paths, but it doesn't hold the list of directories searched for modules. Option D, `pypi.path`, is incorrect because `pypi` refers to the Python Package Index, which is a repository of software packages, and doesn't have a direct path attribute. Option E, `pylib.source`, is incorrect as `pylib` isn't a standard python module, and it doesn't contain information on module source paths.

`sys.path` is crucial for managing module resolution in Python environments, especially within cloud environments like Databricks where custom libraries or modules may be stored in non-standard locations. Modifying `sys.path` allows you to import modules from specific directories, which can be useful for organizing project dependencies and ensuring that the correct versions of libraries are used. In Databricks, you often deal with managing various dependencies to support data engineering pipelines, and `sys.path` enables flexibility in locating and importing these required modules.

For more information, refer to the Python documentation on the `sys` module and the `import` statement.<https://docs.python.org/3/library/sys.html><https://docs.python.org/3/reference/import.html>

Question: 55

Deep Dumps

Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code. Which statement describes a main benefit that offset this additional effort?

- A.Improves the quality of your data
- B.Validates a complete use case of your application
- C.Troubleshooting is easier since all steps are isolated and tested individually
- D.Yields faster deployment and execution times
- E.Ensures that all steps interact correctly to achieve the desired end result

Answer: C

Explanation:

The correct answer is C: "Troubleshooting is easier since all steps are isolated and tested individually." Here's why:

Unit testing inherently focuses on isolating individual components (functions, classes, or small units of code) within a larger application and verifying their correct behavior independently. This is a fundamental principle of software engineering. In a PySpark application, this translates to testing individual transformations, data cleansing operations, or aggregation steps.

By isolating and testing each step, you can quickly identify the exact location where an error occurs. If a unit test fails, it pinpoints the specific piece of code responsible, reducing the scope of your investigation. This contrasts sharply with debugging a complex, monolithic Spark job where tracing errors through multiple interconnected transformations can be time-consuming and difficult.

Options A, B, D, and E are less directly related to the main benefit that offsets the effort of incorporating unit tests. While unit tests can indirectly contribute to data quality (A) by ensuring individual data transformations are correct, that's not the primary benefit. Validating a complete use case (B) is more the domain of integration or end-to-end tests. Unit tests don't inherently yield faster deployment times (D), although they can indirectly improve code quality which might lead to more stable deployments. Ensuring steps interact correctly (E) is again more the domain of integration testing; unit tests focus on individual components, not their interactions.

The effort in upfront design or refactoring to enable unit testing in PySpark is compensated by the significantly improved ease of troubleshooting. When issues arise (and they inevitably do in complex data pipelines), unit tests provide a fast and targeted way to locate and fix the root cause, saving valuable debugging time. This makes the entire development and maintenance process more efficient.

For further reading, explore these resources:

Spark Testing Base: <https://github.com/holdenk/spark-testing-base> (A library facilitating unit testing of Spark applications)

Testing in PySpark: Search for "PySpark unit testing" or "testing Spark applications" online for various blog posts and tutorials.

In summary, the major advantage of incorporating unit tests into a PySpark application is the significant reduction in troubleshooting time and effort due to the isolation and independent verification of individual code components.

Question: 56

Deep Dumps

Which statement describes integration testing?

- A. Validates interactions between subsystems of your application
- B. Requires an automated testing framework
- C. Requires manual intervention
- D. Validates an application use case
- E. Validates behavior of individual elements of your application

Answer: A

Explanation:

The correct answer is A: Validates interactions between subsystems of your application.

Integration testing focuses on verifying the communication and data flow between different modules or components of a software system. It aims to uncover defects that arise when independently developed units are combined. These defects often relate to interface issues, data inconsistencies, or unexpected interactions.

Option B is incorrect because integration testing doesn't necessarily require an automated framework, though automation significantly improves its efficiency and repeatability, particularly in complex systems. Manual integration testing is feasible for smaller systems.

Option C is incorrect because while manual intervention can be part of integration testing, it's not a requirement. Automated tests can be designed to run without manual intervention, providing quicker feedback.

Option D is incorrect because validating an application use case is more closely aligned with end-to-end or system testing. End-to-end testing ensures the entire application works as expected from start to finish, mimicking real-world user scenarios.

Option E is incorrect because validating the behavior of individual elements of the application describes unit testing. Unit testing focuses on testing individual functions, classes, or modules in isolation.

In essence, integration testing bridges the gap between unit testing (individual components) and system testing (the entire application). It validates that the different parts of the system work together harmoniously. A strong integration testing strategy is critical to building reliable and robust software.

For further reading on Integration Testing, refer to the following resources:

Guru99: <https://www.guru99.com/integration-testing.html>

BrowserStack: <https://www.browserstack.com/guide/integration-testing>

Question: 57

Deep Dumps

Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

- A. /jobs/runs/list
- B. /jobs/runs/get-output
- C. /jobs/runs/get
- D. /jobs/get
- E. /jobs/list

Answer: D

Explanation:

The correct REST API call to review notebooks configured as tasks within a multi-task Databricks job is /jobs/get. This is because the /jobs/get endpoint retrieves the full details of a specified job, including the configuration of each task within the job. That configuration would include the notebook path or other task details, allowing you to review which notebooks are used.

Option A, /jobs/runs/list, is used to list the historical runs of a job, not the job's configuration. Option B, /jobs/runs/get-output, is designed to retrieve the output of a specific job run, not the underlying task definitions. Similarly, /jobs/runs/get (Option C) retrieves information about a specific job run, not the job configuration itself. Finally, /jobs/list (Option E) only provides a listing of all jobs, without detailed task configurations for individual jobs. Therefore, to inspect the configuration of a job, including the notebooks used in tasks, /jobs/get is the only appropriate REST API endpoint.

Authoritative documentation can be found at [Databricks Jobs API](#). Specifically, examine the section detailing the GET /jobs/ job_id endpoint. This provides the specifications for extracting job configurations, which will contain the notebook definitions.

Question: 58

Deep Dumps

A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A. If tasks A and B complete successfully but task C fails during a scheduled run, which statement describes the resulting state?

- A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.
- B. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.
- C. All logic expressed in the notebook associated with task A will have been successfully completed; tasks B and C will not commit any changes because of stage failure.
- D. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
- E. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.

Answer: A

Explanation:

The correct answer is **A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.**

Here's a detailed justification:

Databricks Jobs execute tasks based on a defined dependency graph. In this scenario, Task A runs independently, and its successful completion is a prerequisite for Tasks B and C. Since Tasks B and C run in parallel after Task A, the successful completion of A means all operations within its associated notebook have been executed and committed. Task B also completes successfully, indicating its notebook's logic was entirely executed and its results committed.

Task C failing doesn't automatically roll back the successful operations of Tasks A and B. Databricks doesn't employ an all-or-nothing, atomic transaction model across independent job tasks by default. While Databricks supports ACID transactions within a given task, it doesn't guarantee that a failure in one task will roll back the results of other, already completed tasks in a job.

Task C's failure indicates that the notebook associated with it encountered an error during execution. Some operations within the task C notebook might have been executed and have produced side effects before the point of failure. Therefore, the statement "some operations in task C may have completed successfully" is accurate.

Options B, C, D, and E are incorrect because they assume a level of atomicity and rollback capability across independent job tasks that Databricks doesn't provide by default. While transaction control is possible within a Databricks task using techniques like BEGIN TRANSACTION, COMMIT, and ROLLBACK in Spark SQL or similar approaches, a simple task failure doesn't inherently trigger a rollback of successful tasks that depend on it.

In summary, the partial completion of Task C, along with the successful completion of tasks A and B, demonstrates how independent tasks within a Databricks job can execute and commit independently of each other's success, which means option A is the only possibility.

Further research:

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html>

Delta Lake ACID Transactions: <https://docs.databricks.com/delta/transaction-isolation.html>

Question: 59

Deep Dumps

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

- A. The table reference in the metastore is updated and no data is changed.
- B. The table name change is recorded in the Delta transaction log.
- C. All related files and metadata are dropped and recreated in a single ACID transaction.
- D. The table reference in the metastore is updated and all data files are moved.

E. A new Delta transaction log is created for the renamed table.

Answer: A

Explanation:

A. The table reference in the metastore is updated and no data is changed.

When an ALTER TABLE ... RENAME TO ... command is executed on a Delta Lake table, it is a metadata-only operation within the metastore (e.g., Hive Metastore or Unity Catalog). The physical data files and their storage location in cloud object storage are not moved or altered. The table's new name is updated as a pointer to the existing data location in the metastore, which is then recorded in the Delta transaction log.

Question: 60

Deep Dumps

The data engineering team maintains a table of aggregate statistics through batch nightly updates. This includes total sales for the previous day alongside totals and averages for a variety of time periods including the 7 previous days, year-to-date, and quarter-to-date. This table is named store_sales_summary and the schema is as follows:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd FLOAT,
avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT, avg_daily_sales_7d
FLOAT, updated TIMESTAMP
```

The table daily_store_sales contains all the information needed to update store_sales_summary. The schema for this table is: store_id INT, sales_date DATE, total_sales FLOAT

If daily_store_sales is implemented as a Type 1 table and the total_sales column might be adjusted after manual data auditing, which approach is the safest to generate accurate reports in the store_sales_summary table?

- A. Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and overwrite the store_sales_summary table with each Update.
- B. Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and append new rows nightly to the store_sales_summary table.
- C. Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and use upsert logic to update results in the store_sales_summary table.
- D. Implement the appropriate aggregate logic as a Structured Streaming read against the daily_store_sales table and use upsert logic to update results in the store_sales_summary table.
- E. Use Structured Streaming to subscribe to the change data feed for daily_store_sales and apply changes to the aggregates in the store_sales_summary table with each update.

Answer: E

Explanation:

Use Structured Streaming to subscribe to the change data feed for daily_store_sales and apply changes to the aggregates in the store_sales_summary table with each update.

Question: 61

Deep Dumps

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

Cmd 1

```
rawDF = spark.table("raw_data")
```

Cmd 2

```
rawDF.printSchema()
```

Cmd 3

```
flattenedDF = rawDF.select("*", "values.*")
```

Cmd 4

```
finalDF = flattenedDF.drop("values")
```

Cmd 5

```
finalDF.explain()
```

Cmd 6

```
display(finalDF)
```

Cmd 7

```
finalDF.write.mode("append").saveAsTable("flat_data")
```

Which command should be removed from the notebook before scheduling it as a job?

- A.Cmd 2
- B.Cmd 3
- C.Cmd 4
- D.Cmd 5
- E.Cmd 6

Answer: E

Explanation:

Correct answer is E:Cmd 6.

When scheduling a Databricks notebook as a job, it's generally recommended to remove or modify commands that involve displaying output, such as using the `display()` function. Displaying data using `display()` is an

interactive feature designed for exploration and visualization within the notebook interface and may not work well in a production job context.

The `finalDF.explain()` command, which provides the execution plan of the DataFrame transformations and actions, is often useful for debugging and optimizing queries. While it doesn't display interactive visualizations like `display()`, it can still be informative for understanding how Spark is executing the operations on your DataFrame.

Question: 62

Deep Dumps

The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

- A. Manually trigger a job anytime the business reporting team refreshes their dashboards
- B. Schedule a job to execute the pipeline once an hour on a new job cluster
- C. Schedule a Structured Streaming job with a trigger interval of 60 minutes
- D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster
- E. Configure a job that executes every time new data lands in a given directory

Answer: B

Explanation:

The correct answer is B. Here's why:

Service-Level Agreement (SLA) Compliance: The business requirement is to update the dashboards hourly. Option B directly addresses this by scheduling a job to run every hour.

Cost Efficiency:

Option B (New Job Cluster): This approach creates a new Databricks cluster specifically for each hourly run, executes the pipeline, and then shuts down the cluster. Databricks clusters are billed by the second, but only when running. Since the pipeline only takes 10 minutes, the cluster will only run for that duration, minimizing cost.

Option A (Manual Trigger): Requires manual intervention, which is unsustainable and error-prone. It is also difficult to enforce the hourly SLA.

Option C (Structured Streaming): Structured Streaming is designed for continuous data ingestion and processing. While you could use a trigger interval of 60 minutes, it may not be ideal for batch-oriented pipelines. Additionally, it's usually associated with a long-running cluster, potentially leading to higher costs.

Option D (Dedicated Interactive Cluster): Keeps a Databricks cluster running 24/7. This is the most expensive option because you pay for the cluster even when it's idle for 50 minutes each hour.

Option E (Trigger on Data Arrival): The data arrival pattern may not be consistent with the hourly requirement. If data arrives more or less frequently than once an hour, the dashboards will not be updated as required.

In summary, scheduling a Databricks job to execute the pipeline once an hour on a new job cluster (Option B) meets the SLA requirements at the lowest cost because it only runs the cluster when processing data, avoiding unnecessary idle time charges.

Further research:

Databricks Jobs: <https://docs.databricks.com/jobs.html>

Databricks Cost Management: <https://docs.databricks.com/administration-guide/account-settings/cost-management.html>

Spark Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Question: 63

Deep Dumps

A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

```
SELECT COUNT (*) FROM table -
```

Which of the following describes how results are generated each time the dashboard is updated?

- A. The total count of rows is calculated by scanning all data files
- B. The total count of rows will be returned from cached results unless REFRESH is run
- C. The total count of records is calculated from the Delta transaction logs
- D. The total count of records is calculated from the parquet file metadata
- E. The total count of records is calculated from the Hive metastore

Answer: C

Explanation:

The correct answer is C: "The total count of records is calculated from the Delta transaction logs."

Here's a detailed justification:

Delta Lake enhances Parquet data files with a transaction log (Delta log). This log meticulously records every change made to the Delta table, including additions, deletions, and modifications. Crucially, it also tracks metadata about the table, including the total number of records.

When a `SELECT COUNT(*) FROM table` query is executed against a Delta Lake table, Databricks can efficiently retrieve the row count from the Delta log, rather than scanning the entire dataset. This optimization is possible because the Delta log maintains a consistent and up-to-date record of the table's metadata, including the total row count after each transaction. This is significantly faster than scanning all the underlying Parquet files, which would be necessary in a traditional data lake.

The other options are incorrect because:

A. The total count of rows is calculated by scanning all data files: This is inefficient and defeats the purpose of Delta Lake's optimizations. Delta Lake aims to avoid full table scans for simple aggregate queries like `COUNT(*)`.

B. The total count of rows will be returned from cached results unless REFRESH is run: While caching can improve performance, the initial count must still come from somewhere. The Delta log is the source. Also, dashboards typically refresh periodically, so relying solely on cached results could lead to outdated information. While result caching exists within Databricks SQL, it doesn't invalidate the core logic of how `COUNT(*)` is optimized with Delta Lake.

D. The total count of records is calculated from the parquet file metadata: Parquet file metadata typically stores statistics about the data within that specific file, not the total number of records in the table. While Parquet stores row counts in the file footer, aggregating these counts from all files is still more expensive and

less efficient than retrieving the count from the Delta Log.

E. The total count of records is calculated from the Hive metastore: The Hive metastore primarily stores schema information and table locations, not detailed record counts. While integrations may extract basic stats, relying on the metastore for an accurate COUNT(*) would lack the transactional guarantees and freshness provided by the Delta Log. The Hive metastore has no understanding of delta logs' fine-grained change tracking and versioning.

Therefore, Delta Lake leverages its transaction log to provide a fast and accurate response to COUNT(*) queries, making option C the most accurate description of the process.

Here are some authoritative links for further research:

Delta Lake Documentation: ACID Transactions on Apache Spark™:

<https://docs.databricks.com/delta/index.html>

Understanding Delta Lake Performance: <https://www.databricks.com/blog/2019/08/22/diving-into-delta-lake-unpacking-the-transaction-log.html>

Question: 64

Deep Dumps

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

DROP TABLE prod.sales_by_store -

If this statement is executed by a workspace admin, which result will occur?

- A.Nothing will occur until a COMMIT command is executed.
- B.The table will be removed from the catalog but the data will remain in storage.
- C.The table will be removed from the catalog and the data will be deleted.
- D.An error will occur because Delta Lake prevents the deletion of production data.
- E.Data will be marked as deleted but still recoverable with Time Travel.

Answer: C

Explanation:

The table will be removed from the catalog and the data will be deleted.

Question: 65

Deep Dumps

Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with `dbutils.fs.mount()`.

Which of the following statements is correct?

- A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
- B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
- C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
- D. Neither the DBFS root nor mounted storage can be accessed when using `%sh` in a Databricks notebook.
- E. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

Answer: A

Explanation:

The correct answer is A: DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.

Here's a detailed justification:

DBFS Abstraction: Databricks File System (DBFS) is an abstraction layer on top of object storage (like AWS S3 or Azure Blob Storage). It provides a file system interface to interact with data, regardless of the underlying storage. This allows users to use familiar file system commands (like `ls`, `cp`, `mv`) to manage data.

Unix-like Syntax: DBFS aims to provide a user experience similar to traditional Unix file systems. While not a perfect match, the basic operations and hierarchical structure are intended to be familiar.

Underlying Object Storage: The actual data is not stored on the Databricks compute instances themselves. It resides in the configured object storage. DBFS translates file system operations into object storage API calls.

Why other options are incorrect:

B: DBFS root and mounted data sources do not default to only being accessible to workspace administrators. Permissions can be configured with access control lists (ACLs), but by default, access is more open within the workspace.

C: The DBFS root is not the most secure by default. In fact, mounted storage often offers more fine-grained control over access permissions through cloud provider IAM roles and policies. Furthermore, mounted storage doesn't require full public read and write permissions. You can configure specific permissions through service principals, IAM roles, or similar mechanisms.

D: Both the DBFS root and mounted storage can be accessed when using `%sh` in a Databricks notebook. The shell environment has access to the DBFS CLI, allowing interaction with the file system.

E: The DBFS root does not store files in ephemeral block volumes attached to the driver. As stated earlier, it resides in object storage. Furthermore, both the DBFS root and mounted directories persist data to external storage between sessions. Data written to either location will persist after the cluster is terminated.

Supporting Links:

Databricks DBFS Documentation: <https://docs.databricks.com/data/databricks-file-system.html>

Databricks Access Control: <https://docs.databricks.com/security/access-control/index.html>

Question: 66**Deep Dumps**

The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

- A. %sh triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
- B. Instead of cloning, the code should use %sh pip install so that the Python code can get executed in parallel across all nodes in a cluster.
- C. %sh does not distribute file moving operations; the final line of code should be updated to use %fs instead.
- D. Python will always execute slower than Scala on Databricks. The run.py script should be refactored to Scala.
- E. %sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Answer: E**Explanation:**

%sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Question: 67**Deep Dumps**

The data science team has requested assistance in accelerating queries on free form text from user reviews. The data is currently stored in Parquet with the below schema:

item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING

The review column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

- A. Delta Lake statistics are not optimized for free text fields with high cardinality.
- B. Text data cannot be stored with Delta Lake.
- C. ZORDER ON review will need to be run to see performance gains.
- D. The Delta log creates a term matrix for free text fields to support selective filtering.
- E. Delta Lake statistics are only collected on the first 4 columns in a table.

Answer: A**Explanation:**

The correct response to the junior data engineer's suggestion is **A. Delta Lake statistics are not optimized for**

free text fields with high cardinality.

Here's a detailed justification:

Delta Lake enhances data reliability and performance on data lakes. It offers features such as ACID transactions, schema evolution, data versioning, and improved data quality. Delta Lake also automatically collects statistics on the data, like min/max values and null counts, to enable query optimization through data skipping.

However, the effectiveness of Delta Lake's statistics for query optimization is heavily influenced by the characteristics of the data. Delta Lake statistics excel when applied to columns with low cardinality (few distinct values) or when used for equality predicates in WHERE clauses. They help the query engine skip irrelevant data files, significantly speeding up queries.

In this scenario, the review column contains free-form text. This type of field has very high cardinality, meaning the column contains numerous unique values. Delta Lake does collect statistics on all columns. However the benefits of delta lake statistics for query optimization are less apparent on high cardinality columns.

Therefore, the statistics collected on the review column would likely be much less effective at enabling data skipping. The min/max values or histograms captured for such a high-cardinality column offer limited value in determining which data files contain the 30 keywords the data science team is searching for. Essentially, every data file might potentially contain one or more of these keywords, rendering data skipping less effective.

Option B is incorrect because Delta Lake can store text data. Option C is incorrect because ZORDER requires low cardinality. Option D is incorrect, Delta lake does not create a term matrix. Option E is incorrect, statistics are not limited to the first 4 columns.

For further research, consider exploring:

Databricks Delta Lake Documentation: <https://docs.databricks.com/delta/index.html> (Focus on statistics and data skipping)

Cardinality in Data Warehousing: Search for articles and blogs discussing the impact of cardinality on query performance and indexing techniques in data warehouses.

Question: 68

Deep Dumps

Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

- A.configure
- B.fs
- C.jobs
- D.libraries
- E.workspace

Answer: D

Explanation:

The correct Databricks CLI command for uploading a custom Python Wheel to object storage (mounted with DBFS) for use with a production job is libraries. Here's why:

The libraries command directly manages libraries within Databricks, including uploading custom wheels.

Databricks libraries can be installed on clusters to provide dependencies for notebooks and jobs. Uploading a wheel using libraries ensures it's accessible and can be readily installed on the cluster(s) executing your production job. This enables reproducible and reliable execution of code.

Option A (configure) is used for configuring the Databricks CLI itself (authentication, default workspace URL, etc.), not for uploading files. Option B (fs) is for interacting directly with DBFS file system operations like copying files, listing directories, etc., but not optimized for library management. While you could use fs to upload a wheel, libraries is the more direct and appropriate approach for this use case. Option C (jobs) is used for managing Databricks Jobs, such as creating, running, and monitoring them, but not for directly handling library uploads. Option E (workspace) manages Databricks workspace objects such as notebooks, folders, and repos.

Using `databricks libraries install --wheel <path_to_wheel>` will upload the wheel and install it on a given cluster. This ensures that when the production job runs on that cluster, the necessary dependencies are available. This process contributes to improved environment management and deployment.

For further information, see:

Databricks CLI documentation: <https://docs.databricks.com/en/dev-tools/cli/index.html>

Databricks Libraries: <https://docs.databricks.com/en/libraries/index.html>

Question: 69

Deep Dumps

The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named `products_per_order`, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

- A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
- B. Use Structured Streaming to configure a live dashboard against the `products_per_order` table within a Databricks notebook.
- C. Configure a webhook to execute an incremental read against `products_per_order` each time the dashboard is refreshed.
- D. Use the Delta Cache to persist the `products_per_order` table in memory to quickly update the dashboard with each query.
- E. Define a view against the `products_per_order` table and define the dashboard against this view.

Answer: A

Explanation:

Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.

Question: 70

Deep Dumps

A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

- A. Set `spark.sql.files.maxPartitionBytes` to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
- B. Set `spark.sql.shuffle.partitions` to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
- C. Set `spark.sql.adaptive.advisoryPartitionSizeInBytes` to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.
- D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.
- E. Set `spark.sql.shuffle.partitions` to 512, ingest the data, execute the narrow transformations, and then write to parquet.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best choice, along with relevant links for further research:

The goal is to write a 1TB JSON dataset to Parquet with a target part-file size of 512MB without shuffling the data unnecessarily. Shuffling is a costly operation in Spark as it involves data movement across the network.

Option A leverages `spark.sql.files.maxPartitionBytes` to control the initial partitioning of the data when it's read in. By setting this to 512 MB, Spark will attempt to create partitions of approximately that size during the initial read of the JSON files. Narrow transformations (those that can be applied to each partition independently without requiring data from other partitions) can then be executed efficiently. When writing to Parquet, Spark will, ideally, create files that are close to the specified partition size. This approach minimizes shuffling as the initial partitioning is aligned with the desired output file size.

Option B involves setting `spark.sql.shuffle.partitions` and sorting the data. Sorting always involves a shuffle, which is what we are trying to avoid for performance reasons. Even though it aims to repartition, the shuffle makes it inefficient.

Option C uses `spark.sql.adaptive.advisoryPartitionSizeInBytes` which is related to Adaptive Query Execution (AQE). While AQE can dynamically adjust partition sizes, coalesce to a specific number of partitions still involves a data reorganization, although less aggressive than repartition. Coalesce is mainly for reducing number of partition not to get to the desired partition size.

Option D involves a repartition which explicitly causes a full shuffle of the data, making it the least performant option.

Option E is incorrect as setting `spark.sql.shuffle.partitions` to 512 will not guarantee a 512MB parquet file sizes per partition. The number of partitions only influences operations that require a shuffle.

Therefore, option A provides the most efficient approach by influencing the initial partitioning without a costly shuffle, aligning it with the desired output file size.

Key Concepts:

Shuffling: A costly operation in Spark involving data movement across executors.

Partitioning: Dividing data into smaller chunks for parallel processing.

Narrow Transformations: Operations that can be applied independently to each partition.

spark.sql.files.maxPartitionBytes: Controls the maximum number of bytes to pack into a single partition when reading files.

Authoritative Links:

Spark Configuration: <https://spark.apache.org/docs/latest/configuration.html> (Search for spark.sql.files.maxPartitionBytes and spark.sql.shuffle.partitions on this page)

Adaptive Query Execution (AQE): <https://spark.apache.org/docs/latest/sql-performance-tuning.html#adaptive-query-execution>

Question: 71

Deep Dumps

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df._____  
    .groupBy(  
        window("event_time", "5 minutes").alias("time"),  
        "device_id"  
    )  
    .agg(  
        avg("temp").alias("avg_temp"),  
        avg("humidity").alias("avg_humidity")  
    )  
    .writeStream  
    .format("delta")  
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A.withWatermark("event_time", "10 minutes")

B.awaitArrival("event_time", "10 minutes")

C.await("event_time + '10 minutes'")

D.slidingWindow("event_time", "10 minutes")

E.delayWrite("event_time", "10 minutes")

Answer: A

Explanation:

withWatermark("event_time", "10 minutes").

Question: 72

Deep Dumps

A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in bold.

Original query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
        mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
        mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

.start("/item_agg")

Which step must also be completed to put the proposed query into production?

- A.Specify a new checkpointLocation
- B.Increase the shuffle partitions to account for additional aggregates
- C.Run REFRESH TABLE delta.'/item_agg'
- D.Register the data in the "/item_agg" directory to the Hive metastore
- E.Remove .option('mergeSchema', 'true') from the streaming write

Answer: A

Explanation:

A. Specify a new checkpointLocation.

To put the proposed streaming query (which involves an aggregation, specifically `count("promo_code" = 'NEW MEMBER')`) into production, a new checkpointLocation is required. The addition of a new aggregated field is considered an incompatible schema change in Structured Streaming when using the same checkpoint location, as the existing checkpoint metadata is tied to the previous schema. By specifying a new checkpoint location, you signal to Spark Structured Streaming to start the new job with a fresh state, thereby avoiding schema mismatch issues and ensuring proper recovery and fault tolerance in a production environment. The `mergeSchema` option should generally remain enabled to handle schema evolution gracefully.

Question: 73

Deep Dumps

A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

- A. Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
- B. Increase the number of shuffle partitions to maximize parallelism, since the trigger interval cannot be modified without modifying the checkpoint directory.
- C. Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.
- D. Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.
- E. Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.

Answer: C

Explanation:

The correct answer is C because it directly addresses the core issue of excessive microbatch executions, specifically the empty ones, which lead to unnecessary API calls to the source storage and higher storage costs. The problem statement highlights that the job is processing empty microbatches at least 12 times a minute. These empty batches still trigger read operations on the source storage, incurring charges even though no actual data is being processed.

Option A is incorrect because reducing the trigger interval would actually increase the number of microbatches and API calls, exacerbating the cost issue. Option B is also incorrect; while increasing shuffle partitions might improve performance for batches with data, it won't address the problem of frequent empty batches and associated API costs. Option D suggests decreasing the trigger interval, which, as with Option A, would only increase costs by creating more frequent microbatches. Option E, while seemingly reducing compute costs, isn't practical for the business requirement of data being processed in less than 10 minutes. Trigger.Once processes data exactly one time and then shuts down which is not viable for streaming jobs.

Setting the trigger interval to 10 minutes (Option C) reduces the frequency with which the source storage is

queried. By grouping more data into each microbatch, the job makes fewer API calls, leading to significant cost savings, especially in scenarios with frequent empty batches. While this slightly increases latency, it remains within the 10-minute requirement. This approach aligns with the principle of optimizing cloud resource utilization by minimizing unnecessary I/O operations.

Relevant links for further research:

Structured Streaming Programming Guide: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (Specifically, the "Triggers" section).

Databricks Structured Streaming: <https://docs.databricks.com/structured-streaming/index.html>

Question: 74

Deep Dumps

Which statement describes the correct use of `pyspark.sql.functions.broadcast`?

- A. It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.
- B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
- C. It caches a copy of the indicated table on attached storage volumes for all active clusters within a Databricks workspace.
- D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.
- E. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.

Answer: D

Explanation:

The correct answer is D because `pyspark.sql.functions.broadcast(df)` is specifically used to hint to Spark that the specified DataFrame `df` should be broadcast to all executor nodes in the cluster. This optimization is most effective when joining a relatively small DataFrame (the broadcast DataFrame) with a much larger DataFrame. By broadcasting the smaller DataFrame, Spark can avoid shuffling data from the larger DataFrame across the network during the join operation. Instead, each executor has a complete copy of the smaller DataFrame, allowing it to perform the join locally with its partition of the larger DataFrame.

Options A and B are incorrect because broadcast operates on entire DataFrames, not individual columns. It doesn't analyze column cardinality or store individual column values in memory. Option C is wrong as broadcasting doesn't involve attached storage volumes or persist data across multiple clusters. Option E is also incorrect because broadcast doesn't cache data persistently for the entire cluster lifetime or for all future queries necessarily; the broadcast effect typically lasts for the duration of the query it is applied to. Caching is separate from broadcasting. broadcast improves join performance in appropriate scenarios by distributing a small DataFrame to each executor's memory before the join, enabling faster local joins and minimizing data shuffling. This technique is a common and crucial optimization for Spark workloads that involve joining datasets of varying sizes.

Refer to the official Apache Spark documentation for more detailed information about broadcast and join optimizations:

[Apache Spark SQL Performance Tuning](#) (specifically, look for sections on Broadcast Join)
`pyspark.sql.functions.broadcast`

Question: 75

Deep Dumps

A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

- A. Set the configuration `delta.deduplicate = true`.
- B. VACUUM the Delta table after each batch completes.
- C. Perform an insert-only merge with a matching condition on a unique key.
- D. Perform a full outer join on a unique key and overwrite existing data.
- E. Rely on Delta Lake schema enforcement to prevent duplicate records.

Answer: C

Explanation:

The correct answer is C: "Perform an insert-only merge with a matching condition on a unique key." This approach leverages the MERGE operation in Delta Lake, which is ideal for handling upserts and deduplication scenarios involving potentially late-arriving or duplicate data.

Here's a detailed breakdown:

Handling Late-Arriving/Duplicate Records: The core problem is addressing records that arrive out of order or are duplicated across different processing runs.

MERGE Operation: The MERGE statement allows you to specify conditions for matching existing records in the target Delta table based on a unique key with incoming data.

Insert-Only Logic: By setting the merge condition appropriately (e.g., `WHEN NOT MATCHED THEN INSERT *`), the data engineer ensures that only new, unique records (those not already present based on the unique key) are inserted into the Delta table.

Deduplication: This mechanism effectively deduplicates data against previously processed records, as only records with unique keys that do not already exist in the Delta table are inserted. This provides a reliable way to deduplicate even when dealing with late-arriving data.

ACID Properties: Delta Lake's ACID properties guarantee that the merge operation is atomic, ensuring data consistency and preventing partial updates in case of failures. This is crucial for data integrity in data engineering pipelines.

Alternatives:

A (`delta.deduplicate = true`) is not a valid Delta Lake configuration option.

B (VACUUM) only removes old versions of data files, not duplicate data.

D (Full outer join) can be used to detect duplicates, but overwriting data can lead to data loss or unintended consequences if not handled carefully. It is also not efficient for deduplication as it requires reprocessing all records in the table every time.

E (Schema enforcement) prevents incompatible data types but does not address duplicate records.

Using MERGE with the `WHEN NOT MATCHED THEN INSERT` clause is the recommended approach in Databricks for deduplicating data against previously processed records in a Delta table due to its efficiency, ACID properties, and suitability for handling late-arriving data.

Authoritative Links:

Delta Lake MERGE INTO: <https://docs.databricks.com/delta/delta-update.html>

Databricks Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

- A. New fields will not be computed for historic records.
- B. Spark cannot capture the topic and partition fields from a Kafka source.
- C. New fields cannot be added to a production Delta table.
- D. Updating the table schema will invalidate the Delta transaction log metadata.
- E. Updating the table schema requires a default value provided for each field added.

Answer: A

Explanation:

The correct answer is A: New fields will not be computed for historic records.

Here's a detailed justification:

The core issue revolves around the immutability of existing data. When new columns (current timestamp, Kafka topic, partition) are added to the Delta table's schema, those columns are populated for newly ingested records going forward. However, the existing records in the Delta table, representing the data from the past three months, will not be retroactively updated with these new fields. This is because Delta Lake, like most data warehousing solutions, stores data physically as it arrives. Adding columns doesn't trigger a re-processing or rewriting of historical data to fill in those new columns. Consequently, the newly added metadata will only be available for analysis from the point the schema change was implemented. This is a fundamental principle in data warehousing and lakehouse architecture.

Therefore, when the data engineering team attempts to diagnose the historical latency issues, they will find that the `current_timestamp`, `topic`, and `partition` fields are null or missing for the data ingested before the schema change. This limits their ability to correlate latency with specific Kafka topics, partitions, or the exact ingestion time as recorded by Spark for those older records. They can only analyze the new data being ingested after the schema change.

Option B is incorrect because Structured Streaming with Kafka can capture topic and partition information.

Option C is incorrect. Delta Lake supports schema evolution, allowing new fields to be added to production tables.

Option D is incorrect. Adding columns is a schema evolution operation that is handled by Delta Lake's transaction log and doesn't invalidate the log; it's a valid change recorded within the log.

Option E is partially correct in some cases, but not always required. Delta Lake supports adding columns without requiring a default value if the column is nullable.

Supporting Links:

Delta Lake Schema Evolution: <https://docs.databricks.com/delta/delta-batch.html#schema-evolution> - This documentation describes how Delta Lake handles schema evolution, including adding columns.

Structured Streaming with Kafka: <https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html> - This documentation showcases how Spark Structured Streaming can be used to extract details such as topic and partition information.

Question: 77**Deep Dumps**

In order to facilitate near real-time workloads, a data engineer is creating a helper function to leverage the schema detection and evolution functionality of Databricks Auto Loader. The desired function will automatically detect the schema of the source directly, incrementally process JSON files as they arrive in a source directory, and automatically evolve the schema of the table when new fields are detected.

The function is displayed below with a blank:

```
def auto_load_json(source_path: str,
                   checkpoint_path: str,
                   target_table_path: str):
    (spark.readStream
     .format("cloudFiles")
     .option("cloudFiles.format", "json")
     .option("cloudFiles.schemaLocation", checkpoint_path)
     .load(source_path)
     _____
    )
```

Which response correctly fills in the blank to meet the specified requirements?

- A.

```
.writeStream
.option("mergeSchema", True)
.start(target_table_path)
.writeStream
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.trigger(once=True)
.start(target_table_path)
```
- B.

```
.write
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.outputMode("append")
.save(target_table_path)
```
- C. _____

```
.write
.option("mergeSchema", True)
.mode("append")
D. .save(target_table_path)
.writeStream
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
E. .start(target_table_path)
```

Answer: E

Explanation:

Correct answer is E, it is a streaming write, and the default outputMode is Append (so if it's optional in this case)

Question: 78

Deep Dumps

The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
 .groupBy("customer_id")
 .agg(
     F.min("sale_date").alias("first_transaction_date"),
     F.max("sale_date").alias("last_transaction_date"),
     F.mean("sale_total").alias("average_sales"),
     F.countDistinct("order_id").alias("total_orders"),
     F.sum("sale_total").alias("lifetime_value")
 ).write
 .mode("overwrite")
 .table("gold_customer_lifetime_sales_summary")
 )
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A.The silver_customer_sales table will be overwritten by aggregated values calculated from all records in the gold_customer_lifetime_sales_summary table as a batch job.
- B.A batch job will update the gold_customer_lifetime_sales_summary table, replacing only those rows that have different values than the current version of the table, using customer_id as the primary key.
- C.The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.
- D.An incremental job will leverage running information in the state store to update aggregate values in the gold_customer_lifetime_sales_summary table.
- E.An incremental job will detect if new rows have been written to the silver_customer_sales table; if new rows are detected, all aggregates will be recalculated and used to overwrite the gold_customer_lifetime_sales_summary table.

Answer: C

Explanation:

The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.

Question: 79

Deep Dumps

The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

- A.When a database is being created, make sure that the LOCATION keyword is used.
- B.When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
- C.When data is saved to a table, make sure that a full file path is specified alongside the Delta format.
- D.When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.
- E.When the workspace is being configured, make sure that external cloud object storage has been mounted.

Answer: A

Explanation:

A. When a database is being created, make sure that the LOCATION keyword is used.

Why this is correct

In Databricks / Delta Lake:

If a database (schema) is created with a LOCATION, all tables created inside that database are external (unmanaged) by default.

The data lives in user-managed cloud object storage, and Databricks does not control the data lifecycle.

This is the only option that enforces the requirement globally, without relying on developers to remember table-level rules.

Question: 80

Deep Dumps

The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales

org.

Which of the following solutions addresses the situation while emphasizing simplicity?

- A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
- B. Create a new table with the required schema and use Delta Lake's DEEP CLONE functionality to sync up changes committed to one table to the corresponding table.
- C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
- D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.
- E. Instruct the marketing team to download results as a CSV and email them to the sales organization.

Answer: A

Explanation:

The best solution is **A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.**

Here's why:

Simplicity: Views are lightweight and don't involve data duplication or complex data pipelines. They are simply stored queries that present data from the underlying table.

Data Governance/Security: Creating a view allows you to explicitly control which columns are exposed to the sales team, enforcing data governance policies and preventing unauthorized access to sensitive marketing data. The view acts as a filter and restricts data based on the agreed-upon fields.

Schema Transformation: Views can easily alias column names, mapping marketing-specific field names to the sales organization's conventions. This resolves the field name mismatch issue directly within the view definition.

Real-time/Near Real-time access: Changes made to the underlying marketing table are immediately reflected in the view (upon querying it), providing the sales team with up-to-date aggregate data.

Low Overhead: Views consume minimal resources compared to creating and managing separate tables.

The other options are less suitable:

B. DEEP CLONE is usually used to migrate a table from one platform to another.

C. CTAS creates a new table, duplicating data and requiring a pipeline to maintain consistency. This adds complexity and storage overhead.

D. Parallel table write also creates a new table, requiring changes to an existing production pipeline. This is more complex than creating a view.

E. CSV and email is a manual process and unsuitable for regular data sharing and collaboration. It also has data security and compliance implications.

Authoritative Links:

Databricks Views: <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html>

Delta Lake DEEP CLONE: <https://docs.databricks.com/delta/clone.html>

Question: 81

Deep Dumps

A CHECK constraint has been successfully added to the Delta table named activity_details using the following logic:


```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where latitude = 45.50 and longitude = 212.67.

Which statement describes the outcome of this batch insert?

- A.The write will fail when the violating record is reached; any records previously processed will be recorded to the target table.
- B.The write will fail completely because of the constraint violation and no records will be inserted into the target table.
- C.The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
- D.The write will include all records in the target table; any violations will be indicated in the boolean column named valid_coordinates.
- E.The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.

Answer: B

Explanation:

The write will fail completely because of the constraint violation and no records will be inserted into the target table.

Question: 82

Deep Dumps

A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

- A.Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
- B.The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
- C.Other than the default "admins" group, only individual users can be granted privileges on jobs.
- D.A user can only transfer job ownership to a group if they are also a member of that group.
- E.Only workspace administrators can grant "Owner" privileges to a group.

Answer: A

Explanation:

The correct answer is **A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.**

Here's a detailed justification:

Databricks' access control model for Jobs is designed around the concept of a single, definitive owner for each job. This single owner has full administrative control over the job, including the ability to modify its configuration, manage its permissions, and delete it. While you can grant other permissions like "Can Manage" or "Can View" to groups or individual users, the "Owner" privilege is specifically reserved for a single user identity. This design helps to establish clear accountability and responsibility for the job's lifecycle.

The reason "Owner" can't be assigned to a group lies in the inherent ambiguity it would create. If a group were the "Owner," it would be unclear which member of the group would be ultimately responsible for actions requiring ownership privileges. It also prevents potential conflicts if multiple members of the group tried to modify owner-specific settings concurrently. Databricks requires a single point of control to ensure consistent and predictable job management.

Options B, C, D, and E are incorrect because they present scenarios that don't accurately reflect how Databricks Jobs access control functions.

B: The creator of a job isn't permanently bound to the "Owner" role; the ownership can be transferred to another user, but not a group.

C: You can grant permissions on jobs to groups, just not the "Owner" permission.

D: A user doesn't need to be a member of a group to transfer permissions (excluding Owner) to that group.

E: While workspace admins have elevated privileges, the core issue is that the "Owner" privilege cannot be assigned to groups by anyone.

The fundamental restriction lies in Databricks' design principle of maintaining a single, responsible individual (user) as the owner of each job. This prevents a group from being assigned the "Owner" privilege.

Relevant Resources:

Databricks Jobs Access Control: <https://docs.databricks.com/en/workflows/jobs/jobs-acl.html>

Databricks Access Control Overview: <https://docs.databricks.com/en/security/access-control/index.html>

Question: 83

Deep Dumps

All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG

There are 5 unique topics being ingested. Only the "registration" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which of the following solutions meets the requirements?

A.All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.

B.Data should be partitioned by the registration field, allowing ACLs and delete statements to be set for the PII directory.

C.Because the value field is stored as binary data, this information is not considered PII and no special precautions should be taken.

D.Separate object storage containers should be specified based on the partition field, allowing isolation at the storage level.

E.Data should be partitioned by the topic field, allowing ACLs and delete statements to leverage partition boundaries.

Answer: E

Explanation:

The correct answer is **E. Data should be partitioned by the topic field, allowing ACLs and delete statements to leverage partition boundaries.**

Here's why:

Requirement 1: Restricting access to PII: The problem explicitly states that only the "registration" topic contains PII. By partitioning the Delta Lake table by the topic field, you create separate directories for each topic's data. This partitioning allows you to apply Access Control Lists (ACLs) at the directory level, specifically restricting access to the "registration" topic's directory to authorized personnel only.

Requirement 2: PII data retention: Partitioning by topic also enables targeted deletion of PII data. You can use Delta Lake's DELETE command with a WHERE clause that specifies the "registration" topic and a timestamp condition to remove records older than 14 days. This selectively deletes PII data while leaving the other topics untouched. For example: `DELETE FROM delta_table WHERE topic = 'registration' AND timestamp < (current_timestamp() - interval '14' day)`

Why other options are incorrect:

A: Deleting all data bi-weekly and relying on time travel is not a scalable and well-managed approach for long term data storage. It would make querying and analysis of non-PII data beyond biweekly periods cumbersome and inefficient. Additionally, deleting all data is likely not acceptable for the indefinite retention of non-PII data.

B: Partitioning by a registration field is nonsensical as there is only one topic which contains PII.

C: The fact that value is stored as binary data doesn't negate PII concerns. The data within the binary field could very well contain PII. Ignoring this potential is a security risk.

D: While isolating storage containers based on partitioning can achieve the desired result, this is unnecessarily complex and inefficient. Managing multiple containers adds operational overhead compared to using partition ACLs.

Authoritative links:

Delta Lake Partitioning: <https://docs.databricks.com/delta/best-practices.html#partition-data>

Delta Lake DELETE command: <https://docs.databricks.com/delta/delta-update.html#delete-from-a-table>

Access Control Lists (ACLs) in Databricks: <https://docs.databricks.com/security/access-control/index.html>

Question: 84

Deep Dumps

The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;  
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group and that these users are not workspace administrators, which statement describes their privileges?

- A. Group members have full permissions on the prod database and can also assign permissions to other users or groups.
- B. Group members are able to list all tables in the prod database but are not able to see the results of any queries on those tables.
- C. Group members are able to query and modify all tables and views in the prod database, but cannot create new tables or views.
- D. Group members are able to query all tables and views in the prod database, but cannot create or edit anything in the database.
- E. Group members are able to create, query, and modify all tables and views in the prod database, but cannot define custom functions.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

The question focuses on understanding table access control in Databricks, particularly the implications of GRANT USAGE and GRANT SELECT privileges on a database. The eng group has been granted these two specific privileges on the prod database.

USAGE privilege: GRANT USAGE ON DATABASE prod TO eng allows the eng group to access the database prod. This is a foundational privilege. Without USAGE, the group wouldn't even be able to see the database or its contents. It is required to perform any operation on objects within the database.

(<https://docs.databricks.com/en/security/access-control/table-acls/object-privileges.html#database-privileges>)

SELECT privilege: GRANT SELECT ON DATABASE prod TO eng allows the eng group to query tables and views within the prod database. This means they can execute SELECT statements to retrieve data.

(<https://docs.databricks.com/en/security/access-control/table-acls/object-privileges.html#table-privileges>)

However, the provided logic doesn't grant any privileges to modify, create, or alter objects within the prod database. Specifically:

They cannot create new tables or views because CREATE privilege hasn't been granted.

They cannot modify existing tables or views because MODIFY privilege (or specific privileges like INSERT, UPDATE, DELETE) hasn't been granted.

They cannot assign permissions to other users or groups.

Given these limitations, the eng group can only query (read) the data present in the tables and views within the prod database. They lack the authority to make any changes or create new objects. Therefore, option D accurately reflects their granted permissions.

The other options are incorrect because they attribute permissions that were not explicitly granted:

A: Incorrect because it states they have full permissions. They do not have permissions to modify, create, or assign permissions.

B: Incorrect because GRANT SELECT allows them to see the results of queries.

C: Incorrect because it mentions modifying tables and views, which is not granted.

E: Incorrect because it mentions creating and modifying objects.

Question: 85

Deep Dumps

A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to

evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

- A. Workspace audit logs
- B. Driver's log file
- C. Ganglia
- D. Cluster Event Log
- E. Executor's log file

Answer: D

Explanation:

The correct answer is **D. Cluster Event Log**. Here's a detailed justification:

The primary goal is to understand why a Databricks cluster is scaling up – is it user concurrency or resource-intensive queries? To achieve this, you need a timeline of cluster resizing events correlated with potential triggers.

Why Cluster Event Log is the best choice:

Dedicated to Cluster Events: The Cluster Event Log is specifically designed to record lifecycle events of a Databricks cluster, including start, stop, resize (upscale and downscale), configuration changes, and errors.

Detailed Timeline: The log entries are time-stamped, providing a clear sequence of events related to cluster activity. This is crucial for understanding when the cluster scaled up.

Contextual Information: The logs often include information about the reason for scaling events. While it might not directly say "user concurrency" or "resource-intensive queries," it might show metrics or events that point to these causes (e.g., high CPU utilization before upscaling).

Databricks Specific: This is a first-party Databricks feature built precisely for monitoring and troubleshooting cluster behavior.

Why the other options are not the best:

A. Workspace Audit Logs: These logs focus on user actions and security events (e.g., user login, permission changes, workspace object access). While useful for security, they don't directly track cluster scaling events with sufficient detail.

B. Driver's log file & E. Executor's log file: These log files are useful for debugging the execution of Spark jobs. While resource-intensive queries would eventually result in changes in the driver's and executors logs, they wouldn't give a concise and clear timeline of upscaling events with information about why the cluster scaled. You would need to manually analyse these logs and try to correlate them to cluster behaviour.

C. Ganglia: Ganglia is a distributed monitoring system, and while Databricks can integrate with it to provide metrics, it's not the primary place to find a chronological log of cluster resizing events with accompanying explanations (if any). Ganglia focuses on real-time metric collection and historical performance data, but it doesn't provide as much structured information about cluster lifecycle events as the Cluster Event Log.

In summary, the Cluster Event Log provides the most direct and comprehensive record of cluster scaling events, enabling the administrator to analyze the timeline and identify the most likely triggers for upscaling, such as bursts of user activity or computationally expensive queries.

Authoritative Links:

Databricks Cluster Event Log: <https://docs.databricks.com/clusters/clusters-manage.html#cluster-event-log>

Databricks Monitoring: <https://docs.databricks.com/administration-guide/clusters/monitoring.html>

Question: 86**Deep Dumps**

When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

- A.The five Minute Load Average remains consistent/flat
- B.Bytes Received never exceeds 80 million bytes per second
- C.Network I/O never spikes
- D.Total Disk Space remains constant
- E.CPU Utilization is around 75%

Answer: E

Explanation:

The most indicative metric of proper VM resource utilization in a Databricks cluster with executors is a consistently high CPU utilization, ideally around 75%. This signals that the executors are actively processing data and maximizing the computational power of the VMs. Load average (option A) provides a system-wide snapshot but can be misleading if individual cores aren't fully engaged; a flat load average doesn't guarantee full resource use. Network I/O thresholds (options B and C) depend heavily on the workload and data movement patterns, making fixed limits like 80 MBps or constant I/O impractical indicators of proper utilization. Constant total disk space (option D) is irrelevant to CPU utilization. High CPU utilization, on the other hand, implies that the processors are engaged in executing tasks, maximizing the cost-effectiveness of the VM. A utilization of 75% balances efficient resource use with headroom for bursts and prevents resource exhaustion. Too high a utilization, such as consistently 100%, might lead to performance degradation and task queuing, while significantly lower utilization represents underutilization of paid-for resources. Observing CPU utilization alongside other metrics provides a more complete picture of performance, allowing for adjustments in cluster configuration or code optimization if needed. Optimizing for CPU efficiency directly translates to lower cloud costs and faster job completion times.

Refer to Databricks documentation on monitoring clusters and Apache Spark performance tuning for detailed explanations of metrics and optimization strategies.

[Databricks Monitoring](#)
[Spark Performance Tuning](#)

Question: 87**Deep Dumps**

Which of the following technologies can be used to identify key areas of text when parsing Spark Driver log4j output?

- A.Regex
- B.Julia
- C.pyspark.ml.feature
- D.Scala Datasets
- E.C++

Answer: A

Explanation:

The correct answer is **A. Regex (Regular Expressions)**. Here's why:

Spark Driver logs contain a vast amount of text data, often unstructured. Identifying specific patterns, error

messages, or performance bottlenecks requires a method to efficiently search and extract relevant information. Regular expressions are a powerful and standard way to define search patterns within text. They allow you to specify rules for matching specific character sequences, keywords, or structures within the log files.

Options B, C, D, and E are not ideally suited for this task. Julia is a programming language, but not intrinsically designed for log parsing. `pyspark.ml.feature` (C) is a PySpark module for machine learning feature engineering, irrelevant for simple text pattern matching. Scala Datasets (D) are useful for structured data manipulation within Spark, but you first need to extract and structure the log data, which Regex helps with. C++ (E), while capable of text processing, is a lower-level language and less efficient for pattern matching in this context compared to Regex.

Regex allows for flexible and precise pattern definitions, allowing you to isolate key areas of the log. You could use Regex, for example, to find all lines containing the word "ERROR" with the level of log severity and corresponding class that raised it, extract stack traces, or identify stages of the Spark job execution. You can combine Regex with tools like `grep`, `awk`, or scripting languages to automate the log analysis process. The Spark Driver itself uses Log4j, which can be configured to use patterns for log formatting, further supporting the use of Regex for parsing. Regex parsing enables extracting information necessary for debugging, monitoring, and performance tuning of Spark applications.

Here are some resources that can help with learning more about Regex and Spark log analysis:

Regular Expressions Tutorial: <https://regexone.com/>

Spark Logging: <https://spark.apache.org/docs/latest/monitoring.html#logging>

Log4j: <https://logging.apache.org/log4j/2.x/>

Question: 88

Deep Dumps

You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test would the above line exemplify?

- A. Unit
- B. Manual
- C. Functional
- D. Integration
- E. End-to-end

Answer: A

Explanation:

The correct answer is A. Unit. Here's a detailed justification:

The provided code snippet `assert(myIntegrate(lambda x: xx, 0, 3) [0] == 9)` demonstrates a unit test. Unit tests are designed to verify the functionality of individual components or units of code in isolation. In this case, the 'unit' under test is the `myIntegrate` function. The test focuses solely on whether `myIntegrate` correctly calculates the integral of a specific function (xx) within the defined boundaries (0 to 3).

The test is isolated because it doesn't involve any external dependencies (like databases or other services). It directly invokes `myIntegrate` with specific inputs and checks if the output matches the expected value (9). The `assert` statement is a standard way to check if a condition is true; if the integral is not equal to 9, the test will

fail, indicating a problem with the myIntegrate function.

Functional tests, on the other hand, would evaluate the overall behavior of a function or a group of functions. Integration tests would verify that different components of the system work correctly together. End-to-end tests would simulate a user's workflow and test the entire system from start to finish. Manual tests would require human intervention to verify the functionality.

In summary, because the provided test focuses on a single function, verifying its isolated behavior with a specific input and expected output, it exemplifies a unit test.

Further reading on unit testing:

https://en.wikipedia.org/wiki/Unit_testing

<https://testing.googleblog.com/2009/04/what-is-unit-test.html>

Question: 89

Deep Dumps

A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on Task A.

If task A fails during a scheduled run, which statement describes the results of this run?

- A. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
- B. Tasks B and C will attempt to run as configured; any changes made in task A will be rolled back due to task failure.
- C. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task A failed, all commits will be rolled back automatically.
- D. Tasks B and C will be skipped; some logic expressed in task A may have been committed before task failure.
- E. Tasks B and C will be skipped; task A will not commit any changes because of stage failure.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most accurate description of what happens when Task A fails in the described Databricks job scenario, along with supporting reasoning and authoritative links:

Justification:

The core concept here revolves around how Databricks Jobs handle task dependencies and failure scenarios. Option D, "Tasks B and C will be skipped; some logic expressed in task A may have been committed before task failure," accurately reflects this behavior.

1. **Dependency Graph:** Databricks Jobs indeed operate with a dependency graph, which means tasks are executed based on their defined relationships. Tasks B and C are configured to depend on the successful completion of Task A.
2. **Task A Failure:** When Task A fails during execution, the Databricks job scheduler recognizes that Tasks B and C cannot proceed because their dependency (Task A) hasn't been satisfied.
3. **Skipping Dependent Tasks:** Because of the dependency relationship, Tasks B and C will be skipped. The job scheduler will not attempt to execute them, preventing potential errors or inconsistent state due to missing data or transformations that Task A was supposed to provide.
4. **Partial Commits in Task A:** Critically, Databricks Jobs do not provide automatic rollback capabilities like atomic transactions across multiple tasks. If Task A performs operations that directly modify

data in the Lakehouse (e.g., writing data to Delta tables) before the point of failure, those changes will likely be committed. Databricks does not, by default, provide the functionality to automatically undo or rollback those changes if Task A fails.

5. **No Automatic Rollback:** Options A, B, and C are incorrect because they imply that Databricks Jobs have built-in rollback mechanisms for failed tasks to ensure all or nothing guarantees across the entire job. This is not generally the case. While it's possible to implement custom rollback logic within the notebooks themselves, Databricks doesn't provide automatic rollback functionality.
6. Option E is incorrect because task A can commit changes before the stage failure.

In summary: The task dependency is respected, causing the dependent tasks (B and C) to be skipped. However, any writes or modifications to the Lakehouse performed by Task A before the failure occurred are likely to persist. The user is responsible for implementing any rollback or error-handling logic if desired.

Authoritative Links:

Databricks Jobs documentation: <https://docs.databricks.com/workflows/jobs/index.html>

Understanding Task Dependencies: While the above documentation doesn't explicitly detail a lack of rollback capabilities, the general architecture implies that state management and rollbacks must be handled at the notebook level.

Delta Lake ACID Properties: <https://docs.databricks.com/delta/index.html> (Delta Lake guarantees ACID properties, but ACID guarantees are enforced within a single Delta operation not between Databricks Job Tasks.)

These links provide insight into how Databricks Jobs are designed to handle task execution and dependency management, which help to reinforce the explanation above. They also demonstrate that ACID guarantees within Delta Lake tables apply to individual write operations and don't automatically translate into a job-level rollback mechanism.

Question: 90

Deep Dumps

Which statement regarding Spark configuration on the Databricks platform is true?

- A. The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
- B. Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive cluster.
- C. Spark configuration properties can only be set for an interactive cluster by creating a global init script.
- D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.
- E. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer regarding Spark configuration on Databricks, along with supporting explanations and links.

Justification for Option D:

Option D, "Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster," is the most accurate statement. When you configure Spark properties

using the Databricks Clusters UI, you're essentially setting cluster-level configurations. These settings become the default configurations for all SparkSessions created within that cluster, regardless of which notebook initiates them. This means every notebook attached to that cluster will inherit and use those settings unless explicitly overridden at a lower level (like within the notebook itself). This behavior is consistent with the hierarchical nature of Spark configuration and how Databricks manages cluster settings.

Why other options are incorrect:

Option A: Modifying Spark configuration properties for an interactive cluster without interrupting jobs is not generally possible. Changes to Spark configuration typically require a restart of the SparkContext, which would interrupt running jobs. While Databricks might offer rolling restarts in some scenarios, A doesn't account for potential interruption.

Option B: Spark configurations set within a notebook usually affect only the SparkSession created within that notebook. They don't typically cascade to all other SparkSessions attached to the same cluster, unless those other sessions are explicitly configured to inherit those settings (which is not the default behavior). Each notebook has its own SparkSession unless explicitly reusing an existing one.

Option C: Spark configuration properties can be set in multiple ways, not only via global init scripts. The Clusters UI, cluster-scoped init scripts, and notebook-level configurations are all valid methods. Global init scripts affect all clusters in the workspace.

Option E: When the same Spark configuration property is set at both the cluster level and within a notebook, the notebook setting will generally take precedence. The notebook-level configuration overrides the cluster-level configuration for the SparkSession created within that notebook.

Supporting Cloud Computing Concepts:

Hierarchical Configuration: Spark configurations follow a hierarchical model. Cluster-level settings are the default, and more specific settings (e.g., notebook-level) override the defaults.

Cluster Management: Databricks clusters provide a managed environment for running Spark jobs. Centralized configuration at the cluster level simplifies management and ensures consistency across multiple notebooks.

SparkSession: A SparkSession represents the entry point to Spark functionality. Configurations set within a notebook affect the SparkSession created in that notebook.

Authoritative Links:

Databricks Documentation - Spark Configuration: <https://docs.databricks.com/clusters/configure.html>

Databricks Documentation - Init Scripts: <https://docs.databricks.com/clusters/init-scripts.html>

Apache Spark Documentation: <https://spark.apache.org/docs/latest/configuration.html>

In summary, the cluster-level configuration using the Databricks UI serves as the default Spark configuration for all notebooks attached to that cluster, making option D the correct answer.

Question: 91

Deep Dumps

A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the main branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

A. Use Repos to checkout all changes and send the git diff log to the team.

B. Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.

C. Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.

D. Use Repos to merge all differences and make a pull request back to the remote repository.

E. Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.

Answer: E

Explanation:

Here's a detailed justification for why option E is the correct approach for a developer to share code updates when they lack privileges to directly modify the main branch in a Databricks Repos environment connected to a Git repository:

The core problem is that the developer cannot directly commit to the main branch. Git best practices and collaborative workflows dictate that direct commits to a shared branch like main should be restricted, particularly in team environments, to maintain code quality and stability. Pull requests are essential for code review and integration.

Option E, "Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository," aligns perfectly with the standard Git workflow for contributing changes in such a scenario. Creating a new branch allows the developer to isolate their modifications without affecting the main branch or the work of other team members. After making changes within this new branch, committing them records the changes locally. Then, pushing the branch to the remote repository makes the branch and its associated commits visible and accessible to the rest of the team. This is the prerequisite for creating a pull request.

Options A, C, and D are flawed. Option A suggests sending a git diff log, which is cumbersome and doesn't leverage the pull request workflow. Option C attempts to exploit a non-existent branch creation feature from pulling; Git doesn't work this way. Option D talks about merging all differences and making a pull request, which presupposes write access to main (which the developer does not have). Option B, forking the repository, is a valid alternative but is generally preferred when contributing to open-source projects the user does not have existing write access to. In an internal team project like Databricks Repos, branching is simpler and preferred.

The subsequent step after pushing the branch (which option E does) is to create a pull request (PR) to the main branch (or another designated branch) on the remote repository. The PR then triggers a code review process, where other team members can examine the changes, provide feedback, and ensure the code meets quality standards before it's merged into the main branch. This whole process follows the industry best practices.

Therefore, option E is the most appropriate and secure method for sharing code updates, ensuring collaboration, and avoiding the risk of overwriting existing work. **Supporting Links:**

Git Branching: <https://www.atlassian.com/git/tutorials/using-branches>

Git Pull Requests: <https://www.atlassian.com/git/tutorials/making-a-pull-request>

Databricks Repos: <https://docs.databricks.com/repos/index.html>

Question: 92

Deep Dumps

In order to prevent accidental commits to production data, a senior data engineer has instituted a policy that all development work will reference clones of Delta Lake tables. After testing both DEEP and SHALLOW CLONE, development tables are created using SHALLOW CLONE.

A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly

Changing Dimension (SCD) stop working. The transaction logs for the source tables show that VACUUM was run the day before.

Which statement describes why the cloned tables are no longer working?

- A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
- B. Running VACUUM automatically invalidates any shallow clones of a table; DEEP CLONE should always be used when a cloned table will be repeatedly queried.
- C. Tables created with SHALLOW CLONE are automatically deleted after their default retention threshold of 7 days.
- D. The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command.
- E. The data files compacted by VACUUM are not tracked by the cloned metadata; running REFRESH on the cloned table will pull in recent changes.

Answer: B

Explanation:

The correct answer is **D. The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command.**

Here's why:

Shallow Clones vs. Deep Clones: A shallow clone shares the underlying data files with the source table, while a deep clone creates a completely independent copy. The metadata of a shallow clone points to the same data files as the original table.

VACUUM Command: The VACUUM command in Delta Lake removes data files that are no longer needed by the Delta table. This typically includes files that have been superseded by newer versions due to updates or deletes, based on the retention period.

Impact on Shallow Clones: When VACUUM is run on the source table, it physically deletes the older data files. Since the shallow clone's metadata still references these now-deleted files, the shallow clone becomes unusable for those deleted time periods, and data queries will fail or return incomplete results.

Type 1 SCD and VACUUM: While Type 1 SCDs overwrite existing records, the VACUUM command doesn't selectively delete data based on SCD type. It deletes files based on the Delta table's transaction log and configured retention period. This means that when Vacuum is run it purges old data files referenced by the original table. Since the Shallow Clone is simply referencing those same files, the data is deleted there as well, which makes the Shallow Clone become invalid.

Why other options are incorrect:

A: Type 1 changes alone don't invalidate cloned tables. It's the interaction with VACUUM that causes the issue with shallow clones.

B: While deep cloning is a way to prevent data deletion for clones in instances like this, running VACUUM itself does not invalidate the clone. The files must be deleted for it to invalidate the clone.

C: Shallow clones are not automatically deleted after 7 days. The retention period is configurable and defaults to much longer (e.g., 30 days).

E: Refreshing a shallow clone won't bring back the data files purged by VACUUM. REFRESH updates the metadata to reflect the current state of the underlying data, but it cannot recover deleted files.

Therefore, the VACUUM command removing the underlying data files referenced by the shallow clone's metadata is the direct cause of the cloned tables becoming unusable.

Authoritative Links for Further Research:

Delta Lake Cloning: <https://docs.databricks.com/delta/delta-clone.html>

Delta Lake VACUUM: <https://docs.databricks.com/delta/delta-utility.html#remove-files-no-longer-referenced-by-a-delta-table>

Question: 93

Deep Dumps

You are performing a join operation to combine values from a static userLookup table with a streaming DataFrame streamingDF.

Which code block attempts to perform an invalid stream-static join?

- A.userLookup.join(streamingDF, ["userid"], how="inner")
- B.streamingDF.join(userLookup, ["user_id"], how="outer")
- C.streamingDF.join(userLookup, ["user_id"], how="left")
- D.streamingDF.join(userLookup, ["userid"], how="inner")
- E.userLookup.join(streamingDF, ["user_id"], how="right")

Answer: B

Explanation:

The correct answer is B: streamingDF.join(userLookup, ["user_id"], how="outer"). This operation is invalid because Spark Structured Streaming only supports stream-static joins where the static DataFrame (in this case, userLookup) is on the right side of the join. Spark Structured Streaming has limitations to ensure the state can be managed efficiently in a streaming context. Supporting a static table on the left side of the join and a streaming table on the right optimizes for scenarios where you're looking up information from a static, relatively small dataset based on data arriving in a stream.

An "outer" join, whether left, right, or full, requires the system to maintain state for both sides of the join. In a stream-static join scenario, maintaining state on the static side isn't necessary because the static data doesn't change. However, maintaining state on the streaming side is essential to handle incoming data. When the streaming DataFrame is on the left with an outer join, Spark must track which rows in the streaming DataFrame have already been matched with the static DataFrame, which becomes a significant state management burden, especially for infinite streams.

Options A, D, and E all have userLookup on the right side of the join, potentially making them valid stream-static joins (assuming appropriate schema and key names). The how parameter being "inner" or "right" does not invalidate the operation from a stream-static perspective if the static table is on the correct side of the join. Option C streamingDF.join(userLookup, ["user_id"], how="left") is invalid for the same reason as B, left outer joins involving a static table on the right side is not supported. However, the most voted answer is B.

Therefore, the operation streamingDF.join(userLookup, ["user_id"], how="outer") is an invalid stream-static join because it places the streaming DataFrame on the left and attempts an outer join. This requires Spark to maintain state for the incoming stream against a static dataset which leads to high state management overhead for an infinite data stream, which is not a supported operation.

Relevant Documentation:

[Spark Structured Streaming Programming Guide - Joins](#)
[Databricks documentation on stream-static joins](#)

Question: 94

Deep Dumps

Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

- A. Query's detail screen and Job's detail screen
- B. Stage's detail screen and Executor's log files
- C. Driver's and Executor's log files
- D. Executor's detail screen and Executor's log files
- E. Stage's detail screen and Query's detail screen

Answer: B

Explanation:

The correct answer is B: Stage's detail screen and Executor's log files. Here's why:

Spilling to disk happens when a stage in a Spark job requires more memory than is available on an executor. Wide transformations like `groupBy` or `join` often cause this as they shuffle data across partitions. The Spark UI is crucial for diagnosing performance issues like spilling.

The **Stage's detail screen** provides aggregate metrics for each stage. Specifically, you can view metrics like "Disk Bytes Spilled". A high value here indicates that the executors are writing data to disk due to memory pressure during that stage. Analyzing which stages are spilling is the first step in identifying problematic transformations.

The **Executor's log files** provide detailed information about what's happening on each executor. You might see entries specifically mentioning "spilling" or memory-related errors. These logs are invaluable for pinpointing exactly why the executor is running out of memory and leading to spilling. For instance, they may reveal a skewed distribution of data in a partition or exceptionally large records exceeding executor memory limits. This helps in understanding which specific tasks within a stage are causing the issue.

The Query's detail screen provides a high-level overview, but it doesn't directly show spill metrics. The Job's detail screen summarizes job-level information, but the stage-level detail is more crucial for spill diagnosis. Driver logs are typically more focused on the overall application execution and less about individual executor memory usage. Executor detail screens (if they existed as a dedicated page – which they don't in the standard Spark UI) would be useful, but the critical diagnostic information related to why the spill is occurring is still in the Executor logs. Combining the high-level "Disk Bytes Spilled" from the stage detail screen and the granular executor logs gives the best insight.

Therefore, the Stage detail screen and the Executor log files are the best combination to proactively diagnose spill in Spark.

Reference:

[Spark UI Overview](#)
[Monitoring and Instrumentation](#)

Question: 95

Deep Dumps

A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at `/mnt/raw_orders/`. After this data is written, a Databricks job containing the following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order, and that the `time` field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

- A. Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same `customer_id` and `order_id`.
- B. All records will be held in the state store for 2 hours before being deduplicated and committed to the orders table.
- C. The orders table will contain only the most recent 2 hours of records and no duplicates will be present.
- D. Duplicate records arriving more than 2 hours apart will be dropped, but duplicates that arrive in the same batch may both be written to the orders table.
- E. The orders table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

Answer: A

Explanation:

Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same `customer_id` and `order_id`.

Question: 96

Deep Dumps

A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

- A. Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
- B. Databricks supports Spark SQL and JDBC; all logic can be directly migrated from the source system without refactoring.
- C. Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.

D.All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.

E.Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.

Answer: D

Explanation:

The correct answer is D because it accurately reflects the behavior of Delta Lake and its implications for migrating relational workloads to Databricks. Here's a detailed justification:

Delta Lake offers ACID (Atomicity, Consistency, Isolation, Durability) guarantees for transactions. However, these guarantees are scoped to individual table operations. While you can perform complex operations like updates and deletes atomically within a single table, Delta Lake doesn't inherently enforce foreign key constraints to maintain referential integrity across multiple tables.

Relational database systems rely heavily on foreign key constraints to ensure data consistency by preventing orphaned records and maintaining relationships between tables. Multi-table inserts leverage these constraints during write operations to validate data before committing changes.

In the context of migrating to Databricks, this difference in behavior is crucial. The junior data engineer cannot simply replicate the relational database's constraint-based validation logic. Databricks will not automatically enforce foreign key relationships or rollback transactions involving multiple tables if constraint violations occur.

Alternatives such as A, B, C, and E are incorrect:

A: While hashing can be used to improve performance, foreign key constraints are not inherently limited to hashed identifiers in Databricks. More importantly, Databricks doesn't enforce them.

B: Simply migrating the SQL without refactoring is not feasible. The absence of enforced foreign key constraints and the behavior of multi-table operations necessitate adjustments to the data pipeline.

C: While locking is involved in concurrent operations, the primary issue here is the lack of built-in support for ACID transactions across multiple tables to enforce constraints, not just deadlock issues.

E: While upsert operations are possible, they don't replace the need for application-level handling of referential integrity in the absence of enforced foreign key constraints.

Therefore, the junior data engineer needs to implement alternative mechanisms to maintain data integrity during the migration. This could involve custom data quality checks, validation steps within the data pipeline, or utilizing features like Delta Lake's CHECK constraints (which are not foreign key constraints). These custom solutions will ensure the integrity and quality of data within the Databricks Lakehouse, mimicking the behavior enforced by foreign keys in the original relational database.

Authoritative Links:

Delta Lake ACID Properties: <https://docs.databricks.com/delta/index.html>

Delta Lake Constraints: <https://docs.databricks.com/delta/constraints.html>

Question: 97

Deep Dumps

A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in the customers table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table

will provide better performance and scalability.

Which piece of information is critical to this decision?

- A.Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
- B.Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
- C.Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
- D.Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.
- E.Delta Lake only supports Type 0 tables; once records are inserted to a Delta Lake table, they cannot be modified.

Answer: D

Explanation:

The correct answer is D because the scalability of Delta Lake's time travel feature is crucial when considering long-term auditing requirements. While Delta Lake provides versioning, relying solely on it for Type 1 updates over extended periods can become expensive and slow. Each update generates a new version, accumulating metadata and potentially requiring scans across many versions to reconstruct historical states. Type 2 tables, conversely, maintain historical data directly, enabling efficient querying of past states without extensive time travel calculations.

Option A is incorrect because while update operations can have transactional considerations, this isn't the primary concern when comparing the scalability of Type 1 vs. Type 2 for long-term auditing. Delta Lake ACID transactions ensure data consistency.

Option B is incorrect because shallow clones are more applicable to efficiently creating copies of tables for testing or experimentation, not necessarily for optimizing historical queries in Type 1 tables that rely on time travel.

Option C is incorrect; Delta Lake time travel can be used with Type 1 changes. The issue is the performance and cost implications of relying on it extensively for long-term history.

Option E is incorrect; Delta Lake supports updates and deletes.

In summary, the critical factor is the scalability of Delta Lake's time travel functionality versus the performance advantages of directly storing historical data in a Type 2 table. Type 2 tables are inherently optimized for this use case. Therefore, understanding the limitations of time travel is vital in deciding the best approach for long-term auditability.

Relevant links for further research:

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Updates and Deletes: <https://docs.databricks.com/delta/delta-update.html>

Type 1 vs Type 2 dimensions: Search for "Type 1 vs Type 2 dimensions data warehousing" for general Data Warehousing concept.

Question: 98

Deep Dumps

A table named user_ltv is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The user_ltv table has the following schema:

email STRING, age INT, ltv INT

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
    CASE
        WHEN is_member("auditing") THEN TRUE
        ELSE age >= 18
    END
```

An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

- A. All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.
- B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in user_ltv.
- C. All values for the age column will be returned as null values, all other columns will be returned with the values in user_ltv.
- D. All records from all columns will be displayed with the values in user_ltv.
- E. All columns will be displayed normally for those records that have an age greater than 18; records not meeting this condition will be omitted.

Answer: A

Explanation:

All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.

Question: 99

Deep Dumps

The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the user_lookup table to the user_aggregates table.


```

(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
  DELETE FROM user_aggregates
  WHERE user_id IN (
    SELECT user_id
    FROM changes
    WHERE _change_type='delete'
  )
""")

```

Assuming that user_id is a unique identifying key and that all users that have requested deletion have been removed from the user_lookup table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the user_aggregates table are no longer accessible and why?

- A.No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- B.No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.
- C.Yes; the change data feed uses foreign keys to ensure delete consistency throughout the Lakehouse.
- D.Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- E.No; the change data feed only tracks inserts and updates, not deleted records.

Answer: B

Explanation:

No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Question: 100

Deep Dumps

The data engineering team has been tasked with configuring connections to an external database that does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

- A. "Manage" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
 - B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
 - C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
 - D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
- No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.

Answer: C

Explanation:

The correct answer is C: "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

Here's a detailed justification:

The principle of least privilege dictates granting users only the permissions necessary to perform their tasks. In this scenario, teams need to use the credentials to access the external database, not manage or alter them. Therefore, "Manage" permissions are unnecessarily broad.

Secrets in Databricks are organized within secret scopes. Scopes provide a namespace for secrets and control access. Option C proposes creating separate scopes for each team, containing only the credentials relevant to that team. By granting "Read" permissions to a team on their specific secret scope, users can access the credentials defined within that scope. They are prevented from accessing credentials intended for other teams, enforcing security and isolation.

Option A and D suggest granting "Manage" permissions, which would allow teams to modify or delete the secrets, violating the principle of least privilege. Option B grants "Read" permissions on a specific secret key, which is less ideal because each team is expected to use one new credential. The proposed solution in option C encapsulates the credential within a scope and assigns the permission to the scope itself. The final statement, implying administrator access negates security concerns, is incorrect. Workspace administrators may have global control, but the goal here is to implement granular access control based on team membership, which relies on proper secret scope configuration and access controls.

Therefore, configuring a secret scope per team, storing team-specific credentials within that scope, and granting "Read" permissions to the corresponding Databricks group on that scope offers the most secure and manageable solution, adhering to the principle of least privilege.

Further research:

Databricks Secrets: <https://docs.databricks.com/security/secrets/index.html>

Databricks Secret Scopes: <https://docs.databricks.com/security/secrets/secret-scopes.html>

Databricks Access Control: <https://docs.databricks.com/security/access-control/index.html>

Question: 101

Deep Dumps

Which indicators would you look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally? Assume you are using Spark's MEMORY_ONLY storage level.

- A. Size on Disk is < Size in Memory
- B. The RDD Block Name includes the "*" annotation signaling a failure to cache
- C. Size on Disk is > 0
- D. The number of Cached Partitions > the number of Spark Partitions
- E. On Heap Memory Usage is within 75% of Off Heap Memory Usage

Answer: C

Explanation:

The correct answer is **C. Size on Disk is > 0**. Here's why:

When using `MEMORY_ONLY` storage level in Spark, data is supposed to be stored entirely in memory. If Size on Disk is greater than 0, it indicates that Spark has spilled some of the cached data to disk because it ran out of memory. This is a clear sign that the caching strategy is not performing optimally. Spilling to disk significantly slows down data access because disk I/O is much slower than memory access. The whole point of caching is to avoid disk reads. If Spark resorts to disk, the cached table isn't serving its purpose efficiently, defeating the advantages gained through caching.

Option A is incorrect because Size on Disk should ideally be 0 when using `MEMORY_ONLY`.

Option B is incorrect because the "*" annotation in the RDD Block Name indicates that the RDD might have been partially cached, not that caching definitely failed. A failure to cache results in no caching, not partial caching with disk usage in `MEMORY_ONLY`.

Option D is incorrect because the number of cached partitions should typically match the number of Spark partitions for optimal parallelism. An unequal number doesn't inherently signal a performance problem related to caching performance given `MEMORY_ONLY`. It might indicate an imbalance in the partitioning scheme itself, but not necessarily that caching is failing.

Option E is incorrect because the ratio of On Heap to Off Heap Memory Usage doesn't directly indicate if a `MEMORY_ONLY` cached table is performing optimally. It mainly reflects how memory is allocated among driver and executor processes. When spilling to disk, we can consider it, but not an evident sign.

In summary, Size on Disk > 0 when `MEMORY_ONLY` is used directly indicates memory pressure and the necessity to spill to disk. The whole purpose of caching is to prevent disk reads.

Further reading on Spark caching:

Spark Documentation on Caching: <https://spark.apache.org/docs/latest/rdd-programming-guide.html#rdd-persistence>

Understanding Spark Memory Management: <https://databricks.com/blog/2015/05/28/tuning-apache-spark-memory-management-for-large-scale-applications.html>

Question: 102

Deep Dumps

What is the first line of a Databricks Python notebook when viewed in a text editor?

- A. `%python`
- B. `// Databricks notebook source`
- C. `# Databricks notebook source`
- D. `-- Databricks notebook source`
- E. `# MAGIC %python`

Answer: C

Explanation:

The correct answer is **C. # Databricks notebook source**. Here's why:

Databricks notebooks, while interactive environments for code execution, are ultimately stored as text files. When you download a Databricks notebook (e.g., in .dbc format and then extract to a .py file), the first line you will consistently observe in a text editor is `# Databricks notebook source`. This serves as a marker indicating that the file originates from the Databricks environment and contains metadata or formatting specific to Databricks.

Options A, B, D, and E are incorrect because they are not the standard initial lines found when inspecting Databricks notebooks in a text editor. `%python` is a magic command used within Databricks notebooks to explicitly designate a cell as a Python cell (although usually unnecessary). The others are simply incorrect syntax. The initial comment serves primarily as an identifier for Databricks' internal processing of the file.

Therefore, understanding the structure of Databricks notebooks as they are stored in plain text is crucial. This includes recognizing the initial header comment, which is indicative of the notebook's origin and can be important for automated processing or version control systems that may interact with these files.

Further research:

While not explicitly documented, downloading and inspecting Databricks notebooks as .py files consistently reveals `# Databricks notebook source` as the first line. Experimentation is the best way to verify this.

Databricks workspace documentation: <https://docs.databricks.com/en/workspace/index.html>

Question: 103

Deep Dumps

Which statement describes a key benefit of an end-to-end test?

- A. Makes it easier to automate your test suite
- B. Pinpoints errors in the building blocks of your application
- C. Provides testing coverage for all code paths and branches
- D. Closely simulates real world usage of your application
- E. Ensures code is optimized for a real-life workflow

Answer: D

Explanation:

End-to-end (E2E) tests are designed to validate the complete flow of an application from start to finish, simulating a user's interaction with the system. The most significant benefit of E2E tests is their ability to closely simulate real-world usage of the application (D). This is because E2E tests cover multiple components and systems working together, mimicking how a user would actually interact with the application in a production environment.

Option A is less accurate because while E2E tests can be automated, that's not their defining benefit. Unit and integration tests are often easier to automate. Option B is incorrect because E2E tests typically identify failures across the system, not pinpoint specific errors in individual building blocks (unit tests focus on that). Option C is incorrect because achieving 100% code coverage is challenging, and E2E tests don't guarantee coverage of all paths and branches. They focus on critical user flows. Option E, optimization, isn't directly addressed by E2E tests, although they can indirectly highlight inefficiencies if workflows take an unacceptable amount of time.

In cloud computing environments, E2E tests are critical for verifying that various services and components (e.g., data ingestion pipelines, data processing jobs, data warehousing queries, and presentation layers) integrate seamlessly. They help uncover issues related to network latency, data consistency, and service dependencies that might not be apparent in unit or integration testing. For example, an E2E test might simulate a user submitting a form, triggering a data ingestion job, processing that data with Spark, storing the results in a data lake, and displaying the updated information in a dashboard. This complete workflow verification ensures the entire system behaves as expected.

Therefore, simulating real-world usage is the primary advantage of E2E tests, making option D the most accurate.

For more information, refer to these resources:

End-to-End Testing: <https://www.browserstack.com/guide/end-to-end-testing>

Testing Strategies: <https://martinfowler.com/bliki/TestPyramid.html>

Question: 104

Deep Dumps

The Databricks CLI is used to trigger a run of an existing job by passing the `job_id` parameter. The response that the job run request has been submitted successfully includes a field `run_id`.

Which statement describes what the number alongside this field represents?

- A. The `job_id` and number of times the job has been run are concatenated and returned.
- B. The total number of jobs that have been run in the workspace.
- C. The number of times the job definition has been run in this workspace.
- D. The `job_id` is returned in this field.
- E. The globally unique ID of the newly triggered run.

Answer: E

Explanation:

The correct answer, E, states that the `run_id` represents the globally unique ID of the newly triggered run. This is the most accurate and practical interpretation in the context of job execution within Databricks.

Here's a detailed justification:

When a job is triggered via the Databricks CLI, the system needs a mechanism to track and manage the specific execution instance. The `run_id` serves precisely this purpose. It uniquely identifies a single, distinct execution of a job, differentiating it from other executions of the same job or any other job in the workspace. Each time a job runs, it is assigned a new `run_id`.

Options A, B, C, and D are incorrect for the following reasons:

A: The `job_id` and number of times the job has been run are concatenated and returned. While the number of times a job is run might be a component of an internal tracking system, the external `run_id` is specifically designed for identifying a single run. Concatenating job ID and a run counter would not guarantee global uniqueness, especially across different workspaces.

B: The total number of jobs that have been run in the workspace. The `run_id` is specific to one job run, not a cumulative count across all jobs. A workspace-wide job count has a different scope and purpose.

C: The number of times the job definition has been run in this workspace. This is related to the frequency of job execution, but not a unique identifier for an instance of a particular execution. The run ID is for a particular run.

D: The `job_id` is returned in this field. The `run_id` is distinct from the `job_id`. The `job_id` identifies the job

definition, while the `run_id` identifies the execution of that job. Returning the `job_id` in the `run_id` field would negate the purpose of having a unique identifier for each execution.

The `run_id` is crucial for:

Monitoring: Tracking the status (e.g., running, success, failed) of a specific job execution.

Logging: Retrieving logs associated with a particular job run.

Debugging: Investigating issues within a specific execution context.

Reproducibility: Ensuring the ability to replicate a particular run, if necessary.

API Interactions: Using the `run_id` to programmatically interact with the job execution via the Databricks REST API or CLI.

In essence, the `run_id` acts as the primary key for a job run, providing a unique and reliable way to reference and manage each individual execution instance within the Databricks environment.

Authoritative Links:

Databricks Jobs API: <https://docs.databricks.com/api/workspace/jobs> (While this doesn't explicitly define "globally unique", it describes how the run ID is used to manage individual runs via the API.)

Databricks CLI documentation: <https://docs.databricks.com/en/dev-tools/cli/index.html> (Shows how the run id is obtained upon execution).

Question: 105

Deep Dumps

The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type DOUBLE.

The following code correctly imports the production model, loads the customers table containing the `customer_id` key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "customer_id LONG, predictions DOUBLE"?

- A. `df.map(lambda x:model(x[columns])).select("customer_id, predictions")`
- B. `df.select("customer_id", model(*columns).alias("predictions"))`
- C. `model.predict(df, columns)`
- D. `df.select("customer_id", pandas_udf(model, columns).alias("predictions"))`
- E. `df.apply(model, columns).select("customer_id, predictions")`

Answer: B

Explanation:

This code block applies the Spark UDF created from the MLflow model to the DataFrame `df` by selecting the existing `customer_id` column and the new column produced by the model, which is aliased to `predictions`. The `model(*columns)` part is where the UDF is applied to the columns specified in the `columns` list, and `alias("predictions")` is used to name the output column of the model's predictions. This will result in a DataFrame with the desired schema: "customer_id LONG, predictions DOUBLE".

A nightly batch job is configured to ingest all data files from a cloud object storage container where records are stored in a nested directory structure YYYY/MM/DD. The data for each date represents all records that were processed by the source system on that date, noting that some records may be delayed as they await moderator approval. Each entry represents a user review of a product and has the following schema:

user_id STRING, review_id BIGINT, product_id BIGINT, review_timestamp TIMESTAMP, review_text STRING

The ingestion job is configured to append all data for the previous date to a target table reviews_raw with an identical schema to the source system. The next step in the pipeline is a batch write to propagate all new records inserted into reviews_raw to a table where data is fully deduplicated, validated, and enriched.

Which solution minimizes the compute costs to propagate this batch of data?

- A. Perform a batch read on the reviews_raw table and perform an insert-only merge using the natural composite key user_id, review_id, product_id, review_timestamp.
- B. Configure a Structured Streaming read against the reviews_raw table using the trigger once execution mode to process new records as a batch job.
- C. Use Delta Lake version history to get the difference between the latest version of reviews_raw and one version prior, then write these records to the next table.
- D. Filter all records in the reviews_raw table based on the review_timestamp; batch append those records produced in the last 48 hours.
- E. Reprocess all records in reviews_raw and overwrite the next table in the pipeline.

Answer: B

Explanation:

The most cost-effective approach to propagate new records from reviews_raw to a deduplicated, validated, and enriched table is option B: Configure a Structured Streaming read against the reviews_raw table using the trigger once execution mode to process new records as a batch job.

Here's why:

Structured Streaming with trigger once is designed for exactly this type of batch processing scenario on append-only data. It reads new data that has arrived since the last trigger, processes it, and then stops. This avoids repeatedly processing the entire reviews_raw table, thus minimizing compute costs. It's more efficient than other options since it inherently supports append-only processing. Other batch options would perform a full scan which is unnecessary given the append-only ingestion process.

Option A involves an insert-only merge operation, which is viable but potentially more expensive than Structured Streaming with trigger once. Merge operations can be computationally intensive, especially with large tables. While the natural key helps identify duplicates, the merge itself consumes resources.

Option C, using Delta Lake version history, could work, but it adds unnecessary complexity for a simple append-only scenario. Version history diffing is useful for more complex data changes but adds overhead for this use case. It also relies on properly configured Delta history retention.

Option D, filtering by review_timestamp, is problematic. Since moderator approval delays some records, filtering by a fixed time window (last 48 hours) risks missing delayed entries. This leads to data inconsistencies and inaccuracies in the final table.

Option E, reprocessing all records, is the most computationally expensive and least efficient choice. It ignores the fact that only new records need processing. It's unnecessary and wasteful.

Structured Streaming provides a more efficient and targeted approach. By setting up a streaming job with trigger once, the system only processes new records that have arrived since the last run, optimizing resource

utilization and minimizing costs. This leverages the append-only nature of the source data.

Authoritative links:

Spark Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Structured Streaming Trigger Once: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#triggers>

Delta Lake Merge: <https://docs.delta.io/latest/delta-update.html>

Question: 107

Deep Dumps

Which statement describes Delta Lake optimized writes?

- A. Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.
- B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- C. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
- D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
- E. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.

Answer: E

Explanation:

The correct answer is E: "A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions."

Here's why: Delta Lake optimized writes address the small file problem common in data lakes. Small files lead to increased metadata overhead, slower query performance, and higher storage costs. Optimized writes, specifically through techniques like bin-packing, aim to reduce the number of small files created during data ingestion or transformation processes.

Option E accurately describes this mechanism. A shuffle operation redistributes the data across partitions based on some criteria (often partitioning columns or other relevant data characteristics). This allows Spark executors to write larger, more consolidated files because executors receive more data that naturally belongs together within a partition. Without this shuffle, each executor would independently write files based on the data it locally processes, leading to many small files. The shuffling process reduces the volume of file output from individual executors and also reduces the quantity of distinct file outputs during the writing phase. By grouping like data together before the write operation occurs, Delta Lake is able to write larger files and reduce the total count of files.

Let's examine why the other options are incorrect:

A: While running OPTIMIZE after jobs can improve performance, it's not directly part of the write process itself, and the suggestion about automatic execution before cluster termination is not generally a standard feature.

B: This is similar to A. Post-write compaction is achieved by OPTIMIZE. While OPTIMIZE does compact, it is not an integral part of the optimized write procedure itself.

C: This describes a messaging queue architecture, not optimized writes in Delta Lake. While messaging queues can be part of a larger data pipeline, they are not specifically related to Delta Lake optimized writes.

D: Delta Lake still leverages directory partitioning, and the concept of "logical partitions" isn't accurate in this

context. While Delta Lake improves metadata management, it doesn't fundamentally replace directory partitioning with a solely metadata-driven approach.

Authoritative Links:

Delta Lake Optimization: <https://docs.databricks.com/delta/optimizations/index.html> (Specifically, look for sections on file sizing and bin-packing)

Delta Lake Best Practices: <https://docs.databricks.com/delta/best-practices.html> (This provides general information on data layout and optimization)

Question: 108

Deep Dumps

Which statement describes the default execution mode for Databricks Auto Loader?

- A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; the target table is materialized by directly querying all valid files in the source directory.
- B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
- C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.
- D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.
- E. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.

Answer: E

Explanation:

The correct answer is E because it accurately describes Auto Loader's default execution mode leveraging cloud-native features for efficiency and reliability.

Auto Loader, by default, utilizes cloud vendor-specific queue storage and notification services (like AWS SQS and SNS, Azure Queue Storage and Event Grid, or Google Cloud Pub/Sub and Cloud Storage Notifications). These services are configured to monitor the input directory for new file arrivals, ensuring real-time or near real-time data ingestion. This avoids expensive and potentially inconsistent directory listings.

Furthermore, Auto Loader incrementally and idempotently loads these new files into the target Delta Lake table. "Incrementally" means it only processes new data since the last run, saving resources and time. "Idempotently" ensures that even if the same file is processed multiple times (due to, for example, a retry after a failure), it will only be added to the target table once, preventing duplicates and data corruption. This idempotency is crucial for data quality in streaming scenarios. The target table is a Delta Lake table to leverage ACID transactions, versioning, and schema evolution capabilities. It does not directly query all files in the source directory for each update in this default mode; that would negate the benefits of incremental loading.

Option A is incorrect because while it mentions cloud vendor services, it states the target table is materialized by querying all files, negating incremental loading. Option B is wrong because it describes directory listing, not cloud services, for new file identification. Option C talks about webhooks triggering jobs and automatic merging, which is not the default behavior and can be misleading. Option D is partially correct about incremental loading but misses the crucial aspect of leveraging cloud notification services by default.

Therefore, E best captures the default operation by combining cloud notifications with incremental and idempotent loading into Delta Lake.

Further Reading:

Databricks Auto Loader Documentation: <https://docs.databricks.com/delta/auto-loader.html>

Delta Lake: <https://delta.io/>

Question: 109

Deep Dumps

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

- A.post_time
- B.latitude
- C.post_id
- D.user_id
- E.date

Answer: E

Explanation:

The best column for partitioning the Delta Lake table is date. Partitioning in data warehousing and data lakes like Delta Lake involves physically dividing the data based on column values to improve query performance. Choosing the right partition column is crucial.

Option A (post_time) is less ideal than date because it's a timestamp, leading to many small partitions and potential performance degradation (the "small files problem"). Timestamps have high cardinality, meaning many unique values, which defeats the purpose of partitioning, which is to reduce the amount of data scanned for a given query.

Options B (latitude) and C (post_id) are also not good choices. latitude is a continuous numerical value, and like post_time, it has the potential for high cardinality, resulting in numerous small partitions. post_id is likely to be unique for each post, leading to extreme cardinality and rendering partitioning useless.

Option D (user_id) could be considered, but it depends on the data distribution. If some users generate significantly more posts than others, it could lead to unevenly sized partitions (data skew), which can also negatively impact query performance. However, if the user base is balanced with posts, that would be an okay choice.

Option E (date) is the most appropriate because it provides a balance between granularity and cardinality. Users often query data by date ranges (e.g., "show me posts from the last week"). Partitioning by date will allow Delta Lake to quickly filter out partitions that don't fall within the specified date range, significantly reducing the amount of data that needs to be scanned. The number of unique dates is far less than the other continuous numerical and unique options, but much more appropriate than partitioning on a value with extreme data skew. This approach aligns well with common time-series analysis and reporting use cases. It strikes a good balance between reducing data scanning and avoiding an excessive number of small partitions.

Therefore, date is the ideal candidate for partitioning as it enables efficient filtering based on time ranges, a common query pattern, while avoiding the issues associated with high-cardinality or skewed columns.

Reference links for more in-depth information:

Question: 110

Deep Dumps

A large company seeks to implement a near real-time solution involving hundreds of pipelines with parallel updates of many tables with extremely high volume and high velocity data.

Which of the following solutions would you implement to achieve this requirement?

- A. Use Databricks High Concurrency clusters, which leverage optimized cloud storage connections to maximize data throughput.
- B. Partition ingestion tables by a small time duration to allow for many data files to be written in parallel.
- C. Configure Databricks to save all data to attached SSD volumes instead of object storage, increasing file I/O significantly.
- D. Isolate Delta Lake tables in their own storage containers to avoid API limits imposed by cloud vendors.
- E. Store all tables in a single database to ensure that the Databricks Catalyst Metastore can load balance overall throughput.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most appropriate solution for a near real-time, high-volume, high-velocity data processing scenario using Databricks, and why the other options are less suitable.

Option A, "Use Databricks High Concurrency clusters, which leverage optimized cloud storage connections to maximize data throughput," is the best choice because it directly addresses the core challenges of high-volume and high-velocity data ingestion and processing. High Concurrency clusters are designed to handle concurrent queries and data updates, crucial for near real-time scenarios. They are optimized for cloud storage access, using techniques like vectorized reads/writes and optimized connectors, to minimize I/O bottlenecks when reading from and writing to object storage (e.g., AWS S3, Azure Blob Storage, Google Cloud Storage). This is essential for performance.

Option B, "Partition ingestion tables by a small time duration to allow for many data files to be written in parallel," can help with parallel writes, but it can also lead to a large number of small files (the small file problem). While Delta Lake can handle this to some extent, managing a huge number of partitions efficiently can become complex and may not be as effective as addressing the core concurrency issues directly.

Option C, "Configure Databricks to save all data to attached SSD volumes instead of object storage, increasing file I/O significantly," is not recommended. While SSDs offer faster I/O compared to object storage, they are typically more expensive and less scalable. Object storage is designed for massive scalability and durability, and optimized Databricks clusters (Option A) can leverage it efficiently. Storing data on ephemeral SSDs attached to compute nodes can also lead to data loss if a node fails.

Option D, "Isolate Delta Lake tables in their own storage containers to avoid API limits imposed by cloud vendors," is a good practice in general for very large datasets, but isn't the primary concern here. While API limits can become a problem, optimizing concurrency and I/O (as in Option A) will have a more immediate and greater impact. Sharding is more of a scaling consideration than a near real-time ingestion issue per se.

Option E, "Store all tables in a single database to ensure that the Databricks Catalyst Metastore can load balance overall throughput," is generally a bad idea. Putting all tables into a single database doesn't inherently improve Metastore throughput and can make management and permissions more complex. It also could lead to naming conflicts and doesn't address the core problem of concurrent data ingestion.

In summary, Option A provides the most direct and effective solution by leveraging Databricks' optimized clusters for high concurrency and efficient cloud storage access, essential for near real-time data processing at scale.

Authoritative Links:

Databricks High Concurrency Clusters: <https://docs.databricks.com/clusters/configure.html#--cluster-mode>
Delta Lake on Databricks: <https://docs.databricks.com/delta/index.html>

Question: 111

Deep Dumps

Which describes a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

- A.Run `source env/bin/activate` in a notebook setup script
- B.Use `b` in a notebook cell
- C.Use `%pip install` in a notebook cell
- D.Use `%sh pip install` in a notebook cell
- E.Install libraries from PyPI using the cluster UI

Answer: C

Explanation:

The correct answer is **C. Use `%pip install` in a notebook cell.**

Here's a detailed justification:

Databricks notebooks provide "magic commands," which are special commands that enhance notebook functionality. `%pip` is a Databricks magic command that allows you to install Python packages directly from a notebook cell. When you use `%pip install <package_name>`, Databricks installs the package on all driver and worker nodes of the currently active cluster. This ensures that the package is available for use in any notebook attached to that cluster. The installation is scoped to the cluster, meaning that the installed package will persist as long as the cluster is active and won't affect other clusters. Importantly, the installed package is available to all notebooks using the active cluster.

Option A (Run `source env/bin/activate` in a notebook setup script) is generally used for activating virtual environments locally, not for installing packages cluster-wide in Databricks. Setup scripts for init scripts run on the cluster, but this isn't the simplest method for notebook-scoped installs.

Option B is incomplete and doesn't specify a valid command.

Option D (Use `%sh pip install` in a notebook cell) is also incorrect. While `%sh` lets you execute shell commands, using it with `pip install` would install the package on the driver node of the cluster, not on all nodes, leading to inconsistencies. `pip` invoked this way also often utilizes a system python installation, not the Databricks environment, potentially creating conflicts.

Option E (Install libraries from PyPI using the cluster UI) is a valid method for installing libraries on a cluster. However, the question specifically asks about a method from the notebook. While using the cluster UI impacts all notebooks using a cluster, the question is explicitly asking for a method within a notebook cell.

Using `%pip` provides a convenient and quick way to manage package dependencies within a Databricks notebook environment, ensuring that all nodes in the cluster have the necessary packages for execution. This directly addresses the requirement of installing a Python package scoped at the notebook level to all nodes in the currently active cluster.

For more information, refer to the official Databricks documentation on managing Python packages with %pip and %conda:

[Databricks documentation: %pip](#)

[Databricks documentation: Libraries](#)

Question: 112

Deep Dumps

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

- A. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- B. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores / Executor
- C. • Total VMs: 1
 - 400 GB per Executor
 - 160 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores / Executor
- E. • Total VMs: 2
 - 200 GB per Executor
 - 80 Cores / Executor

Answer: B

Explanation:

The correct answer is B. Here's why:

The key factor for guaranteed job completion in the face of VM failures is fault tolerance. Fault tolerance in distributed computing environments like Spark (used in Databricks) is primarily achieved through replication and distribution of data and computation across multiple nodes. A cluster with more VMs provides higher fault tolerance because if one or more VMs fail, the remaining VMs can still process the data, assuming sufficient resources and task distribution.

Option C with only one VM offers no fault tolerance. If that single VM fails, the entire job fails. Options A, D, and E offer some fault tolerance, but less than option B.

Option B, with 16 VMs, has the highest degree of redundancy. The job's tasks are spread across these 16 executors. Even if several VMs fail, the remaining VMs are more likely to have the resources to pick up the failed tasks and continue processing. Spark's scheduler is designed to redistribute tasks from failed executors to available executors automatically, given sufficient resources are available.

The smaller executors (25 GB RAM, 10 cores) in Option B allows for a finer-grained distribution of tasks. This better distribution means a failure is less likely to overwhelm the remaining resources compared to configurations with larger executors and fewer VMs.

The ability of the cluster to tolerate failures depends on how Spark has distributed the tasks. Spark automatically redistributes incomplete tasks to other workers as long as there are sufficient resources on the remaining workers to complete them. In conclusion, the higher number of VMs in option B provides a

significantly better chance of tolerating VM failures and guaranteeing job completion, making it the most suitable choice. For further research, refer to these resources:

Apache Spark Fault Tolerance: <https://spark.apache.org/docs/latest/> (Search for "fault tolerance" within the Spark documentation)

Databricks documentation on cluster management: <https://docs.databricks.com/clusters/index.html>

Question: 113

Deep Dumps

A Delta Lake table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

- A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and time travel functionality.
- B. Parse the Delta Lake transaction log to identify all newly written data files.
- C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
- D. Execute `DESCRIBE HISTORY customer_churn_params` to obtain the full operation metrics for the update, including a log of all records that have been added or modified.
- E. Use Delta Lake's change data feed to identify those records that have been updated, inserted, or deleted.

Answer: E

Explanation:

The correct method for determining the difference between the new and previous versions of the `customer_churn_params` Delta Lake table, given the nightly overwrite implementation, is **E. Use Delta Lake's change data feed to identify those records that have been updated, inserted, or deleted.**

Here's a detailed justification:

1. **Change Data Feed (CDF) Purpose:** Delta Lake's Change Data Feed is specifically designed to track row-level changes (inserts, updates, and deletes) between versions of a Delta table. It provides a stream of records representing these changes.
2. **Overwrite Operation and CDF:** Even though the table is overwritten each night, CDF captures the changes between the current version (after overwrite) and the immediately preceding version. The overwrite operation effectively behaves like a complete replacement, making all the previously existing rows effectively deleted and replaced with the new set of rows. CDF is the most straightforward and efficient way to discern the differences in this scenario.
3. **Why other options are not suitable:**
 - A. Querying using Time Travel and Diff:** While Delta Lake's time travel allows querying previous versions, calculating the difference manually by querying both versions and performing a `UNION ALL` and aggregation or `EXCEPT` operation is inefficient and complex, especially with large datasets. This becomes increasingly challenging with larger datasets.
 - B. Parsing Transaction Log:** Parsing the Delta Lake transaction log directly is generally not recommended for change tracking. While the transaction log contains information about file

additions and removals, it doesn't directly give you row-level changes. Extracting row-level changes requires significant effort and understanding of the Delta log format. It is also prone to become unstable as Delta Lake evolves.

C. Parsing Spark Event Logs: Spark event logs primarily capture execution details of Spark jobs, such as task completion times and resource usage. These logs do not contain row-level data modification information. Therefore, parsing Spark event logs is inadequate for identifying row-level changes in a Delta table.

D. DESCRIBE HISTORY Limitations: DESCRIBE HISTORY provides metadata about table versions, including operation metrics and user information. While it shows the operation type (e.g., Overwrite), it doesn't give you a detailed record of each row that was added, modified, or deleted. It provides a higher-level overview and is not intended for detailed change tracking.

4. **Efficiency and Ease of Use:** CDF is the most performant solution because the data changes are tracked and stored efficiently alongside the Delta table itself. Delta Lake is optimized for these types of operations.
5. **Implementation:** The data engineering team can enable CDF on the customer_churn_params Delta table. After each successful overwrite, they can query the CDF to retrieve the changes (inserted, updated, deleted rows) between the current and previous versions. Here are some relevant authoritative links:

Delta Lake Change Data Feed: <https://docs.databricks.com/delta/delta-change-data-feed.html>

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Question: 114

Deep Dumps

A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in bold.

Original query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"),
       count("promo_code = 'NEW_MEMBER'").alias("new_member_promo"))
  .writeStream
  .outputMode("complete")
  .option('mergeSchema', 'true')
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Which step must also be completed to put the proposed query into production?

- A. Specify a new checkpointLocation
- B. Remove .option('mergeSchema', 'true') from the streaming write
- C. Increase the shuffle partitions to account for additional aggregates
- D. Run REFRESH TABLE delta.`/item\_agg`

Answer: A

Explanation:

Answer A When updating the schema of a streaming job, specifying a new checkpoint location ensures that the streaming query starts fresh with the new schema. This avoids issues that might arise from schema mismatches between the previous state and the new schema. This is especially relevant when adding new fields because the existing state might not be compatible with the new schema.

Question: 115

Deep Dumps

When using CLI or REST API to get results from jobs with multiple tasks, which statement correctly describes the response structure?

- A. Each run of a job will have a unique job_id; all tasks within this job will have a unique job_id
- B. Each run of a job will have a unique job_id; all tasks within this job will have a unique task_id
- C. Each run of a job will have a unique orchestration_id; all tasks within this job will have a unique run_id
- D. Each run of a job will have a unique run_id; all tasks within this job will have a unique task_id
- E. Each run of a job will have a unique run_id; all tasks within this job will also have a unique run_id

Answer: D

Explanation:

Let's break down why option D is the correct answer when retrieving results from Databricks jobs with multiple tasks using the CLI or REST API.

The core concept is understanding how Databricks structures runs and tasks. When you execute a Databricks job, it's considered a "run." Each time a job is executed, it's assigned a unique identifier: the run_id. This run_id distinguishes one execution of the job from another, even if the job definition remains the same.

Within a job run, you have individual tasks. These tasks are the distinct units of work that make up the overall job. Each task within a specific run is assigned its own unique identifier: the task_id. This ensures that each

task within that particular job run can be individually tracked and managed.

The `job_id` represents the overall definition of the job (its configuration, the code to be executed), not a specific execution instance. Therefore, it's not unique per run.

Options A, B, C, and E are incorrect because they misuse the terms `job_id`, `orchestration_id`, or `run_id` in relation to the uniqueness of runs and tasks. Option A incorrectly attributes uniqueness of `job_id` to individual tasks. Option B incorrectly states uniqueness of `job_id` to run and `task_id` to tasks. Option C invents a term `orchestration_id`. Option E incorrectly asserts tasks share a single `run_id` with their parent run.

Therefore, the structure of the response from the CLI or REST API reflects that each execution of a job has a unique `run_id`, and each task within that run has a unique `task_id`. This allows for precise identification and retrieval of results for each task within each job execution.

In summary, Option D, "Each run of a job will have a unique `run_id`; all tasks within this job will have a unique `task_id`," is the correct description.

Supporting documentation:

Databricks Jobs API: <https://docs.databricks.com/api/workspace/jobs> - This is a comprehensive resource describing all the API endpoints, parameters and response structures.

Databricks CLI: <https://docs.databricks.com/en/dev-tools/cli/index.html> - This is a resource describing how to use the Databricks CLI.

These official Databricks resources validate the concepts of `run_id` and `task_id` and how they are used to identify and retrieve results from jobs.

Question: 116

Deep Dumps

The data engineering team is configuring environments for development, testing, and production before beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

- A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
- B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
- C. As long as code in the development environment declares `USE dev_db` at the top of each notebook, there is no possibility of inadvertently committing changes back to production data sources.
- D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.
- E. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.

Answer: B

Explanation:

The correct answer is B because it emphasizes data security and isolation, crucial best practices in data

engineering, especially when dealing with production data.

Justification:

Option B advocates for granting only read permissions to production data in interactive environments like development and testing. This prevents accidental or malicious modifications to the live production data. Allowing write access in these environments creates a significant risk of data corruption, unintended changes, or even data loss.

Furthermore, isolating databases for each environment (development, testing, production) is a fundamental principle of environment segregation. This ensures that code executed in one environment does not inadvertently affect the data or processes in another. It provides a safeguard against unintended consequences arising from bugs, misconfigurations, or even malicious activities.

Options A, C, D, and E are flawed because they downplay the risks associated with exposing production data to development and testing environments without adequate safeguards. Option A suggests combining all environments, which is a terrible practice concerning security and stability. Option C's reliance on `USE dev_db` is insufficient, as it is easily overlooked and doesn't prevent accidental operations on mounted production data. Option D incorrectly assumes Delta Lake completely eliminates risks from user error. Time travel is helpful for recovery, but prevention is better. Option E is incorrect because passthrough credentials primarily manage access to storage, not the environment's inherent risks.

In essence, answer B highlights the importance of the principle of least privilege (granting only necessary permissions) and the benefits of environment isolation, both crucial for maintaining data integrity and security in a data engineering pipeline. Implementing these measures minimizes the impact of potential errors and reduces the risk of data breaches or corruption.

Supporting Links:

Data Security Best Practices: <https://docs.databricks.com/security/index.html>

Environment Isolation: (Search for "environment isolation best practices" on AWS, Azure, or GCP documentation for detailed guidance relevant to your cloud provider.)

Question: 117

Deep Dumps

A data engineer, User A, has promoted a pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

A workspace admin, User C, inherits responsibility for managing this pipeline. User C uses the Databricks Jobs UI to take "Owner" privileges of each job. Jobs continue to be triggered using the credentials and tooling configured by User B.

An application has been configured to collect and parse run information returned by the REST API. Which statement describes the value returned in the `creator_user_name` field?

- A. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User A's email address will appear in this field.
- B. User B's email address will always appear in this field, as their credentials are always used to trigger the run.
- C. User A's email address will always appear in this field, as they still own the underlying notebooks.
- D. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User B's email address will appear in this field.
- E. User C will only ever appear in this field if they manually trigger the job, otherwise it will indicate User B.

Answer: C

Explanation:

The correct answer is C: User A's email address will always appear in this field, as they still own the underlying notebooks. Here's why:

The `creator_user_name` field in the Databricks Jobs API, specifically when retrieving run information, reflects the original creator of the job, not the user who currently owns or triggers the job. Taking ownership of the job in the Databricks UI only grants administrative privileges, it doesn't change the historical creator information tied to the job's origin. This is a common pattern in cloud platforms to preserve audit trails and accountability. The fact that User B's credentials are used to trigger the run is irrelevant; the `creator_user_name` specifically refers to the user who created the job definition. The creator is tracked at the job definition level, which remains unchanged when User C takes ownership or User B triggers runs. The ownership change by User C only affects who can manage the job's settings and permissions, not its historical creation metadata. The underlying notebooks' ownership is also irrelevant to `creator_user_name`.

Therefore, User A's email will persist in the `creator_user_name` field, regardless of subsequent ownership changes or job executions triggered by different users.

For further information on the Databricks Jobs API, consult the official documentation:

[Databricks Jobs API](#)

Question: 118**Deep Dumps**

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

Cmd 1

```
rawDF = spark.table("raw_data")
```

Cmd 2

```
rawDF.printSchema()
```

Cmd 3

```
flattenedDF = rawDF.select("*", "values.*")
```

Cmd 4

```
finalDF = flattenedDF.drop("values")
```

Cmd 5

```
display(finalDF)
```

Cmd 6

```
finalDF.write.mode("append").saveAsTable("flat_data")
```

Which command should be removed from the notebook before scheduling it as a job?

- A.Cmd 2
- B.Cmd 3
- C.Cmd 4
- D.Cmd 5

Answer: D

Explanation:

Correct answer is D:Cmd 5.

Question: 119

Deep Dumps

Which statement regarding Spark configuration on the Databricks platform is true?

- A.The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
- B.Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive

cluster.

C. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.

D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

Option D accurately describes how Spark configurations work in Databricks interactive clusters. When you configure Spark properties using the Clusters UI (or the Databricks CLI/API used to manage clusters), these settings are applied at the cluster level. Since notebooks are attached to a cluster, they inherit the cluster's Spark configuration. This means every SparkSession created within any notebook running on that cluster will be affected by the properties configured at the cluster level. This provides a centralized and consistent way to manage Spark behavior across all workloads running on the same cluster.

Option A is incorrect because while the Databricks REST API can modify Spark configuration, changes to an interactive cluster will often interrupt running jobs. Modifying Spark configurations usually requires restarting the Spark context, which disrupts any ongoing Spark operations.

Option B is incorrect because Spark configurations set within a notebook only affect the SparkSession that is created or modified in that specific notebook. Configuration is not implicitly propagated to other notebooks attached to the same cluster. Each notebook essentially has its own isolated SparkSession unless specifically configured to share one.

Option C is also incorrect. When a configuration property is set both at the cluster level and within a notebook, the notebook setting generally takes precedence over the cluster setting for the SparkSession created in that notebook. This allows for specific notebooks to override the cluster-wide configuration when necessary.

In summary, Databricks uses a hierarchical configuration model. Cluster-level settings provide a baseline configuration, and notebook-level settings can override these defaults for a more tailored approach. The Clusters UI serves as a centralized control panel for cluster-wide settings. Therefore, D is the most correct answer.

Further Research:

Databricks Documentation on Spark Configuration: <https://docs.databricks.com/clusters/spark-config.html>

Databricks REST API: <https://docs.databricks.com/dev-tools/api/latest/index.html>

Question: 120

Deep Dumps

The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts, transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

A. Configure a job that executes every time new data lands in a given directory

B. Schedule a job to execute the pipeline once an hour on a new job cluster

C. Schedule a Structured Streaming job with a trigger interval of 60 minutes

D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective solution for the Databricks scenario:

Option B: Schedule a job to execute the pipeline once an hour on a new job cluster

This approach offers the best balance between meeting the hourly SLA and minimizing costs. Let's break down why:

1. **SLA Compliance:** By scheduling the job to run every hour, the business reporting team's requirement for hourly data updates is directly satisfied. The pipeline completes within 10 minutes, well within the hourly window.
2. **Cost Optimization:** A new job cluster spins up only when the pipeline needs to run. Once the ETL process completes (in 10 minutes), the cluster shuts down. This "pay-as-you-go" model is significantly cheaper than maintaining a constantly running cluster.
3. **Resource Efficiency:** Using a job cluster ensures that resources are allocated only when needed, minimizing wasted computing power and associated costs.
4. **Isolation:** Each pipeline run has its own isolated environment, reducing the risk of interference or conflicts between different ETL runs. This is especially important for stability.
5. **Clean State:** Starting with a fresh cluster each time guarantees a clean state, preventing issues related to accumulated temporary files or stale configurations impacting subsequent runs.

Let's examine why other options are less suitable:

Option A: Configure a job that executes every time new data lands in a given directory: This trigger-based approach can lead to excessive pipeline runs if data lands too frequently, even within the same hour, increasing costs unnecessarily. It might also not be ideal if the pipeline requires aggregating data from multiple sources that don't all land simultaneously.

Option C: Schedule a Structured Streaming job with a trigger interval of 60 minutes: Structured Streaming is designed for continuous data processing, not batch jobs like this one. While possible, it's an overkill. A streaming job would continuously consume resources, even when idle, leading to higher costs. Plus, if the pipeline isn't actually "streaming," this solution doesn't align well with the use case and is inefficient.

Option D: Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster: An interactive cluster runs continuously, whether or not the pipeline is active. This is the most expensive option as you're paying for the cluster 24/7, even though it's only processing data for 10 minutes per hour.

Authoritative Links:

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html>

Databricks Clusters: <https://docs.databricks.com/clusters/index.html>

Databricks Cost Management: <https://www.databricks.com/product/cost-management>

Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

In summary, option B provides the most cost-effective and efficient solution by leveraging Databricks job clusters for hourly ETL execution, ensuring SLA compliance while minimizing resource consumption and associated expenses.

Question: 121

Deep Dumps

A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

SELECT COUNT (*) FROM table -

Which of the following describes how results are generated each time the dashboard is updated?

- A.The total count of rows is calculated by scanning all data files
- B.The total count of rows will be returned from cached results unless REFRESH is run
- C.The total count of records is calculated from the Delta transaction logs
- D.The total count of records is calculated from the parquet file metadata

Answer: C

Explanation:

The correct answer is C: The total count of records is calculated from the Delta transaction logs. Here's why:

Delta Lake enhances data lakes with ACID transactions, schema enforcement, and time travel capabilities. A key component is the transaction log (Delta Log), which meticulously records every change made to the Delta table. This log maintains a sequential record of all actions, including additions, deletions, and updates to the data.

When a query like `SELECT COUNT(*) FROM table` is executed against a Delta table, Databricks SQL leverages the Delta Log to efficiently determine the total number of records. Instead of performing a full scan of all data files, which would be resource-intensive and slow (option A), the query engine can quickly retrieve the current table state from the Delta Log. The Delta Log effectively acts as a highly optimized index for metadata, including the number of records.

Option B is partially correct, as Databricks SQL does employ caching mechanisms to improve query performance. However, the source of truth for the `COUNT(*)` is always derived from the Delta Log. The cache might hold a previous result, but the Delta Log is consulted to ensure the data is up-to-date. `REFRESH` statement invalidates any cached copies and forces an update from the Delta Log. Thus, while caching plays a role in performance, the foundational data source for the count is the Delta Log, which is always checked for data consistency.

Option D, calculating the count solely from Parquet file metadata, is insufficient. While Parquet files store metadata about the data they contain, this metadata doesn't reflect the current state of the Delta table after various transactions (inserts, deletes, updates). The Delta Log is crucial for capturing these changes and providing an accurate record count. The Parquet metadata would only provide a file-level record count, which isn't the table level count that accounts for deletions or updates handled through the Delta Log.

Therefore, Databricks SQL leverages the Delta Log's transactional history to efficiently and accurately determine the total number of records without the need for a full data scan, making option C the most accurate description of the process. This optimization is a significant benefit of using Delta Lake.

Reference links for further reading:

Delta Lake Overview: <https://delta.io/>

Databricks Delta Lake Documentation: <https://docs.databricks.com/delta/index.html>

Delta Lake Transaction Log: <https://docs.databricks.com/delta/delta-intro.html#what-is-delta-lake>
(Specifically look for the benefits of Transaction Logs for improved performance)

Question: 122

Deep Dumps

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

DROP TABLE prod.sales_by_store -

If this statement is executed by a workspace admin, which result will occur?

- A.Data will be marked as deleted but still recoverable with Time Travel.
- B.The table will be removed from the catalog but the data will remain in storage.
- C.The table will be removed from the catalog and the data will be deleted.
- D.An error will occur because Delta Lake prevents the deletion of production data.

Answer: C

Explanation:

The table will be removed from the catalog and the data will be deleted.

Question: 123

Deep Dumps

A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the main branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

- A.Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.
- B.Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.
- C.Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.
- D.Use Repos to merge all differences and make a pull request back to the remote repository.

Answer: A

Explanation:

The correct answer is A because it directly addresses the developer's constraints and provides a safe, collaborative workflow. The developer lacks write access to the main branch, so directly pushing changes there isn't possible. Option A, creating a new branch, addresses this limitation by providing a personal workspace for changes.

Branches in Git (and therefore Repos) are designed for isolated development. This allows developers to work on features or bug fixes without impacting the main codebase or other team members. By creating a new

branch, the developer isolates their changes.

Committing changes to the new branch saves the developer's work and prepares it for sharing. Pushing the branch to the remote repository makes the changes visible to the rest of the team. From there, a pull request can be created, initiating a code review process where teammates can inspect, comment on, and eventually merge the code into the main branch.

Option B, forking the repository, is generally used when contributing to a project where you don't have direct write access and are likely to be making substantial contributions. For smaller contributions or contributions to a project where you are part of the team, branching is more suitable. Forking creates a completely independent copy of the repository under the developer's own account, adding unnecessary complexity for a team member making standard contributions.

Option C incorrectly describes how Git pull works. A pull operation doesn't magically create new branches reflecting the incoming changes. It integrates changes from a remote branch into the current branch.

Option D suggests merging all differences before a pull request. The developer hasn't identified specific remote changes they want to integrate into their local branch. This creates unnecessary risks. It should be the reviewer (or the developer if instructed during the review) of the pull request who decides when and how to merge code.

Therefore, creating a new branch, committing changes, and pushing it (option A) is the standard, most effective, and safest way to collaborate in this scenario, respecting the developer's limited permissions and enabling team review before merging. It aligns with common Git workflows for collaborative development and code review practices.

Further reading:

Git Branching: <https://www.atlassian.com/git/tutorials/using-branches>

Git Pull Requests: <https://www.atlassian.com/git/tutorials/making-a-pull-request>

Databricks Repos: <https://docs.databricks.com/repos/index.html>

Question: 124

Deep Dumps

The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.

After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
      .read
      .format("jdbc")
      .option("url", connection)
      .option("dbtable", tablename)
      .option("user", username)
      .option("password", password)
      )
```

Which statement describes what will happen when the above code is executed?

- A.The connection to the external table will succeed; the string "REDACTED" will be printed.
- B.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
- C.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
- D.The connection to the external table will succeed; the string value of password will be printed in plain text.

Answer: A

Explanation:

The connection to the external table will succeed; the string "REDACTED" will be printed.

Question: 125

Deep Dumps

The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type DOUBLE.

The following code correctly imports the production model, loads the customers table containing the customer_id key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf (spark,
model_uri="models:/churn/prod")

df = spark.table("customers")

columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "customer_id LONG, predictions DOUBLE"?

- A.df.map(lambda x:model(x[columns])).select("customer_id, predictions")
- B.df.select("customer_id", model(*columns).alias("predictions"))
- C.model.predict(df, columns)

```
D.df.apply(model, columns).select("customer_id, predictions")
```

Answer: B

Explanation:

Correct Answer: B This option uses select to specify columns from the DataFrame and applies the model to the specified columns (columns). The output of the model is aliased as "predictions", which ensures the output DataFrame will have the column names "customer_id" and "predictions" with appropriate data types assuming the model returns a double type. This syntax aligns with PySpark's DataFrame transformations and is a typical way to apply a machine learning model to specific columns in Databricks.

Question: 126

Deep Dumps

A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the geo_lookup table.

Before executing the code, running SHOW TABLES on the current database indicates the database contains only two tables: geo_lookup and sales.

Cmd 1

```
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]
```

Cmd 2

```
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

What will be the outcome of executing these command cells in order in an interactive notebook?

- A. Both commands will succeed. Executing SHOW TABLES will show that countries_af and sales_af have been registered as views.
- B. Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named countries_af: if this entity exists, Cmd 2 will succeed.
- C. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable representing a PySpark DataFrame.
- D. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Answer: D

Explanation:

Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

The data science team has requested assistance in accelerating queries on free-form text from user reviews. The data is currently stored in Parquet with the below schema:

item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING

The review column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

- A. Delta Lake statistics are not optimized for free text fields with high cardinality.
- B. Delta Lake statistics are only collected on the first 4 columns in a table.
- C. ZORDER ON review will need to be run to see performance gains.
- D. The Delta log creates a term matrix for free text fields to support selective filtering.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct response to the junior data engineer's suggestion, and why the other options are incorrect:

Justification for Option A (Correct):

Option A states: "Delta Lake statistics are not optimized for free text fields with high cardinality." This is the most accurate and relevant reason why simply converting to Delta Lake might not automatically accelerate the queries significantly for keyword searches within the review column.

Delta Lake leverages data skipping using statistics (min/max values, null counts) to avoid reading irrelevant files during queries. However, these statistics are most effective when applied to numerical or low-cardinality categorical columns. Free text fields, like the review column, typically have very high cardinality (many unique values). Because of this high cardinality, the min/max statistics don't effectively narrow down the data scanned during queries for specific keywords. For instance, knowing the lexicographical minimum and maximum review string won't help determine which reviews contain specific keywords. Consequently, data skipping's benefits are significantly diminished in this scenario. Other techniques such as vector embeddings and specialized indexes would be far more applicable, which are outside the scope of what a naive Delta Lake conversion provides.

Why Other Options are Incorrect:

Option B: "Delta Lake statistics are only collected on the first 4 columns in a table." This is incorrect. Delta Lake collects statistics on all columns by default, unless sampling is configured to limit statistic gathering. The number of columns is not a constraint.

Option C: "ZORDER ON review will need to be run to see performance gains." While Z-Ordering is a data optimization technique in Delta Lake for co-locating related data for better data skipping, Z-Ordering on a free-text field (review) would not provide meaningful performance improvements for keyword searches. Z-Ordering is effective when there is spatial locality or correlation between the ordering column and query patterns. Free text lacks such correlation.

Option D: "The Delta log creates a term matrix for free text fields to support selective filtering." This is factually incorrect. The Delta log primarily tracks changes (additions, deletions, updates) to the Delta table; it does not generate a term matrix or specialized indexes for full-text search capabilities. Such functionality would typically require separate indexing solutions designed for text (e.g., Elasticsearch, Solr, or even feature engineering like TF-IDF followed by ML models for keyword classification).

Supporting Links:

Databricks Delta Lake Documentation: <https://docs.databricks.com/delta/index.html> - Provides comprehensive information on Delta Lake features, including data skipping and Z-Ordering. Specifically, look for sections describing data skipping and how statistics are used to optimize query performance.

Data Skipping in Delta Lake: Search the Databricks documentation for "Delta Lake data skipping" to find details on how statistics are leveraged.

Z-Ordering in Delta Lake: Search the Databricks documentation for "Delta Lake Z-Ordering" for a detailed explanation of this data layout optimization technique.

Question: 128

Deep Dumps

The data engineering team has configured a job to process customer requests to be forgotten (have their data deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.

The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of VACUUM commands on all Delta Lake tables throughout the organization.

The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.

Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

- A. Because the VACUUM command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
- B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the VACUUM job is run the following day.
- C. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the VACUUM job is run 8 days later.
- D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.

Answer: C

Explanation:

The correct answer is C because it accurately describes Delta Lake's default data retention behavior and its impact on data availability after deletions. Let's break down why the other options are incorrect and why option C is the right choice.

Option A is incorrect because the retention period isn't simply "around 24 hours." Delta Lake defaults to a 7-day retention period for time travel. The VACUUM job on Monday will indeed remove files, but only those older than the retention interval set for the table (defaulting to 7 days). The delete job runs on Sunday, so deleted data will be available to time travel for nearly a full week, assuming no interim VACUUM operations.

Option B is incorrect because the data retention threshold isn't simply "24 hours". Delta Lake maintains a default history for 7 days (168 hours), making this statement inaccurate. Files will be retained based on the retention interval, not just until the next day.

Option D is incorrect because while Delta Lake's DELETE operations have ACID guarantees, these guarantees pertain to the consistency and reliability of the deletion operation itself. It means that the delete operation is atomic, consistent, isolated, and durable. However, ACID guarantees do not equate to immediate and permanent physical purging from storage. The data files themselves are not immediately purged; they remain available for time travel until the retention period expires and VACUUM is executed. Delta Lake manages data deletion by marking records as deleted and rewriting Parquet files as needed. Old versions of files are kept to

enable time travel.

Option C is correct because it highlights the crucial relationship between Delta Lake's default retention period and the timing of the VACUUM operation. The compliance officer's concern is valid because deleted records will be accessible via time travel until the VACUUM command removes the older file versions. The default retention period in Delta Lake is 7 days. The customer data is deleted on Sunday, and the VACUUM job is run the following Monday, which is 8 days later (7 full days + Sunday). Therefore, the data files containing the deleted records are retained for approximately 8 days, making the data accessible via time travel during that period. Only after VACUUM is executed, can time travel access to the deleted data be prevented.

Here are some authoritative links for further research:

Delta Lake Documentation on Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Documentation on VACUUM: <https://docs.databricks.com/delta/optimizations/vacuum.html>

Delta Lake ACID Guarantees: <https://delta.io/> (refer to the key features section highlighting ACID transactions)

Question: 129

Deep Dumps

Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

- A.configure
- B.fs
- C.workspace
- D.libraries

Answer: B

Explanation:

The correct Databricks CLI command for uploading a custom Python Wheel to object storage mounted with DBFS is fs. Here's why:

The Databricks File System (DBFS) acts as a distributed file system abstraction layer over cloud object storage (like AWS S3, Azure Blob Storage, or Google Cloud Storage). To interact with the DBFS, including uploading and downloading files, the dbfs or fs command is used in the Databricks CLI.

Option A, configure, is used to configure the Databricks CLI with authentication information, such as your Databricks host and access token or user credentials. It's not for uploading files.

Option C, workspace, is used for managing Databricks workspaces, including importing and exporting notebooks, managing repos, and creating and deleting jobs. It doesn't directly handle file uploads to DBFS.

Option D, libraries, is primarily used for managing libraries (including uploading them) specifically for clusters or jobs, not for directly uploading files to DBFS storage locations. While you could upload a wheel using the libraries API eventually, the task specifically requests uploading to DBFS.

The fs command is designed for file system operations within DBFS. Specifically, the dbfs cp or databricks fs cp sub-command copies files to and from DBFS. This directly addresses the requirement of uploading a Python Wheel to a DBFS mount point. For instance, `databricks fs cp my_wheel.whl dbfs:/path/to/wheel/` will upload `my_wheel.whl` to the specified DBFS path.

Here are some authoritative links for further research:

Databricks CLI documentation: <https://docs.databricks.com/en/dev-tools/cli/index.html>

Databricks File System (DBFS): <https://docs.databricks.com/en/dbfs/>

Databricks CLI fs command: <https://docs.databricks.com/en/dev-tools/cli/dbfs-cli.html>

Question: 130

Deep Dumps

The following table consists of items found in user carts within an e-commerce website.

```
Carts (id LONG, items ARRAY<STRUCT<id: LONG, count: INT>>)
id  items                                                    email
1001[{id: "DESK65", count: 1}]                               "u1@domain.com"
1002[{id: "KYBD45", count: 1}, {id: "M27", count: 2}]        "u2@domain.com"
1003[{id: "M27", count: 1}]                                   "u3@domain.com"
```

The following MERGE statement is used to update this table using an updates view, with schema evolution enabled on this table.

```
MERGE INTO carts c
USING updates u
ON c.id = u.id
WHEN MATCHED
  THEN UPDATE SET *
```

How would the following update be handled?

```
(new nested field, missing existing column)
id  items
1001[{id: "DESK65", count: 2, coupon: "BOGO50"}]
```

- A.The update throws an error because changes to existing columns in the target schema are not supported.
- B.The new nested Field is added to the target schema, and dynamically read as NULL for existing unmatched records.
- C.The update is moved to a separate "rescued" column because it is missing a column expected in the target schema.
- D.The new nested field is added to the target schema, and files underlying existing records are updated to include NULL values for the new field.

Answer: B

Explanation:

The new nested Field is added to the target schema, and dynamically read as NULL for existing unmatched records.

Question: 131

Deep Dumps

An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field pk_id.

For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system. For analytical purposes, only the most recent value for each record needs to be recorded. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

- A. Iterate through an ordered set of changes to the table, applying each in turn to create the current state of the table, (insert, update, delete), timestamp of change, and the values.
- B. Use merge into to insert, update, or delete the most recent entry for each pk_id into a table, then propagate all changes throughout the system.
- C. Deduplicate records in each batch by pk_id and overwrite the target table.
- D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.

Answer: D

Explanation:

The correct answer is D because it leverages Delta Lake's Change Data Feed (CDF) to efficiently handle the CDC requirements. Here's a detailed breakdown:

Delta Lake CDF directly addresses the scenario of capturing and processing changes from an upstream system. It allows you to track row-level changes (inserts, updates, deletes) made to a Delta table. This directly satisfies the requirement of maintaining a history of all valid values for auditing purposes.

By enabling CDF on the Delta table where you ingest the CDC logs, every change is automatically recorded. This ensures that the data governance team has a full record of all changes. Furthermore, using MERGE INTO with the CDF data allows the construction of a separate table representing the current state (latest values) for analytical purposes, fulfilling the second requirement. CDF helps avoid complex and potentially inefficient iterative processing or deduplication logic. Also, change propagation can be managed via other Delta tables with CDF enabled if needed.

Option A is less efficient as it involves manually iterating through changes, which can be slow and resource-intensive, especially with large datasets. Option B doesn't fully leverage the advantages of Delta Lake and requires additional custom logic to propagate changes. Option C loses historical data by overwriting the target table.

Therefore, leveraging Delta Lake's CDF provides the most efficient and comprehensive solution for capturing, storing, and processing CDC data while meeting both auditing and analytical requirements.

Relevant Links:

[Delta Lake Change Data Feed](#)
[MERGE INTO](#)

Question: 132

Deep Dumps

An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The user_id field represents a unique key for the data, which has the following schema:

user_id BIGINT, username STRING, user_utc STRING, user_region STRING, last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT

New records are all ingested into a table named account_history which maintains a full record of all data in the

same schema as the source. The next table in the system is named `account_current` and is implemented as a Type 1 table representing the most recent value for each unique `user_id`.

Which implementation can be used to efficiently update the described `account_current` table as part of each hourly batch job assuming there are millions of user accounts and tens of thousands of records processed hourly?

- A. Filter records in `account_history` using the `last_updated` field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each username.
- B. Use Auto Loader to subscribe to new files in the `account_history` directory; configure a Structured Streaming trigger available job to batch update newly detected files into the `account_current` table.
- C. Overwrite the `account_current` table with each batch using the results of a query against the `account_history` table grouping by `user_id` and filtering for the max value of `last_updated`.
- D. Filter records in `account_history` using the `last_updated` field and the most recent hour processed, as well as the max `last_login` by `user_id` write a merge statement to update or insert the most recent value for each `user_id`.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most efficient implementation for updating the `account_current` table as a Type 1 table, along with supporting concepts and links:

Justification for Option D:

Option D provides the most efficient and accurate approach for updating the `account_current` table, given the specific requirements and constraints:

1. **Incremental Processing:** By filtering `account_history` using the `last_updated` field and the most recent hour processed, we limit the amount of data we need to process in each batch. This minimizes computational overhead compared to processing the entire `account_history` table.
2. **Key-Based Deduplication and Correctness:** The key to Type 1 updates is using the correct key to identify rows to update. `user_id` is defined as the unique key for the data. Deduplication is based on the `last_login` timestamp per `user_id` to ensure that we only consider the most recent record for each user within the processed batch. This is critical for maintaining the correct state of the `account_current` table.
3. **Merge Statement Efficiency:** Using a MERGE statement in Spark SQL provides an optimized way to either update existing rows (if a `user_id` already exists in `account_current`) or insert new rows (if a `user_id` is new). The MERGE statement avoids the need for separate update and insert operations, improving performance.
4. **Type 1 Conformance:** Option D effectively implements a Type 1 Slowly Changing Dimension (SCD) approach by overwriting older data with the most recent data for each `user_id`.

Why other options are less suitable:

Option A: Deduplicating on username is incorrect because the question explicitly states `user_id` is the unique key. This will lead to incorrect updates.

Option B: While Auto Loader and Structured Streaming can be used for incremental ingestion, they are not necessarily the most efficient way to handle batch updates for a Type 1 SCD. Using a MERGE statement with a targeted filter is often more efficient than a continuous streaming job, especially when you have a natural batch boundary (hourly in this case). Structured Streaming might add more complexity than needed for a simple hourly batch process.

Option C: Overwriting the entire account_current table with each batch is highly inefficient. It requires processing the entire account_history table, even if only a small percentage of records have changed. This is unnecessary and will significantly increase processing time and resource consumption.

Conceptual Support and Relevant Links:

Type 1 SCDs: Understand the concept of Type 1 Slowly Changing Dimensions, where existing records are simply overwritten with new data. <https://www.databricks.com/glossary/slowly-changing-dimension-scd>

Delta Lake MERGE Statement: The MERGE statement provides efficient upsert functionality.

<https://docs.databricks.com/delta/delta-update.html>

Incremental Data Processing: Learn about efficient data processing by only considering the changes in data (the hourly batch). <https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/batch-processing>

Structured Streaming: Examine Structured Streaming for real-time data pipelines.

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

In conclusion, option D provides the most efficient and accurate solution for updating the account_current table as a Type 1 SCD within an hourly batch job by leveraging incremental processing, key-based deduplication, and the optimized MERGE statement.

Question: 133

Deep Dumps

The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd  
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,  
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named products_per_order, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,  
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

- A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
- B. Use Structured Streaming to configure a live dashboard against the products_per_order table within a Databricks notebook.
- C. Define a view against the products_per_order table and define the dashboard against this view.
- D. Use the Delta Cache to persist the products_per_order table in memory to quickly update the dashboard with each query.

Answer: A

Explanation:

Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten

with each update.

Question: 134

Deep Dumps

A Delta lake table with CDF enabled table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

- A. Apply the churn model to all rows in the `customer_churn_params` table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.
- B. Convert the batch job to a Structured Streaming job using the complete output mode; configure a Structured Streaming job to read from the `customer_churn_params` table and incrementally predict against the churn model.
- C. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.
- D. Modify the overwrite logic to include a field populated by calling `spark.sql.functions.current_timestamp()` as data are being written; use this field to identify records written on a particular date.

Answer: C

Explanation:

The correct answer is C because it leverages Delta Lake's Change Data Feed (CDF) functionality to efficiently identify changed records, aligning directly with the requirement of predicting churn only for records modified in the last 24 hours. Option A, while functional, is inefficient because it applies the model to all rows, including unchanged ones, which is wasteful of compute resources. Option B, using Structured Streaming in complete output mode, would reprocess the entire table for each micro-batch, similar to the initial overwrite, failing to isolate only the changed records effectively. Option D, using `current_timestamp()`, captures the time of the ETL job's execution rather than the actual data change time from upstream systems. This leads to inaccurate identification of changed records.

Option C directly addresses the need by replacing the overwrite with a MERGE statement which will insert, update, or delete records based on the changes in upstream sources. CDF then allows easy identification of these changed records as part of that merge process. This isolates records relevant for the churn prediction model, optimizing resource usage and model execution time. By focusing only on changed records identified through CDF, this approach provides the most efficient and accurate solution. The Data Engineering team can then provide the machine learning team a stream or batch view of only changed records.

Here are some authoritative links for further research:

Delta Lake Change Data Feed: <https://docs.databricks.com/delta/delta-change-data-feed.html>

Delta Lake MERGE: <https://docs.databricks.com/delta/delta-update.html#upsert-into-a-table-using-merge>

Question: 135

Deep Dumps

A view is registered with the following code:

```
CREATE VIEW recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
     FROM users) a
  INNER JOIN
    (SELECT user_id, order_id, order_date
     FROM orders
     WHERE order_date >= (current_date() - 7)) b
  ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables.

Which statement describes the results of querying recent_orders?

- A.All logic will execute when the view is defined and store the result of joining tables to the DBFS; this stored data will be returned when the view is queried.
- B.Results will be computed and cached when the view is defined; these cached results will incrementally update as new records are inserted into source tables.
- C.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
- D.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Answer: D

Explanation:

All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Question: 136

Deep Dumps

A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

- A.Set spark.sql.files.maxPartitionBytes to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
- B.Set spark.sql.shuffle.partitions to 2,048 partitions (1TB*1024*1024/512), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
- C.Set spark.sql.adaptive.advisoryPartitionSizeInBytes to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions (1TB*1024*1024/512), and then write to parquet.
- D.Ingest the data, execute the narrow transformations, repartition to 2,048 partitions (1TB* 1024*1024/512), and then write to parquet.

Answer: A

Explanation:

The best strategy for writing a 1TB JSON dataset to Parquet with 512MB part-file sizes without shuffling during the write operation is **A. Set `spark.sql.files.maxPartitionBytes` to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.**

Here's why:

spark.sql.files.maxPartitionBytes: This configuration directly controls the maximum size of a single partition when reading data from files. By setting it to 512MB before ingesting the data, you are effectively guiding Spark to create partitions that are roughly the desired output size. This happens during the initial data read from the JSON files, avoiding the need for a separate, explicit repartition later. It works by Spark splitting the input files into partitions respecting the `maxPartitionBytes` setting.

Narrow Transformations: The prompt specifies executing "narrow transformations." These are operations like `map`, `filter`, and `withColumn` that process each row independently within a partition without needing to shuffle data across partitions. Executing these before writing to Parquet is key.

Avoiding Shuffles: The critical requirement is to avoid shuffling. Shuffling is an expensive operation where data needs to be redistributed across the cluster based on some key. Options B, C, and D all involve explicit repartitioning using `repartition` or `coalesce` after the initial ingest. This forces a full shuffle of the data, which is precisely what the prompt seeks to avoid. Even B's sorting optimization repartitions, negating the 'no shuffle' requirement.

Why other options are suboptimal:

B: Using `spark.sql.shuffle.partitions` and sorting is useful for overall parallelism, but not the best way to control the number of output files directly. More importantly, sorting requires a shuffle, which is prohibited.

C: `spark.sql.adaptive.advisoryPartitionSizeInBytes` only suggests a partition size during adaptive query execution (AQE). `coalesce`, while reducing the number of partitions, still involves shuffling, even though it attempts to minimize it. It's also typically used to reduce partitions, not create them from a larger single partition. It's better suited after a shuffle operation, if necessary.

D: Explicitly repartitioning with `repartition` forces a full shuffle.

How option A achieves the goal: By controlling the initial partition size during ingestion, you shape the data early on into the desired partition sizes. The subsequent narrow transformations preserve this partitioning. Therefore, when the data is written to Parquet, each partition becomes a separate Parquet file (or multiple files if a single partition is larger than the `parquet.block.size` setting within the partition).

In summary, option A provides the most efficient solution by directly controlling the input partition size, minimizing the need for expensive data shuffling operations, and enabling the creation of Parquet files with the target size.

Authoritative Links:

Spark Configuration: <https://spark.apache.org/docs/latest/configuration.html> (Search for `spark.sql.files.maxPartitionBytes` and `spark.sql.shuffle.partitions`)

Adaptive Query Execution: <https://spark.apache.org/docs/latest/sql-performance-tuning.html#adaptive-query-execution>

Repartitioning in Spark: <https://sparkbyexamples.com/spark/spark-repartition-vs-coalesce/>

Question: 137

Deep Dumps

Which statement regarding stream-static joins and static Delta tables is correct?

- A.The checkpoint directory will be used to track updates to the static Delta table.
- B.Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
- C.The checkpoint directory will be used to track state information for the unique keys present in the join.
- D.Stream-static joins cannot use static Delta tables because of consistency issues.

Answer: B

Explanation:

The correct statement regarding stream-static joins and static Delta tables is **B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.**

Here's a detailed justification:

Stream-static joins, a fundamental operation in structured streaming, combine streaming data (a constantly updating stream) with a static dataset (data that doesn't change during the stream's execution). Delta Lake, being a reliable and performant storage layer built on top of data lakes, is often used for these static datasets. The key consideration here is data consistency during the streaming process.

Statement B accurately describes how Spark Structured Streaming handles stream-static joins with Delta tables. When the streaming job initializes, it reads the current version of the static Delta table. For each subsequent microbatch processed by the streaming query, it uses that same initial snapshot of the static Delta table. It does not pick up any changes made to the Delta table after the stream started. This ensures consistency: each microbatch is joined against the same static data, avoiding data inconsistencies from changes to the Delta table mid-stream. This is a crucial aspect of ensuring deterministic results in streaming applications.

Option A is incorrect because the checkpoint directory is primarily used for fault tolerance in the streaming job itself, tracking the progress of the stream and allowing it to recover from failures. It does not track changes to the static Delta table.

Option C is misleading. While checkpoints are used to track state information, this applies to stateful operations within the stream, like aggregations or windowing. It doesn't directly relate to updates within the static Delta table.

Option D is false. Stream-static joins can absolutely use static Delta tables, and they are a very common use case. Delta Lake's ACID properties and versioning make it a good choice for a static dataset in these scenarios.

In summary, the static Delta table's snapshot used for the stream-static join is determined when the streaming job starts, providing consistent results for each microbatch. Changes made to the Delta table after the stream starts will not be reflected in the stream's processing until the stream is restarted.

Further Research:

Structured Streaming Programming Guide: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (Look for sections on joins and state management)

Delta Lake Documentation: <https://docs.delta.io/latest/index.html> (Focus on features like ACID properties and versioning)

Databricks documentation also has dedicated articles detailing streaming and delta lake, useful for this topic.

Question: 138

Deep Dumps

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using

DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df.withWatermark("event_time", "10 minutes")
  .groupBy(
    _____,
    "device_id"
  )
  .agg(
    avg("temp").alias("avg_temp"),
    avg("humidity").alias("avg_humidity")
  )
  .writeStream
  .format("delta")
  .saveAsTable("sensor_avg")
```

Which line of code correctly fills in the blank within the code block to complete this task?

- A.to_interval("event_time", "5 minutes").alias("time")
- B.window("event_time", "5 minutes").alias("time")
- C."event_time"
- D.lag("event_time", "10 minutes").alias("time")

Answer: B

Explanation:

This is the standard syntax to do non-overlapping time interval Window-ed grouping by the time field in a dataset in Structured Streaming. .withWatermark() function defines the staging buffers after which delayed records will be dropped/ignored.

Question: 139

Deep Dumps

A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

- A.Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
- B.Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this

approach minimizes costs for both compute and storage.

C.Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.

D.Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.

Answer: C

Explanation:

The correct answer is C: "Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost."

Here's a detailed justification:

The problem states that the streaming job processes empty microbatches at least 12 times per minute, meaning the job is frequently checking the source for updates even when there's no new data. This frequent checking triggers API calls to the source storage, which contributes significantly to cloud storage costs.

Option A is incorrect because reducing the trigger interval would increase the frequency of checks and API calls, exacerbating the problem, not solving it. It also incorrectly claims the default trigger consumes too many records, leading to spills. The problem explicitly states batches are processed in 3 seconds, indicating no resource contention.

Option B suggests using trigger once with a job scheduled every 10 minutes. While this could reduce compute costs, it fundamentally changes the nature of the job from near real-time to a batch process running every 10 minutes. The requirements specify records need to be processed in less than 10 minutes. It also doesn't directly address the API call issue, as the trigger once still has to query the data source. In addition, trigger once is designed for a single execution of the query. Re-running it as a scheduled job with the trigger once option will not maintain the state of the Structured Streaming query, therefore will not correctly process the data.

Option D proposes a smaller trigger interval of 500ms. While seemingly reducing latency, this will dramatically increase the number of API calls to the source storage and thus significantly increase storage costs. This option is the opposite of the desired effect.

Option C directly addresses the root cause: the frequent API calls. By setting the trigger interval to 10 minutes, the maximum allowed while still meeting the latency requirement of processing records in under 10 minutes, we reduce the number of checks to the source storage to the minimum necessary. This directly translates to fewer API calls and lower storage costs. Structured Streaming's `Trigger.ProcessingTime(interval)` allows you to specify the interval for micro-batch execution.

Therefore, reducing the trigger frequency (but still meeting latency requirements) is the most direct and effective way to minimize cloud storage costs associated with API calls in this scenario.

Relevant documentation:

Databricks Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (Specifically, the section on Triggers).

Structured Streaming Trigger Options in Databricks: <https://docs.databricks.com/structured-streaming/triggers.html>

Question: 140

Deep Dumps

Which statement describes Delta Lake optimized writes?

A.Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.

- B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- C. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.
- D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.

Answer: C

Explanation:

The correct answer is C because Delta Lake optimized writes aim to reduce the number of small files written to storage. This is achieved by performing a shuffle operation before writing data. During the shuffle, data with similar characteristics, often based on partitioning columns, is grouped together. This grouping allows each writer task to write larger, more consolidated files, rather than many small files scattered across the storage system. The result is a significant reduction in the number of files and improved query performance, especially when reading data based on the partitioning scheme used in the optimized write process.

Option A is incorrect. While OPTIMIZE does compact files, optimized writes are a pre-write process, not a post-termination one. Relying on OPTIMIZE alone, especially after every job termination, would be inefficient.

Option B is also incorrect. While an asynchronous process like this could be implemented, it isn't how Delta Lake optimized writes work by default. Optimized writes are about influencing how data is initially written, not relying on a subsequent compaction process.

Option D is incorrect as well. While Delta Lake's metadata management contributes to performance, optimized writes are primarily focused on how data is physically laid out, and don't solely rely on logical partitions to reduce small file sizes. They work hand-in-hand with directory partitioning by attempting to write fewer, larger files within those directory partitions. The core functionality is reducing the total number of files written during the write process, not changing the partitioning mechanism itself.

Here are a couple of resources for further reading on Delta Lake and optimized writes:

Databricks Documentation on Delta Lake Optimized Writes: (Search for "Optimize writes using optimized autoscaling" on the Databricks website) - Provides a comprehensive overview of how Delta Lake optimized writes function.

Delta Lake Official Website: (<https://delta.io/>) - Offers general information and architecture details about Delta Lake.

Question: 141

Deep Dumps

Which statement characterizes the general programming model used by Spark Structured Streaming?

- A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
- B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
- C. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.
- D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.

Answer: D

Explanation:

The correct answer is D because Spark Structured Streaming treats a data stream as an unbounded,

continuously growing table. This fundamentally changes how streaming data is processed. Instead of dealing with individual events or micro-batches in isolation, Structured Streaming applies relational queries on this conceptually infinite table. New data arriving in the stream is effectively treated as new rows appended to the table. This allows users to define streaming computations using familiar SQL-like syntax or DataFrame APIs.

Options A, B, and C are incorrect. A is incorrect because while Spark can leverage GPUs in some contexts (like deep learning), Structured Streaming itself doesn't inherently rely on or optimize for GPU processing. Its parallelism comes from distributing the workload across Spark's cluster nodes. B is incorrect because Structured Streaming is not derived from Apache Kafka or implemented as a messaging bus; it's a separate component built on top of the Spark engine. While it can consume data from Kafka, it's not dependent on Kafka's architecture. C is also incorrect. While Spark does use caching for performance optimization, Structured Streaming doesn't primarily rely on a distributed network of nodes holding incremental state values for cached stages as its core programming model. Its state management is more nuanced and supports features like stateful aggregations. Spark Structured Streaming does use state management, but it is related to operations like groupByKey and updateStateByKey, not cached stages.

The key principle of Structured Streaming is treating the stream as a table. This enables batch and stream processing code reuse and simplifies stream processing logic via higher-level declarative operations on the unbounded table. This approach provides fault tolerance and exactly-once processing guarantees. It relies on Spark's fault tolerance mechanisms to handle failures and checkpointing to maintain state across restarts. Structured Streaming divides the stream into micro-batches and uses the Spark SQL engine to process each micro-batch as a query against the unbounded table. The result of each query updates the unbounded table.

Further information can be found in the official Apache Spark documentation:

[Structured Streaming Programming Guide](#)
[Spark SQL, DataFrames and Datasets Guide](#)

Question: 142

Deep Dumps

Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

- A.spark.sql.files.maxPartitionBytes
- B.spark.sql.autoBroadcastJoinThreshold
- C.spark.sql.adaptive.advisoryPartitionSizeInBytes
- D.spark.sql.adaptive.coalescePartitions.minPartitionNum

Answer: A

Explanation:

The correct answer is **A. spark.sql.files.maxPartitionBytes**.

Here's why:

spark.sql.files.maxPartitionBytes directly controls the maximum number of bytes Spark will pack into a single partition when reading data from file-based data sources (like Parquet, CSV, etc.). When reading data, Spark attempts to split the input files into partitions. This parameter dictates the maximum size of each of these partitions before any further processing takes place. This influences the initial data distribution across the Spark cluster. A larger value generally results in fewer, larger partitions; a smaller value results in more, smaller partitions. If not set, the default value is usually equal to **spark.sql.adaptive.advisoryPartitionSizeInBytes**, which is also 64MB.

spark.sql.autoBroadcastJoinThreshold affects query optimization. It determines the maximum size, in bytes,

of a table that will be broadcast to all worker nodes when performing a join operation. It does not affect the size of partitions when initially ingesting data.

spark.sql.adaptive.advisoryPartitionSizeInBytes influences the target size of shuffle partitions when adaptive query execution (AQE) is enabled. AQE dynamically optimizes query plans during runtime, and this setting advises Spark on a suitable partition size. It has some affect on the partitioning, but is not the initial source of truth.

spark.sql.adaptive.coalescePartitions.minPartitionNum specifies the minimum number of partitions after coalescing partitions during adaptive query execution. While relevant to partitioning, it's also part of the AQE framework and comes into play after the initial data ingestion.

In summary, **spark.sql.files.maxPartitionBytes** is the primary configuration parameter that directly impacts the initial size of Spark partitions created during data ingestion from files. The other options are more related to query optimization, adaptive execution, or shuffle operations that occur after data is initially read into Spark.

Authoritative Links:

Apache Spark Configuration Documentation: <https://spark.apache.org/docs/latest/configuration.html>
(Search for "spark.sql.files.maxPartitionBytes", "spark.sql.autoBroadcastJoinThreshold",
"spark.sql.adaptive.advisoryPartitionSizeInBytes", and
"spark.sql.adaptive.coalescePartitions.minPartitionNum")

Databricks Documentation on AQE: <https://docs.databricks.com/spark/latest/spark-sql/aqe/index.html> (For more information on Adaptive Query Execution)

Question: 143

Deep Dumps

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.

Which situation is causing increased duration of the overall job?

- A.Task queueing resulting from improper thread pool assignment.
- B.Spill resulting from attached volume storage being too small.
- C.Network latency due to some cluster nodes being in different regions from the source data
- D.Skew caused by more data being assigned to a subset of spark-partitions.

Answer: D

Explanation:

Here's a detailed justification for why option D (Skew caused by more data being assigned to a subset of spark-partitions) is the most likely cause of the described performance issue, along with supporting details:

The key observation is the extreme difference between the minimum/median task duration and the maximum task duration (100x longer). This pattern strongly suggests data skew. Data skew occurs when data is unevenly distributed across partitions. Some partitions contain significantly more data than others. Spark processes each partition in parallel, so tasks processing the smaller partitions complete quickly (hence the low min/median). However, tasks processing the larger, skewed partitions take much longer (hence the high max).

Let's analyze why the other options are less likely:

A. Task queueing resulting from improper thread pool assignment: While improper thread pool assignment can cause delays, it typically affects all tasks to some degree, leading to a more uniform increase in duration

rather than extreme variations. The UI would show a more general slowdown rather than a few tasks taking extremely long.

B. Spill resulting from attached volume storage being too small: Spilling to disk due to insufficient memory certainly degrades performance. However, it tends to affect tasks more consistently because all tasks are impacted as memory gets tight, though some might be more affected. The extreme variance in task duration is a less common consequence of spilling. The slow tasks in the skewed data situation are slow because they are processing a huge volume of data before spilling occurs, so while the two could be related (a large partition will spill), skew is the root cause in the question scenario.

C. Network latency due to some cluster nodes being in different regions from the source data: Network latency can be a problem, but it typically impacts all tasks similarly. The observed pattern of some tasks completing very quickly while others take extremely long is not the signature of uniform network latency across the cluster. Also Spark's scheduler tends to try to schedule tasks to nodes local to the data, so the effect is mitigated.

In summary: Data skew perfectly explains the observed behavior. A few partitions contain significantly more data than others, leading to long task durations for those partitions while the rest complete quickly.

Supporting Concepts:

Spark Partitions: Data in Spark is divided into partitions, which are the fundamental units of parallelism.

Data Skew: Uneven distribution of data across partitions, leading to performance bottlenecks.

Spark UI: A tool for monitoring and diagnosing Spark applications. It provides detailed information about stages, tasks, and execution times.

Authoritative Links:

Spark Performance Tuning: <https://spark.apache.org/docs/latest/tuning.html>

Data Skew in Spark: This is a widely discussed topic, searching for "Spark Data Skew" will provide ample resources from Databricks, Stack Overflow, and other sources.

Question: 144

Deep Dumps

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

- A. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- B. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores / Executor
- C. • Total VMs: 1
 - 400 GB per Executor
 - 160 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores / Executor

Answer: B

Explanation:

The correct answer is B because it provides the highest level of fault tolerance among the options.

Here's a detailed justification:

Fault Tolerance: In distributed computing environments, fault tolerance is crucial for long-running jobs. It refers to the ability of a system to continue operating properly even in the event of the failure of some of its components.

VM Failures: In cloud environments, VM failures are a reality. Factors such as hardware issues, software bugs, or network problems can cause VMs to become unavailable.

Impact on Job Completion: If a VM running a part of a job fails, the entire job can fail unless the system is designed to handle such failures.

Cluster Configuration and Fault Tolerance: The number of VMs in a cluster directly impacts its fault tolerance. A cluster with more VMs has a higher chance of surviving a VM failure without interrupting the job.

Option Analysis:

Option C (1 VM): The worst option. If that single VM fails, the entire job fails. No fault tolerance.

Option A (8 VMs): Better than C, but a single VM failure removes a relatively large portion of the resources (12.5% of the total RAM and cores).

Option D (4 VMs): Better than A, but a single VM failure removes a relatively large portion of the resources (25% of the total RAM and cores).

Option B (16 VMs): The best option. A single VM failure has the least impact, removing only 6.25% of the total RAM and cores. The job is more likely to survive with minimal disruption and continue execution on the remaining VMs. This distribution minimizes the impact of a single point of failure.

Redundancy and Distribution: Option B provides the most redundancy through a greater number of VMs. The workload is more evenly distributed, reducing the risk associated with any single VM.

Spark's Fault Tolerance: Spark, a common data processing engine, has built-in fault tolerance mechanisms. When a task fails on one executor, Spark can reschedule it on another available executor. Having more executors (as with Option B) improves the chances of successful rescheduling.

Resource Utilization: While all options have the same total resources, the distribution across more VMs can lead to better overall resource utilization.

Authoritative Links:

Fault Tolerance in Cloud Computing: <https://www.bmc.com/blogs/fault-tolerance/>

Spark's Fault Tolerance: <https://spark.apache.org/docs/latest/> (Refer to the Spark documentation for details on fault tolerance mechanisms)

Question: 145

Deep Dumps

A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at /mnt/raw_orders/. After this data is written, a Databricks job containing the following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order, and that the `time` field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

- A. Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same `customer_id` and `order_id`.
- B. All records will be held in the state store for 2 hours before being deduplicated and committed to the orders table.
- C. The orders table will contain only the most recent 2 hours of records and no duplicates will be present.
- D. The orders table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

Answer: A

Explanation:

Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same `customer_id` and `order_id`.

Question: 146

Deep Dumps

A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

- A. Rely on Delta Lake schema enforcement to prevent duplicate records.
- B. VACUUM the Delta table after each batch completes.
- C. Perform an insert-only merge with a matching condition on a unique key.
- D. Perform a full outer join on a unique key and overwrite existing data.

Answer: C

Explanation:

The correct approach for deduplicating late-arriving or duplicate records against previously processed data in a Delta table is option C: Performing an insert-only merge with a matching condition on a unique key. Here's a detailed justification:

Delta Lake's merge operation is specifically designed for upsert scenarios, which naturally handle deduplication. An insert-only merge allows us to update the target Delta table based on conditions met in a source dataframe.

In this context, the source dataframe represents the new batch of data, which may contain duplicates or late-arriving records. The "matching condition" is defined on a unique key that identifies each record within the dataset.

When a new record in the source dataframe doesn't match any existing record in the Delta table (based on the unique key condition), the new record is inserted (insert-only). However, if a matching record already exists (indicating a duplicate or late-arriving record that has already been processed), the merge statement can be configured to either ignore it, update it, or flag the record as a duplicate (depending on the business requirements). In this case, we are focusing on avoiding inserting duplicates, so the "ignore it" approach would apply.

This approach provides an efficient and transactional way to manage duplicates as data arrives. Delta Lake's ACID properties ensure data consistency throughout the merge operation. Other approaches are less effective: Schema enforcement (A) only validates data types, not duplicate data. Vacuuming (B) simply removes old versions of the Delta table and doesn't handle de-duplication. Full outer join (D) coupled with overwriting can potentially lead to data loss or inconsistencies if not handled carefully, and is more complex than a merge. A merge operation is the best approach as it allows for handling of late arriving data in a controlled, auditable, and transactional way.

Authoritative Links:

Delta Lake Merge: <https://docs.databricks.com/delta/delta-update.html>

Upsert with Delta Lake: <https://www.databricks.com/blog/2019/01/30/optimizing-upserts-in-delta-lake.html>

Question: 147

Deep Dumps

A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a bronze table created with the property `delta.enableChangeDataFeed = true`. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
 .option("readChangeFeed", "true")
 .option("startingVersion", 0)
 .table("bronze")
 .filter(col("_change_type").isin(["update_postimage", "insert"]))
 .write
 .mode("append")
 .table("bronze_history_type1")
)
```

Which statement describes the execution and results of running the above query multiple times?

- A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
- B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
- C. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.
- D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.

Answer: B

Explanation:

Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.

Question: 148

Deep Dumps

A DLT pipeline includes the following streaming tables:

- raw_iot ingests raw device measurement data from a heart rate tracking device.
- bpm_stats incrementally computes user statistics based on BPM measurements from raw_iot.

How can the data engineer configure this pipeline to be able to retain manually deleted or updated records in the raw_iot table, while recomputing the downstream table bpm_stats table when a pipeline update is run?

- A. Set the pipelines.reset.allowed property to false on raw_iot
- B. Set the skipChangeCommits flag to true on raw_iot
- C. Set the pipelines.reset.allowed property to false on bpm_stats
- D. Set the skipChangeCommits flag to true on bpm_stats

Answer: B

Explanation:

The correct answer is **B. Set the skipChangeCommits flag to true on raw_iot**. Here's why:

The core problem revolves around retaining manual modifications (deletions or updates) made directly to the raw_iot table while still allowing the pipeline to recompute derived tables like bpm_stats. Delta Live Tables (DLT) pipelines are designed to incrementally process data changes. By default, DLT tracks change commits and reprocesses data based on these changes during pipeline updates.

Setting pipelines.reset.allowed to false prevents the entire pipeline from being reset and reprocessed from scratch, which isn't the objective here. We want to recompute bpm_stats based on the existing (potentially modified) raw_iot data. Options A and C, that involve this property, are incorrect because they affect the pipeline's overall reset behavior, not the specific handling of change commits in the source table.

The skipChangeCommits flag, when set to true on the raw_iot table, tells DLT to ignore the change commit log for that specific table. In effect, DLT will not propagate delete or update operations from raw_iot downstream to bpm_stats unless a full refresh is triggered. Thus manual changes to raw_iot (deletions, updates) are preserved. During a pipeline update, bpm_stats will be recomputed based on the current state of raw_iot which includes any manual edits. This retains manual modifications to the raw data while still allowing for recalculation of derived statistics. Setting this flag on bpm_stats (option D) would have no effect on preserving changes in raw_iot. The pipeline would still propagate all updates from the raw data, including manual

deletions, downstream.

In summary, `skipChangeCommits = true` on the source table (`raw_iot`) allows us to retain manual data modifications while enabling recomputation of dependent tables (`bpm_stats`) based on the current, modified state of the raw data.

For further reading, consult Databricks documentation on DLT settings:

[Delta Live Tables settings](#) (specifically look for the `skipChangeCommits` and `pipelines.reset.allowed` properties)
[How Delta Live Tables handles updates](#)

Question: 149

Deep Dumps

A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

- A. New fields will not be computed for historic records.
- B. Spark cannot capture the topic and partition fields from a Kafka source.
- C. Updating the table schema requires a default value provided for each field added.
- D. Updating the table schema will invalidate the Delta transaction log metadata.

Answer: A

Explanation:

The correct answer is A: "New fields will not be computed for historic records."

Here's a detailed justification:

The data engineering team is adding new columns (timestamp, topic, partition) to the Delta Lake table to help diagnose latency issues. These new columns are populated based on information available at the time of ingestion. However, once data is written to the bronze table, the original Kafka messages are not re-processed to retroactively fill in these new columns for existing (historic) data.

This is because Delta Lake stores the data as it was ingested. Updating the schema only changes the schema definition for future writes. The existing data remains as it was originally written, without the new columns' values populated. If the team tries to query the new columns for older data, they will typically see NULL values or the default value (if one was specified during the schema evolution) since the values were not recorded at the time of the original ingestion. Therefore, the team will be unable to use these newly added fields to diagnose the latency problems affecting data ingested prior to the schema change. They will only be able to analyze the performance for data ingested after the new fields were introduced and populated.

Option B is incorrect because Spark Structured Streaming can access the topic and partition information from Kafka. This is a standard feature when reading from a Kafka source. The `kafkaUtils.createDirectStream` method (or its equivalent) provides access to these metadata fields.

Option C is partially correct. When adding a new column to a Delta table with existing data, and without specifying a default value, Delta Lake requires a non-null constraint to be dropped or a default value to be provided. However, it is not always required; you can add columns that are nullable without a default value.

This isn't the main limitation affecting the team's ability to diagnose historic latency.

Option D is incorrect. Updating the table schema using Delta Lake's schema evolution features does not invalidate the transaction log metadata. Schema evolution is a core feature of Delta Lake, and the transaction log is specifically designed to track schema changes, ensuring data consistency and allowing for time travel. The transaction log is updated to reflect the schema change, but it's not invalidated.

Supporting Links:

Delta Lake Schema Evolution: <https://docs.databricks.com/delta/delta-batch.html#schema-evolution>
Structured Streaming with Kafka: <https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html>

Question: 150

Deep Dumps

A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
     .format("parquet")
     .load(f"/mnt/daily_batch/{year}/{month}/{day}")
     .select("*",
             time_col.alias("ingest_time"),
             input_file_name().alias("source_file")
            )
     .write
     .mode("append")
     .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

def new_records():

A.return spark.readStream.table("bronze")

B.return spark.read.option("readChangeFeed", "true").table ("bronze")

```
    return (spark.read
            .table("bronze")
            .filter(col("ingest_time") == current_timestamp())
```

C.)

D.

```
return (spark.read
        .table("bronze")
        .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
    )
```

Answer: B

Explanation:

```
return spark.read.option("readChangeFeed", "true").table ("bronze")
```

Question: 151

Deep Dumps

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
- C. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
- D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.

Answer: D

Explanation:

The correct answer is D because it highlights the importance of manually defining schemas for data quality and consistency, especially when dealing with complex data structures in a Lakehouse environment. Here's a detailed justification:

Option A is incorrect. While Databricks and its underlying Apache Spark engine handle string data efficiently, suggesting that strings are "always most efficient" is misleading. Storing numeric or boolean data as strings can negatively impact performance and storage efficiency. The Tungsten engine focuses on optimized memory management and code generation across various data types, not just strings.

<https://spark.apache.org/docs/latest/>

Option B is incorrect. While Delta Lake leverages Parquet for storage, data type evolution isn't as simple as modifying file footers. Changing a data type requires rewriting the affected data. Delta Lake supports schema evolution, but it involves updating the Delta log and potentially rewriting data based on compatibility rules.

<https://docs.databricks.com/delta/delta-batch.html#schema-evolution>

Option C is incorrect. Schema inference is convenient, but it's not foolproof. The inferred types are based on the data observed, and if the data is inconsistent or incomplete, the inferred schema may not accurately reflect the intended data types for all scenarios or the expectations of downstream systems. Relying solely on

schema inference can introduce data quality issues.

Option D is the most accurate. When dealing with complex, nested data and a Lakehouse environment where data is consumed by various applications, manual schema declaration is crucial for data quality. Databricks infers schemas based on the data it observes. It chooses the data type that accommodates all observed data, which might lead to choosing a more generic data type (e.g., string) instead of a more specific and efficient one (e.g., integer) if there's even one non-integer value in a column. By manually defining the schema, the data engineer can enforce specific data types, catch data quality issues early, and ensure consistency across downstream applications. This proactive approach is better than reactive troubleshooting.

Therefore, manually declaring the schema provides better control over data types and data quality compared to relying solely on schema inference, especially when a well-defined and consistent schema is vital for downstream applications like dashboards and production models. This aligns with best practices for building robust and reliable data pipelines in a Lakehouse architecture.

Question: 152

Deep Dumps

The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
.select(
    orderDF.accountID,
    orderDF.itemID,

    itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
.select(
    orderWithItemDF["*"],

    accountDF.city))

(finalDF.write
    .mode("overwrite")
    .table("enriched_itemized_orders_by_account"))
```

Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. A batch job will update the `enriched_itemized_orders_by_account` table, replacing only those rows that have different values than the current version of the table, using `accountID` as the primary key.
- B. The `enriched_itemized_orders_by_account` table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.
- C. No computation will occur until `enriched_itemized_orders_by_account` is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.
- D. An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the `enriched_itemized_orders_by_account` table.

Answer: B

Explanation:

The `enriched_itemized_orders_by_account` table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.

Question: 153

Deep Dumps

The data engineering team is configuring environments for development, testing, and production before beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

- A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
- B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
- C. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.
- D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.

Answer: B

Explanation:

The correct answer is B because it emphasizes data security and risk mitigation, crucial aspects of responsible data engineering practices in cloud environments.

Here's why:

Principle of Least Privilege: Granting only read permissions to production data in development and testing environments adheres to the principle of least privilege. This limits the potential for accidental or malicious data modification or deletion by developers during interactive coding sessions. This directly contradicts the junior data engineer's suggestion that production data should be directly mounted with potentially unrestricted access.

Data Isolation: Creating isolated databases for each environment (development, testing, production) is a foundational practice in data engineering. It prevents unintended cross-environment contamination and ensures that development and testing activities do not inadvertently impact production data or systems. This aligns with the need for data segregation to minimize risks.

Security Risks: Mounting production data directly to a development environment, especially one with admin privileges for all users, is a significant security risk. Development environments are often less strictly controlled than production, making them vulnerable to security breaches. A compromised development environment could then provide a pathway to access and potentially damage or expose production data.

Auditing and Traceability: Restricting access and segregating data allows for better auditing and traceability. It becomes easier to track who accessed which data and when, which is essential for compliance and security incident investigations.

Options A, C, and D are incorrect:

A: Unified Workspace Inefficiency: Combining all environments into a single workspace is not a best practice. It creates a chaotic environment and increases the risk of unintended modifications to production data.

C: Passthrough Credential Fallacy: Passthrough credentials do not eliminate the risk of data modification or deletion in development. A compromised development environment could still use these credentials to access and potentially damage production data.

D: Delta Lake Versioning Limitations: While Delta Lake provides versioning, it does not provide complete immunity from data loss or corruption, especially if someone with admin privileges intentionally or unintentionally deletes or modifies data at a fundamental level. Versioning helps recover from accidental deletions, but it does not mitigate the risk of malicious actions or extensive data corruption. Moreover, relying solely on versioning is reactive, not proactive.

Authoritative Links:

Principle of Least Privilege: https://csrc.nist.gov/glossary/term/least_privilege (NIST definition)

Data Security Best Practices: <https://cloud.google.com/security/> (Google Cloud Security documentation, relevant to general cloud security principles)

AWS Security Best Practices: <https://aws.amazon.com/security/> (AWS Security documentation, covering cloud security)

Question: 154

Deep Dumps

The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. Whenever a database is being created, make sure that the LOCATION keyword is used.
- B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
- C. Whenever a table is being created, make sure that the LOCATION keyword is used.
- D. When tables are created, make sure that the UNMANAGED keyword is used in the CREATE TABLE statement.

Answer: A

Explanation:

When a database (schema) is created with a LOCATION, all tables created inside that database are external (unmanaged) Delta tables by default.

The table data is stored in user-managed cloud object storage, not in Databricks' managed storage.

This approach enforces the requirement globally, instead of relying on every developer to remember table-level settings.

Question: 155

Deep Dumps

The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales org.

Which of the following solutions addresses the situation while emphasizing simplicity?

- A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
- B. Create a new table with the required schema and use Delta Lake's DEEP CLONE functionality to sync up changes committed to one table to the corresponding table.
- C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
- D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.

Answer: A

Explanation:

The best solution is A, creating a view on the marketing table with aliased and selected fields. Views offer several advantages in this scenario, aligning with the principle of simplicity highlighted in the problem.

A view is a virtual table defined by a query. It doesn't store data physically; it dynamically reflects the underlying table's content. This characteristic addresses the requirement of sharing aggregated data without creating unnecessary copies.

The approach fulfills the need for renaming fields through aliasing in the SELECT statement. Aliasing changes field names in the view's schema, aligning them with the sales organization's naming conventions.

By carefully selecting only approved fields in the SELECT statement of the view definition, sensitive marketing-specific fields can be excluded, ensuring data governance.

The simplicity arises because no additional storage or data synchronization mechanisms are needed. The view relies directly on the marketing table, minimizing operational overhead. Delta Lake views inherit the guarantees of the underlying Delta table, ensuring consistency and reliability. Views are easy to create and manage via SQL, fitting within the Databricks ecosystem seamlessly.

Option B using DEEP CLONE is overkill, creating a full copy of the table and adding complexity for continuous updates. Options C and D, utilizing CTAS and parallel table writes, introduce data duplication and add complexity to the data pipeline, which goes against the need for simplicity. Views are preferable for simple data transformations and access control.

For further reading on Databricks Views and Delta Lake features, refer to these links:

[Databricks Views](#)

[Databricks Delta Lake](#)

Question: 156**Deep Dumps**

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

This table is partitioned by the date column. A query is run with the following filter:

longitude < 20 & longitude > -20

Which statement describes how data will be filtered?

- A. Statistics in the Delta Log will be used to identify partitions that might include files in the filtered range.
- B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
- C. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.
- D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.

Answer: D**Explanation:**

The correct answer is D because Delta Lake leverages its metadata, specifically statistics stored in the Delta Log, for data skipping. These statistics include min/max values for columns within each file. When a query with filters is executed, Delta Lake first consults the Delta Log to determine which files might contain data that satisfies the filter conditions. In this case, the longitude < 20 & longitude > -20 filter is applied to the longitude column.

Delta Lake checks the min/max longitude values stored for each data file in the Delta Log. If the filter's range overlaps with the range of longitude values in a file (min <= 20 and max >= -20), that file is considered a potential candidate and will be scanned. Files whose longitude range falls completely outside the filter range (e.g., min > 20 or max < -20) are skipped entirely, improving query performance.

Option A is incorrect because while statistics are used, they are used at the file level, not partition level for non-partitioning columns. Option B is incorrect because Delta Lake does utilize statistics for non-partitioning columns when available. Option C is incorrect because Delta Lake does not scan parquet file footers for each row; it uses the metadata in the Delta Log for data skipping before even accessing the parquet files themselves. The key is that Delta Log contains pre-computed aggregate statistics about the data files, allowing for skipping entire files. This significantly reduces the amount of data that needs to be read from storage.

Here are some authoritative links for further research:

Delta Lake Documentation on Data Skipping: <https://docs.databricks.com/delta/data-skipping.html>

Databricks Delta Lake Overview: <https://databricks.com/product/delta-lake>

Question: 157**Deep Dumps**

A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.

The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.

Assuming that all data governance considerations are accounted for, which statement accurately informs this

decision?

- A.Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.
- B.Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.
- C.Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.
- D.Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

Answer: C

Explanation:

The correct answer is C because it directly addresses the performance and cost implications of data locality in cloud computing. Deploying compute resources (Databricks workspace) in a different region than the data storage can introduce significant latency due to the physical distance the data must travel during read and write operations. This increased latency can negatively impact the performance of data pipelines and AI applications, making them slower and less responsive. Moreover, cross-region data transfer incurs network egress costs, which can quickly accumulate, especially with large datasets.

Option A is incorrect because while Databricks can interact with cloud storage that often leverages a distributed file system-like architecture similar to HDFS, it doesn't inherently run HDFS on cloud volume storage. Databricks abstracts away the underlying storage details, allowing it to work with various storage solutions. The physical location of VMs is still important for performance and cost.

Option B is incorrect because the location of the Databricks workspace significantly impacts performance and cost, making it far from a decision based solely on the administrator's convenience. Ignoring regional considerations can lead to substantial penalties.

Option D is incorrect because while security is a valid concern, the primary driver for workspace location in this scenario is performance and cost related to data access. Databricks uses secure communication channels and doesn't rely on sending all code over the open internet. Furthermore, user proximity to the workspace isn't as critical as data proximity for data engineering pipelines.

Therefore, collocating compute and data is a best practice in cloud environments to minimize latency and cost, aligning perfectly with answer choice C.

Supporting Links:

Azure Data Regions: <https://azure.microsoft.com/en-us/global-infrastructure/regions/> - Demonstrates the importance of region selection.

AWS Regions: https://aws.amazon.com/about-aws/global-infrastructure/regions_az/ - Demonstrates the importance of region selection.

Google Cloud Regions: <https://cloud.google.com/about/locations> - Demonstrates the importance of region selection.

Data Egress Costs: Search "[cloud provider] data egress costs" to find official documentation regarding data transfer pricing for specific cloud providers, for example: Azure, AWS and GCP.

Question: 158

Deep Dumps

A CHECK constraint has been successfully added to the Delta table named activity_details using the following logic:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where latitude = 45.50 and longitude = 212.67.

Which statement describes the outcome of this batch insert?

- A.The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.
- B.The write will fail completely because of the constraint violation and no records will be inserted into the target table.
- C.The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
- D.The write will include all records in the target table; any violations will be indicated in the boolean column named valid_coordinates.

Answer: B

Explanation:

The write will fail completely because of the constraint violation and no records will be inserted into the target table.

Question: 159

Deep Dumps

A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

- A.Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
- B.Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.
- C.Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.
- D.All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.

Answer: D

Explanation:

The correct answer is D because it accurately reflects how Delta Lake and Databricks handle data integrity constraints and transactions. Here's a detailed justification:

Delta Lake, the storage layer of Databricks, offers ACID (Atomicity, Consistency, Isolation, Durability) properties for individual table transactions. This means that writes to a single Delta table are reliable and consistent. However, Databricks, by default, does not enforce foreign key constraints. The relational database's reliance on foreign keys and multi-table inserts for validation represents a significant architectural difference.

Option A is incorrect because Databricks doesn't inherently restrict foreign keys to hashed identifiers. In fact, it doesn't actively enforce foreign keys at all.

Option B is also incorrect. While Delta Lake does support MERGE operations for upsert functionality, it doesn't require that multi-table inserts utilize it. Moreover, the primary concern is the lack of enforced foreign keys, not how they might relate to upserts.

Option C presents a misleading concern. While managing transactions across multiple tables can introduce complexities in any system, Delta Lake's challenge is the lack of native support for transactions spanning multiple tables with enforced referential integrity. The possibility of deadlocks is a general database concurrency issue but not the core reason why the original system's validation logic needs to be re-evaluated in Databricks.

Therefore, the junior data engineer must adapt the validation logic previously handled by foreign key constraints and multi-table inserts. They might need to implement these checks within their data pipelines using Databricks' capabilities, such as data quality checks with Delta Live Tables or custom validation code using Spark. This might involve checking for referential integrity within the data pipeline itself, rather than relying on the database system to enforce it.

Further resources:

Delta Lake ACID Transactions: <https://docs.databricks.com/delta/index.html>

Data Quality with Delta Live Tables: <https://docs.databricks.com/delta-live-tables/index.html>

Question: 160

Deep Dumps

A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in the customers table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table will provide better performance and scalability.

Which piece of information is critical to this decision?

- A. Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
- B. Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
- C. Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
- D. Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.

Answer: D

Explanation:

The correct answer is D because it highlights the fundamental limitation of relying solely on Delta Lake time travel for long-term auditing with Type 1 tables. While Delta Lake's time travel feature is powerful, it's not designed to be a primary mechanism for maintaining a complete, scalable, and cost-effective historical record

for auditing purposes, especially over extended periods. Type 1 tables, by overwriting data in place, rely entirely on Delta Lake's transaction log and data file versioning for historical reconstruction. Over time, querying older versions of a Type 1 table can become inefficient and expensive as Delta Lake needs to reconstruct the data state from numerous transaction log entries and potentially multiple versions of data files.

Type 2 tables, which maintain a complete history of changes within the table itself using start and end dates, offer better performance and scalability for historical queries because the history is readily available within the table structure. While maintaining a Type 2 table increases storage costs, it mitigates the performance and cost overhead associated with long-term time travel queries. The primary concern of Type 1 tables is less about data corruption or inability to use time travel at all (as options A and C incorrectly state), but rather the practical limitations in scalability and cost-effectiveness for long-term historical analysis and auditing. Option B is also incorrect because shallow clones are more useful for creating isolated copies of the table at a specific point in time, and don't improve the cost or latency of time travel.

For more information on Delta Lake time travel and table types, refer to the official Databricks documentation:

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Scalability: <https://www.databricks.com/blog/2019/04/24/diving-into-delta-lake-unpacking-the-transaction-log.html> (This blog post illustrates how the transaction log is used and its impact on query performance for historical versions).

Question: 161

Deep Dumps

A data engineer wants to join a stream of advertisement impressions (when an ad was shown) with another stream of user clicks on advertisements to correlate when impressions led to monetizable clicks.

In the code below, Impressions is a streaming DataFrame with a watermark ("event_time", "10 minutes")

```
.groupBy(
  window("event_time", "5 minutes"),
  "id")
.count()
).      withWatermark("event_time", 2 hours)
impressions.join(clicks, expr("clickAdId = impressionAdId"), "inner")
```

The data engineer notices the query slowing down significantly.

Which solution would improve the performance?

- A. Joining on event time constraint: `clickTime >= impressionTime AND clickTime <= impressionTime interval 1 hour`
- B. Joining on event time constraint: `clickTime + 3 hours < impressionTime - 2 hours`
- C. Joining on event time constraint: `clickTime == impressionTime` using a `leftOuter` join
- D. Joining on event time constraint: `clickTime >= impressionTime - interval 3 hours` and removing watermarks

Answer: A

Explanation:

Joining on event time constraint: `clickTime >= impressionTime AND clickTime <= impressionTime interval 1 hour`.

Question: 162**Deep Dumps**

A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

- A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
- B. The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
- C. Only workspace administrators can grant "Owner" privileges to a group.
- D. A user can only transfer job ownership to a group if they are also a member of that group.

Answer: A**Explanation:**

The correct answer is A, stating that Databricks jobs must have exactly one owner, and "Owner" privileges cannot be assigned to a group. This is because Databricks job ownership is designed around individual accountability and responsibility. The "Owner" has full control over the job, including the ability to modify, delete, and manage permissions. Granting ownership to a group would create ambiguity as to which individual within the group is ultimately responsible. The Databricks documentation emphasizes individual user roles and permissions for resource management.

Option B is incorrect because while the creator initially has "Owner" privileges, these can be transferred to another individual. Option C is incorrect as workspace administrators do not generally need to be involved in the transfer of ownership from one user to another. Option D is also incorrect. Job ownership transfer is not contingent on the current owner being a member of the target group. The key constraint lies in assigning ownership to a group instead of an individual. Databricks access control focuses on precise individual permissions to maintain security and auditability. The "Owner" role requires a single accountable individual, and assigning it to a group would violate this principle. Therefore, the "Owner" privilege can only be assigned to a single user, not a group. This design reflects best practices for access control in cloud environments.

Supporting Documentation:

While specific documentation explicitly stating "Owner cannot be a group" is difficult to pinpoint, the general context of Databricks access control and user roles strongly supports this claim. Examining Databricks documentation on Access Control and Job Management provides this context:

Databricks Access Control: <https://docs.databricks.com/en/security/access-control/index.html>

Databricks Job Management: <https://docs.databricks.com/en/jobs/index.html>

Question: 163**Deep Dumps**

A table named user_ltv is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The user_ltv table has the following schema:

email STRING, age INT, ltv INT

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
    CASE
        WHEN is_member("auditing") THEN TRUE
        ELSE age >= 18
    END
```

An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

- A. All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.
- B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in user_ltv.
- C. All values for the age column will be returned as null values, all other columns will be returned with the values in user_ltv.
- D. All columns will be displayed normally for those records that have an age greater than 18; records not meeting this condition will be omitted.

Answer: A

Explanation:

All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.

Question: 164

Deep Dumps

All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG

There are 5 unique topics being ingested. Only the "registration" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which solution meets the requirements?

- A. All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.
- B. Data should be partitioned by the registration field, allowing ACLs and delete statements to be set for the PII directory.
- C. Data should be partitioned by the topic field, allowing ACLs and delete statements to leverage partition

boundaries.

D. Separate object storage containers should be specified based on the partition field, allowing isolation at the storage level.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution, along with supporting information:

The core requirements are twofold: restricting access to PII within a Delta Lake table and implementing different retention policies based on whether data contains PII (specifically from the "registration" topic) or not.

Option C, partitioning the Delta Lake table by the topic field, directly addresses these needs. Partitioning splits the data into separate directories based on the value of the specified partition column (in this case, the topic). This enables granular access control. Using the topic field for partitioning isolates PII records (within the "registration" partition) from non-PII records (partitions for the other topics).

With partitioned data, you can apply Access Control Lists (ACLs) to restrict access to the "registration" partition, effectively limiting access to PII. Furthermore, using DELETE statements targeting the "registration" partition will allow compliant removal of PII records after 14 days based on the timestamp field. This targeted delete does not affect other topics.

The non-PII partitions ("topic" partitions other than "registration") can be retained indefinitely by simply not applying the retention policy to them. No data deletion is required.

Option A is not suitable because deleting all data biweekly and relying solely on Time Travel for non-PII data is overly aggressive. Time Travel should not be used as a primary retention strategy, as it can have performance implications and adds complexity.

Option B, partitioning by a non-existent "registration" field (instead of "topic"), is incorrect. The data contains a "topic" column, and not a "registration" field as the question describes.

Option D, separating object storage containers based on the partition field, could work, but is not ideal. It adds operational complexity compared to leveraging partitioning within a single Delta Lake table. Managing separate storage containers introduces more administrative overhead and may complicate data integration and querying. It also doesn't directly leverage the features of Delta Lake for data management and governance.

Therefore, option C leverages Delta Lake's partitioning capabilities combined with ACLs and targeted DELETE operations to efficiently and effectively meet both data access restriction and differential retention requirements.

Supporting documentation:

Delta Lake Partitioning: <https://docs.databricks.com/delta/best-practices.html#partition-data>

Delta Lake Time Travel: <https://docs.databricks.com/delta/versioning.html>

Delta Lake Data Governance: <https://docs.databricks.com/data-governance/index.html>

Question: 165

Deep Dumps

The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the user_lookup table to the user_aggregates table.

```
(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
  DELETE FROM user_aggregates
  WHERE user_id IN (
    SELECT user_id
    FROM changes
    WHERE _change_type='delete'
  )
  """)
```

Assuming that user_id is a unique identifying key and that all users that have requested deletion have been removed from the user_lookup table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the user_aggregates table are no longer accessible and why?

- A.No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- B.No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.
- C.No; the change data feed only tracks inserts and updates, not deleted records.
- D.Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.

Answer: B

Explanation:

No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Question: 166

Deep Dumps

An external object storage container has been mounted to the location /mnt/finance_eda_bucket.

The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db  
LOCATION '/mnt/finance_eda_bucket';  
GRANT USAGE ON DATABASE finance_eda_db TO finance;  
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS  
SELECT *  
FROM sales  
WHERE state = "TX";
```

If all users on the finance team are members of the finance group, which statement describes how the tx_sales table will be created?

- A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
- B. An external table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- C. A managed table will be created in the DBFS root storage container.
- D. A managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

Answer: D

Explanation:

An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

Question: 167

Deep Dumps

The data engineering team has been tasked with configuring connections to an external database that does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

- A. No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.
- B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
- C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
- D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

Answer: C

Explanation:

Here's a breakdown of why option C is the correct answer and why the others are not, along with supporting information:

Justification for Option C: "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

This option aligns perfectly with the principle of least privilege and secure credential management in Databricks. Secret scopes provide a namespace for storing secrets (like database credentials). By creating a secret scope for each team containing only the credentials that team needs, you isolate access. Granting "Read" permission to a team on their designated secret scope allows them to retrieve the credentials necessary to connect to the external database. Critically, they cannot manage (create, modify, delete) those credentials, enforcing a separation of duties. This approach ensures that each team only has access to the secrets required for their tasks and prevents unauthorized access to other teams' credentials. Utilizing secret scopes in this way offers granular control, minimizing the blast radius in case of a security breach and facilitating easier auditing and compliance. <https://docs.databricks.com/security/secrets/index.html>

Why other options are incorrect:

A. No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added. This is a massive security risk. Granting all users admin privileges violates the principle of least privilege and exposes all secrets in the workspace to everyone. It defeats the purpose of using secret scopes to control access.

B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team. While setting "Read" permissions is correct, applying it on a secret key level is cumbersome and less scalable. If multiple teams need different sets of credentials, managing permissions individually for each key would become very complex. Secret scopes provide a higher level of abstraction that simplifies management and makes it easier to enforce access control policies. Also, this would require multiple secret scopes.

D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team. "Manage" permissions are far too permissive. Giving teams the ability to manage the secrets in their scopes allows them to modify, delete, or even create new credentials. This significantly increases the risk of unauthorized access or data breaches. "Read" permission is sufficient for connecting to a database.

In summary, secret scopes combined with appropriate "Read" permissions provide the most secure and manageable way to grant teams access to the necessary credentials for connecting to the external database, while adhering to the principles of least privilege and secure credential management.

Question: 168

Deep Dumps

What is the retention of job run history?

- A. It is retained until you export or delete job run logs
- B. It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3
- C. It is retained for 60 days, during which you can export notebook run results to HTML
- D. It is retained for 60 days, after which logs are archived

Answer: C

Explanation:

The correct answer is C because Databricks retains job run history for a specific duration to allow for

debugging, monitoring, and historical analysis. While logs themselves may be streamed to persistent storage, the run history accessible directly through the Databricks UI or API has a retention period. Option C accurately reflects the documented retention policy, stating job run history is retained for 60 days, allowing you to export notebook run results to HTML during this time. Options A and B are incorrect because while exporting or delivering logs to persistent storage is a common practice for long-term archiving, it does not affect the default retention period of the run history within the Databricks platform itself. Option D is also incorrect, as the job run history is not archived after 60 days; it is deleted. The ability to export results to HTML is a specific feature relevant to preserving notebook execution outcomes tied to the run. This retention policy ensures efficient resource management within the Databricks environment while providing a reasonable window for operational tasks. Specifically the ability to download results to HTML is valid for notebook job types within the 60 day window. The documentation confirms this behavior and highlights features related to exporting and managing job outputs during this period.

Refer to the following Databricks documentation for more information on job run history and logging:

[Databricks Jobs Documentation](#)

[Databricks Logging](#) (while focusing on cluster logging, it indirectly supports the need for a retention policy for associated job run metadata)

Question: 169

Deep Dumps

A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

Which statement describes the contents of the workspace audit logs concerning these events?

- A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
- B. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.
- C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
- D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer and why the other options are incorrect:

Justification for Option C: Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.

Audit logs in Databricks, and cloud environments in general, are designed to track actions performed by specific identities. When using personal access tokens (PATs) via the REST API, the system will log the actions taken using that token as being performed by the user associated with the token. Job creation and job execution (triggering runs) are distinct events. Therefore, the audit log will record the user whose PAT was used for each action separately. User A created the jobs, meaning User A's PAT was used during the creation process. Hence, User A's identity will be logged for the job creation events. User B, the DevOps engineer, triggered the job runs using their own PAT through the external orchestration tool. This means User B's identity will be associated with the job run (triggering) events. This approach provides clear accountability and traceability for auditing purposes. Separate logging allows administrators to pinpoint who created which jobs

and who initiated specific runs, which is critical for debugging, security, and compliance.

Why other options are incorrect:

A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events. This is generally incorrect. Service Principals are separate identities often used for automated processes or applications and are manually configured. Using the REST API doesn't automatically switch to a Service Principal unless one is explicitly configured and its credentials are used. In this scenario, PATs are used, thus logging actions under those user identities.

B. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events. This is incorrect because User B is triggering the job using their token. While User A created the job, the actual run is triggered by User B, and that action is recorded against User B's identity.

D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs. This is incorrect. Audit logs are specifically designed to capture the identity of the user or service performing an action. If the REST API usage didn't track user identity, it would defeat the purpose of auditing. Databricks, like most cloud services, strives for granular audit trails.

Authoritative Links:

Databricks Audit Logging: <https://docs.databricks.com/administration-guide/audit-logs/> (This link provides comprehensive documentation on Databricks audit logging, what events are captured, and how to interpret the logs.)

Databricks REST API Authentication: <https://docs.databricks.com/dev-tools/api/latest/authentication.html> (This link explains how authentication works with Databricks REST API and the use of PATs)

Question: 170

Deep Dumps

A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

- A. Workspace audit logs
- B. Driver's log file
- C. Ganglia
- D. Cluster Event Log

Answer: D

Explanation:

The correct answer is **D. Cluster Event Log**. Here's why:

The key is pinpointing where Databricks records cluster-specific operational events, especially resizing (upscaling/downscaling). The Cluster Event Log directly addresses this need. It provides a chronological record of actions performed on a specific cluster, including start, stop, resize (upscaling and downscaling), configuration changes, node additions/removals, and other relevant lifecycle events. These events are associated with timestamps, making it easy to correlate resizing events with potential triggers like concurrent user activity or resource-intensive query execution.

Workspace audit logs (A) capture workspace-level activities (e.g., user logins, notebook creation, permissions

changes) but generally don't provide granular details on the specific reasons why a particular cluster resized. They focus on user actions at a higher level, not cluster internals.

Driver's log file (B) is useful for debugging Spark applications running on the cluster. While resource usage could be inferred, analyzing individual queries' impact on cluster scaling would be extremely tedious and impractical. It doesn't explicitly show scaling events and their causes in a readily understandable way. It's primarily used for debugging Spark code.

Ganglia (C) is a distributed monitoring system that collects metrics like CPU utilization, memory usage, and network traffic. While helpful for identifying resource contention and performance bottlenecks at a point in time, it doesn't directly log the events of cluster resizing, nor does it correlate these events with specific user actions or queries in a straightforward manner. Ganglia is focused on the what (resource usage) and less on the why (scaling decisions).

The Cluster Event Log is specifically designed to track and provide insights into the operational history of a given cluster, including scaling events, which makes it the perfect place to begin your investigation. You can examine the events leading up to a resizing event to determine if it coincides with a surge in user activity or the execution of particularly resource-intensive queries.

Authoritative Links:

Databricks Cluster Event Log: <https://docs.databricks.com/clusters/clusters-manage.html#cluster-event-log>

Databricks Auditing: <https://docs.databricks.com/administration-guide/account-settings/audit-logs.html>

Question: 171

Deep Dumps

When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

- A. The five Minute Load Average remains consistent/flat
- B. CPU Utilization is around 75%
- C. Network I/O never spikes
- D. Total Disk Space remains constant

Answer: B

Explanation:

The correct answer, CPU Utilization is around 75%, indicates proper VM resource utilization in a Databricks cluster with Ganglia metrics.

Here's why: A consistently high CPU utilization (around 75%) suggests that the VM is actively processing workloads and not sitting idle. This is generally desirable, as it means you're getting the most out of the compute resources you're paying for.

A flat Five Minute Load Average (A) could indicate a consistently low workload, meaning the VM is underutilized. A high load average might indicate the VM is overloaded.

Constant Total Disk Space (D) primarily reflects that the data is not changing, not resource utilization.

Network I/O never spiking (C) is not a reliable indicator of resource utilization. Spikes in Network I/O are normal for data-intensive operations, and limiting that is not always optimal.

Ideally, CPU usage should remain at a high-sustained percentage to maximize resource utilization. Aiming for around 75% allows for headroom for unexpected spikes in processing demand without severely impacting performance. Exceeding a certain utilization rate may create bottlenecks and slowdowns. Monitoring CPU utilization is essential to determine the suitable node size for the workload. [Databricks Monitoring](#)

[Ganglia Metrics](#) allows the evaluation of the cluster performance of CPU, Memory, I/O and other resources. By

monitoring the cluster, the efficiency of the cluster may be further optimized.

Question: 172

Deep Dumps

The data engineer is using Spark's MEMORY_ONLY storage level.

Which indicators should the data engineer look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally?

- A. On Heap Memory Usage is within 75% of Off Heap Memory Usage
- B. The RDD Block Name includes the "*" annotation signaling a failure to cache
- C. Size on Disk is > 0
- D. The number of Cached Partitions > the number of Spark Partitions

Answer: B

Explanation:

The correct answer is B: The RDD Block Name includes the "*" annotation signaling a failure to cache.

Here's a detailed justification:

Spark's MEMORY_ONLY storage level attempts to store RDD partitions in memory as deserialized Java objects. This provides the fastest access but is limited by available memory. When using MEMORY_ONLY, monitoring the Storage tab in the Spark UI is crucial for performance tuning.

If Spark fails to cache an RDD partition in memory when using MEMORY_ONLY, it indicates that there was insufficient memory available at the time of caching. The presence of a * annotation in the RDD Block Name signifies that some or all of the partition data was evicted or could not be cached at all. This is a clear indicator that the MEMORY_ONLY storage level is not performing optimally because the data is not resident in memory as intended. This leads to recomputation or fetching from slower storage on subsequent accesses, negatively affecting performance.

Option A is incorrect because the relationship between on-heap and off-heap memory usage doesn't directly signal caching performance issues when MEMORY_ONLY is in use. While memory fragmentation or other issues might occur, the asterisk is a direct flag.

Option C is incorrect because if Size on Disk is greater than zero when using MEMORY_ONLY, it suggests data spilling to disk because of insufficient memory. While spilling reduces performance, the primary indicator in the Storage tab pointing specifically to caching failure is the * annotation. MEMORY_ONLY is designed to avoid disk usage unless memory is critically low.

Option D is incorrect as the relation between cached and spark partitions does not suggest non-optimal caching by itself. The number of cached partitions and Spark partitions being different might be due to filtering or other transformations reducing the number of active partitions. This is normal and not a direct sign of failing to cache.

In summary, the presence of the * annotation is the most direct signal in the Spark UI's Storage tab indicating a problem with the MEMORY_ONLY storage level failing to cache data due to insufficient memory.

For further research, refer to the official Apache Spark documentation on storage levels: <https://spark.apache.org/docs/latest/rdd-programming-guide.html#rdd-persistence>

Also, reviewing documentation on using the Spark UI will be helpful: <https://spark.apache.org/docs/latest/monitoring.html>

Question: 173**Deep Dumps**

Review the following error traceback:

```
-----
AnalysisException                                Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
    1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
    1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
    1693         return DataFrame(jdf, self.sql_ctx)
    1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
    1302
    1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
    1305             answer, self.gateway_client, self.target_id, self.name)
    1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
    121         # Hide where the exception came from that shows a non-Pythonic
    122         # JVM exception message.
--> 123         raise converted from None
    124     else:
    125         raise

AnalysisException: cannot resolve '`heartrateheartrateheartrate`' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet
```

Which statement describes the error being raised?

- A. There is a syntax error because the heartrate column is not correctly identified as a column.
- B. There is no column in the table named heartrateheartrateheartrate
- C. There is a type error because a column object cannot be multiplied.
- D. There is a type error because a DataFrame object cannot be multiplied.

Answer: B**Explanation:**

There is no column in the table named heartrateheartrateheartrate.

Question: 174**Deep Dumps**

What is a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

- A.Run source env/bin/activate in a notebook setup script
- B.Install libraries from PyPI using the cluster UI
- C.Use %pip install in a notebook cell
- D.Use %sh pip install in a notebook cell

Answer: C

Explanation:

The correct answer is **C. Use %pip install in a notebook cell**. This method offers the most straightforward and effective way to install Python packages scoped to the notebook and, crucially, propagates those packages to all nodes within the Databricks cluster currently associated with that notebook.

Here's a breakdown of why:

%pip vs. !pip and %sh pip: Databricks notebooks support "magic commands" prefixed with % or %%. %pip is a Databricks magic command specifically designed for package installation. It automatically integrates with the Databricks environment, ensuring packages are installed within the notebook's Python environment and made available across the cluster. !pip or %sh pip execute the pip command directly through the shell on the driver node. While seemingly similar, they do not guarantee consistent installation across all worker nodes, leading to dependency discrepancies and runtime errors.

Cluster-Wide Propagation: When you use %pip install, Databricks automatically handles the distribution of the installed package to each worker node in the cluster. This ensures that all nodes have access to the same dependencies, preventing "ModuleNotFoundError" exceptions and other environment-related issues when executing distributed tasks (e.g., using Spark). This is crucial for scalable data engineering workflows.

Notebook Scope and Isolation: Packages installed using %pip are scoped to the notebook's environment and the associated cluster. Other notebooks connected to the same cluster can also access the packages installed by %pip. This promotes code reusability and consistent environments across related tasks. However, installations persist only as long as the cluster is running. Upon cluster restart or termination, these packages must be reinstalled.

Alternatives and Their Limitations:

A. Run source env/bin/activate in a notebook setup script: This is not a suitable solution for Databricks. The concept of manually activating virtual environments is relevant in local development but not when working with managed Spark clusters like those in Databricks. Databricks manages the Python environment behind the scenes. Attempting to manually activate a virtual environment can interfere with Databricks' environment management, causing unpredictable behavior.

B. Install libraries from PyPI using the cluster UI: Installing libraries through the cluster UI does make libraries available across the cluster, but this approach applies the packages at the cluster level, affecting all notebooks attached to that cluster. This method provides cluster-level scope, not notebook-level scope. If different notebooks require conflicting package versions, this can create problems. While a valid option, it doesn't fulfill the "notebook level" scoping requirement.

D. Use %sh pip install in a notebook cell: This command executes pip from the shell on the driver node only, not the worker nodes. The package would be installed on the driver but not on the executor nodes, leading to errors when executing code. Therefore, this isn't cluster-wide.

In summary, %pip install offers the best balance of ease of use, cluster-wide propagation, and notebook-level scoping, making it the preferred approach for managing Python package dependencies within a Databricks notebook environment.

Authoritative Links:

Databricks Documentation - Libraries: <https://docs.databricks.com/libraries/index.html>

Databricks Documentation - %pip: <https://docs.databricks.com/notebooks/notebook-use.html#install-a-library-from-a-pip-compatible-package-repository>

Question: 175

Deep Dumps

What is the first line of a Databricks Python notebook when viewed in a text editor?

- A.%python
- B.// Databricks notebook source
- C.# Databricks notebook source
- D.-- Databricks notebook source

Answer: C

Explanation:

The correct answer is indeed # Databricks notebook source. This line acts as a special marker embedded in the notebook file. It signifies to the Databricks platform (and potentially other tools interacting with the file) that this is a Databricks notebook file.

The presence of this marker is vital for proper notebook parsing and execution within the Databricks environment. When a notebook is saved, Databricks embeds this marker. This ensures when the notebook is subsequently loaded, Databricks recognizes it as a notebook and handles it accordingly, maintaining cell structure, language contexts (Python, Scala, SQL, R, etc.), and metadata.

Alternatives are incorrect. %python is a magic command used within a Databricks notebook cell to explicitly specify that the cell should be executed using the Python interpreter. It's not the first line of the file itself. // and -- are line comment prefixes in other languages (like Java/C++ and SQL, respectively), but not the initial identifier for a Databricks notebook file. The // is also incorrect syntax for a line comment in python (which uses #).

The primary function of this marker is to differentiate Databricks notebook files from other types of text files, preventing potential misinterpretation or processing errors. This is part of Databricks' approach to managing and executing collaborative data engineering and data science workflows effectively.

This embedded marker is a specific implementation detail of the Databricks platform, and it is not a standard convention in general programming. Understanding its presence is crucial when working with Databricks notebooks in various contexts, such as version control systems or automated deployment pipelines.

Further research and authoritative documentation can be found on the official Databricks website and documentation portals, particularly those sections covering notebook structure, import/export functionality, and version control integration. You might want to investigate the Databricks CLI and its integration with notebook management.

Question: 176

Deep Dumps

Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.

Which benefit offsets this additional effort?

- A.Improves the quality of your data

- B. Validates a complete use case of your application
- C. Troubleshooting is easier since all steps are isolated and tested individually
- D. Ensures that all steps interact correctly to achieve the desired end result

Answer: C

Explanation:

The correct answer is **C. Troubleshooting is easier since all steps are isolated and tested individually.**

Unit testing focuses on individual components or functions within the PySpark application. By isolating and testing each unit (e.g., a specific transformation or data cleansing function), developers can quickly pinpoint the source of errors when issues arise. When a test fails, the scope of the problem is limited to the unit being tested, significantly reducing the debugging time compared to tracing errors through the entire application. This modularity simplifies the identification and correction of defects, making the overall development and maintenance process more efficient. While unit testing can indirectly contribute to data quality, it's not its primary focus. Unit tests do not validate entire use cases, which is the domain of integration or end-to-end testing. They also don't guarantee that all steps interact correctly as that is the purpose of integration tests.

Why the other options are less suitable:

A. Improves the quality of your data: While unit tests can indirectly improve data quality by ensuring data transformations are performed correctly, their primary focus is on the functionality of the code, not the data itself. Data quality is more directly addressed by data validation techniques and data quality monitoring.

B. Validates a complete use case of your application: Unit tests test individual units, not the entire application's use case. Integration or end-to-end tests validate complete use cases.

D. Ensures that all steps interact correctly to achieve the desired end result: Unit tests focus on the individual functions of the PySpark job. Integration tests are used to verify the interactions between different parts of the system.

Authoritative Links:

Testing and Debugging Spark Applications: <https://spark.apache.org/docs/latest/testing.html> (This is a general introduction, but doesn't focus on unit testing specifically)

Unit Testing in PySpark (Example): While an official Apache Spark doc on unit testing strategy is scarce, various tutorials demonstrate its usefulness. Here is one useful tutorial that focuses on unit testing in PySpark <https://towardsdatascience.com/how-to-unit-test-apache-spark-pyspark-a9b6888ef558>

Question: 177

Deep Dumps

What describes integration testing?

- A. It validates an application use case.
- B. It validates behavior of individual elements of an application,
- C. It requires an automated testing framework.
- D. It validates interactions between subsystems of your application.

Answer: D

Explanation:

Integration testing focuses on verifying the communication and data flow between different components or modules of a software application. It ensures that when these individual units are combined and work together, they function as intended and meet the overall system requirements. Option A, validating an

application use case, aligns more closely with user acceptance testing or end-to-end testing, which assesses the entire workflow from the user's perspective. Option B, validating the behavior of individual elements, describes unit testing, where each component is tested in isolation. While automated testing frameworks (Option C) can be used for integration testing, they aren't a requirement for this type of testing; manual integration testing is also possible, especially in smaller systems. The essence of integration testing lies in confirming that interfaces between different parts of the system are working correctly, ensuring smooth data exchange and proper function calls across module boundaries. This involves testing the interaction between databases, APIs, microservices, and other subsystems. A successful integration test suite identifies issues arising from combined components that wouldn't surface during unit testing, leading to a more robust and reliable final product. In cloud environments, where applications are often distributed and composed of numerous interconnected services, integration testing is crucial for verifying interactions between those services and ensuring they function harmoniously. For more information, refer to the following resources:

Software Testing Fundamentals: <https://www.guru99.com/software-testing.html>

Integration Testing: <https://www.softwaretestinghelp.com/integration-testing/>

Question: 178

Deep Dumps

The Databricks CLI is used to trigger a run of an existing job by passing the `job_id` parameter. The response that the job run request has been submitted successfully includes a field `run_id`.

Which statement describes what the number alongside this field represents?

- A. The `job_id` and number of times the job has been run are concatenated and returned.
- B. The globally unique ID of the newly triggered run.
- C. The number of times the job definition has been run in this workspace.
- D. The `job_id` is returned in this field.

Answer: B

Explanation:

The correct answer is B: The globally unique ID of the newly triggered run. Let's dissect why.

When you use the Databricks CLI to initiate a job run, the system needs a way to track and manage that specific instance of the job. The `run_id` serves precisely this purpose. It's not merely a counter, a derivative of the `job_id`, or the `job_id` itself. Instead, it's a distinct identifier assigned to that particular execution of the job. This is crucial for several reasons:

1. **Uniqueness:** Each job run, even if initiated from the same job definition, is treated as a separate event. The `run_id` ensures each execution is uniquely identifiable within the Databricks environment.
2. **Tracking and Monitoring:** You use the `run_id` to track the status, logs, and results of a specific job execution. Without a unique identifier, it would be impossible to differentiate between different runs of the same job.
3. **Reproducibility and Auditing:** The `run_id` allows you to pinpoint the exact configuration and data used for a particular run, facilitating reproducibility and auditing efforts.
4. **API Interactions:** When interacting with the Databricks API, the `run_id` is essential for targeting specific job runs for actions like canceling a run, retrieving its details, or accessing its output.

Options A, C, and D are therefore incorrect because they do not accurately describe the function of `run_id`. Option A suggests concatenation, which is not how the `run_id` is determined. Option C is incorrect since the `run_id` tracks individual run instances not the job definition. Option D is wrong because `run_id` and `job_id` are different and have unique purposes. The `run_id` identifies the instance of a run, whereas the `job_id` identifies the job itself.

For further research, refer to the official Databricks documentation on Jobs API and related CLI commands:

Databricks Jobs API: <https://docs.databricks.com/api/workspace/jobs>

Databricks CLI: <https://docs.databricks.com/cli/index.html>

Question: 179

Deep Dumps

A Databricks job has been configured with three tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.

What will be the resulting state if tasks A and B complete successfully but task C fails during a scheduled run?

A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.

B. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.

C. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.

D. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.

Answer: A

Explanation:

The correct answer is A: All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully. Here's why:

Databricks jobs execute tasks independently, based on their dependencies. Task A, having no dependencies, runs first and completes successfully, meaning all its defined operations are executed and any writes it performs are committed. Subsequently, tasks B and C, dependent on A, begin execution in parallel. Task B completes successfully, implying its operations are also executed and committed. Crucially, the failure of Task C does not automatically roll back the successful operations of Tasks A and B.

Databricks jobs do not inherently provide an automatic, global transaction rollback mechanism that spans across all tasks in the event of a single task failure unless explicitly coded for in the notebooks themselves. Each task's notebook code runs in its own Spark context and writes its results independently. Task dependency defines the execution order but not necessarily transactional atomicity at the job level. If Task C fails, the Databricks job runner will mark it as failed and report the error. However, the successfully completed operations of Tasks A and B remain committed. Some operations in Task C may have completed successfully before the failure occurred. It will be determined by the point of failure within the notebook.

Option B is incorrect because Databricks does not automatically roll back all changes across all tasks simply because one task failed. Rollback behavior must be explicitly implemented within the notebooks using transactional mechanisms if desired. Option C is wrong because the dependency graph manages execution order, not transactional consistency across the entire graph unless explicitly programmed. Option D is partially correct in stating A and B are successful but incorrectly implies that C's changes are rolled back automatically. Rollback is not automatic; it requires explicit coding.

Here are some resources that further clarify these concepts:

Databricks Jobs: <https://docs.databricks.com/workflows/jobs/index.html> - This documentation explains how Databricks Jobs manage task dependencies and execution.

Spark Transactions (Delta Lake): <https://docs.databricks.com/delta/index.html> - To achieve transactional

behavior, consider using Delta Lake, which provides ACID properties, but the transactional logic needs to be implemented within the notebooks performing the operations. This doesn't happen automatically at the job level based on failure.

Question: 180

Deep Dumps

When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

- A.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- B.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
- C.Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- D.Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1

Answer: A

Explanation:

Here's a detailed justification for why option A is the most appropriate configuration for scheduling production Structured Streaming jobs in Databricks, focusing on automatic recovery and cost efficiency:

Option A (Cluster: New Job Cluster; Retries: Unlimited; Maximum Concurrent Runs: 1) provides a good balance between fault tolerance and cost management. Using a New Job Cluster means that a dedicated cluster is spun up only when the streaming job needs to run. This is more cost-effective than keeping an Existing All-Purpose Cluster running 24/7, even when the streaming job is idle. The ephemeral nature of the job cluster also avoids resource contention with other workloads.

Retries: Unlimited ensures that the streaming job will automatically restart in case of failure (e.g., transient network issues, temporary resource unavailability). This automated recovery is crucial for production systems, minimizing manual intervention. Databricks automatically restarts the cluster in case of failures to maintain data flow.

Limiting Maximum Concurrent Runs: 1 prevents multiple instances of the same streaming job from running simultaneously, which could lead to data duplication and increased costs. This configuration ensures that only one instance of the job is active at any given time, thereby controlling resource usage and avoiding conflicts. Each run uses only resources it needs and gets killed once it finished.

Option B (Cluster: New Job Cluster; Retries: Unlimited; Maximum Concurrent Runs: Unlimited) is less desirable as "Unlimited" concurrent runs can lead to resource exhaustion and conflicts, potentially disrupting the streaming pipeline.

Option C (Cluster: Existing All-Purpose Cluster; Retries: Unlimited; Maximum Concurrent Runs: 1) can be costly because the All-Purpose cluster must be kept running continuously, even when the streaming job isn't actively processing data.

Option D (Cluster: New Job Cluster; Retries: None; Maximum Concurrent Runs: 1) lacks fault tolerance. If the job fails for any reason, it will not be automatically restarted, requiring manual intervention and potentially leading to data loss.

In summary, Option A leverages ephemeral job clusters for cost efficiency and unlimited retries for automatic recovery, which is perfect for production use cases for structured streaming.

[Databricks Documentation on Job Clusters](#)[Databricks Documentation on Jobs](#)

Question: 181

Deep Dumps

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

- A. The table reference in the metastore is updated.
- B. All related files and metadata are dropped and recreated in a single ACID transaction.
- C. The table name change is recorded in the Delta transaction log.
- D. A new Delta transaction log is created for the renamed table.

Answer: A

Explanation:

A. The table reference in the metastore is updated.

ALTER TABLE ... RENAME TO ... on a Delta Lake table is a metadata-only operation.

The physical data files and storage location remain unchanged.

Only the table name in the metastore (Hive Metastore or Unity Catalog) is updated to point to the same underlying data.

Question: 182

Deep Dumps

The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The recent_sensor_recordings table contains an identifying sensor_id alongside the timestamp and temperature for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when mean (temperature) > 120. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

- A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
- B. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query
- C. The source query failed to update properly for three consecutive minutes and then restarted
- D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query

Answer: B

Explanation:

The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query.

Question: 183

Deep Dumps

A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named dev-2.3.9 is not available from the branch selection dropdown.

Which approach will allow this developer to review the current logic for this notebook?

- A. Use Repos to make a pull request use the Databricks REST API to update the current branch to dev-2.3.9
- B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.
- C. Use Repos to checkout the dev-2.3.9 branch and auto-resolve conflicts with the current branch
- D. Use Repos to merge the current branch and the dev-2.3.9 branch, then make a pull request to sync with the remote repository

Answer: B

Explanation:

B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.

In Databricks Repos, if a developer wants to review the current logic for a specific branch (e.g., dev-2.3.9), they need to:

Pull the latest changes from the remote Git repository to make sure their local Repos view is up to date.

Check out the branch they want to work with or review — in this case, dev-2.3.9.

This allows them to view the notebooks and code as they exist in that branch without affecting or merging anything.

Question: 184

Deep Dumps

Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with `dbutils.fs.mount()`.

Which of the following statements is correct?

- A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
- B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
- C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
- D. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

Answer: A

Explanation:

A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.

Databricks File System (DBFS) is an abstraction layer that lets users interact with object storage (like AWS S3, Azure Blob Storage, or GCP Cloud Storage) using file system semantics, such as ls, cp, rm, etc., similar to a Unix file system.

DBFS allows developers and data scientists to access, manage, and use data seamlessly from notebooks and jobs without having to write cloud-storage-specific code.

The interface is familiar: paths like /dbfs/mnt/... are used.

Question: 185

Deep Dumps

An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code:

```
df = spark.read.format("parquet").load(f"/mnt/source/(date)")
```

Which code block should be used to create the date Python variable used in the above code block?

- A. `date = spark.conf.get("date")`
- B. `import sys`
`date = sys.argv[1]`
- C. `date = dbutils.notebooks.getParam("date")`
- D. `dbutils.widgets.text("date", "null")`
`date = dbutils.widgets.get("date")`

Answer: D

Explanation:

In Databricks notebooks, when you want to pass parameters (like date) into a notebook — for example, during job runs or when calling another notebook using `dbutils.notebooks.run()` — you use widgets.

Question: 186

Deep Dumps

The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

- A. "Can Manage" privileges on the required cluster
- B. Cluster creation allowed, "Can Restart" privileges on the required cluster
- C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
- D. "Can Restart" privileges on the required cluster

Answer: D

Explanation:

The correct answer is D: "Can Restart" privileges on the required cluster. Here's why:

The scenario focuses on users needing to use pre-configured, existing interactive clusters that auto-terminate after inactivity, not creating new ones. Therefore, cluster creation permissions (options B and C) are unnecessary.

"Can Manage" (option A) provides broad control over a cluster, including configuration changes, which exceeds the minimal required permissions for simply starting and attaching. The problem statement asks for the minimal set of permissions.

Since the clusters are set to auto-terminate after inactivity, users frequently need to restart them to use them. The "Can Restart" permission is essential for this. Restarting a terminated cluster brings it back to an active state, allowing users to attach and execute workloads.

Once the cluster is running (either after creation or restart), the "Can Attach To" permission is needed. However, by default, users typically have "Can Attach To" if they have "Can Restart." In this specific case, simply granting "Can Restart" handles both requirements.

Therefore, granting "Can Restart" privileges on the designated cluster enables users to bring it back online and subsequently attach to it, meeting the requirement of executing workloads at any time without granting excessive privileges.

Authoritative Links:

Databricks Cluster Permissions: <https://docs.databricks.com/administration/permissions/cluster-permissions.html> - This document details the various cluster permissions available in Databricks and their corresponding functionalities. Pay close attention to the "Can Restart" and "Can Attach To" sections.

Question: 187

Deep Dumps

The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named preds with the schema "customer_id LONG, predictions DOUBLE, date DATE".

```

from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
    "customer_id",
    model(*columns).alias("predictions"),
    current_date().alias("date")
))

```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A. `preds.write.mode("append").saveAsTable("churn_preds")`

B. `preds.write.format("delta").save("/preds/churn_preds")`

```

(preds.writeStream
    .outputMode("append")
    .option("checkpointPath", "/_checkpoints/churn_preds")
    .table("churn_preds")

```

C.)

```

(preds.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("churn_preds")

```

D.)

Answer: A

Explanation:

`preds.write.mode("append").saveAsTable("churn_preds").`

Question: 188

Deep Dumps

The following code has been migrated to a Databricks notebook from a legacy workload:

```

%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data

```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

- A. %sh triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
- B. Instead of cloning, the code should use %sh pip install so that the Python code can get executed in parallel across all nodes in a cluster.
- C. %sh does not distribute file moving operations; the final line of code should be updated to use %fs instead.
- D. %sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Answer: D

Explanation:

D. %sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

In Databricks, the %sh magic command is used to execute shell commands in notebooks. However:

These commands are run only on the driver node, not distributed across the Spark cluster.

As a result, large data processing tasks (like moving, extracting, or transforming ~1 GB of data) done via %sh are bottlenecked by the driver's resources.

This means the operation is not parallelized, and does not benefit from Databricks' distributed computing capabilities, which is why it can take 20+ minutes.

Question: 189

Deep Dumps

A Delta table of weather records is partitioned by date and has the below schema:

date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT

To find all the records from within the Arctic Circle, you execute a query with the below filter:

latitude > 66.3

Which statement describes how the Delta engine identifies which files to load?

- A. All records are cached to an operational database and then the filter is applied
- B. The Parquet file footers are scanned for min and max statistics for the latitude column
- C. The Hive metastore is scanned for min and max statistics for the latitude column
- D. The Delta log is scanned for min and max statistics for the latitude column

Answer: D

Explanation:

The correct answer is D, which states that the Delta log is scanned for min and max statistics for the latitude column to identify which files to load. This process is known as partition pruning. Let's break down why this is the case and why the other options are incorrect.

Delta Lake leverages its transaction log, or Delta log, which meticulously records every change made to the table, including file additions and removals. Importantly, the Delta log also stores metadata about the data, including min/max statistics for each partition column (in this case, latitude).

When a query includes a filter on a partition column, Delta Lake intelligently consults the Delta log to determine which partitions (and therefore which files) could potentially contain data that satisfies the filter condition (latitude > 66.3). By examining the min/max latitude values stored in the Delta log for each partition, Delta Lake can quickly eliminate entire partitions that do not overlap with the specified latitude range. This effectively prunes the search space, reducing the number of files that need to be read, leading to significant performance gains.

Option A is incorrect because caching all records to an operational database before filtering would negate the benefits of partitioning and could be extremely inefficient for large datasets.

Option B is incorrect because while Parquet files do contain footers with min/max statistics, Delta Lake relies on the Delta log for this information, providing a consistent and reliable view of the table's metadata across all versions. Scanning Parquet footers directly would bypass the Delta Lake's transactional guarantees and potentially lead to inconsistencies.

Option C is incorrect because while the Hive metastore stores schema information and partitioning definitions, it doesn't inherently store min/max statistics for the data within the partitions for Delta tables. Delta Lake uses its own log for managing such metadata. While integrated with the Hive metastore for schema information and table location, Delta does not rely on it for partitioning metadata for filtering.

In summary, Delta Lake leverages the metadata stored in its transaction log to perform partition pruning, significantly optimizing query performance by reducing the number of files that need to be accessed. This is fundamental to understanding Delta Lake's query optimization capabilities.

For further research, consult the official Databricks documentation on Delta Lake and partition pruning:

Delta Lake Partitioning: <https://docs.databricks.com/delta/best-practices.html#partition-data>

Delta Lake Data Skipping: <https://docs.databricks.com/delta/data-skipping.html>

Question: 190

Deep Dumps

In order to prevent accidental commits to production data, a senior data engineer has instituted a policy that all development work will reference clones of Delta Lake tables. After testing both DEEP and SHALLOW CLONE, development tables are created using SHALLOW CLONE.

A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly Changing Dimension (SCD) stop working. The transaction logs for the source tables show that VACUUM was run the day before.

Which statement describes why the cloned tables are no longer working?

- A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
- B. Running VACUUM automatically invalidates any shallow clones of a table; DEEP CLONE should always be used when a cloned table will be repeatedly queried.
- C. The data files compacted by VACUUM are not tracked by the cloned metadata; running REFRESH on the cloned table will pull in recent changes.
- D. The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command.

Answer: D

Explanation:

The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command."

Delta Lake supports shallow and deep clones of tables. Here's how they differ:

Shallow Clone:

Copies only the metadata, not the actual data files.

Still references the original data files from the source table.

Deep Clone:

Copies both metadata and physical data files.

The clone is completely independent of the original table.

Question: 191

Deep Dumps

A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint `2.0/jobs/create`.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Assuming that all configurations and referenced resources are available, which statement describes the result of executing this workload three times?

- A. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.
- B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
- C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
- D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.

Answer: C

Explanation:

Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.

Question: 192

Deep Dumps

A Delta Lake table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

- A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and time travel functionality.
- B. Parse the Delta Lake transaction log to identify all newly written data files.
- C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
- D. Execute DESCRIBE HISTORY customer_churn_params to obtain the full operation metrics for the update, including a log of all records that have been added or modified.

Answer: A

Explanation:

Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and time travel functionality.

Delta Lake supports versioning and time travel, which allows you to query a table as it existed at a specific version or timestamp. This is extremely helpful when you want to:

Compare data between two versions.

Audit changes.

Debug data pipeline issues.

Perform rollback operations.

Question: 193

Deep Dumps

A view is registered with the following code:

```
CREATE VIEW recent_orders AS (  
  SELECT a.user_id, a.email, b.order_id, b.order_date  
  FROM  
    (SELECT user_id, email  
     FROM users) a  
    INNER JOIN  
    (SELECT user_id, order_id, order_date  
     FROM orders  
     WHERE order_date >= (current_date() - 7)) b  
    ON a.user_id = b.user_id  
)
```

Both users and orders are Delta Lake tables.

Which statement describes the results of querying recent_orders?

- A. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.

B.All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.

C.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.

D.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Answer: D

Explanation:

All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Question: 194

Deep Dumps

A data engineer is performing a join operation to combine values from a static userLookup table with a streaming DataFrame streamingDF.

Which code block attempts to perform an invalid stream-static join?

A.userLookup.join(streamingDF, ["user_id"], how="right")

B.streamingDF.join(userLookup, ["user_id"], how="inner")

C.userLookup.join(streamingDF, ["user_id"], how="inner")

D.userLookup.join(streamingDF, ["user_id"], how="left")

Answer: D

Explanation:

Here's a detailed justification of why option D is the correct answer, explaining why stream-static joins in Spark Structured Streaming have limitations, particularly with left joins, and providing links to official documentation.

The core concept here is the limitation of stream-static joins within Spark Structured Streaming. Spark Structured Streaming allows joining a static DataFrame with a streaming DataFrame, but it enforces specific constraints to guarantee correct and efficient incremental processing of the streaming data. A crucial restriction is that the streaming DataFrame must be on the left side of the join. This restriction exists because the streaming DataFrame represents an unbounded stream of data that is continuously arriving.

When performing a left join where the static DataFrame is on the left, the streaming DataFrame acts as the right side. To determine all possible matches for each row in the static DataFrame, the streaming DataFrame would need to be fully scanned. This implies storing the entire history of the streaming data to handle late-arriving data or updates, violating the principles of continuous, incremental processing that underpin Structured Streaming. Such a full scan would require keeping track of all possible future streaming data, which is impossible and defeats the purpose of streaming.

Options A, B, and C place the streaming DataFrame on the left side of the join, which is permitted. Option A uses a right join, but since the streaming DataFrame is on the right, it's implicitly treating the static DataFrame as the driving side, which avoids the problem of needing to retain all streaming data. Options B and C use an inner join and place the streaming DataFrame on the left, making them valid within the constraints of Structured Streaming.

Option D, `userLookup.join(streamingDF, ["user_id"], how="left")`, is invalid because it attempts a left join where

the static userLookup DataFrame is on the left and the streaming streamingDF is on the right. This requires Spark to retain potentially unbounded information about the stream to determine all possible matches for each row in userLookup, violating the streaming processing model.

Therefore, only option D represents an invalid stream-static join within Spark Structured Streaming. The other options are valid or, more precisely, don't violate the specific stream-static join limitations regarding join order in Structured Streaming.

Authoritative Links:

Spark Structured Streaming Documentation - Joins: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#joins> This is the primary source for understanding join operations within Structured Streaming.

Databricks Documentation - Stream-Static Joins: <https://docs.databricks.com/structured-streaming/stream-static-joins.html> Databricks documentation provides a concise explanation of stream-static joins.

Question: 195

Deep Dumps

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df._____  
    .groupBy(  
        window("event_time", "5 minutes").alias("time"),  
        "device_id"  
    )  
    .agg(  
        avg("temp").alias("avg_temp"),  
        avg("humidity").alias("avg_humidity")  
    )  
    .writeStream  
    .format("delta")  
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

- A.withWatermark("event_time", "10 minutes")
- B.awaitArrival("event_time", "10 minutes")
- C.await("event_time + '10 minutes'")
- D.slidingWindow("event_time", "10 minutes")

Answer: A

Explanation:

`withWatermark("event_time", "10 minutes")`

Question: 196

Deep Dumps

A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:

```
./bronze
├── _checkpoint
├── _delta_log
├── year_week=2020_01
├── year_week=2020_02
└── ...
```

Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

- A.No; Delta Lake manages streaming checkpoints in the transaction log.
- B.Yes; both of the streams can share a single checkpoint directory.
- C.No; only one stream can write to a Delta Lake table.
- D.No; each of the streams needs to have its own checkpoint directory.

Answer: D

Explanation:

No; each of the streams needs to have its own checkpoint directory.

Question: 197

Deep Dumps

A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.

Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

- A.Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
- B.Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.
- C.The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
- D.Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.

Answer: B

Explanation:

The correct answer is B: "Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill."

Here's a detailed justification:

The primary problem is that microbatches during peak hours are exceeding the desired processing time of 10 seconds. This indicates that the current trigger interval of 10 seconds is insufficient to handle the increased data volume during peak periods. A longer processing time for a microbatch means more data accumulates before the next batch starts processing.

Option B directly addresses this issue. By decreasing the trigger interval to 5 seconds, the Structured Streaming job will initiate processing more frequently. This reduces the amount of data accumulated in each microbatch. Smaller batches are generally processed faster, mitigating the risk of individual batches exceeding the 10-second processing limit. Smaller batches also prevent large data backups that may cause spilling issues to disk and hence, improve performance.

Option A suggests that idle executors will help with processing, but it is not the core problem. Even with idle executors, large batches will still cause performance issues.

Option C is incorrect. The trigger interval can be modified without impacting stream state or requiring changes to the checkpoint directory. The checkpoint directory stores metadata about the stream's progress, not information tightly coupled to a specific trigger interval. Increasing the number of shuffle partitions might help with parallelism but is not the primary solution for handling data accumulation.

Option D is also incorrect. Using trigger once and scheduling a job is not ideal for continuous data processing. trigger once processes the available data and then stops. The streaming nature is lost as it only runs and computes once. Thus, it will require additional orchestration, which is not cost-effective, and might still be delayed if the accumulated data exceeds processing capabilities.

In summary, decreasing the trigger interval helps to process data in smaller, more manageable batches, preventing delays and ensuring that records are processed within the required timeframe.

Refer to the following resources for further information:

1. **Apache Spark Structured Streaming Programming Guide:** <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> - Explains the fundamentals of Structured Streaming, including trigger intervals.
2. **Databricks Documentation on Structured Streaming:** <https://docs.databricks.com/structured-streaming/> - Provides comprehensive information on using Structured Streaming within the Databricks environment.

Question: 198

Deep Dumps

Which statement describes the default execution mode for Databricks Auto Loader?

- A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.
- B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
- C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.

D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.

Answer: D

Explanation:

The correct answer is D because it accurately describes Auto Loader's default execution mode. Auto Loader, by default, operates by discovering new files through directory listing. It systematically examines the input directory to identify files that haven't been processed yet. These newly discovered files are then incrementally loaded into the target Delta Lake table. The loading process is designed to be idempotent, meaning that even if a file is processed multiple times, the result in the Delta Lake table remains consistent and avoids duplication.

Option A is incorrect because while Auto Loader can be configured to use cloud vendor-specific queue storage and notification services (like AWS SQS/SNS or Azure Event Grid), this is not the default behavior. The default is directory listing.

Option B is incorrect because Auto Loader doesn't materialize the entire target table by directly querying all valid files in the source directory for each update. It incrementally loads new files. While an initial load might involve reading existing files, subsequent operations primarily focus on newly arrived data.

Option C is incorrect because while webhooks can be used in conjunction with Databricks jobs, they are not part of Auto Loader's default behavior. The default mechanism for finding new files is directory listing, not event-triggered execution via webhooks.

In summary, Auto Loader's default mode prioritizes simplicity and ease of use, relying on directory listing to detect new files. This approach allows for a quick and easy setup, especially when cloud-based notification services are not readily available or desired. The incremental and idempotent loading ensure data consistency and prevent data duplication in the target Delta Lake table.

Relevant documentation:

[What is Auto Loader?](#) (Refer to the sections describing file discovery and incremental loading.)

Question: 199

Deep Dumps

Which statement describes the correct use of `pyspark.sql.functions.broadcast`?

- A. It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.
- B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
- C. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.
- D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.

Answer: D

Explanation:

D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.

`pyspark.sql.functions.broadcast()` is a join hint. When you apply it to a DataFrame, you're telling Spark that this DataFrame is small enough to be replicated (broadcast) to all executors. Spark can then perform a broadcast hash join, avoiding an expensive shuffle of the larger table.

Question: 200

Deep Dumps

Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

- A.Stage's detail screen and Query's detail screen
- B.Stage's detail screen and Executor's log files
- C.Driver's and Executor's log files
- D.Executor's detail screen and Executor's log files

Answer: B

Explanation:

The correct answer is B, Stage's detail screen and Executor's log files. Here's a detailed justification:

When Spark executes wide transformations (e.g., joins, aggregations, shuffles), it may need to spill data to disk if the data exceeds the available memory of the executors. Identifying these spills is crucial for performance tuning.

The **Stage's detail screen** in the Spark UI provides aggregated metrics about the stage's execution, including the amount of data spilled to disk. By examining metrics like "Disk Bytes Spilled", you can quickly see if a stage is experiencing spillover, indicating that the amount of data being processed is exceeding the memory capacity of the executors during the stage. You can look at these metrics on the stage detail page of the Spark UI.

However, the Stage's detail screen only presents a high-level overview. To gain more granular insights into which executors are experiencing spills and potential causes, you need to examine the **Executor's log files**. These logs often contain messages related to memory usage and disk spilling. Looking for log messages containing phrases like "spilling", "disk persistence", or "out of memory" can provide valuable information. You can use the logs to track the specific executor that is spilling to disk and also the size of disk persistence involved.

Option A, Stage's detail screen and Query's detail screen, is incorrect because the Query detail screen primarily focuses on the logical and physical execution plans of the query. While it shows a breakdown of stages, it doesn't directly provide information on individual executor spills.

Option C, Driver's and Executor's log files, is partly correct since Executor logs are relevant. However, the driver logs generally do not provide detailed information about disk spilling within executors. The primary responsibility of the driver is orchestrating the execution, not managing the memory of individual tasks.

Option D, Executor's detail screen and Executor's log files, is partially correct but not as comprehensive as option B. While the executor's detail page shows metrics for individual executors, the Stage's detail provides a better overall view and summary of spillover metrics across all executors involved in that stage. Using both the aggregate view of a stage and the specific logs of the executor provides more complete information.

In summary, the Stage's detail screen provides a high-level indication of spillover, and the Executor's log files offer granular details to pinpoint the source and cause of the spilling.

Here's a reference on how to monitor Spark jobs using the Spark UI, including information on Stages and Executors:

[Spark Monitoring and Instrumentation: Spark UI](#)

Question: 201**Deep Dumps**

An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the date variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order.

If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

- A. Each write to the orders table will only contain unique records, and only those records without duplicates in the target table will be written.
- B. Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.
- C. Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.
- D. Each write to the orders table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

Answer: C**Explanation:**

C. Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.

When `customer_id` and `order_id` serve as a composite key to uniquely identify each order, any duplicate entries for a single order will be identified by their composite key. If the upstream system occasionally produces duplicates, the correct behavior is to overwrite the existing record in the target table with the new record having the same composite key. This ensures that the orders table always contains the most up-to-date information for each unique order, while maintaining the uniqueness constraint imposed by the composite key.

Question: 202**Deep Dumps**

A junior data engineer on your team has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
    INSERT *
```

The view new_events contains a batch of records with the same schema as the events Delta table. The event_id field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same event_id as an existing record?

- A.They are merged.
- B.They are ignored.
- C.They are updated.
- D.They are inserted.

Answer: B

Explanation:

B. They are ignored .

Then the query likely involves an INSERT operation with the "IGNORE" option or an INSERT OVERWRITE where duplicates are not allowed.

Delta Lake does not enforce primary key constraints by default, so if you execute a simple INSERT INTO, duplicates would be inserted.

However, if the query is using MERGE INTO with the WHEN NOT MATCHED clause, or an INSERT with an ignore mechanism, then duplicate records (same event_id as existing records) would be ignored.

Question: 203

Deep Dumps

A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

- A.The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
- B.Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
- C.Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
- D.Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

Answer: D

Explanation:

The correct answer, D, highlights Delta Lake's capability to create a persistent, replayable history of data by ingesting raw data from Kafka into a bronze Delta table. This approach directly addresses the data loss

scenario described.

Here's why this works and why the other options are less suitable:

Bronze Layer as Raw Data Repository: A bronze layer in a data lake architecture acts as the initial ingestion point for data. It's designed to land data in its rawest form, without significant transformation or cleaning. In this case, landing the raw Kafka data, including metadata, into a bronze Delta table ensures that a complete record of the original data is preserved.

Replayability: Because the bronze table contains the raw data, you can always replay the data ingestion process from this point. Even if a downstream pipeline misses a field initially, you can reprocess the data in the bronze table to include the missing field. This is crucial when issues are discovered after the Kafka retention period.

Auditing and Lineage: The bronze table provides a valuable audit trail. You can track the original state of the data and understand how it has been transformed through the pipeline.

Delta Lake's Versioning: Delta Lake's transaction log and versioning capabilities make this replayability feasible. You can revert to a specific version of the bronze table, ensuring consistency.

Why the other options are less suitable:

A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer: The Delta log tracks changes to the Delta table itself, not the Kafka producer's entire history. Structured Streaming checkpoints manage the state of the streaming job, but they don't inherently store the complete raw data history of the Kafka stream. While these are important, they don't solve the specific problem of raw data loss.

B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source: Delta Lake schema evolution primarily deals with adding new columns and managing schema changes to the Delta table. It can't magically calculate missing data from a source it never ingested in the first place. If the data wasn't ingested to begin with, schema evolution is irrelevant.

C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer: Delta Lake doesn't enforce schema validation at the ingestion stage in the sense that it will reject data if fields are missing from the target table. While schema enforcement is possible, it must be configured; it isn't automatic by default.

In Summary: Capturing the raw Kafka data into a bronze Delta table is the proactive approach that provides the most robust protection against data loss in the described scenario. By preserving the original data state, the data engineer can always reprocess it if necessary.

Authoritative Links:

Delta Lake Architecture: <https://docs.databricks.com/delta/index.html>

Data Lake Architecture: <https://www.databricks.com/glossary/data-lakehouse> (Lakehouse architecture, including the bronze/silver/gold layers concept)

Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Question: 204

Deep Dumps

The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
 .groupBy("customer_id")
 .agg(
     F.min("sale_date").alias("first_transaction_date"),
     F.max("sale_date").alias("last_transaction_date"),
     F.mean("sale_total").alias("average_sales"),
     F.countDistinct("order_id").alias("total_orders"),
     F.sum("sale_total").alias("lifetime_value")
 ).write
 .mode("overwrite")
 .table("gold_customer_lifetime_sales_summary")
)
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. The silver_customer_sales table will be overwritten by aggregated values calculated from all records in the gold_customer_lifetime_sales_summary table as a batch job.
- B. A batch job will update the gold_customer_lifetime_sales_summary table, replacing only those rows that have different values than the current version of the table, using customer_id as the primary key.
- C. The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.
- D. An incremental job will detect if new rows have been written to the silver_customer_sales table; if new rows are detected, all aggregates will be recalculated and used to overwrite the gold_customer_lifetime_sales_summary table.

Answer: C

Explanation:

The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.

Question: 205

Deep Dumps

The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels.

The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

- A. Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.
- B. Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.
- C. Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.
- D. Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.

Answer: A

Explanation:

The correct answer is A because it aligns with best practices for data governance, security, and collaboration in a Lakehouse architecture, particularly when dealing with tiered data quality and sensitive information.

Isolating tables into separate databases (bronze, silver, gold) provides a logical separation of data based on its quality and intended use. This separation enables granular access control using database ACLs (Access Control Lists). Each database can have different permissions, allowing you to grant access to the bronze database only to data engineering teams, the silver database to data engineering and machine learning teams, and the gold database to business intelligence and reporting teams. This minimizes the risk of unauthorized access to sensitive data.

Physical separation of storage locations for managed tables, configured at the database level, allows for better cost management and compliance. Bronze data might be stored in cheaper, less-performant storage, while gold data might require higher performance storage for faster query execution. This segregation contributes to enhanced data security and auditability.

Option B is incorrect because databases in Databricks are more than just logical constructs. They are fundamental for organizing data assets, managing permissions, and controlling storage locations. Therefore, database organization significantly impacts security and discoverability.

Option C is flawed because storing all production tables in a single database, while simplifying discoverability with broad "view" privileges, compromises security. Granting wide access to a single database containing all production data exposes sensitive information and violates the principle of least privilege.

Option D is incorrect because using the default Databricks database does not provide the greatest security. It is generally recommended to create specific databases for different purposes and apply appropriate security measures to those databases. Storing data in the DBFS root is often discouraged for production workloads due to governance and security concerns.

In summary, isolating data by quality tier into separate databases and leveraging database ACLs, along with physical storage segregation, is the most secure and manageable approach for implementing a bronze-silver-gold architecture in Databricks, especially when handling PII and promoting collaboration among different teams.

Relevant links:

Databricks Unity Catalog: <https://www.databricks.com/product/unity-catalog> (Focuses on data governance and security)

Databricks Access Control Lists (ACLs): <https://docs.databricks.com/security/access-control/index.html>

Databricks Lakehouse Architecture: <https://www.databricks.com/glossary/what-is-data-lakehouse>

"unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. When a database is being created, make sure that the LOCATION keyword is used.
- B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
- C. When data is saved to a table, make sure that a full file path is specified alongside the USING DELTA clause.
- D. When tables are created, make sure that the UNMANAGED keyword is used in the CREATE TABLE statement.

Answer: A

Explanation:

A. When a database is being created, make sure that the LOCATION keyword is used.

In Databricks / Delta Lake:

Creating a database (schema) with a LOCATION ensures that all tables created in that database are external (unmanaged) by default.

The data resides in user-managed cloud object storage, and Databricks does not manage the data lifecycle.

This is the only option that enforces the requirement consistently across all tables, without relying on individual developers to remember table-level settings.

Question: 207

Deep Dumps

To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.

The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.

Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

- A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
- B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
- C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
- D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most suitable solution, along with supporting concepts and links:

Option B offers the most balanced approach to addressing the new requirements while minimizing disruption and maintaining a streamlined data architecture. By creating a new table with the necessary field renames

and additions for the customer-facing application, it isolates the impact of the changes. This avoids directly altering the existing aggregate table, which is used by multiple teams and applications.

To prevent breaking existing queries, a view is created. This view maintains the original table name and schema by aliasing the relevant fields from the new table. This ensures that other teams and applications using the original table name can continue to function without modification. The view acts as a compatibility layer.

This approach adheres to best practices for data governance and change management. Modifying a widely used table directly can lead to unexpected issues and data quality problems across multiple downstream systems. Creating a view mitigates this risk. It also avoids the data duplication and increased storage costs associated with option C (using Delta Lake's deep clone). Option A requires external logic to be used by other teams, which is not scalable or desirable. Option D introduces overhead on view creation.

Delta Lake concepts such as schema evolution and versioning are relevant here. However, the key advantage of option B is that it minimizes the reliance on these features for resolving the specific problem, thus simplifying the overall solution.

Here's why the other options are less ideal:

Option A: Sending users notice and providing the logic to revert the new schema puts the burden of managing the change on each individual user. This approach is prone to errors, increases support overhead, and can lead to inconsistent data usage.

Option C: Using Delta Lake's deep clone functionality introduces unnecessary data duplication. This increases storage costs and requires additional management overhead to ensure the two tables remain synchronized.

Option D: Replacing the current table with a view may impact performance, as the view's underlying query may need to be executed every time the table is accessed.

Supporting Links:

Views: <https://spark.apache.org/docs/latest/sql-ref-syntax-ddl-create-view.html>

Delta Lake Schema Evolution: <https://docs.databricks.com/delta/delta-batch.html#schema-evolution>

Delta Lake Deep Clone: <https://docs.databricks.com/delta/clone.html>

In summary, option B strikes the best balance between meeting the new requirements, minimizing disruption, and optimizing data architecture and cost. The use of a view provides a clean and effective way to maintain compatibility with existing systems.

Question: 208

Deep Dumps

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

- A.post_time
- B.date
- C.post_id
- D.user_id

Answer: B

Explanation:

The best candidate for partitioning the Delta Lake table is the date column. Partitioning is a crucial optimization technique for Delta Lake, allowing queries to skip irrelevant data files, significantly improving performance, especially when querying large datasets. The date column is a suitable choice for several reasons.

First, the date column has relatively low cardinality compared to other columns like `user_id` or `post_id`. Low cardinality implies a smaller number of distinct values, which prevents the creation of an excessive number of small partitions, a scenario known as the "small file problem." This problem can degrade performance due to the overhead of managing numerous files.

Second, queries are highly likely to filter data based on date ranges. Consider scenarios like analyzing daily post activity, generating reports for specific dates, or training machine learning models on time-windowed data. Partitioning by date allows Delta Lake to efficiently locate the relevant data files based on the query's date filters, bypassing the need to scan the entire table.

Third, using `post_time` directly could lead to too many partitions as the granularity is at the timestamp level, resulting in the small file problem. While `user_id` could be used, it depends on the query patterns and the number of unique users. If queries rarely filter by specific users, partitioning by `user_id` may not be beneficial. Similarly, `post_id` is a unique identifier for each post, leading to extremely high cardinality and an unmanageable number of partitions.

Partitioning by date aligns with common data warehousing practices and offers a balance between data granularity and query performance. It improves query speed, reduces I/O operations, and makes the table more efficient to manage.

For further reading, refer to the official Databricks documentation on Delta Lake partitioning:

Databricks - Partition data: <https://docs.databricks.com/delta/optimizations/partitioning.html>

Delta Lake: High Cardinality Partitioning: <https://medium.com/towards-data-science/delta-lake-high-cardinality-partitioning-eb27858ab98a>

Question: 209

Deep Dumps

The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid latitude and longitude values in the `activity_details` table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add CHECK constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

A. The current table schema does not contain the field `valid_coordinates`; schema evolution will need to be enabled before altering the table to add a constraint.

- B.The activity_details table already exists; CHECK constraints can only be added during initial table creation.
- C.The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.
- D.The activity_details table already contains records; CHECK constraints can only be added prior to inserting values into a table.

Answer: C

Explanation:

C. The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.

When you add a CHECK constraint to an existing table (like activity_details), Delta Lake (and many other database systems) requires that:

All existing rows in the table must satisfy the new constraint, or the ALTER TABLE operation will fail.

This ensures data consistency and avoids invalid data persisting alongside new constraints.

Question: 210

Deep Dumps

What is true for Delta Lake?

- A.Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
- B.Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
- C.Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
- D.Z-order can only be applied to numeric values stored in Delta Lake tables.

Answer: C

Explanation:

The correct answer is C. Let's break down why, and why the other options are incorrect:

Why C is correct: Delta Lake does indeed automatically collect statistics on the first 32 columns of each table. These statistics (min, max, null counts, etc.) are crucial for data skipping. When a query includes a filter (e.g., WHERE column_name > 10), Delta Lake uses these statistics to efficiently determine which data files within the Delta table need to be scanned. If a data file's statistics show that the column values within that file are all less than or equal to 10, the entire file is skipped, significantly improving query performance.

Why A is incorrect: Views in a data lakehouse, while offering a logical abstraction over the underlying data, do not maintain a persistent cache of the most recent versions of source tables. Views are defined as queries, and their results are computed dynamically when the view is queried. While some systems might employ caching strategies, this is not an inherent characteristic of views in the Lakehouse concept, nor is it explicitly managed by Delta Lake. Views provide a simplified, secure and unified layer, but they rely on the underlying data freshness, not cached copies.

Why B is incorrect: Delta Lake supports defining primary and foreign key constraints. However, Delta Lake, by itself, does not enforce these constraints to prevent duplicate values from being inserted. It provides the mechanism to define them and potentially check them (using features like CHECK constraints or custom validation logic), but it doesn't automatically enforce uniqueness at write time. Enforcement requires additional mechanisms like adding validation logic in the write process or using merge operations with

appropriate conditions. The constraints are more for metadata and potentially for optimization than for real-time prevention of duplicates.

Why D is incorrect: Z-ordering (also known as space-filling curve ordering or multi-dimensional clustering) is a data optimization technique used in Delta Lake to improve query performance by colocating related data on disk. While Z-ordering is particularly effective on numeric values with good cardinality, it is not exclusively limited to numeric data types. It can also be applied to string or other data types after appropriate transformations (e.g., converting string to numerical representation). The key is to have attributes that when ordered using Z-order, meaningfully group together records that are frequently queried together.

Supporting Links:

Delta Lake Statistics and Data Skipping: <https://docs.databricks.com/delta/data-skipping.html>

Delta Lake Constraints: <https://docs.databricks.com/delta/constraints.html>

Delta Lake Z-Ordering: <https://docs.databricks.com/delta/optimizations/z-order.html>

Question: 211

Deep Dumps

The view updates represents an incremental batch of all newly ingested data to be inserted or updated in the customers table.

The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_ey, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
null)
```

Which statement describes this implementation?

- A. The customers table is implemented as a Type 2 table; old values are overwritten and new customers are appended.
- B. The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.
- C. The customers table is implemented as a Type 0 table; all writes are append only with no changes to existing values.
- D. The customers table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.

Answer: B

Explanation:

B. The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

SCD Type 2 is used when you want to track historical changes to a record.

When a change occurs:

The existing row is preserved, but marked as no longer current (often using an `is_current` flag or an `end_date`).

A new row is inserted with the updated values and a new `start_date`.

This allows you to retain a full history of customer data over time.

Question: 212

Deep Dumps

A team of data engineers are adding tables to a DLT pipeline that contain repetitive expectations for many of the same data quality checks. One member of the team suggests reusing these data quality rules across all tables defined for this pipeline.

What approach would allow them to do this?

- A. Add data quality constraints to tables in this pipeline using an external job with access to pipeline configuration files.
- B. Use global Python variables to make expectations visible across DLT notebooks included in the same pipeline.
- C. Maintain data quality rules in a separate Databricks notebook that each DLT notebook or file can import as a library.
- D. Maintain data quality rules in a Delta table outside of this pipeline's target schema, providing the schema name as a pipeline parameter.

Answer: C

Explanation:

The correct answer is C because it promotes modularity and reusability, key principles in software and data engineering. Storing data quality rules in a separate notebook and importing it as a library allows multiple DLT notebooks within the same pipeline to access and utilize the same expectations without code duplication. This significantly improves maintainability and reduces the risk of inconsistencies across different tables. Changes to the data quality rules only need to be made in one central location (the library notebook), and all pipelines that import the library will automatically inherit the updates.

Option A is not ideal because accessing pipeline configuration files and using an external job to apply constraints increases complexity. It adds an unnecessary layer of indirection, making the process harder to manage and debug. It also separates the definition of the data quality rules from the DLT pipeline definition, reducing clarity.

Option B, using global Python variables, is generally discouraged as it can lead to namespace pollution and unintended side effects, especially in larger projects. It also reduces the clarity of the code, making it harder to understand where the expectations are defined. The scope and lifetime of these variables might cause problems.

Option D, storing rules in a Delta table and providing the schema name as a parameter, is not suited for this scenario. This approach would require reading data quality rules from a table during pipeline execution, which is less efficient than directly importing them as code. Furthermore, data quality rules are typically expressed as code (e.g., Python functions with expect decorators in DLT), not as data. Using a Delta table for storing code would complicate version control and management of the rules. In essence, storing data quality rules as data within a delta table and then interpreting them is not a practical approach.

In conclusion, creating a central library notebook promotes code reusability, enhances maintainability, and ensures consistency in data quality checks across the DLT pipeline.<https://docs.databricks.com/delta-live-tables/index.html><https://docs.databricks.com/notebooks/index.html>

Question: 213

Deep Dumps

The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.

What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

- A.Can manage
- B.Can edit
- C.Can run
- D.Can read

Answer: D

Explanation:

D, "Can Read." Here's why:

The core objective is to grant access for reviewing production logic without risking unintended modifications to the code or the resulting data. The principle of least privilege dictates granting only the minimum necessary permissions to achieve the task.

A. Can Manage: "Manage" permissions provide the user with complete control over the notebook. They can edit, run, delete, change permissions, and essentially do anything with the notebook. This clearly violates the requirement to prevent accidental changes to production code.

B. Can Edit: "Edit" permissions, as the name suggests, allow the user to modify the notebook's content. This directly contradicts the requirement to avoid accidental code changes. Granting "Edit" access presents a high risk of inadvertently altering the production logic, which is unacceptable.

C. No permissions: This would prevent the new hire from reviewing the code, making it impossible for them to onboard or understand the production logic, therefore, failing to meet the user's requirements.

D. Can Read: "Read" permissions allow the user to view the notebook's contents, including the code and any markdown explanations. They cannot modify the notebook in any way. This is the ideal permission level for code review as it satisfies the requirement to review the production logic while preventing any accidental changes to production notebooks and data.

E. Can Run: "Run" permissions allows the user to execute the notebook. While they cannot change the code, running the production notebook could inadvertently trigger data processing or modifications within the environment, which can lead to unintended side effects in the production system. Since the requirement is only to review the code, running the notebook is not necessary and adds unnecessary risk.

Therefore, "Can Read" is the most restrictive permission level that still allows the user to accomplish the requested task of reviewing the notebook content. This aligns with the principle of least privilege and safeguards the production environment.

Relevant Documentation:

Databricks Notebook Permissions: While specific documentation directly detailing the impacts of each

permission level on Jobs is limited, general Databricks ACL and Permissions documentation can give insight. Searching the Databricks documentation for "Access Control Lists" or "ACLs" and "Notebook Permissions" will provide related information.

Question: 214

Deep Dumps

A table named `user_ltv` is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The `user_ltv` table has the following schema:

`email` STRING, `age` INT, `ltv` INT

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
    is_member('marketing') THEN email
    ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv -
```

Which statement describes the results returned by this query?

- A. Three columns will be returned, but one column will be named "REDACTED" and contain only null values.
- B. Only the email and ltv columns will be returned; the email column will contain all null values.
- C. The email and ltv columns will be returned with the values in `user_ltv`.
- D. Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

Answer: D

Explanation:

D. Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

The query likely selects only the email and ltv columns.

The email column is overwritten or replaced by the literal string "REDACTED" in each returned row.

The ltv column contains the actual values from the data (e.g., from a table or subquery called `user_ltv`).

Question: 215**Deep Dumps**

The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PII) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property "contains_pii" = true.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

- A.DESCRIBE EXTENDED dev.pii_test
- B.DESCRIBE DETAIL dev.pii_test
- C.SHOW TBLPROPERTIES dev.pii_test
- D.DESCRIBE HISTORY dev.pii_test

Answer: A**Explanation:**

A. DESCRIBE EXTENDED dev.pii_test

Shows detailed table metadata: schema, location, and some properties.

Good for seeing current schema and configuration info.

Does not show the full transaction history or commit details.

Question: 216**Deep Dumps**

The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named users.

```
DELETE FROM users
WHERE user_id IN
(SELECT user_id FROM delete_requests)
```

Assuming that user_id is a unique identifying key and that delete_requests contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

- A.Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- B.No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

C.Yes; the Delta cache immediately updates to reflect the latest data files recorded to disk.

D.No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.

Answer: B

Explanation:

B. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

When you delete records in Delta Lake, the DELETE command logically removes the records and updates the transaction log, so they no longer appear in normal queries.

However, the actual data files containing those records are not immediately removed.

Delta Lake retains these old files to support time travel (querying older snapshots).

These files are only physically deleted when you run the VACUUM command, which cleans up files no longer referenced by any active snapshots.

Until VACUUM is run, deleted data files remain accessible if someone queries using time travel.

Question: 217

Deep Dumps

The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;  
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group and that these users are not workspace administrators, which statement describes their privileges?

A.Group members are able to create, query, and modify all tables and views in the prod database, but cannot define custom functions.

B.Group members are able to list all tables in the prod database but are not able to see the results of any queries on those tables.

C.Group members are able to query and modify all tables and views in the prod database, but cannot create new tables or views.

D.Group members are able to query all tables and views in the prod database, but cannot create or edit anything in the database.

Answer: D

Explanation:

The answer is D because it accurately reflects the privileges granted by the provided SQL statements. Let's break down why:

GRANT USAGE ON DATABASE prod TO eng; This statement grants the eng group the ability to access the prod database. Without USAGE privilege, users cannot even access the database or any of its contents. The USAGE privilege is a prerequisite for any further actions within the database.

GRANT SELECT ON DATABASE prod TO eng; This statement grants the eng group the ability to query data from tables and views within the prod database. SELECT permission allows users to read data, but it does not permit data modification (e.g., INSERT, UPDATE, DELETE), creation of new tables or views, or alteration of existing tables.

Given these two grants, the eng group can see the table structure in the database and execute SELECT queries to retrieve data from those tables. However, they lack privileges for:

Creation: They cannot create new tables or views because they don't have CREATE privilege.

Modification: They cannot modify data in existing tables or views because they don't have INSERT, UPDATE, or DELETE privileges.

Alteration: They cannot alter the structure of existing tables or views (e.g., add columns) because they don't have ALTER privilege.

Therefore, the only permitted action is querying data, making option D the correct answer: "Group members are able to query all tables and views in the prod database, but cannot create or edit anything in the database."

Here's why the other options are incorrect:

A: Incorrect because the group cannot create or modify anything. The USAGE and SELECT privileges don't grant CREATE, INSERT, UPDATE, or DELETE privileges.

B: Incorrect because the SELECT permission on the database allows seeing the results of queries, not just listing the tables.

C: Incorrect because the group can query, but not modify, due to the lack of INSERT, UPDATE, and DELETE privileges.

Authoritative Links:

Databricks SQL Privileges and Securable Objects: <https://docs.databricks.com/security/access-control/sql-privileges/index.html>

Question: 218

Deep Dumps

A user wants to use DLT expectations to validate that a derived table report contains all records from the source, included in the table validation_copy.

The user attempts and fails to accomplish this by adding an expectation to the report table definition.

```
CREATE LIVE TABLE report (  
    CONSTRAINT no_missing_records EXPECT (key IN validation_copy)  
)  
AS SELECT <...>
```

Which approach would allow using DLT expectations to validate all expected records are present in this table?

- A. Define a temporary table that performs a left outer join on validation_copy and report, and define an expectation that no report key values are null
- B. Define a SQL UDF that performs a left outer join on two tables, and check if this returns null values for report key values in a DLT expectation for the report table
- C. Define a view that performs a left outer join on validation_copy and report, and reference this view in DLT expectations for the report table
- D. Define a function that performs a left outer join on validation_copy and report, and check against the result in

a DLT expectation for the report table

Answer: A

Explanation:

Define a temporary table that performs a left outer join on validation_copy and report, and define an expectation that no report key values are null.

Question: 219

Deep Dumps

A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using display() calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.

Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

- A. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.
- B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.
- C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.
- D. Calling display() forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

Answer: B

Explanation:

The provided answer, **B**, is the most accurate for measuring production-like performance in Databricks during development, even though other options have some merit. Here's why:

Production-like Environment is Key: The primary reason for slow execution times in production often stems from the sheer scale of data and the configuration of the cluster. Development environments typically use smaller datasets and less powerful clusters. This means that execution times observed during cell-by-cell execution are unlikely to represent how the code will behave with large-scale data and resources.

Data Size Impact: Larger datasets result in more data shuffling and processing, which highlights bottlenecks and inefficiencies that are not apparent with smaller sample datasets.

Cluster Configuration Matters: Production clusters are often configured with specific driver/worker node types, autoscaling rules, and Spark configurations to optimize for performance and cost. These configurations significantly impact how code executes.

"Run All" Simulates Production Workflow: Running all cells sequentially mimics the typical flow of a data pipeline in a production environment. This allows you to observe how transformations interact and how caching (or lack thereof) affects performance across the entire pipeline.

Why Other Options are Less Effective:

A: While running a notebook as a job through the Jobs UI is useful, Photon is typically enabled by default in Databricks runtimes and is available across different use cases, including interactive clusters, not solely for

scheduled jobs. This option doesn't directly address the core issue of using production-sized data and clusters.

C: Using a local build of open-source Spark and Delta Lake can be helpful for unit testing, but it won't accurately reflect the Databricks environment's nuances (optimizations, managed services, specific configurations, etc.). Databricks has its own optimized Spark distribution, which makes direct comparison misleading.

D: The `display()` function undoubtedly triggers a job execution, and caching can influence repeated executions. However, this option doesn't offer a solution to obtain a more accurate production performance measurement.

In conclusion, to accurately measure production performance during development, you must run your Databricks notebooks with realistic data sizes on clusters that are configured similarly to your production environment. Using "Run All" provides a more holistic and accurate view than cell-by-cell executions.

Relevant Links:

Databricks Performance Optimization: <https://www.databricks.com/solutions/performance-optimization>

Photon Engine: <https://www.databricks.com/product/photon>

Spark Configuration: <https://spark.apache.org/docs/latest/configuration.html>

Question: 220

Deep Dumps

Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

- A. In the Executor's log file, by grepping for "predicate push-down"
- B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
- C. In the Query Detail screen, by interpreting the Physical Plan
- D. In the Delta Lake transaction log, by noting the column statistics

Answer: C

Explanation:

The correct answer is C, diagnosing predicate pushdown issues in the Spark UI via the Query Detail screen by interpreting the Physical Plan. Here's why:

Predicate pushdown is a crucial optimization technique that pushes filtering operations closer to the data source, reducing the amount of data that needs to be read and processed. A failure to leverage predicate pushdown leads to increased I/O and data shuffling, resulting in slower query execution.

The Spark UI provides several avenues for performance analysis. Option A (Executor logs) might contain relevant information, but it's less targeted and more cumbersome to analyze than the physical plan. Option B (Stage Details) provides aggregated data size read information, indicating excessive I/O, but not why the I/O is excessive; it doesn't pinpoint predicate pushdown failures directly. Option D (Delta Lake transaction log) provides information about data changes and metadata, but doesn't give a detailed view of the execution plan.

The **Query Detail screen** shows the physical plan, which outlines the sequence of operations Spark performs to execute the query. By examining the physical plan, you can:

1. Identify whether filtering operations (Filter or Project nodes) are located near the data source (Scan node). If the Filter is located higher up in the plan, significantly after the Scan, it indicates that data was read before being filtered, signifying a potential lack of predicate pushdown.
2. Look for full table scans when only a subset of data is needed. If the plan shows a full table scan when a filter should have reduced the scanned data, that suggests a problem.

3. Assess whether data sources (e.g., Parquet files, Delta tables) support predicate pushdown and if it's being used.

Thus, the Query Detail screen and the Physical Plan it displays provide the most direct and insightful means of diagnosing predicate pushdown problems in Spark. By examining the plan, one can visually determine if the filtering operations are occurring at the optimal stage.

Relevant resources for more information include:

Apache Spark Documentation: <https://spark.apache.org/docs/latest/> (Focus on sections related to query optimization and the Spark UI.)

Databricks Documentation on Query Optimization: <https://www.databricks.com/> (Search for "query optimization," "predicate pushdown," and "Spark UI.")

Question: 221

Deep Dumps

A data engineer needs to capture pipeline settings from an existing setting in the workspace, and use them to create and version a JSON file to create a new pipeline.

Which command should the data engineer enter in a web terminal configured with the Databricks CLI?

- A. Use list pipelines to get the specs for all pipelines; get the pipeline spec from the returned results; parse and use this to create a pipeline
- B. Stop the existing pipeline; use the returned settings in a reset command
- C. Use the get command to capture the settings for the existing pipeline; remove the pipeline_id and rename the pipeline; use this in a create command
- D. Use the clone command to create a copy of an existing pipeline; use the get JSON command to get the pipeline definition; save this to git

Answer: C

Explanation:

Here's a detailed justification for why option C is the best approach for capturing pipeline settings and creating a new, versioned pipeline using the Databricks CLI:

Option C proposes a workflow that directly addresses the need to capture, modify, and reuse pipeline settings. It leverages the get command to extract the configuration of an existing pipeline. This allows the data engineer to obtain a JSON representation of the pipeline's settings, which can then be saved and versioned. Before using these settings to create a new pipeline, the pipeline_id needs to be removed because it's unique to the original pipeline. Also renaming the pipeline ensures a unique identifier. Finally, the modified JSON can be used with the create command to instantiate a new pipeline. This approach allows for controlled modifications and avoids relying on implicit or potentially inaccurate data.

Option A, while listing pipelines, doesn't directly give a detailed configuration for a specific pipeline suitable for recreating it. Listing only provides an overview.

Option B, stopping and resetting, focuses more on manipulating an existing pipeline rather than creating a new one based on the configuration of another. "Reset" operations in data pipelines typically involve clearing data or restarting from a known state, not capturing configuration settings.

Option D, cloning is conceptually related, but the suggested use of get JSON after cloning isn't a standard or necessary step. The direct get command approach is more efficient for extracting the configuration. The intent of a clone is to usually copy and run another instance of the same pipeline rather than alter the existing pipeline configuration.

Therefore, extracting the configuration, modifying (removing the ID and renaming), and creating a new pipeline is the most direct and controlled method, making option C the best choice.

Here are relevant links for further research:

Databricks CLI Documentation: <https://docs.databricks.com/en/dev-tools/cli/>

Databricks REST API (Pipelines): <https://docs.databricks.com/api/workspace/pipelines> (The CLI often wraps REST API calls.)

Question: 222

Deep Dumps

Which Python variable contains a list of directories to be searched when trying to locate required modules?

- A.`importlib.resource_path`
- B.`sys.path`
- C.`os.path`
- D.`pypi.path`

Answer: B

Explanation:

The correct answer is **B. `sys.path`**.

`sys.path` is a Python variable that holds a list of directory paths that the interpreter searches when importing modules. When you use the `import` statement, Python iterates through the directories listed in `sys.path` to find the requested module. The order of the directories in `sys.path` is significant; Python searches them in the order they appear in the list. Typically, the first entry is the directory of the script being executed, followed by standard library paths and other user-defined paths.

Option A, `importlib.resource_path`, is used for accessing resources (like data files) contained within a package, not for module search paths.

Option C, `os.path`, is a module containing functions for manipulating pathnames, but it does not store the module search paths. While `os.path` helps construct paths that can be added to `sys.path`, it doesn't inherently contain the list of directories Python uses for imports.

Option D, `pypi.path`, is incorrect as `pypi` (Python Package Index) is the repository for Python packages, not a module within the Python standard library, and it doesn't have a path attribute directly related to module search. `Pip`, the package installer for Python, interacts with `PyPI` but doesn't directly modify or reveal the `sys.path`. The location of installed packages is added to `sys.path` during installation.

In cloud environments like Databricks, understanding `sys.path` is crucial because custom libraries or modules installed as part of a cluster's initialization script or through Databricks libraries management need to be accessible by the Python code running in notebooks or jobs. If a module is not found, it likely means its installation directory is not present in `sys.path`. You can inspect and modify `sys.path` within a Databricks notebook to ensure that custom modules are found during import. This might involve adding the path to the directory where you've placed custom modules to `sys.path` using `sys.path.append('/path/to/my/modules')`.

Authoritative links:

Python documentation on the `sys` module: <https://docs.python.org/3/library/sys.html>

Python documentation on the import system: <https://docs.python.org/3/reference/import.html>

Question: 223**Deep Dumps**

You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test would the above line exemplify?

- A. Unit
- B. Manual
- C. Functional
- D. Integration

Answer: A**Explanation:**

Here's a detailed justification for why the provided code snippet exemplifies a unit test:

The core of unit testing is isolating and verifying the behavior of individual, small components of code. In this case, `myIntegrate(lambda x: xx, 0, 3)` represents a single unit of code – the *myIntegrate* function that calculates the area under a curve. The *lambda x: xx* part further isolates the test to a specific curve (x squared).

The assert statement directly checks if the `myIntegrate` function returns the expected result (9) when given the specified input. This assertion acts as a mini-specification for the function's intended behavior. There's no involvement of external systems, databases, or other modules within this test. It focuses solely on the `myIntegrate` function's internal logic.

The scope is deliberately narrow. If the assertion fails, it immediately points to an issue within the `myIntegrate` function itself. The use of a lambda function to define the curve dynamically also contributes to the flexibility and isolatability that unit tests provide. It's designed to be fast and run frequently as developers build and modify the `myIntegrate` function, ensuring it consistently produces the correct output. The function doesn't require any complex environment setup or mocking other systems, making it a classic example of a unit test.

For further research, consider these resources:

1. **Unit Testing:**
2. Wikipedia: https://en.wikipedia.org/wiki/Unit_testing
3. Microsoft Documentation: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>
4. **Python's assert statement:**
5. Python Documentation: https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement

Question: 224**Deep Dumps**

What is a key benefit of an end-to-end test?

- A. It makes it easier to automate your test suite
- B. It pinpoints errors in the building blocks of your application
- C. It provides testing coverage for all code paths and branches
- D. It closely simulates real world usage of your application

Answer: D

Explanation:

The correct answer is D: It closely simulates real-world usage of your application.

End-to-end (E2E) tests, also known as integration tests or system tests, validate the entire application flow from the user interface (UI) or entry point through all underlying layers and components, including databases, third-party services, and APIs. This holistic approach mimics how a user would interact with the application in a production environment, providing a high level of confidence that the system functions correctly under realistic conditions.

While options A, B, and C have some relevance to testing in general, they are not the key benefit of E2E tests. E2E tests can be more complex to automate than unit tests (contradicting A), and they are not designed to pinpoint errors in individual building blocks like unit tests do (contradicting B). Option C, comprehensive code coverage, is more closely associated with unit testing or a combination of testing strategies, rather than being the primary advantage of E2E tests.

The strength of E2E tests lies in their ability to verify that all parts of the application work together as intended. By mimicking real user workflows, E2E tests can uncover integration issues, data inconsistencies, performance bottlenecks, and other problems that might not be apparent from unit or component tests alone. This comprehensive validation ensures that the application meets user expectations and business requirements, reducing the risk of production failures. For example, in a Databricks-based data pipeline, an E2E test would verify that data ingested from a source system is correctly processed through transformations, stored in the target data lake, and accessible by downstream applications – simulating an entire real-world data journey.

Therefore, while other testing types are essential, the defining advantage of an end-to-end test is that it validates the complete application functionality by simulating real-world user scenarios and interactions.

Relevant links:

1. [End-to-end testing - Wikipedia](#)
2. [What is End-to-End Testing?: A Practical Guide - BrowserStack](#)
3. [End-to-End Tests | Cypress Documentation](#)

Question: 225

Deep Dumps

Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

- A. /jobs/runs/list
- B. /jobs/list
- C. /jobs/runs/get
- D. /jobs/get

Answer: D

Explanation:

The correct answer is D, /jobs/get. This REST API endpoint is specifically designed to retrieve detailed information about a Databricks job. Within the returned JSON structure from /jobs/get, you can find the tasks array. Each element within this array represents a task in the job, and for notebook tasks, you'll find details such as the notebook_task specification including the notebook_path. Therefore, /jobs/get directly provides the information needed to review the notebooks associated with tasks in a multi-task job.

Option A, `/jobs/runs/list`, is used to list the runs of a job, not the job's configuration itself. It provides information on past executions, including status and start/end times. Option B, `/jobs/list`, lists all jobs in a workspace but does not provide detailed configuration information about the tasks within each job. You'd still need to call `/jobs/get` for each job to see the task details. Option C, `/jobs/runs/get`, retrieves information about a specific job run, not the job's configuration or the notebooks associated with its tasks. It focuses on the details and metrics of a single execution.

Therefore, only `/jobs/get` gives you direct access to the tasks configuration where notebook paths for tasks are listed, making it the optimal choice for reviewing configured notebooks.

Further Reading: [Databricks Jobs API Reference](#)

Question: 226

Deep Dumps

A Data Engineer wants to run unit tests using common Python testing frameworks on Python functions defined across several Databricks notebooks currently used in production.

How can the data engineer run unit tests against functions that work with data in production?

- A. Define and import unit test functions from a separate Databricks notebook
- B. Define and unit test functions using Files in Repos
- C. Run unit tests against non-production data that closely mirrors production
- D. Define unit tests and functions within the same notebook

Answer: B

Explanation:

The most appropriate answer is **C. Run unit tests against non-production data that closely mirrors production**. Here's a detailed justification:

Running unit tests directly against production data is highly discouraged due to the risk of data corruption, performance degradation, and unexpected application behavior. Unit tests should be isolated and repeatable, and modifying production data defeats this purpose.

Option A, defining and importing unit test functions from a separate Databricks notebook, is a valid technique for organizing code, but it doesn't address the core issue of testing against production data. It simply separates the test code from the functions being tested but still leaves open the possibility of accidentally manipulating production data within the imported functions.

Option B, defining and unit test functions using Files in Repos, primarily provides version control and collaboration features. It doesn't inherently solve the problem of avoiding direct interaction with production data during unit testing. While helpful for development, it's not the primary solution.

Option C is the most appropriate because it advocates for testing against a **non-production data environment** that closely resembles the production environment. This ensures that the tests are relevant and realistic, exercising the functions with data similar to what they would encounter in production, but without the risk of impacting the actual production system. This minimizes the chance of unintended side effects.

By utilizing a mirror environment, Data Engineers can write robust unit tests that target the specific logic and behavior of functions working with the data structures. The data in the test environment will be used as a mock environment that allows to properly unit test functions without impacting the Production environment. This ensures that the code behaves as expected under realistic conditions before deployment.

Consider using tools and techniques like:

1. **Data Cloning:** Create a point-in-time copy of your production data.
2. **Data Masking/Anonymization:** Protect sensitive data in the non-production environment.
3. **Data Subsetting:** Use a representative sample of the production data.

Therefore, running unit tests against a closely mirrored non-production environment ensures the safety and integrity of the production data while allowing for robust testing of data-processing functions.

Question: 227

Deep Dumps

A data engineer wants to refactor the following DLT code, which includes multiple table definitions with very similar code.

```
@dlt.table(name=f"t1_dataset")
def t1_dataset():
    return spark.read.table(t1)

@dlt.table(name=f"t2_dataset")
def t2_dataset():
    return spark.read.table(t2)

@dlt.table(name=f"t3_dataset")
def t3_dataset():
    return spark.read.table(t3)

...
```

In an attempt to programmatically create these tables using a parameterized table definition, the data engineer writes the following code.

```
tables = ["t1", "t2", "t3"]

for t in tables:
    @dlt.table(name=f"{t}_dataset")
    def new_table():
        return spark.read.table(t)
```

The pipeline runs an update with this refactored code, but generates a different DAG showing incorrect configuration values for these tables.

How can the data engineer fix this?

- A.Wrap the for loop inside another table definition, using generalized names and properties to replace with those from the inner table definition.
- B.Convert the list of configuration values to a dictionary of table settings, using table names as keys.

C. Move the table definition into a separate function, and make calls to this function using different input parameters inside the for loop.

D. Load the configuration values for these tables from a separate file, located at a path provided by a pipeline parameter.

Answer: C

Explanation:

Move the table definition into a separate function, and make calls to this function using different input parameters inside the for loop.

Question: 228

Deep Dumps

A data engineer has created a 'transactions' Delta table on Databricks that should be used by the analytics team. The analytics team wants to use the table with another tool which requires Apache Iceberg format.

What should the data engineer do?

- A. Require the analytics team to use a tool which supports Delta table.
- B. Create an Iceberg copy of the 'transactions' Delta table which can be used by the analytics team.
- C. Convert the 'transactions' Delta to Iceberg and enable uniform so that the table can be read as a Delta table.
- D. Enable uniform on the transactions table to 'iceberg' so that the table can be read as an Iceberg table.

Answer: D

Explanation:

D is correct because enabling the table's uniform format to "iceberg" lets Databricks expose the same underlying storage and transactional state through Iceberg metadata without duplicating data or altering producer semantics.

Reasoning: enabling uniform format is a metadata-level interoperability feature that maps the Delta table's committed state to an Iceberg-compatible view, preserving ACID semantics, avoiding a full data copy, and maintaining a single source of truth for governance, lineage, and access controls; this minimizes operational risk, latency, and storage costs compared with creating a parallel dataset. From an exam perspective, the correct solution should satisfy three constraints simultaneously: (1) let the analytics tool use Iceberg natively, (2) avoid data duplication or conversion-induced downtime, and (3) preserve existing Delta producers and governance — uniform format satisfies all three. Technically, the feature exposes table metadata and manifests in an Iceberg-compatible layout while leaving the authoritative Delta transactional log intact for writers; consumers using Iceberg clients read the same committed snapshots without breaking Delta workflows.

Why the other options are incorrect:

A — Forcing the analytics team to change tools is operationally unrealistic and fails the exam requirement to enable native Iceberg access without disrupting consumers.

B — Creating an Iceberg copy produces redundant storage, introduces synchronization and freshness challenges, and doubles governance surface area, which violates maintainability and single-source-of-truth best practices.

C — Converting Delta to Iceberg is destructive/irreversible in many workflows, can disrupt Delta-based producers and governance, and contradicts the goal of providing Iceberg access while preserving the Delta table for existing consumers.

Question: 229

Deep Dumps

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream for highly selective joins on a number of fields, and will also be leveraged by the machine learning team to filter on a handful of relevant fields. In total, 15 fields have been identified that will often be used for filter and join logic.

The data engineer is trying to determine the best approach for dealing with these nested fields before declaring the table schema.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. Because Delta Lake uses Parquet for data storage, Dremel encoding information for nesting can be directly referenced by the Delta transaction log.
- B. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
- C. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- D. By default, Delta Lake collects statistics on the first 32 columns in a table; these statistics are leveraged for data skipping when executing selective queries.

Answer: D

Explanation:

D is correct because Delta Lake stores and uses column-level file statistics (min/max/null counts) for data skipping and, by default, collects those statistics only for the first 32 columns, which directly impacts selectivity and query pruning.

Reasoning: Delta's transaction log records per-file column statistics that the query engine leverages to skip files during predicate evaluation; the practical consequence is that only statistics on a limited set of leading columns (32 by default) are automatically available for that pruning, so choosing which 15 frequently-filtered fields appear among those columns (or using Z-ordering or explicit statistics collection) materially affects read I/O and join/selectivity performance. The default 32-column behavior therefore guides schema design: promote frequently-filtered/joined fields to early physical column positions or use explicit optimization (optimize zorder, vacuum, or compute statistics) to restore effective skipping. This explanation relies on how Delta persists file-level stats and how the optimizer uses them, not on any inferred semantics of nested JSON.

Why the other options are incorrect:

A is incorrect: Delta uses Parquet storage with Parquet's definition/repetition levels for nested structures and stores file-level column statistics in the Delta log; there is no concept of "Dremel encoding" being directly recorded in the Delta transaction log for use in pruning.

B is incorrect: schema inference and evolution are convenience features but are not guarantees of consistent downstream types — inference can misclassify fields, and evolution requires deliberate policies (enforce or allow changes, explicit merges or casts) to ensure downstream compatibility.

C is incorrect: Tungsten refers to Spark's execution and memory/code-generation optimizations (binary row format, off-heap memory, codegen), not a string-specific storage optimization; keeping data as JSON strings prevents Parquet's columnar encoding and predicate pushdown, so parsing into typed columns or structured Parquet is usually far more efficient for selective queries.

References: <https://docs.delta.io/latest/delta-optimizations.html#data-skipping><https://docs.delta.io/latest/delta-schema-management.html#schema-enforcement-and-evolution><https://parquet.apache.org/documentation/latest/>

Question: 230

Deep Dumps

A platform engineer is creating catalogs and schemas for the development team to use.

The engineer has created an initial catalog, Catalog_A, and initial schema, Schema_A. The engineer has also granted USE CATALOG, USE SCHEMA, and CREATE TABLE to the development team so that the engineer can begin populating the schema with new tables.

Despite being owner of the catalog and schema, the engineer noticed that they do not have access to the underlying tables in Schema_A.

What explains the engineer's lack of access to the underlying tables?

- A. The owner of the schema does not automatically have permission to tables within the schema, but can grant them to themselves at any point.
- B. Users granted with USE CATALOG can modify the owner's permissions to downstream tables.
- C. Permissions explicitly given by the table creator are the only way the Platform Engineer could access the underlying tables in their schema.
- D. The platform engineer needs to execute a REFRESH statement as the table permissions did not automatically update for owners.

Answer: A

Explanation:

A is correct because Unity Catalog enforces independent object-level privileges: owning a catalog or schema does not automatically grant access to child tables.

Unity Catalog models catalogs, schemas, and tables as separate securable objects where privileges are evaluated at the object being accessed rather than inherited from parent scopes. When a user creates a table they become the table owner and that table-level ownership and privileges govern SELECT, MODIFY, and GRANT on the table. Consequently, schema ownership confers namespace and CREATE rights but does not bestow implicit table-level access to tables created by other principals. The engineer can obtain access only by receiving explicit table privileges from the table owner or via administrative actions (transfer of ownership or grant operations) permitted by higher-level admin rights.

Why the other options are incorrect (structured):

- B — Incorrect: USE CATALOG only permits entering the catalog namespace and does not include rights to alter or override table-level permissions; modifying downstream object privileges requires explicit privileges on those objects or administrative authority.
- C — Incorrect: While table creators commonly grant access, platform administrators or principals with appropriate ownership/management privileges can change ownership or grant table privileges without relying solely on the original creator.
- D — Incorrect: REFRESH affects metadata caching and visibility of metadata changes, it does not change access control or create privileges on objects.

References: <https://docs.databricks.com/security/unity-catalog/access-control.html><https://docs.databricks.com/security/unity-catalog/permissions.html><https://docs.databricks.com/security/unity-catalog/grant-revoke-privileges.html>

Question: 231**Deep Dumps**

A data engineer has created a new cluster using shared access mode with default configurations. The data engineer needs to allow the development team access to view the driver logs if needed.

What are the minimal cluster permissions that allow the development team to accomplish this?

- A. CAN VIEW
- B. CAN RESTART
- C. CAN ATTACH TO
- D. CAN MANAGE

Answer: D

Explanation:

D is correct because CAN MANAGE is the only cluster ACL that grants the full cluster metadata and operational control necessary to expose driver logs in shared access mode.

Reasoning: in shared access mode driver runtime artifacts and aggregated driver logs are treated as cluster-level sensitive outputs; Databricks implements a permission matrix where CAN VIEW provides metadata-only visibility, CAN ATTACH TO grants execution context for notebooks but not cluster-level diagnostics, and CAN RESTART allows a single operational action without exposing internal logs. CAN MANAGE, by design, is the elevated permission that includes the ability to view, download, and configure cluster diagnostics and logging endpoints, which is required to access driver logs when clusters are shared. From a least-privilege and auditability perspective, driver logs reveal runtime state and potentially tenant-specific data; therefore Databricks restricts them to users who are explicitly authorized to manage the cluster rather than merely attach or view.

Why other options are incorrect:

- A. CAN VIEW — allows inspection of cluster metadata and status but does not grant access to cluster diagnostic artifacts or download privileges for driver logs.
- B. CAN RESTART — authorizes a restart operation only and does not expand visibility into cluster internals or log retrieval capabilities.
- C. CAN ATTACH TO — permits attaching notebooks and submitting commands to the cluster execution context but intentionally does not provide authority to retrieve cluster-managed logs or change cluster logging configuration.

References <https://docs.databricks.com/security/access-control/cluster-acl.html> <https://docs.databricks.com/clusters/cluster-access-modes.html> <https://docs.databricks.com/clusters/troubleshooting/driver-logs.html>

Question: 232**Deep Dumps**

A data engineer wants to create a cluster using the Databricks CLI for a big ETL pipeline. The cluster should have five workers and one driver of type i3.xlarge and should use the '14.3.x-scala2.12' runtime.

Which command should the data engineer use?

- A. `databricks compute add 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster`
- B. `databricks clusters create 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster`

C. databricks compute create 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster

D. databricks clusters add 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster

Answer: B

Explanation:

B is correct because the Databricks CLI organizes resources under the "clusters" namespace and uses the "create" action to provision clusters with flags such as --num-workers, --node-type-id, and --cluster-name that match the intent of the command shown.

Reasoning: the CLI's canonical resource/action pattern is databricks clusters create, which maps directly to the Clusters REST API create endpoint and supports cluster configuration flags required for programmatic ETL cluster creation; this makes B the syntactically and semantically appropriate choice for creating a cluster. Using the clusters resource ensures the command invokes the managed cluster lifecycle operations (create/start/terminate) rather than a non-existent or different resource namespace. The flags shown (--num-workers, --node-type-id, --cluster-name) correspond to standard cluster configuration parameters exposed by the CLI and API, so the structure of B aligns with documented usage for automated cluster provisioning. From an exam and production perspective, choosing the correct resource/action pair (clusters + create) is the decisive criterion: it guarantees the CLI call will be routed to the cluster creation API rather than failing due to an invalid subcommand or resource.

Why the other options are incorrect: A. Uses "compute add" — "compute" is not the clusters resource in the Databricks CLI and "add" is not the documented action for cluster lifecycle; this combination will not invoke the clusters create endpoint.

C. Uses "compute create" — although "create" is the right action name, the top-level namespace must be "clusters"; "compute" is not the proper CLI group for Databricks cluster operations.

D. Uses "clusters add" — while the namespace is correct, "add" is not a supported cluster action; the CLI expects "create" for provisioning, so this will fail or be rejected.

References: <https://docs.databricks.com/dev-tools/cli/clusters.html> <https://docs.databricks.com/dev-tools/cli/index.html> <https://docs.databricks.com/dev-tools/api/latest/clusters.html>

Question: 233

Deep Dumps

A 'transactions' table has been liquid clustered on the columns 'product_id', 'user_id' and 'event_date'.

Which operation lacks support for cluster on write?

- A. CTAS and RTAS statements
- B. `spark.writeStream.format('delta').mode('append')`
- C. `spark.write.format('delta').mode('append')`
- D. INSERT INTO operations

Answer: B

Explanation:

B is correct because streaming Delta writers (`spark.writeStream.format('delta').mode('append')`) do not support the transactional file-rewrite semantics required for cluster-on-write.

Cluster-on-write requires the writer to deterministically reorganize and atomically commit files according to

the table's clustering specification during the same transaction that adds data; that capability is provided by batch CTAS/RTAS, INSERT INTO, and batch DataFrame writes which perform a single transactional commit and can perform file selection/rewrites as part of that commit. Structured streaming's continuous/micro-batch append semantics and its checkpoint-driven incremental commits prevent the single-transaction file-rewrite and deterministic layout guarantees that cluster-on-write depends on; supporting cluster-on-write would require streaming writes to block, buffer and rewrite stream data across commits, which violates streaming throughput/latency and correctness expectations. Consequently, Databricks documents and implementation notes state cluster-on-write is not supported for writeStream append writers while it is available for CTAS/RTAS, batch DataFrame writes, and INSERT INTO table operations.

Why the other options are incorrect:

A. CTAS and RTAS statements: these execute as single batch DDL/DML operations with a transactional commit; they can invoke the clustering-on-write logic at table creation/replacement time, so they are supported.

C. `spark.write.format('delta').mode('append')`: batch `DataFrameWriter` append is a single-transaction batch write that can participate in cluster-on-write file layout and therefore is supported.

D. INSERT INTO operations: INSERT INTO is a transactional table-level operation that runs as a batch commit and can apply cluster-on-write during the insert transaction, so it is supported.

References <https://docs.databricks.com/delta/optimizations/liquid.html> <https://docs.databricks.com/delta/delta-streaming.html> <https://docs.databricks.com/spark/latest/spark-sql/language-manual/sql-ref-syntax-ddl-create-table-as-select.html>

Question: 234

Deep Dumps

The data governance team has instituted a requirement that the "user" table containing Personal Identifiable Information (PII) must have the appropriate masking on the SSN column. This means that anyone outside of the HRAdminGroup should see masked social security numbers as `***-**-****`.

The team created a masking function:

```
CREATE FUNCTION ssn_mask(ssn STRING)
  RETURN CASE WHEN is_member(HRAdminGroup) THEN ssn ELSE '***-**-****' END;
```

What does the data governance team need to do next to achieve this goal?

- A. CREATE TABLE users -
(name STRING);
ALTER TABLE users CREATE COLUMN ssn CREATE MASK ssn_mask;
- B. CREATE TABLE users -
(name STRING, int STRING);
ALTER TABLE users ALTER COLUMN ssn CREATE MASK if is_member('HRAdminGroup');
- C. CREATE TABLE users -
(name STRING, ssn INT MASKED ssn_mask);
- D. CREATE TABLE users -
(name STRING, ssn STRING);
ALTER TABLE users ALTER COLUMN ssn SET MASK ssn_mask;

Answer: D

Question: 235

Deep Dumps

A data engineer needs to create an application that will collect information about the latest job run including the repair history.

How should the data engineer format the request?

- A. Call `/api/2.1/jobs/runs/list` with the `run_id` and `include_history` parameters
- B. Call `/api/2.1/jobs/runs/get` with the `run_id` and `include_history` parameters
- C. Call `/api/2.1/jobs/runs/get` with the `job_id` and `include_history` parameters
- D. Call `/api/2.1/jobs/runs/list` with the `job_id` and `include_history` parameters

Answer: B

Explanation:

Because the `jobs/runs/get` endpoint is designed to return the full metadata for a single run (addressable by `run_id`) and supports the `include_history` flag to return repair/attempt history, option B is correct.

Reasoning: `jobs/runs/get` is the targeted retrieval call for an individual run; supplying `run_id` identifies the specific execution instance and `include_history` instructs the API to expand its repair/attempt history into the response, which is required to collect the latest run details plus repair history. Using `run_id` is semantically precise because repair history is tied to a particular run instance rather than the job-level list. The `runs/list` endpoint is intended for enumerating runs for a job (aggregation/pagination) and is not the direct mechanism to fetch a single run's expanded history payload efficiently.

Why the other options are incorrect:

A: Incorrect — `jobs/runs/list` is for enumerating runs and takes `job_id` to list executions; it is not the single-run retrieval endpoint and does not provide the same compact single-run + repair-history semantics as `runs/get` with `run_id`.

C: Incorrect — `jobs/runs/get` requires `run_id` to identify a run; passing `job_id` to `runs/get` is a parameter mismatch and will not retrieve the intended run history.

D: Incorrect — `jobs/runs/list` with `job_id` returns a list of runs for a job (not the detailed, single-run repair history payload that `include_history` on `runs/get` returns), making it the wrong choice for retrieving the latest run plus repair history.

References: <https://docs.databricks.com/dev-tools/api/2.1/jobs.html#runs-get> <https://docs.databricks.com/dev-tools/api/2.1/jobs.html#runs-list> <https://docs.databricks.com/dev-tools/api/latest/index.html>

Question: 236

Deep Dumps

A data engineer is working in an interactive notebook with many transformations before outputting the result from `display(df.collect())`. The notebook includes wide transformations and a cross join.

The data engineer is getting the following error: "The spark driver has stopped unexpectedly and is restarting. Your notebook will be automatically reattached."

Which action should the data engineer take?

- A. Run the notebook on a single node cluster to keep driver from falling.
- B. Rewrite their code to avoid putting memory pressure on the driver node.
- C. Check into the Spark UI to see how many jobs are assigned to each stage as they are employing fewer executors.
- D. Look at the compute metrics UI to see if the executors have higher than 90% memory utilization.

Answer: B

Explanation:

Why B is correct: Rewriting the code to avoid placing large objects into the driver (i.e., avoid `collect()`-ing the full result set) prevents driver heap exhaustion caused by wide transformations and cross joins.

Rewriting is the correct remediation because `collect()` materializes the entire result set in the driver JVM; wide shuffles and a cross join dramatically enlarge the working set and can exceed driver heap, causing a restart. The proper fix is to keep heavy operations distributed (push aggregations and joins into the cluster, write results to distributed storage, use `show()/take(n)/limit` for inspection, or increase parallelism and use partition-aware operations) rather than forcing full materialization on the driver. This is a structural, algorithmic solution that addresses the root cause (driver memory pressure) rather than masking symptoms.

Critical evaluation of alternatives:

A. Run on a single node cluster — incorrect: collapsing driver and executors onto one node concentrates the full cluster working set into one JVM and typically worsens memory pressure; it does not address the underlying algorithmic cause.

C. Check Spark UI for jobs per stage — incorrect as a primary action: the Spark UI is diagnostic and may reveal skew or executor issues, but it does not prevent driver-side OOM when the application explicitly collects large results.

D. Inspect executor memory >90% — partially useful for debugging, but insufficient: high executor memory is an executor-side concern; the observed driver restart implicates driver heap overflow from `collect()`, so executor metrics alone will not resolve the driver failure.

References: <https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions> <https://spark.apache.org/docs/latest/tuning.html> <https://spark.apache.org/docs/latest/api/python>

Question: 237

Deep Dumps

An analytics team wants run an experiment in the short term on the customer transaction Delta table (with 20 billions records) created by the data engineering team in Databricks SQL.

Which strategy should the data engineering team use to ensure minimal downtime and no impact on the ongoing ETL processes?

- A. Deep clone the table for the analytics team.
- B. Create a new table for the analytics team using a CTAS statement.
- C. Shallow clone the table for the analytics team.
- D. Give access to the table for the analytics team.

Answer: C

Explanation:

C is correct because a shallow clone instantaneously creates an isolated metadata snapshot that references the original table's data files (no data copy), enabling fast, low-cost experiments with zero downtime and no interruption to ongoing ETL.

A shallow clone works by copying table metadata and pointers to existing data files rather than physically duplicating 20 billion records, so it completes in seconds/minutes and uses minimal additional storage; this provides a consistent, point-in-time snapshot for analytics while the source table continues to be updated under Delta's transactional snapshot isolation, avoiding locks or ETL pauses. Shallow clones also give the analytics team an independent namespace to run schema/metadata changes or query experiments without altering the production table metadata, but they do require operational awareness because the clone shares

underlying files (so VACUUM/retention policies on the source must be respected).

Critical evaluation of other options:

A. Deep clone the table for the analytics team — Incorrect: deep clone physically copies all data files, incurring massive I/O, storage cost, and long copy time for 20B records; that creates potential resource contention and effective downtime for the copy operation and increases operational complexity.

B. Create a new table for the analytics team using a CTAS statement — Incorrect: CTAS forces a full table scan and write-out of all rows into new files, which is resource- and time-intensive (same drawbacks as deep clone), can contend with ETL workloads, and does not provide the near-instant, low-cost snapshot isolation that a shallow clone offers.

D. Give access to the table for the analytics team — Incorrect: while read permissions allow direct queries, they do not provide an isolated snapshot or separate metadata space; heavy experimental queries run directly against the production table can cause resource contention, risk accidental metadata changes, and do not protect the experiment from subsequent production file GC or compaction effects.

References <https://docs.databricks.com/delta/delta-clone.html> <https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-as-select.html> <https://docs.databricks.com/delta/transaction-log.html>

Question: 238

Deep Dumps

A data team is working to optimize an existing large, fast-growing table 'orders' with high cardinality columns, which experiences significant data skew and requires frequent concurrent writes. The team notice that the columns 'user_id', 'event_timestamp' and 'product_id' are heavily used in analytical queries and filters, although those keys may be subject to change in the future due to different business requirements.

Which partitioning strategy should the team choose to optimize the table for immediate data skipping, incremental management over time, and flexibility?

- A. Partition the table with: ALTER TABLE orders PARTITION BY user_id, product_id, event_timestamp
- B. Use z-order after partiitiing the table: OPTIMIZE orders ZORDER BY (user_id, product_id) WHERE event_timestamp = current date () - 1 DAY
- C. Cluster the table with: ALTER TABLE orders CLUSTER BY user_id, product_id, event_timestamp
- D. Z-order the table with OPTIMIZE orders ZORDER BY (user_id, product_id, event_timestamp)

Answer: C

Explanation:

C is correct because CLUSTER BY co-locates high-cardinality keys into balanced file groups (reducing skew and hot spots) while enabling incremental reclustering and data-skipping without exploding partition directories.

Reasoning: CLUSTER BY hashes and groups rows into file-level clusters rather than creating many directory partitions, which (1) mitigates metadata and small-file overhead under high cardinality, (2) balances concurrent write targets to reduce contention, and (3) produces file-level min/max statistics that support immediate data skipping for the heavily queried columns; because clustering is a table-layout property it can be incrementally maintained (via targeted OPTIMIZE operations) and changed over time with bounded rewrite cost, giving future flexibility when business keys evolve.

Why the other options are incorrect:

A (Partition by user_id, product_id, event_timestamp): creates an enormous number of partitions for high-cardinality keys, causing metadata explosion, many tiny files, write hotspots and poor concurrent-write behavior.

B (OPTIMIZE ... ZORDER BY with a single-day WHERE): z-ordering is an expensive, read-optimization rewrite that must be reapplied to maintain locality; limiting it to one day leaves the rest of the table unoptimized and does not address write contention or skew across the entire dataset.

D (OPTIMIZE ... ZORDER BY all three columns): full-table z-ordering improves multi-dimensional locality for reads but is costly to maintain on a fast-growing table and does not prevent partition/file-level skew or improve concurrent write distribution the way CLUSTER BY does.

References: <https://docs.databricks.com/delta/optimizations/partitioning.html> <https://docs.databricks.com/delta/optimizer.html> <https://docs.databricks.com/delta/optimizations/optimize.html>

Question: 239

Deep Dumps

A faulty IoT sensor in a factory reports a temperature of -500, causing the LDP pipeline to fail the expectation, which only allows values between -100 and 200 degrees Celsius. The data engineer would like to further analyze the faulty data to better understand the reason behind this.

How should the data engineer resolve the faulty data while ensuring data quality standards are maintained?

- A. Remove all expectations from the pipeline to prevent any future failures, regardless of data quality.
- B. Ignore the error and simply re-run the pipeline, as Databricks will automatically skip the problematic record on the next run.
- C. Fix the pipeline code and implement a quarantine logic to isolate the faulty data before re-running the pipeline.
- D. Change the expectation action from fail to warn so that invalid records are included in the output and the pipeline does not fail.

Answer: C

Explanation:

Technical justification

Option C – Fix the pipeline code and implement quarantine logic

Maintains the integrity of the existing LDP expectations (range -100 to 200°C).

Isolates the out-of-range record in a separate “quarantine” table or zone, allowing the data engineer to inspect, diagnose, and correct the sensor fault without discarding the entire dataset.

Preserves downstream data-quality checks for subsequent runs, ensuring that only validated data proceeds through the pipeline.

Aligns with best-practice data-engineering patterns: detect → quarantine → remediate → re-process.

Why the other options are unsuitable

A – Remove all expectations – Eliminates the quality guardrails that protect downstream analytics; it is an over-reaction that sacrifices data-quality standards.

B – Ignore the error and re-run – Databricks does not automatically skip invalid records; the pipeline will still fail on the same bad data unless the expectation is altered or the record is filtered out.

D – Change action from fail to warn – Allows the invalid temperature to propagate downstream, polluting results and breaking the contract of the LDP pipeline; it does not address the root cause of the faulty sensor data.

Conclusion

Implementing a quarantine mechanism (Option C) resolves the immediate failure, enables root-cause analysis of the sensor anomaly, and upholds the pipeline’s data-quality contract.

References

Databricks Lakehouse Platform – Managing data quality with expectations: <https://docs.databricks.com/data-engineering/expectations/index.html>

Best practices for data-quality pipelines in Databricks: <https://databricks.com/blog/2023/03/15/best-practices-data-quality-pipelines.html>

Question: 240

Deep Dumps

A data engineer is optimizing a managed table that suffers from data skew and frequently changing query filter columns. The engineer needs to avoid costly data rewrites when query patterns evolve. The table size is under 1TB.

How should data engineer meet this requirement?

- A. Use Hive-style partitioning, as it provides efficient data skipping and is easy to change partition columns at any time.
- B. Combine partitioning and Z-ordering to maximize flexibility and minimize maintenance as query patterns change.
- C. Enable liquid clustering, as it efficiently handles data skew, allows clustering keys to be changed without rewriting existing data, and adapts to evolving query patterns.
- D. Apply Z-ordering, since it allows flexible reorganization of data layout without rewriting existing and adapts easily to new filter columns.

Answer: C

Explanation:

Why option C is the optimal choice

Liquid clustering stores data in a columnar layout that can be reorganized on-the-fly, allowing the clustering key to be altered without a full table rewrite.

It automatically mitigates **data skew** by distributing skewed values across multiple files, improving read-amplification and query latency.

Because the clustering metadata is lightweight, changing the clustering column to match new filter predicates incurs only a modest metadata update, preserving **query-pattern agility** while keeping maintenance overhead low.

For tables **under 1TB**, the overhead of maintaining clustering is minimal, making it the most cost-effective solution compared with full-scale partitioning or extensive Z-ordering rewrites.

Why the other options are less suitable

A – Hive-style partitioning requires explicit partition column changes and a rewrite of the partition metadata; it does not handle skew gracefully and can become costly when filters evolve.

B – Combining partitioning and Z-ordering adds complexity and still demands periodic rewrites to adjust partition or Z-order keys, increasing maintenance when query patterns shift.

D – Pure Z-ordering improves data skipping but does not address skew and cannot change the ordering key without rewriting the entire dataset, limiting flexibility for evolving filters.

References

Liquid Clustering in Databricks: <https://docs.databricks.com/data/clustering/liquid-clustering.html>

Z-Ordering and its limitations: <https://docs.databricks.com/data/z-ordering.html>

A security team wants to enforce data protection for a customer table containing customer PII data. To comply with local policies, sales team members should only see customers from their region, while non-admin users should have email addresses masked.

Which implementation approach should be used when using Unity Catalog row filters and column masks?

- A. Create SQL UDFs for row filtering based on user region and column masking based on group membership, then apply them using ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK commands.
- B. Use table ACLs to restrict access using tags with GRANT SELECT ON table_name WITH TAG command, and rely on application-level filtering for sensitive data based on user region.
- C. Create a view with dynamic WHERE clauses for region filtering and use string replacement functions for email masking using ALTER COLUMN SET MASK command.
- D. Implement row filters with SQL UDFs based on user region only since column masks cannot be combined with row filters on the same table, then apply them by recreating the table with DROP TABLE and CREATE TABLE SET ROW FILTER commands.

Answer: A

Explanation:

Why option A is the correct implementation

Row-level security with Unity Catalog – Unity Catalog supports row filters that can be defined once and automatically applied to all queries against the table. The filter condition can reference a **SQL UDF** that evaluates the current user's region (e.g., via the CURRENT_USER() or a session-level attribute). This guarantees that only rows matching the user's region are returned, without needing application-side logic.

Column-level masking – Unity Catalog also enables column masks that can be attached to a column (e.g., email). A mask can be defined as a **SQL UDF** that checks group membership (e.g., IS_MEMBER('admin_group')) and returns either the original value or a masked placeholder. Because masks are evaluated per-row, they work together with row filters, ensuring that only non-admin users see the masked value.

Single-statement configuration – Using ALTER TABLE ... SET ROW FILTER and ALTER COLUMN ... SET MASK allows the security team to enforce both policies centrally, keeping the data model unchanged and preserving referential integrity.

Compliance and auditability – The policies are stored as metadata in the catalog, making them visible to auditors and easy to modify without redeploying code.

Why the other options are unsuitable

Option B – Table ACLs with tags only control access to the table; they do not enforce row-level visibility or column masking inside the data engine. Relying on application-level filtering for region logic bypasses Unity Catalog's native security and can lead to inconsistent enforcement.

Option C – Creating a view with dynamic WHERE clauses still requires the application to embed region logic, which is error-prone and not centrally managed. Using string-replacement functions for masking does not leverage Unity Catalog's mask framework and can expose raw data if the view is accessed directly.

Option D – Unity Catalog permits row filters and column masks to coexist on the same table; there is no need to drop and recreate the table. The statement that they "cannot be combined" is false, making this approach unnecessarily complex and disruptive.

References

Unity Catalog Row Filters: <https://docs.databricks.com/en/data-governance/unity-catalog/row-filters.html>

Unity Catalog Column Masks: <https://docs.databricks.com/en/data-governance/unity-catalog/column-masks.html>

A data engineer is evaluating tools to build a production-grade data pipeline. The team must process change data from cloud object storage, filter out or isolate invalid records, and ensure the timely delivery of clean data to downstream consumers. The team is small, under tight deadlines, and wants to minimize operational overhead while keeping pipelines auditable and maintainable.

Which approach should the data engineer implement?

- A. Ingest data directly into Delta tables via Spark jobs, apply data quality filters using UDFs, and use LDP for creating Materialized Views.
- B. Use a hybrid approach: Ingest with Auto Loader into Bronze tables, then process using SQL queries in Databricks Workflows to generate cleaned Silver and Gold tables on a schedule.
- C. Implement ingestion using Auto Loader with Structured Streaming, and manage invalid data handling and table updates using checkpointing and merge logic.
- D. Use LDP to build declarative pipelines with Streaming Tables and Materialized Views, leveraging built-in support for data expectations and incremental processing.

Answer: D

Explanation:

Why option D is the optimal choice

Declarative pipeline definition – LakehouseDeltaPiPe (LDP) lets you describe the entire data flow (ingestion, validation, transformation, and serving) as code, which makes the pipeline auditable, version-controlled, and easy to maintain.

Built-in data-expectations & incremental processing – LDP integrates native support for defining data quality expectations (e.g., schema, nullability, range checks) and automatically applies them during each incremental load, eliminating the need for custom UDFs or ad-hoc filtering logic.

Streaming Tables & Materialized Views – With Streaming Tables you can consume change data from cloud object storage in near-real-time, while Materialized Views keep downstream tables always up-to-date without manual refresh jobs. This satisfies the “timely delivery of clean data” requirement.

Minimal operational overhead – LDP abstracts checkpointing, state management, and job orchestration, so a small team can focus on business logic rather than managing Spark job lifecycles or workflow schedulers.

Scalable and production-grade – The framework is designed for large-scale, fault-tolerant pipelines in Databricks, aligning with the “production-grade” expectation.

Why the other options are less suitable

Option A – Relies on manual Spark jobs and UDFs for data-quality filtering, which adds custom code complexity and makes the pipeline harder to audit and maintain. LDP’s declarative expectations provide a cleaner, more maintainable alternative.

Option B – Uses a hybrid batch-oriented workflow (Auto Loader → Bronze → Silver/Gold) that introduces additional staging tables and scheduled jobs. While viable, it adds operational steps (e.g., workflow scheduling) and does not leverage the full streaming capabilities of LDP for incremental processing.

Option C – Implements Auto Loader with Structured Streaming and custom checkpoint/merge logic, which works but requires the engineer to build and maintain the error-handling and merge logic manually. LDP offers the same functionality with built-in support, reducing development effort and improving reliability.

References

Databricks LakehouseDeltaPiPe (LDP) documentation: <https://learn.databricks.com/pipe>
Streaming Tables and Materialized Views guide: <https://docs.databricks.com/delta/delta-streaming.html#streaming-tables>

A data engineer is using Structured Streaming to read in transaction data from a bronze Delta table. It was discovered that the data has quality issues where sometimes the transaction value is negative, and when that occurs, the rows need to be routed to a separate quarantine table. They have low latency requirements for the good data since it is used by downstream systems, but the bad data will only be analyzed periodically and has no production dependencies. The quarantine job needs to be implemented so that it cannot affect the production processes that depend on the good data, and the cost of the job needs to be minimized.

How should the quarantine process be implemented in order to satisfy these requirements?

- A. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing. Both should run as separate streams on the same cluster to minimize cost.
- B. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. A new boolean column called “quarantine” should be added to the dataframe, and its value should be set to true if the transaction value is less than 0 and false if the transaction value is greater than or equal to 0. Processing and storing all the data together will save costs.
- C. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. Inside a `foreachBatch` function, the dataframe should be filtered so that records with a transaction value greater than or equal to 0 are written to the good data table and records with a transaction value less than 0 are written to a quarantine table. Try/Catch can be added around the writes in the `foreachBatch` function so that the stream can't fail.
- D. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing, and should not share compute with other processes. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing, and should be implemented on a separate small cluster and only run once a day to minimize cost.

Answer: D

Explanation:

Why option D is the correct implementation

Isolation of good-data pipeline – By routing only rows with `transaction_value >= 0` to the production stream, the good-data job never sees corrupted records, eliminating any risk of downstream failures.

Separate compute footprint – Running the quarantine stream on its own small cluster (or a low-cost job schedule) guarantees that its resource consumption does not interfere with the low-latency production stream.

Minimal cost – The quarantine job can be scheduled to run only when new data arrives (e.g., once per day) and can use a smaller VM type, reducing cluster-hour costs while still capturing all bad rows.

Deterministic output – Filtering before write ensures that each stream writes a consistent schema to its target Delta table, avoiding the need for post-hoc cleanup or complex branching logic.

Why the other options are less suitable

Option A – Running two separate streams on the same cluster still shares resources, so a spike in quarantine processing could still impact latency of the good-data stream. It also does not address the need for a distinct, low-cost schedule.

Option B – Adding a quarantine flag and writing both good and bad rows to the same table mixes production and non-production data, violating the requirement that the quarantine data must not affect downstream systems. It also increases storage and processing overhead.

Option C – Using `foreachBatch` with try/catch can mask failures but does not provide isolation of compute; the bad-row writes still compete for the same cluster resources, potentially affecting latency. Moreover, relying on exception handling for normal data routing is not a robust design pattern.

Implementation sketch (aligned with option D)

1. **Good-data stream** – Read the bronze Delta table, apply where transaction_value >= 0, write to the production Delta table using a dedicated streaming query with low-latency settings.
2. **Quarantine stream** – Read the same source, apply where transaction_value < 0, write to a separate quarantine Delta table. Deploy this query on a separate, modest-size cluster and schedule it (e.g., once per micro-batch or once daily) to keep costs low.
3. **Checkpointing & idempotency** – Enable checkpointing for both queries to guarantee exactly-once semantics and safe restart after failures.

References

Delta Lake Structured Streaming documentation: <https://docs.databricks.com/delta/structured-streaming.html>

Using foreachBatch for custom write logic: <https://spark.apache.org/docs/latest/streaming-programming-guide.html#foreachbatch>

These resources detail the recommended patterns for isolating streaming jobs and managing compute costs in Databricks.

Question: 244

Deep Dumps

When monitoring a complex workload, being able to see the query plan is critical to understanding what the workload is doing.

Where can the visualization of the query plan be found?

- A. In the Spart UI, under the Jobs tab
- B. In the Query Profiler, under Query Source
- C. In the Spark UI, under the SQL/DataFrame tab
- D. In the Query Profiler, under the Stages tab

Answer: C

Explanation:

Justification

The Spark UI provides a dedicated **SQL/DataFrame** tab where the execution plan of each query is rendered as a visual DAG. This view shows operators, data movement, and estimated costs, making it the primary place to inspect a query plan in Databricks.

OptionC directly references this location, matching the official Spark UI navigation.

OptionA (“Spart UI, under the Jobs tab”) is not a valid Databricks component; the Jobs tab only lists completed jobs and does not display query-plan visualizations.

OptionB (“Query Profiler, under Query Source”) describes a different diagnostic view that focuses on the raw query text and source, not the graphical execution plan.

OptionD (“Query Profiler, under the Stages tab”) also points to a profiling area that lists stage-level metrics but does not provide the interactive query-plan diagram.

Therefore, the correct answer is **C – In the Spark UI, under the SQL/DataFrame tab.**

References

Spark UI documentation – SQL/Dataset tab: <https://spark.apache.org/docs/latest/sql-ui.html>

Databricks Query Profiler overview: <https://docs.databricks.com/optimization/profiler.html>

Question: 245

Deep Dumps

A data engineer is troubleshooting a slow-running Delta Lake query on Databricks SQL involves complex joins and large datasets. They need to identify whether the root cause is related to poor data skipping, inefficient join strategies, or excessive data shuffling.

Which approach should identify the specific bottlenecks using native Databricks tools?

- A. Analyze the Top Operators panel in the Query Profile to identify high-cost operations like BroadcastNestedLoopJoin
- B. Check the query's execution time in the Jobs UI and correlate it with cluster resource utilization metrics.
- C. Enable the EXPLAIN command to review the parsed logical plan and manually estimate shuffle sizes.
- D. Use the LIMIT clause to run a subset of the query and compare execution times with the full dataset.

Answer: A

Explanation:

Why option A is the best choice

The **Top Operators panel** in the Databricks Query Profile visualises the exact physical operators that consume the most resources (e.g., BroadcastNestedLoopJoin, ShuffleHashJoin, FileScan).

It directly surfaces **data-skipping inefficiencies**, **join strategy choices**, and **shuffle volume** for each step of the execution plan, letting the engineer pinpoint the bottleneck without manual estimation.

Because the information is presented by Databricks natively, it is always in sync with the runtime environment and reflects the actual cost of each operation.

Why the other options are less suitable

B. Jobs UI shows overall query latency and cluster utilization, but it does not break down the cost per operator, so it cannot isolate whether the slowdown is caused by poor data skipping, an inefficient join, or excessive shuffling.

C. EXPLAIN provides the logical plan, but interpreting shuffle sizes and join strategies from a textual plan requires deep expertise and is error-prone; it is not a native, interactive diagnostic tool.

D. Running a LIMIT subset may give a rough timing hint, yet execution characteristics can change dramatically with data size (e.g., different join strategies become viable only at scale), making it an unreliable method for root-cause analysis.

Conclusion – Only the **Top Operators panel** offers a native, granular view of the exact performance-critical operators, making it the recommended approach for diagnosing the specific bottlenecks in a slow-running Delta Lake query.

References

Query Profile – Top Operators: <https://docs.databricks.com/en/query-profile/index.html#top-operators>

Understanding the Query Profile: <https://docs.databricks.com/en/query-profile/index.html#understanding-the-query-profile>

Question: 246

Deep Dumps

A data engineer is designing a secure data sharing strategy for their organization. The company needs to share sensitive customer analytics data with two different partners. Partner A uses Databricks with Unity Catalog enabled, while Partner B uses Apache Spark on AWS without Databricks.

How should the company implement secure data sharing for these scenarios?

A. For Partner A, implement Databricks-to-Databricks sharing (D2D) with Unity Catalog integration and no-token exchange system. For Partner B, use open sharing protocol (D2O) with either bearer tokens or OIDC federation for authentication, ensuring both approaches maintain robust security and governance.

B. Both partners should use the same Delta Sharing approach since security requirements are identical. You should create bearer tokens for both partners and use the open sharing protocol (D2O) for maximum compatibility.

C. Open sharing protocol (D2O) should be used for both partners because it provides better security than D2D sharing. The bearer token approach is always more secure than Unity Catalog's native authentication.

D. Databricks-to-Databricks sharing (D2D) can only be used within the same cloud provider, so you must use open sharing (D2O) for any cross-cloud scenarios. Unity Catalog governance is not available when sharing with external platforms.

Answer: A

Explanation:

Why option A is the correct choice

Partner A (Databricks+Unity Catalog) – Use **Databricks-to-Databricks (D2D) sharing** with Unity Catalog integration. This leverages native fine-grained access controls, row-level security, and column-level masking defined in Unity Catalog, eliminating the need for token exchange while preserving the full governance model.

Partner B (Spark on AWS, no Databricks) – Use **open sharing (Delta Sharing) with bearer-token or OIDC federation**. This approach works across clouds and platforms, provides end-to-end encryption, and can be scoped with the same token-based permissions that mirror Unity Catalog policies.

Why the other options are less suitable

Option B – Proposes identical Delta Sharing for both partners and suggests creating bearer tokens for both. This ignores the opportunity to exploit Unity Catalog's native security model for Partner A, which is more granular and requires no external token management.

Option C – Claims open sharing is universally more secure than D2D and that bearer tokens are always superior to Unity Catalog authentication. Security is context-dependent; Unity Catalog offers richer, centrally managed policies that cannot be fully replicated by generic bearer tokens.

Option D – States D2D is limited to the same cloud provider and that Unity Catalog governance is unavailable externally. In reality, D2D can span clouds when both parties run Databricks (even on different clouds), and Unity Catalog policies can be enforced on shared data regardless of the consumer's environment.

Conclusion – Option A correctly aligns each partner's platform capabilities with the most secure, governance-rich sharing mechanism available.

References

Delta Sharing – Open Protocol: <https://delta.io/sharing>

Databricks Unity Catalog Documentation: <https://docs.databricks.com/en/data-governance/unity-catalog/index.html>

These sources provide the technical details behind Delta Sharing's cross-platform capabilities and Unity Catalog's native security controls.

Question: 247

Deep Dumps

Which approach demonstrates a modular and testable way to use DataFrame transform for ETL code in PySpark?

A.

```
class Pipeline:
    def transform(self, df):
        return df.withColumn("value_upper", upper(col("value")))

pipeline = Pipeline()
assertDataFrameEqual (pipeline.transform(test_input), expected)
```

B.

```
def transform_data(input_df):
    # transformation logic here
    return output_df

test_input = spark.createDataFrame([(1, "a")], ["id", "value"])
assertDataFrameEqual (transform_data (test_input), expected)
```

C.

```
def upper_transform(df):
    return df.withColumn("value_upper", upper(col("value")))

actual = test_input.transform(upper_transform)
assertDataFrameEqual (actual, expected)
```

D.

```
def upper_value (df):
    return df.withColumn("value_upper", upper(col("value")))

def filter_positive(df):
    return df.filter (df["id"] > 0)

pipeline_df = df.transform(upper_value).transform
(filter_positive)
```

Answer: C

Question: 248

Deep Dumps

A company stores account transactions in a Delta Lake table. The company needs to apply frequent account-level correlations (e.g., UPDATE statements) but wants to avoid rewriting entire Parquet files for each change to reduce file churn and improve write performance.

Which Delta Lake feature should they enable?

- A. Enable automatic file compaction on writes
- B. Enable change data feed on the Delta table
- C. Partition the Delta table by account_id
- D. Enable deletion vectors on the Delta table

Answer: D

Explanation:

Correct option: D – Enable deletion vectors on the Delta table

Deletion vectors allow Delta Lake to record the existence of deleted rows at the file-level rather than rewriting whole Parquet files.

When an UPDATE (or DELETE) operation is performed, Delta writes a small “deletion” file that marks the affected row-ids as removed. Subsequent reads can efficiently skip those files, avoiding the need to rewrite the entire dataset for each change.

This reduces **file churn** and **write amplification**, preserving the original file layout and improving write throughput — exactly what the scenario requires for frequent account-level correlations.

Why the other options are less suitable

A. Enable automatic file compaction on writes – Compaction merges many small files after a write, but it still requires a full rewrite of the affected partitions; it does not prevent the initial rewrite caused by each update.

B. Enable change data feed – CDC provides a log of changes for downstream consumption, but it does not alter how updates are stored; the underlying files still need to be rewritten for each mutation.

C. Partition the Delta table by account_id – Partitioning improves query pruning and can reduce the amount of data scanned, yet each UPDATE still rewrites the whole partition file; it does not eliminate file churn for frequent point updates.

Implementation tip

Activate deletion vectors with ALTER TABLE transactions SET TBLPROPERTIES (delta.enableDeletionVectors = true); and ensure that the table’s merge/update operations use the WHERE clause on the primary key (e.g., account_id) to let Delta generate precise deletion vectors.

References

Delta Lake Documentation – Deletion Vectors: <https://docs.databricks.com/delta/delta-batch.html#deletion-vectors>

Delta Lake Documentation – Optimizing Writes: <https://docs.databricks.com/delta/delta-write.html#optimizing-writes-with-deletion-vectors>

Question: 249

Deep Dumps

In a Databricks Asset Bundle project, in the file resources/app.yml, the data engineer would like to deploy a Databricks Apps databricks_app_deployed and Volume volume_deployed and grant the Service Principal behind Databricks Apps permissions to READ and WRITE to the Volume.

How should the data engineer achieve the deployment?

A.

```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_schema
    grants:
      -principal: ${resources.app.databricks_app_deployed.id}
      privileges:
        -READ_VOLUME
        -WRITE_VOLUME
```

B.

```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_schema
    grants:
      -principal: ${resources.app.name-databricks_app.id}
      privileges:
        -READ_VOLUME
        -WRITE_VOLUME
```

C.

```

resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_scheama
    grants:
      -principal: ${resources.app.name-
databricks_app_deployed.id}
      privileges:
        -READ
        -WRITE

```

D.

```

resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name schema
    grants:
      -principal: ${resources.app.name-name-databricks_app.id}
      privileges:
        -READ_VOLUME
        -WRITE_VOLUME

```

Answer: A

Question: 250

Deep Dumps

A data engineering teams needs to implement a tagging system for their tables as part of an automated ETL process, and needs to apply tags programmatically to tables in Unity Catalog.

Which SQL command adds tags to a table programmatically?

- A. ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');
- B. APPLY TAGS ON table_name VALUES ('key1' = 'value1', 'key2' = 'value2')
- C. COMMENT ON TABLE table_name TAGS ('key1' = 'value1', 'key2' = 'value2')

D. SET TAGS FOR table_name AS ('key1' = 'value1', 'key2' = 'value2')

Answer: A

Explanation:

Correct option: A – ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');

Unity Catalog supports table-level tags that can be set, viewed, and removed with the ALTER TABLE ... SET TAGS statement.

The SET TAGS clause accepts a comma-separated list of key = value pairs, exactly as shown in option A. This command can be executed programmatically from any client (SQL, Python, Spark, etc.) and is the only syntax that Databricks recognises for assigning tags to a table.

Why the other options are not valid

B. APPLY TAGS ON table_name VALUES (...) – No such clause exists in the Unity Catalog SQL grammar; APPLY is used for other operations (e.g., APPLY CHANGES) but not for tag assignment.

C. COMMENT ON TABLE table_name TAGS (...) – COMMENT ON is used to add comments to tables, columns, or columns, not tags. Tags are a distinct metadata construct and require the SET TAGS syntax.

D. SET TAGS FOR table_name AS (...) – The preposition FOR and the keyword AS are not part of the supported syntax; the correct keyword is SET TAGS directly after ALTER TABLE.

References

Unity Catalog SQL Reference – Table Properties: <https://docs.databricks.com/en/data-governance/unity-catalog/sql-ref-table-properties.html#tags>

ALTER TABLE ... SET TAGS: <https://docs.databricks.com/en/sql/language/sql/alter-table.html#set-tags>

These sources confirm that ALTER TABLE ... SET TAGS is the official, supported command for programmatically adding tags to Unity Catalog tables.

Question: 251

Deep Dumps

A platform engineer needs to report the resource consumption, categorized by SKU tier, across all workspaces. The engineer decides to use the system.billing.usage system table to create a query.

Which SQL query will accurately return the daily usage by product?

A.


```
SELECT
    DATE_TRUNC('day', usage_start_time) AS day,
    sku_name,
    SUM(dbus) AS daily_dbus
FROM
    system.billing.usage
WHERE
    sku_name IS NOT NULL
GROUP BY
    day, sku_name;
```

B.

```
SELECT
    CAST(usage_start_time AS DATE) AS usage_day,
    sku_name,
    COUNT (usage_quantity) AS dbu_count
FROM
    system, billing.usage
GROUP BY
    usage_day, sku_name
ORDER BY
    usage_day;
```

C.

```
SELECT
    date_trunc('day', usage_start_time) AS usage_day,
    sku_name,
    SUM(usage_quantity) AS total_daily_dbus
FROM
    system.billing.usage
WHERE
    usage_quantity IS NOT NULL
GROUP BY
    usage_day, sku_name
ORDER BY
    usage_day;
```

```
SELECT
    TO_DATE(usage_start_time) AS usage_day,
    sku_name,
    SUM(usage_quantity) AS total_dbus
FROM
    system.billing.usage
GROUP BY
    usage_day
ORDER BY
    usage_day
```

D.

Answer: C

Question: 252

Deep Dumps

A data engineer is running a groupBy aggregation on a massive user activity log grouped by user_id. A few users have millions of records, causing task skew and long runtimes.

Which technique will fix the skew in this aggregation?

- A. Increase the Spark driver memory and retry.
- B. Filter out the skewed users before the aggregation
- C. Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after removing the prefix.
- D. Use reduceByKey instead of groupBy to avoid shuffles.

Answer: C

Explanation:

Why option C is the best solution

Root cause: In a groupBy aggregation, Spark shuffles all rows to the same reducer based on the grouping key (user_id). When a few users generate millions of records, their data ends up on a single executor/task, creating task skew and long runtimes.

Salting technique: By adding a random prefix (e.g., user_id + "_" + random_int) to the key before the shuffle, the skewed keys are distributed across many partitions. After the first aggregation, the prefix is stripped and a second aggregation combines the partial results. This spreads the load evenly and eliminates the hot-spot without losing correctness.

Deterministic outcome: The final aggregation still produces the same grouped results because the prefix is removed before the final groupBy, preserving the original semantics.

Why the other options are unsuitable

A – Increase driver memory: Skew is a task-level issue, not a driver-memory problem. Adding memory does not redistribute data and does not solve uneven reducer loads.

B – Filter out skewed users: Removing users discards valuable activity data and changes the business logic; it is not a generic fix for skew.

D – Use reduceByKey instead of groupBy: Both operations trigger a shuffle; reduceByKey does not prevent skew when the key distribution is highly skewed. The underlying partitioning problem remains.

References

Spark SQL, Grouping and Aggregation: <https://spark.apache.org/docs/latest/sql-ref-grouping.html>
Handling Skewed Data in Spark: <https://spark.apache.org/docs/latest/sql-performance-tuning.html#skewed-data>

Question: 253

Deep Dumps

A job runs four independent tasks (X, Y, Z, W) in parallel to process regional sales data. The Data Engineering team recently updated its cluster policy to ban cost-prohibitive instance types. Task Y now fails due to the newly enforced cluster policy restricting the use of a specific instance type. A data engineer needs to resolve the failure quickly without disrupting the other tasks.

How should the data engineer resolve the failure of tasks?

- A. Delete the failed run, disable the cluster policy, and re-execute all tasks.
- B. Manually create a new cluster for Task Y, update the job configuration, and trigger a full re-run.
- C. Use “Repair run”, override the cluster configuration for Task Y to use a permitted instance type, and let Databricks re-run only Task Y.
- D. Edit the global cluster policy to allow the restricted instance type, then re-run the entire job.

Answer: C

Explanation:

Justification

Option C – Repair run lets the engineer isolate the failed task, override its cluster settings to a permitted instance type, and re-execute only that task. This approach restores the job quickly, preserves the successful completions of tasks X, Z, and W, and complies with the new cost-control policy without affecting the rest of the workflow.

Option A – Deleting the run and disabling the policy would remove the safeguard that prevents costly instance types, potentially leading to future budget overruns; it also requires a full re-execution of all tasks, increasing runtime and cost unnecessarily.

Option B – Creating a separate cluster and re-running the entire job adds operational overhead, risks version drift, and still forces a full re-run, which is slower than a targeted repair.

Option D – Modifying the global policy to allow the prohibited instance type defeats the purpose of the policy and could expose the environment to the same cost-prohibitive configurations that the team is trying to avoid.

Therefore, the most efficient and policy-compliant remediation is to use the Repair run feature to re-run only TaskY with an allowed instance type.

References

Databricks Repair Runs: <https://docs.databricks.com/jobs/repair-runs.html>
Managing Cluster Policies: <https://docs.databricks.com/clusters/cluster-policies.html>

Question: 254

Deep Dumps

A data engineer is analyzing a large, partitioned retail dataset in Databricks, where each row represents a sale made by a salesperson. The dataset contains millions of records with the following schema:

sales_df: [salesperson_id: string, region: string, sale_amount: double, sale_date: date]

The data engineer needs to generate a DataFrame that ranks salespeople within each region based on their total cumulative sales, with the highest seller ranked as 1. If multiple salespeople have the same total sales, they should share the same rank.

The data engineer wants to implement this logic using a PySpark window function and the dense_rank () function.

Which code snippet will perform this ranking?

- A.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

window_spec = Window.partitionBy("region").orderBy(
 F.desc("sale_amount"))

ranked_df= sales_df.withColumn("rank",
 F.dense_rank().over(window_spec))
```
- B.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
    .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.partitionBy("region").orderBy(
    F.desc("total_sales"))

ranked_df= agg_df.withColumn("rank", F.rank().over(window_spec))
```
- C.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
 .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.orderBy (F.desc("total_sales"))

ranked_df= agg_df.withColumn("rank",
 F.dense_rank().over(window_spec))
```
- D.

```

from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
 .agg(F.sum("sale_amount").alias("total_sales"))

window_spec =
Window.partitionBy("region").orderBy(F.desc("total_sales"))

ranked_df=agg_df.withColumn("rank",
F.dense_rank().over(window_spec))

```

Answer: D

### Question: 255

Deep Dumps

What describes a primary technical challenge in ensuring consistent PII masking across all nodes in large-scale, distributed Databricks batch and streaming pipelines?

- A. PII masking is only required for direct identifiers.
- B. Dynamic data masking is applied only at rest, so it does not affect query performance.
- C. Masking functions must be standardized and managed through Unity Catalog, with enforcement applied across all relevant datasets to avoid any data inconsistency.
- D. Native masking in Databricks automatically synchronizes with all downstream external Databricks systems.

Answer: C

Explanation:

#### Technical justification

**OptionC** correctly identifies that a primary challenge is to **standardize masking logic and enforce it uniformly across every dataset** accessed by both batch and streaming workloads. In large-scale Databricks environments, data resides in many tables, views, and external locations; without a centralized mechanism such as Unity Catalog policies, masking rules can diverge, leading to inconsistent PII exposure and compliance gaps.

**OptionA** is inaccurate because masking is required for any data element that can be linked to an individual, not only direct identifiers.

**OptionB** misstates the nature of dynamic data masking; it operates at query-time (in-memory) rather than only at rest, and it can impact performance when applied to large result sets.

**OptionD** overstates native capabilities; Databricks does not automatically propagate masking policies to all downstream external systems, so explicit governance is still necessary.

#### References

Unity Catalog security and data masking: <https://docs.databricks.com/en/data-governance/unity-catalog/security.html>

Dynamic data masking in Databricks SQL: [https://docs.databricks.com/en/sql/data masking/index.html](https://docs.databricks.com/en/sql/data%20masking/index.html)

A data engineer is working on a Databricks notebook that requires several third-party Python libraries. Some of these are available on PyPI, while others are custom-developed and stored as local.wheel (.whl) and source (.tar.gz) files in an S3 bucket. The goal is to ensure all dependencies are installed and correctly available across multiple jobs running on any automated cluster in a Unity Catalog-enabled workspace. The engineer needs to install the required dependencies in a way that ensures a consistent environment setup across interactive notebooks and jobs and complies with workspace security policies (no internet access).

Which approach should the engineer use to install and manage these dependencies while also ensuring reproducibility and compliance?

- A. Use an init script on the cluster to install all dependencies using pip, referencing the local file system.
- B. Install all dependencies manually in the driver node of an interactive cluster, then export the environment and reimport on job clusters using %conda.
- C. Create a Python wheel file for the entire project, upload it to the Databricks Workspace Files or Volumes, and install it using a Cluster Library or pip install in a requirements.txt declared within a Databricks Asset Bundle.
- D. Use %pip install in every notebook and job to install packages directly from PyPI and custom S3 paths.

**Answer: C**

**Explanation:**

**Why option C is the best choice**

**Single source of truth** – By packaging the whole set of dependencies into a Python wheel (or a collection of wheels) and storing it in a Databricks Volume or the workspace file system, the environment definition is version-controlled and can be reused by every job or notebook without manual intervention.

**Reproducibility across clusters** – The wheel can be referenced from a cluster library or via a requirements.txt that is consumed by a Databricks Asset Bundle (DAB). All clusters that are attached to the same library or DAB will install the exact same artifacts, guaranteeing identical package versions and transitive dependencies.

**Compliance with security policies** – No outbound internet access is required; installation happens from an internal location (Workspace Files/Volumes or an S3-mounted location) that is already whitelisted. This satisfies the “no internet” constraint and aligns with Unity Catalog governance because the wheel can be stored in a catalog-controlled location and its provenance tracked.

**Automation-friendly** – In a CI/CD pipeline the wheel can be built once, uploaded to the volume, and then the DAB’s project.yml can declare the wheel as a dependency. Subsequent jobs automatically pick up the updated wheel without any manual steps, reducing drift and operational overhead.

**Why the other options are less suitable**

**Option A** – Init scripts run only at cluster creation and rely on the driver’s local filesystem. When clusters are recreated or scaled, the script must be re-executed, and any change to dependencies requires a new cluster restart. This approach does not provide a reproducible artifact that can be version-controlled or shared across multiple jobs.

**Option B** – Manually installing packages on the driver node and exporting the environment is fragile; the exported state is tied to the specific driver version and Python version, and it cannot be reliably re-imported on job clusters. %conda is primarily for Conda environments and does not handle binary wheels stored in S3, making it an incomplete solution for custom .whl and .tar.gz files.

**Option D** – Using %pip install in every notebook or job forces each run to perform network calls (even to internal S3) and creates non-deterministic installations if the order or environment differs. This violates the “no internet” policy and makes it difficult to enforce consistent package versions across all jobs, especially in a Unity Catalog-enabled workspace where governance expects a single source of truth.

**References**



DatabricksUnity Catalog – Managing libraries and dependencies: <https://docs.databricks.com/en/data-governance/unity-catalog/index.html>

Creating and using Python wheels in Databricks: <https://docs.databricks.com/en/dev-tools/library/creating-wheels.html>

Prepared for Databricks Certified Data Engineer Associate exam review.

### Question: 257

Deep Dumps

A data engineer is using Auto Loader to read in JSON data as it arrives. They have configured Auto Loader to quarantine invalid JSON records. They are noticing that over time, some records are being quarantined even though they are well-formed JSON.

The snippet of code is:

```
df = (spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("badRecordsPath", "/tmp/somewhere/badRecordsPath")
 .schema("a int, b int")
 .load("/Volumes/catalog/schema/raw_data/"))
```

What is the cause of the missing data?

- A. The source data is valid JSON, but doesn't conform to their defined schema in some way
- B. The badRecordsPath location is accumulating many small files
- C. The engineer forgot to set the option "cloudFiles.quarantineMode", "rescue".
- D. At some point, the upstream data provider switched everything to multi-line JSON.

Answer: A

### Question: 258

Deep Dumps

A data engineer wants to enforce the principle of least privilege when configuring ACLs for Databricks jobs in a collaborative workspace.

Which approach should the data engineer use?

- A. Assign users only the minimum permission level (e.g., CAN RUN or CAN VIEW) required for their role on each job.
- B. Use only folder-level permissions and avoid setting permissions on individual jobs.
- C. Grant CAN RUN permission to everyone and CAN MANAGE to a single admin group.
- D. Grant all users CAN MANAGE permission on all jobs to avoid access issues.

Answer: A

Explanation:

Why option A is correct

Implements the principle of least privilege by granting each user only the minimum permission (e.g., **CANRUN** or **CANVIEW**) needed to perform their specific tasks on a job.

Allows granular control at the job-level, ensuring that users cannot accidentally modify or manage resources beyond their scope.

Reduces the attack surface and aligns with Databricks security best practices for collaborative workspaces.

#### Why the other options are unsuitable

**B.** Relying solely on folder-level permissions ignores the finer-grained job-level ACLs that Databricks provides; it can lead to over-privileged access when multiple jobs share a folder.

**C.** Granting **CANRUN** to everyone eliminates differentiation between roles and can permit unintended execution of sensitive jobs; reserving **CANMANAGE** for a single admin group does not enforce least privilege across the broader user base.

**D.** Giving **CANMANAGE** to all users removes any restriction, allowing any user to alter job configurations, schedules, and dependencies — contrary to security and audit requirements.

#### References

Databricks SQL Access Control: <https://docs.databricks.com/security/sql-access-control.html>

Managing Permissions in Databricks Jobs: <https://docs.databricks.com/jobs/permissions.html>

### Question: 259

### Deep Dumps

A data engineering workspace was automatically enabled for Unity Catalog, creating a workspace catalog. New team members report they can create tables in the default schema but cannot access table in other schemas within the same workspace catalog.

Why are the new team members unable to access tables in other schemas?

- A. Workspace catalog permissions are not subject to inheritance rules.
- B. Workspace users receive USE CATALOG and specific privileges on default schema only.
- C. Tables in other schemas require additional BROWSE privileges that new users don't receive automatically.
- D. New users only receive CREATE TABLE privileges on the default schema.

**Answer: B**

**Explanation:**

#### Why option B is the correct answer

In a Unity Catalog-enabled workspace, a **catalog** is created automatically for the workspace.

When a user is added to the workspace, the platform grants them the **USE CATALOG** privilege on that catalog, but **only the default schema** inside the catalog receives additional permissions (typically CREATE and INSERT).

Other schemas remain **invisible** to the user until explicit grants (e.g., SELECT, INSERT, CREATE) are applied to them.

Therefore, new team members can create tables in the default schema because they have CREATE on it, but they cannot read or write tables in any other schema because they lack the necessary grants on those schemas.

#### Why the other options are not appropriate

**A. "Workspace catalog permissions are not subject to inheritance rules."**

Permissions in Unity Catalog do follow inheritance (catalog → schema → table), but the problem here is not about inheritance; it is about the **initial set of grants** that are automatically applied.

**C. “Tables in other schemas require additional BROWSE privileges that new users don’t receive automatically.”**

BROWSE is not a distinct privilege in Unity Catalog; access is controlled by SELECT/INSERT/UPDATE etc., and the core issue is the lack of any schema-level grant, not a missing BROWSE privilege.

**D. “New users only receive CREATE TABLE privileges on the default schema.”**

While it is true they receive CREATE on the default schema, the statement does not explain why they cannot access other schemas. The key point is the **absence of explicit grants on those schemas**, not just the presence of CREATE on the default one.

---

## References

**Unity Catalog documentation – Permissions model:** <https://docs.databricks.com/en/data-governance/unity-catalog/permissions.html>

**Unity Catalog documentation – Default schema behavior:** <https://docs.databricks.com/en/data-governance/unity-catalog/default-schema.html>

## Question: 260

## Deep Dumps

A data engineer is implementing a job to download multiple PDF files from a third-party provided REST API endpoint by specifying different report types. The REST API is time-consuming and encounters intermittent errors, so the engineer wants to track each download activity to know when it fails and to retry partially, while providing scalable throughput. The engineer needs to download ten report types, and the list can be changed over time.

How should the data engineer achieve this?

- A. Use a foreach task with a list of report types as its inputs.
- B. Define ten Notebook tasks to clearly track which report download failed.
- C. Use a Delta Lake table to track each report download status as 10 rows, and use it as a source table to execute the download function as a Pandas UDF.
- D. Define a list variable within a Notebook to loop through the report types to download them, and print the download results. Execute it as a Notebook tasks.

**Answer: A**

**Explanation:**

**Why option A is the best choice**

**Dynamic input handling** – A foreach task can receive a list of report-type identifiers at runtime, so the job automatically adapts when the list changes without requiring code modifications or additional tasks.

**Parallel execution & scalability** – Each iteration of the foreach runs as an independent task (or subtask) that can be dispatched to separate workers, enabling high-throughput, concurrent downloads while still being managed by the job scheduler.

**Built-in failure tracking & retries** – The foreach construct integrates with Databricks job retry policies; failures are logged per-iteration, allowing the engineer to pinpoint exactly which report type failed and to automatically retry only that download.

**Simplified orchestration** – All download logic resides in a single notebook/task, keeping the pipeline concise and maintainable, whereas the other options either fragment the workflow (B, D) or introduce unnecessary complexity (C).

**Why the other options are less suitable**

**B – Ten separate Notebook tasks** – Hard-codes the number of reports; any change in the list forces manual

task updates, breaking scalability and making dynamic list handling impossible.

**C – Delta Lake table + Pandas UDF** – Over-engineers the solution; storing each report as a row in Delta adds latency and complexity for a simple download operation, and a Pandas UDF is unnecessary when the work is I/O-bound external API calls.

**D – List variable loop inside a Notebook** – Executes sequentially in a single executor, limiting throughput and lacking native retry/failure granularity; it also does not leverage Databricks' task-level parallelism.

## References

Databricks Jobs –[foreach task documentation](#)

Retry policies for jobs –[Retry and failure handling](#)

## Question: 261

## Deep Dumps

A data engineer and a platform engineer are working together to automate their system tasks. A script needs to be executed outside of Databricks only if a particular daily Databricks job finishes successfully for the day. Databricks CLI command was used to check the last execution of the job.

What are the required command options for that task?

- A. databricks jobs list-runs --job-id JOB\_ID --start-time-to TODAY\_MIDNIGHT\_EPOCH\_MS --completed-only
- B. databricks jobs list-runs --job-id JOB\_ID --start-time-from TODAY\_MIDNIGHT\_EPOCH\_MS --active-only
- C. databricks jobs list-runs --job-id JOB\_ID --start-time-to TODAY\_MIDNIGHT\_EPOCH\_MS --active-only
- D. databricks jobs list-runs --job-id JOB\_ID --start-time-from TODAY\_MIDNIGHT\_EPOCH\_MS --completed-only

**Answer: D**

**Explanation:**

**Why option D is the correct choice**

The goal is to **inspect only completed runs** of a specific job that started **on or after midnight of the current day**.

databricks jobs list-runs supports the flag --completed-only to filter runs that have finished with a success or error status.

To limit the search to runs that began **from** the start of today, the CLI requires --start-time-from <epoch-ms> (the epoch milliseconds representing midnight of the current day).

Therefore the command must include **both** --start-time-from TODAY\_MIDNIGHT\_EPOCH\_MS **and** --completed-only. Option **D** exactly matches this combination:

databricks jobs list-runs --job-id JOB\_ID --start-time-from TODAY\_MIDNIGHT\_EPOCH\_MS --completed-only.

**Why the other options are unsuitable**

**Option A** uses --start-time-to which defines an upper bound on the start time; it would exclude runs that started after midnight, opposite of the requirement.

**Option B** employs --active-only, which returns only runs that are currently executing, not the historical completed runs we need to evaluate.

**Option C** mixes --start-time-to (wrong direction) with --active-only (wrong state filter), so it cannot correctly isolate completed runs that started today.

Hence, only option **D** provides the precise set of flags needed to retrieve completed runs that began on the current day.

## References

**Databricks CLI – jobs list-runs command:** <https://docs.databricks.com/en/dev-tools/cli/jobs.html#list-runs>  
**CLI options for time filtering:** <https://docs.databricks.com/en/dev-tools/cli/reference/jobs/list-runs.html>

### Question: 262

Deep Dumps

A data engineering team needs to create a SQL Alert that monitors data quality across multiple columns in their customer table. They want to trigger an alert when both the percentage of customers with missing email addresses exceeds 15% AND the percentage of customers with invalid phone number formats exceeds 10%.

Which SQL query pattern is appropriate for implementing this multi-column alert condition?

- A. `SELECT COUNT (*) FROM customers WHERE email IS NULL OR phone_format_invalid = true`
- B. `SELECT email, phone FROM customers WHERE email IS NULL AND phone NOT RLIKE '[0-9-+()\s]+$'`
- C. `SELECT email_null_pct, phone_invalid_pct FROM (SELECT (COUNT(CASE WHEN email IS NULL THEN 1 END) * 100.0 / COUNT (*)) as email_null_pct, (COUNT(CASE WHEN phone NOT RLIKE '[0-9-+()\s]+$' THEN 1 END) * 100.0 / COUNT (*)) as phone_invalid_pct FROM customers)`
- D. `SELECT CASE WHEN email_null_pct > 15 AND phone_invalid_pct > 10 THEN 1 ELSE 0 END FROM (SELECT (COUNT (CASE WHEN email IS NULL THEN 1 END) * 100.0 / COUNT (*)) as phone_invalid_pct FROM customers) metrics`

**Answer: C**

**Explanation:**

**Why option C is the correct pattern**

**Aggregates per-column percentages in a single derived table** – the inner query computes `email_null_pct` and `phone_invalid_pct` using conditional COUNT expressions, then multiplies by 100 to obtain percentages.

**Both percentages are available side-by-side** – the outer query can reference both metrics in the same SELECT list, enabling a logical AND condition (`email_null_pct > 15 AND phone_invalid_pct > 10`).

**Correct syntax for percentage calculation** – uses `COUNT(CASE WHEN ... THEN 1 END) 100.0 / COUNT()` which yields a floating-point percentage; the result is aliased (`email_null_pct`, `phone_invalid_pct`) for clarity.

**Deterministic and portable** – relies only on standard SQL functions (COUNT, CASE, arithmetic) and works in Databricks SQL without external UDFs or complex joins.

**Why the other options are unsuitable**

**Option A** – Returns a single row count, not percentages, and uses an OR condition that would fire when either column is missing, not when both exceed their thresholds.

**Option B** – Performs row-level filtering with `WHERE email IS NULL AND phone NOT RLIKE ...`; it does not compute any percentage and cannot be used to trigger an alert based on thresholds.

**Option D** – Attempts to reference `email_null_pct` without defining it, and the inner query mistakenly calculates only `phone_invalid_pct` while aliasing it as `phone_invalid_pct` but never computes `email_null_pct`. The outer CASE also mixes up the column names.

## References

**Databricks SQL – Conditional Aggregation:** <https://docs.databricks.com/sql/language-manual/sql-ref-functions/aggregate/count.html>

**Databricks SQL – Percent Calculations:** <https://docs.databricks.com/sql/language-manual/sql-ref-functions/aggregate/percent.html>

## Question: 263

## Deep Dumps

A data engineer is bringing an existing production Databricks job under asset bundle management and wants to ensure that:

- The job's current configuration is captured as YAML, and all referenced files are included in their bundle project.
- Future changes to the bundle's YAML will update the existing job in-place (not create a new job)

How should the data engineer successfully move the production job under asset bundle management?

- A. Run Databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deploy to deploy the bundle, which will always update the existing job automatically.
- B. Export the job definition as JSON, convert it to YAML, and place it in your bundle. Then, run Databricks bundle deploy to update the existing job.
- C. Manually create the YAML configuration for the job in your bundle project, ensuring all settings match the existing job. Then, run Databricks bundle deploy the bundle, which will update the existing job in your workspace.
- D. Run databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deployment, bind to link the bundle's job resource to the existing job in Databricks.

**Answer: D**

**Explanation:**

**Why optionD is the correct approach**

**Accurate capture of the current job** – databricks bundle generate job --existing-job-id introspects the existing production job, writes its full YAML definition, and downloads every artifact/file that the job references into the bundle directory. This guarantees that the bundle contains an exact replica of the job's configuration and dependencies.

**In-place update semantics** – After generating the YAML, the engineer runs databricks bundle deployment and explicitly **binds** the newly created job resource to the original job ID (--job-id <existing-job-id>). Binding tells Databricks to treat the bundle's job as the same logical job, so a subsequent bundle deploy updates the existing job rather than creating a new one.

**Idempotent and repeatable** – Because the job is bound, any future change to the bundle's YAML can be redeployed and will overwrite the current job configuration in place, satisfying the requirement for "in-place" updates.

**Why the other options are inferior**

**OptionA** – bundle generate is correctly used, but bundle deploy alone does **not** guarantee an in-place update; without binding the job resource to the existing job, Databricks may create a new job version instead.

**OptionB** – Exporting to JSON and manually converting to YAML bypasses the automated generation step, increasing the risk of missing referenced files or incorrect settings, and it does not provide the binding mechanism needed for in-place updates.

**OptionC** – Manually recreating the YAML requires meticulous hand-copying of all settings and file paths; any omission or mismatch can cause deployment failures, and it does not leverage the built-in binding that ensures the job is updated rather than replaced.

**References**

**Databricks Bundles Documentation** – <https://docs.databricks.com/jobs/bundles.html>

**CLI Command Reference: databricks bundle generate job** – <https://docs.databricks.com/dev-tools/bundles/commands.html#generate-job>



These sources detail the generation of job YAML from an existing job and the binding step that enables in-place updates via bundle deploy.

## Question: 264

## Deep Dumps

A data architect is implementing Delta Sharing as part of their data governance strategy to enable secure data collaboration with external partners and internal business units. The architect must establish a permission framework that allows designated data stewards to create shares for their respective domains while maintaining security boundaries and audit compliance.

Which specific permissions and roles must be assigned to enable users to create, configure, and manage Delta Shares while maintaining proper security governance and access controls?

- A. Only workspace admins can create and manage shares
- B. Users need the MANAGE SHARES permission on the workspace
- C. Users need to be metastore admins or have CREATE SHARE privilege for the metastore
- D. Any user with USE\_CATALOG privilege can create shares

**Answer: C**

**Explanation:**

**Why option C is the correct choice**

**Metastore-level privilege** – In Databricks, creating a Delta Share requires the **CREATESHARE** privilege (or metastore admin rights) on the underlying Unity Catalog metastore. This privilege is granted to specific users or groups, allowing them to register shares without giving them unrestricted admin rights on the entire workspace.

**Security boundary enforcement** – Assigning CREATESHARE (or metastore admin) to designated data stewards ensures that only authorized entities can expose data objects as shares, preserving the principle of least privilege and preventing accidental exposure of sensitive tables.

**Audit compliance** – Because the permission is tied to the metastore, every share-creation event is logged in the metastore audit logs, providing traceability for governance reviews.

**Separation of concerns** – Workspace admins (option A) have broader control and do not need to be involved for each share; granting them sole authority would create bottlenecks.

**Workspace-level vs. metastore-level** – Option B mentions “MANAGESHARES” on the workspace, but Delta Sharing permissions are enforced at the metastore level; workspace-level permissions do not control share creation.

**Insufficient privilege** – Option D’s “USE\_CATALOG” only allows querying a catalog; it does not confer the ability to create or manage shares.

**Why the other options are unsuitable**

**A** – Restricting share creation to only workspace admins eliminates the ability for domain-specific stewards to self-service shares, contradicting the goal of decentralized governance.

**B** – “MANAGESHARES” is not a recognized Databricks permission; share creation is governed by metastore privileges, not a generic workspace permission.

**D** – “USE\_CATALOG” merely permits reading from a catalog; it does not grant the rights needed to define or publish shares.

## References

Databricks Docs: Create and manage shares – <https://learn.databricks.com/share/create-manage>

Databricks Docs: Unity Catalog permissions – <https://learn.databricks.com/unity-catalog/permissions>

## Question: 265

Deep Dumps

A data engineer manages a production Lakeflow Spark Declarative Pipeline that processes customer transaction data. The pipeline includes several data quality expectations, such as: `transaction_amount > 0` and `customer_id IS NOT NULL`.

These expectations are defined using the `EXPECT` clause in SQL. The engineer aims to monitor the pipeline's data quality by analyzing the number of records that passed or failed each expectation during the latest pipeline update. The Lakeflow Spark Declarative Pipelines event logs are stored in a Delta table named `event_log_table`.

For the most recent pipeline update, determine a programmatically approach to extract information like the name of each expectation, associated dataset, count of records that passed the expectation, and count of records that failed the expectation.

Which method retrieves the desired data quality metrics from the Lakeflow Spark Declarative Pipelines event log?

- A. Access the `event_log_table`, filter for events where `event_type = 'flow progress'`, and parse the `details.flow_progress.data_quality.expectations` field to extract the required metrics.
- B. Use the Lakeflow Spark Declarative Pipelines UI to navigate to the specific pipeline, select the dataset, and view the Data Quality tab to manually retrieve the expectation metrics.
- C. Query the `event_log_table` for events with `event_type = 'data_quality'` and directly select the `passed_records` and `failed_records` fields.
- D. Access the `event_log_table`, filter for events where `event_type = 'expectation_result'`, and extract the expectation metrics from the `details` field.

**Answer: A**

**Explanation:**

**Why option A is correct**

The Lakeflow Spark Declarative Pipeline writes its execution events to a Delta table (`event_log_table`).

For data-quality monitoring the relevant events are of type **flow progress**; within the `details.flow_progress.data_quality.expectations` struct each expectation's name, associated dataset, and the counts of `passed_records` and `failed_records` are stored.

By filtering the Delta table on `event_type = 'flow progress'` and then parsing the nested `details.flow_progress.data_quality.expectations` field, a programmatic query can extract exactly the required metrics (expectation name, dataset, passed/failed record counts) in a single, repeatable operation. This approach works entirely within Spark SQL/Databricks SQL, enabling automation, scaling, and integration with downstream analytics or alerting pipelines.

**Why the other options are less suitable**

**B** – Relies on a manual UI interaction; it cannot be automated or programmatically queried, making it unsuitable for repeatable data-quality reporting.

**C** – The `data_quality` event type does not exist in the Lakeflow event log schema; the correct event type for expectation results is `flow progress`. Selecting `data_quality` would return no rows or unrelated data.

**D** – There is no top-level event type called `expectation_result`; the expectation results are nested inside the `flow progress` event. Querying for a non-existent event type would yield empty results.

**Implementation sketch (Spark SQL)**

```
SELECT expectation.name AS expectation_name, expectation.dataset AS dataset_name, expectation.
```

The query above returns one row per expectation with the exact metrics needed for the latest pipeline update.

## References

**Lakeflow Spark Declarative Pipelines – Event Log Schema:** <https://learn.microsoft.com/en-us/azure/databricks/data-engineering/lakeflow/event-log>

**Data Quality Expectations in Lakeflow:** <https://learn.microsoft.com/en-us/azure/databricks/data-engineering/lakeflow/data-quality-expectations>

### Question: 266

Deep Dumps

Predictive Optimization is an automated Databricks service enabled by default for Unity Catalog Managed tables. It helps maintain Delta tables by continuously optimizing them to ensure optimal performance and costs.

Which two operations does Predictive Optimization run to maintain the Delta tables? (Choose two.)

- A. PARTITION BY
- B. COMPACT
- C. ANALYZE
- D. OPTIMIZE
- E. BUCKETING

**Answer: CD**

**Explanation:**

**Why CD are the correct choices**

**ANALYZE** – Predictive Optimization automatically refreshes table statistics (via ANALYZE TABLE ... COMPUTE STATISTICS) so that the query optimizer can generate efficient execution plans. Accurate statistics are essential for cost-based optimization and for deciding when to trigger further maintenance actions.

**OPTIMIZE** – The service periodically rewrites Delta data files (e.g., compaction, Z-ordering, and file size tuning) using the OPTIMIZE command. This reduces the number of small files, improves read throughput, and keeps storage costs low.

**Why the other options are not part of Predictive Optimization**

**PARTITION BY** – Partitioning is a user-defined schema design decision; Predictive Optimization does not create or modify partitions automatically.

**COMPACT** – COMPACT is a manual command that forces file consolidation; it is not invoked by the automated Predictive Optimization service.

**BUCKETING** – Bucketing is a table property set at creation time and is not a maintenance operation performed by the service.

## References

Predictive Optimization: [Databricks Docs – Predictive Optimization](#)

OPTIMIZE and ANALYZE commands: [Databricks Docs – Optimize Delta Tables](#)

Statistics collection: [Databricks Docs – Analyze Delta Tables](#)

### Question: 267

Deep Dumps

A data engineer is building a customer data pipeline in Lakeflow Spark Declarative Pipelines. The source is a cloud-based event stream with limited retention containing inserts, updates, and deletes for customer records. These

changes are being applied using the AUTO CDC INTO syntax to maintain an SCD Type 1 table as the target table, customer\_dim.

How should the data engineer build a downstream job that streams from the customer\_dim table to only act on updates and delete events, processing data incrementally?

- A. Use ignoreChanges flag while streaming from customer\_dim to avoid breaking the pipeline during updates and deletes.
- B. Read change data feed from customer\_dim table and apply filters to incrementally act on the change events.
- C. Streaming from customer\_dim table would only be possible in the case of SCD 2 retention.
- D. When stored as SCD 1, the target of AUTO CDC INTO includes updates and deletes. Streaming from customer\_dim can fail due to these operations. Instead, build another stream from the original source.

**Answer: B**

**Explanation:**

**Why option B is the correct approach**

The AUTO CDC INTO clause populates the target customer\_dim with **change events** (inserts, updates, deletes) that are captured from the source stream.

To process only the **updates and deletes** downstream, the engineer can read the change-data-feed (CDF) from customer\_dim and apply a filter on the operation type (op = 'U' or op = 'D').

This method is **incremental**: it reads only new change rows, respects the limited retention of the source stream, and avoids unnecessary full-table scans.

It leverages the built-in CDC metadata (e.g., op column) that marks each row as insert, update, or delete, enabling precise downstream logic without breaking the pipeline.

**Why the other options are less suitable**

**Option A – ignoreChanges flag:** This flag is used to suppress change-event handling for a particular stream, not to filter CDC events. It would prevent the pipeline from reacting to updates/deletes altogether, defeating the requirement to act on them.

**Option C – SCD2 retention prerequisite:** SCD2 retention is unrelated to CDC consumption. The source stream already provides CDC; there is no need to enforce SCD2 on the target to enable downstream streaming.

**Option D – Build another stream from the original source:** While technically possible, it duplicates effort and bypasses the CDC metadata already embedded in customer\_dim. It also risks missing deletes that are only captured via the CDC flow, making it an inefficient and error-prone solution.

**Implementation sketch**

1. Create a streaming read on customer\_dim that exposes the CDC columns (\_change\_type, op, etc.).
2. Apply a filter such as WHERE op IN ('U','D').
3. Route the filtered rows to the downstream processing logic (e.g., enrich, write to another Delta table, trigger alerts).
4. Ensure the read respects the source stream's retention window to avoid processing stale changes.

**References**

Databricks Auto CDC Syntax: <https://learn.databricks.com/data-engineering/auto-cdc>

Incremental Streaming with Delta Live Tables: <https://docs.databricks.com/data-engineering/delta-live-tables/incremental-processing.html>

A data engineer is designing a system leveraging Lakeflow Declarative Pipeline technology to process real-time truck telemetry data ingested from JSON files in S3 using Auto Loader. The data includes truck\_id, timestamp, location, speed, and fuel\_level. The system must support two use cases:

1. Near-real-time monitoring of the latest location, speed, and fuel\_level per truck\_id for the operations team.
2. Daily aggregated reports of total distance traveled and average fuel efficiency per truck\_id for the management team.

Which approach should the data engineer use for streaming tables and materialized views in the Lakeflow Declarative Pipeline to meet these requirements?

- A. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to compute the daily aggregated distance and fuel efficiency per truck\_id reporting. Create a materialized view to compute the latest location, speed, and fuel\_level per truck\_id for real-time monitoring.
- B. Define a streaming table to ingest and store the raw telemetry data, and create a materialized view to compute the latest location, speed, and fuel\_level per truck\_id for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per truck\_id for reporting.
- C. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to incrementally compute the latest location, speed, and fuel\_level per truck\_id for real-time monitoring. Create a materialized view to compute the daily aggregated distance and fuel efficiency per truck\_id for reporting.
- D. Define a materialized view to ingest and store the raw telemetry data, and create a streaming table to compute the latest location, speed, and fuel\_level per truck\_id for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per truck\_id for reporting.

**Answer: C**

**Explanation:**

**Why option C is the optimal design**

**Streaming table for raw ingestion** –Auto Loader continuously writes the JSON telemetry files into a DeltaLake table that is marked as a streaming table. This preserves the exact arrival order and enables incremental reads for downstream processing.

**Streaming table for real-time metrics** –A second streaming table can be defined with a continuous query that groups by truck\_id and keeps only the most recent record per key (using row\_number() over a watermark-aware window). Because the query is incremental, the latest location, speed, and fuel\_level are always materialised in the table and can be queried with low latency by the operations team.

**Materialized view for daily aggregates** –The daily report requires a batch-style aggregation (total distance, average fuel efficiency) that can be computed once per day. A materialized view is ideal here because it stores the pre-aggregated results, refreshes on schedule, and can be read by downstream analytics without re-scanning the raw stream.

**Separation of concerns** –Using a streaming table for the real-time per-truck state keeps the latency low and guarantees up-to-date rows, while a materialized view handles the heavier, less-frequent aggregation efficiently. This matches the Lakeflow declarative model where streaming tables are used for incremental pipelines and materialized views for fast-lookup aggregates.

**Why the other options are less suitable**

**Option A** –It proposes two streaming tables for the daily aggregates. Streaming tables are designed for continuous updates; aggregating over an entire day would require complex window management and would not benefit from the snapshot semantics of a materialized view, leading to unnecessary compute overhead.

**Option B** –It suggests using materialized views for both the real-time per-truck snapshot and the daily aggregation. Materialized views cannot be incrementally refreshed with the same low-latency guarantees as streaming tables; they are refreshed on demand or on a schedule, which may introduce latency that is unacceptable for near-real-time monitoring.

**Option D** –It treats a materialized view as the ingestion target for raw telemetry, which contradicts the purpose of materialized views (they store derived data, not raw ingest). Moreover, a streaming table cannot be created from a materialized view; the direction of data flow is reversed, making the design logically

inconsistent.

## References

**Lakeflow Declarative Pipelines – Streaming Tables:** <https://learn.databricks.com/lakeflow/streaming-tables>

**Materialized Views in Delta Lake:** <https://docs.databricks.com/delta/materialized-views.html>

### Question: 269

### Deep Dumps

A platform team lead is responsible for automating the individual teams attribution towards SQL Warehouse usage. The requirement is to identify the SQL warehouse usage at the individual user's level and generate a daily report to be shared with an executive team that includes leaders from all business units.

How should the platform lead generate an automated report that can be shared daily?

- A. Use the system tables to capture the audit and billing usage data and share the queries with the executive team. This enables the executives to execute the query and see the latest results any time.
- B. Use the system tables to capture the audit and billing usage data and create a dashboard with daily refresh schedules and shared with the executive team.
- C. Restrict users from running any SQL query unless they provide all the query details so that the attribution can be calculated and shared with the executive team.
- D. Let the users run the SQL query and then directly report the usage to the executives. The ownership of the SQL warehouse usage will be with the individual teams.

**Answer: B**

**Explanation:**

**Why option B is the correct approach**

**Automated aggregation at the user level** – System tables such as `system.query_history` and `system.billing_usage` expose per-query and per-warehouse consumption metrics, including the user column. By querying these tables and grouping on user, the platform can compute each user's SQL-warehouse usage for the previous day.

**Scheduled pipeline** – The aggregation can be implemented as a notebook or a dbt model that runs on a daily schedule (e.g., via Databricks Jobs). The job writes the result to a dedicated table or a Delta Lake location that serves as the source for a report.

**Centralized, shareable output** – The daily result can be visualised in a Databricks SQL dashboard (or exported to an external tool) and shared with the executive team. The dashboard can be configured with a refresh cadence that matches the job schedule, ensuring the executive view is always up-to-date without manual intervention.

**Governance & auditability** – Because the usage data is derived from system tables, the report inherits the same security and audit properties as the underlying warehouse, satisfying compliance requirements.

**Why the other options are unsuitable**

**Option A** – Simply exposing raw system-table queries to executives forces non-technical users to run and interpret SQL themselves, which is error-prone, insecure, and does not provide a ready-to-consume daily report.

**Option C** – Requiring users to supply full query details before execution is not feasible; Databricks does not enforce such a restriction, and it does not address the need for an automated, aggregated usage report.

**Option D** – Allowing each team to manually report their own usage leads to fragmented, inconsistent reporting and places the burden of data collection on the teams, defeating the purpose of a centralized, automated daily executive summary.



## References

Databricks SQL usage and billing tables: <https://docs.databricks.com/en-us/sql/warehouse/usage.html>  
Scheduling notebooks and jobs for automated pipelines: <https://docs.databricks.com/en-us/jobs/schedule-notebooks.html>

### Question: 270

### Deep Dumps

A data engineering team is collaborating on a Databricks project where each team member needs to develop and test code independently before merging changes into the main branch. They want to avoid accidental overwrites or branch switching issues while ensuring that all work is version-controlled and can be integrated into their CI/CD pipeline.

How should the data engineer achieve collaboration?

- A. Each team member creates their own Databricks Git folder, mapped to the same remote Git repository, and works in their own development branch within their personal folder.
- B. All team members work in the same Databricks Git folder and perform Git operations (pull, push, commit, branch switching) directly in that shared folder.
- C. Team members edit notebooks directly in the workspace's shared folder and periodically copy changes into a Git folder for version control.
- D. Team members use the Databricks CLI to clone the Git repository and perform Git operations from a cluster's web terminal.

**Answer: A**

**Explanation:**

**Why option A is the optimal choice**

**Isolated workspaces prevent accidental overwrites** – Each engineer works in a separate Databricks Git folder that points to the same remote repository, so commits from one user cannot unintentionally replace or delete another user's changes.

**Branch isolation mirrors Git best practices** – Developers can create and switch to their own feature branches locally, keeping the main branch untouched until a pull-request is approved and merged.

**Full version-control integration** – All code, notebooks, and configuration files are stored in Git, enabling history tracking, code review, and automated CI/CD triggers (e.g., DatabricksReposCI pipelines).

**Scalable collaboration** – Multiple users can clone the shared remote repo into their own folders, push/pull independently, and merge via pull-requests without contending for the same workspace resources.

**Why the other options are less suitable**

**Option B** – Using a single shared folder forces all users to operate on the same branch and workspace, making concurrent edits highly prone to merge conflicts and accidental overwrites.

**Option C** – Editing notebooks directly in a shared folder bypasses Git for day-to-day work; periodic manual copying introduces loss of granular version history and increases the risk of divergent, untested states.

**Option D** – Relying on the Databricks CLI inside a cluster's web terminal couples version control to runtime resources, limits reproducibility, and does not provide the structured branching and pull-request workflow needed for CI/CD integration.

## References

Databricks Repos and Git Integration: <https://docs.databricks.com/repos/git-integration.html>  
Branching Strategies for Databricks Repos: <https://docs.databricks.com/repos/branching.html>

A data engineer is using the AUTO CDC API in Lakeflow Spark Declarative Pipeline to propagate deletions from a source table (orders\_source) to a target table (orders\_target). The source has Change Data Feed (CDF) enabled, but some delete events arrive out of order due to upstream delays.

How does the AUTO CDC API internally ensure deletions are applied correctly despite out-of-order events?

- A. It ignores deletions if they arrive after updates for the same key.
- B. It manually sorts incoming events by timestamp before applying changes.
- C. It runs VACUUM on the target table to purge conflicting records.
- D. It uses sequence\_by to order events and retains tombstones for deleted rows until older sequences are processed.

**Answer: D**

**Explanation:**

#### Technical justification

The AUTO CDC API in Lakeflow Spark Declarative Pipelines does **not** rely on the arrival order of CDC events. It assigns a **monotonically increasing sequence number** to each change event (via the sequence\_by function) that reflects the logical order of the source change stream, regardless of when the event is received. Deleted rows are represented by **tombstone records** that are kept in the target table until all earlier sequence numbers have been processed. This guarantees that a delete is not overwritten by a later update or that a newer update does not “re-appear” a row that should have been removed.

When the pipeline later processes older sequence numbers, the corresponding tombstones are finally applied, ensuring **exactly-once** semantics for deletions even when events are out-of-order due to network latency or upstream buffering.

#### Why the other options are incorrect

##### A. “It ignores deletions if they arrive after updates for the same key.”

Ignoring late-arriving deletes would leave stale data in the target, violating the invariant that the target must reflect the source’s current state. The CDC engine deliberately **preserves** delete markers, not discards them.

##### B. “It manually sorts incoming events by timestamp before applying changes.”

The pipeline does **not** perform a manual timestamp sort; ordering is achieved through the logical sequence numbers generated by the source change feed, not by wall-clock timestamps.

##### C. “It runs VACUUM on the target table to purge conflicting records.”

VACUUM is a storage-level cleanup operation that reclaims file space; it does not influence the logical ordering of CDC events. Deletion handling is managed by the CDC engine, not by vacuuming.

Hence, option **D** correctly describes the internal mechanism: the API uses sequence\_by to order events and retains tombstones until older sequences are processed, ensuring correct deletion semantics despite out-of-order arrivals.

#### References

**Lakeflow Spark Declarative Pipelines – Change Data Capture (CDC) Overview** – Databricks Docs

<https://learn.databricks.com/lakeflow/cdc-overview>

**Sequence Numbers in Auto CDC** – Databricks Docs

<https://learn.databricks.com/lakeflow/auto-cdc-sequence-number>

These links provide detailed explanations of how the Auto CDC API orders events and manages tombstones

## Question: 272

## Deep Dumps

A data governance team at a large enterprise is improving data discoverability across its organization. The team has hundreds of tables in their Databricks Lakehouse with thousands of columns that lack proper documentation. Many of these tables were created by different teams over several years, with missing context about column meanings and business logic. The data governance team needs to quickly generate comprehensive column descriptions for all existing tables to meet compliance requirements and improve data literacy across the organization. They want to leverage modern capabilities to automatically generate meaningful descriptions rather than manually documenting each column, which would take months to complete. The team is looking for a solution that can understand data patterns, column names, and sample values to create intelligent descriptions.

Which approach should the team use in Databricks to automatically generate column comments and descriptions for existing tables?

- A. Write custom PySpark code using `df.describe()` and `df.schema` to programmatically generate basic statistical descriptions for each column.
- B. Navigate to the table in Databricks Catalog Explorer, select the table schema view, and use the “AI Generate” option which leverages artificial intelligence to automatically create meaningful column descriptions based on column names, data types, sample values, and data patterns.
- C. Use the `DESCRIBE TABLE` command to extract existing schema information and manually write descriptions based on column names and data types.
- D. Use Delta Lake’s `DESCRIBE HISTORY` command to analyze table evolution and infer column purposes from historical changes.

**Answer: B**

**Explanation:**

**Why option B is the correct choice**

**Built-in AI-driven column description** – Databricks Catalog Explorer includes an “AI Generate” button that uses large-language-model inference to synthesize clear, business-oriented descriptions from a column’s name, data type, sample values, and statistical patterns.

**Automation at scale** – The feature can be invoked for an entire table (or across many tables) in a single click, producing a full set of column comments without writing custom code or manually editing each schema.

**Context-aware output** – The generated text incorporates data-type nuances (e.g., timestamp, decimal) and observed value distributions (e.g., “percentage of nulls”, “high-cardinality string”), yielding more meaningful metadata than a plain `df.describe()` output.

**Compliance-ready** – Because the descriptions are stored as column comments in the metastore, they become searchable and auditable, satisfying governance and regulatory requirements.

**Why the other options are less suitable**

**Option A** – `df.describe()` only returns statistical aggregates (count, mean, std, min, max, etc.) and does not produce natural-language explanations. It also requires looping over every table and writing custom logic to convert numbers into readable text, which is time-consuming and error-prone.

**Option C** – `DESCRIBE TABLE` merely lists column names and data types; it offers no inference capability. Any description would still be manually crafted, defeating the goal of rapid, large-scale documentation.

**Option D** – `DESCRIBE HISTORY` reveals Delta log changes (file additions, compaction, schema evolution) but does not expose semantic meaning of individual columns. Historical metadata cannot reliably infer business-level semantics for new documentation.

**Implementation steps (high-level)**

1. Open the table in **Databricks Catalog Explorer**.
2. Click the : **menu** next to the table name → **AIGenerate**.
3. Review the automatically generated column comments; edit any that need refinement.
4. Save the changes – the comments are persisted as column metadata in the Unity Catalog/Metastore, making them visible to all downstream users and tools.

## References

**AI-Generate Column Descriptions in Databricks Catalog** – <https://learn.databricks.com/docs/data-governance/catalog/ai-generate-column-descriptions>

**Managing Table Schema and Comments** – <https://docs.databricks.com/data-governance/catalog/table-metadata.html>

### Question: 273

### Deep Dumps

A company processes semi-structured JSON files from an external source using Auto Loader in a classic Databricks job. Occasionally, records arrive with null critical fields, invalid types, or unexpected nested schema variations. The engineer must ensure that malformed or non-conforming records are not dropped silently and are captured in a separate quarantine table. The pipeline should continue processing good records into the Bronze layer without failing the job, and the approach must support both batch and streaming ingestion.

The data engineer needs to build a robust ingestion pattern that automatically routes bad records to a quarantine Delta table, while still ingesting good records into the Bronze layer for further processing.

Which approach fulfills the quarantine mechanism in this ingestion architecture?

- A. Create a notebook job with `inferSchema= True`, write a streaming query with `.foreachBatch()` and catch exceptions using `try/except` to redirect failed batches to quarantine.
- B. Use Auto Loader with `failFast` mode to set to false, and enable schema evolution; invalid records will be silently ignored during ingestion.
- C. Use Lakeflow Spark Declarative Pipelines with a SQL pipeline; configure it to drop rows with nulls using where `critical_fields` is not null, and rely on audit logs for malformed data.
- D. Use Auto Loader with LDP and implement an `EXPECT ()` constraint with a record audit logic to route bad records.

**Answer: D**

**Explanation:**

**Why option D is the correct choice**

**AutoLoader+LakehouseDeltaProcessing (LDP)** provides a built-in **quarantine** capability through the `EXPECT()` constraint, which can be combined with a **record-audit** clause to capture rows that violate schema, type, or null-ability rules.

The constraint can be defined declaratively (e.g., `EXPECT(record, "critical_field IS NOT NULL AND typeof(critical_field) = 'string'"))` and automatically routes violating rows to a separate Delta table without aborting the job.

This pattern works **both for batch and streaming** ingestion because the same declarative pipeline definition can be executed in Structured Streaming mode; good records flow to the Bronze layer while bad ones are persisted to quarantine for later inspection or re-processing.

The solution is **fault-tolerant**: the job never fails due to malformed records, and the quarantine table can be continuously appended to, enabling downstream cleansing or audit workflows.

**Why the other options are unsuitable**

**OptionA** – Using `foreachBatch()` with manual `try/except` works for streaming but requires custom error handling and does not leverage Delta's native quarantine feature. It adds complexity, can miss schema-evolution events, and is error-prone when scaling.

**OptionB** – `failFast=false` together with `schemaEvolution` only allows new columns; it **silently drops** rows that violate type or null constraints, which contradicts the requirement to capture bad records.

**OptionC** – Declarative pipelines in Lakeflow can filter nulls, but they **do not provide a built-in quarantine mechanism** for malformed JSON; relying on audit logs alone leaves bad rows un-tracked and does not guarantee robust routing.

## References

Auto Loader & Quarantine: <https://docs.databricks.com/data-engineering/auto-loader/auto-loader.html#quarantine>

EXPECT() constraint & record audit: <https://docs.databricks.com/data-engineering/declarative-pipelines/expect-constraint.html>

## Question: 274

## Deep Dumps

A data engineer is analyzing transactional data in a PySpark DataFrame `df` containing `customer_id`, `transaction_timestamp` (precise to milliseconds), and `amount_spent`. The objective is to compute a cumulative sum of `amount_spent` per customer, strictly ordered by `transaction_timestamp`. The cumulative sum must include all transactions from the earliest timestamp up to and including the current row, respecting temporal ordering within each customer partition.

Which PySpark code snippet most accurately constructs the appropriate window specification and applies the aggregation to yield the correct cumulative expenditure per customer?

A.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id")\
 .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))
```

B.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.orderBy("transaction_timestamp")\
 .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))
```

C.



```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id").orderBy("transaction_timestamp")\
 .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))
```

D.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id").orderBy("transaction_timestamp")
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))
```

Answer: C

### Question: 275

Deep Dumps

A data team is implementing an append-only Delta Lake pipeline that needs to handle both batch and streaming data. They want to ensure that schema changes in the source data can be automatically incorporated without breaking the pipeline.

Which configuration should the team use when writing data to the Delta table?

- A. ignoreChanges = false
- B. validateSchema= false
- C. overwriteSchema=true
- D. mergeSchema=true

Answer: D

Explanation:

Justification

**mergeSchema=true** (optionD) tells Delta to automatically merge new columns from incoming data into the target Delta table while preserving existing data. This is the recommended mode for append-only pipelines that must tolerate evolving source schemas without manual intervention.

**OptionA (ignoreChanges = false)** is not a valid Delta write option; it would cause a syntax error and does not control schema evolution.

**OptionB (validateSchema= false)** disables schema validation entirely, which can lead to corrupted data or inconsistent column types, and it does not provide the controlled merging behavior needed for reliable schema evolution.

**OptionC (overwriteSchema=true)** replaces the entire schema of the target table, discarding existing columns and data. This is unsuitable for an append-only pipeline that must retain previously loaded data.

Therefore, using mergeSchema=true is the only configuration that safely incorporates new columns while preserving existing data and maintaining pipeline stability.

References



## Question: 276

## Deep Dumps

A senior data engineer is planning large-scale data workflows. The current task is to identify the considerations that form a foundation for creating scalable data models that are essential for effective management of large datasets. The data engineering team has identified the core capabilities as part of a scalable data model to build a modern data platform and provided their reasoning for considering Delta Lake for review. The senior data engineer is responsible for identifying the recommendations that are not valid.

Which key features can be ignored while evaluating Delta Lake?

- A. Delta Lake works with various data formats (e.g., Parquet, JSON, CSV) and integrates well with Spark and Databricks tools:
- B. Delta Lake optimizes metadata handling, efficiently managing billions of files and facilitating scalability to petabyte-scale datasets.
- C. Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.
- D. Delta Lake provides limited support for monitoring and troubleshooting data pipelines, so relevant partner tools have to be identified and set up for enhanced operational efficiency.

**Answer: D**

**Explanation:**

**Why option D can be ignored**

DeltaLake's built-in monitoring and troubleshooting capabilities are robust (e.g., DESCRIBE DETAIL, OPTIMIZE history, VACUUM metadata, and integration with Unity Catalog). Relying on external partner tools is unnecessary for core pipeline observability, so this limitation does not affect the fundamental scalability assessment.

**Why the other options are essential considerations**

- A** – DeltaLake's native support for open file formats (Parquet, JSON, CSV) and tight integration with Spark/Databricks enables seamless read/write operations and eliminates format-lock-in, a cornerstone for scalable data model design.
- B** – Optimized metadata handling (single transaction log, snapshot isolation) allows DeltaLake to manage billions of files efficiently, directly supporting petabyte-scale workloads and preventing performance degradation.
- C** – Unified batch-and-stream API (Structured Streaming) lets engineers ingest, process, and serve data consistently across both paradigms, simplifying architecture and ensuring end-to-end scalability.

**Conclusion**

Option D describes a non-existent limitation; DeltaLake provides comprehensive monitoring, so this feature need not be evaluated when judging its suitability for large-scale, scalable data models.

**References**

Delta Lake Documentation – "Monitoring & Debugging": <https://docs.databricks.com/delta/monitoring.html>  
Delta Lake Documentation – "Scalability and Performance": <https://docs.databricks.com/delta/performance.html>

**Question: 277****Deep Dumps**

A data engineering team is implementing an append-only data pipeline using Delta Lake, and wants to ensure that data is never modified or deleted once written.

Which Delta Lake feature should the data engineer enable to prevent modifications to existing data?

- A. Delta APPEND\_ONLY
- B. Delta VACUUM
- C. Delta OPTIMIZE
- D. Delta Time Travel

**Answer: A****Explanation:****Justification**

**DeltaAPPEND\_ONLY** – Enables an append-only mode that blocks any write operation that would overwrite, update, or delete existing files. When this mode is active, Delta Lake rejects any attempt to modify data that already resides in the table, guaranteeing immutability for the lifetime of the data.

**DeltaVACUUM** – Performs garbage-collection of old files; it does not restrict writes, and in fact can delete data that is older than the retention period. It is therefore unrelated to preventing modifications.

**DeltaOPTIMIZE** – Rewrites data files to improve read performance (e.g., compaction). It can rewrite existing files, which would violate the “no-modification” requirement, so it does not provide immutability.

**DeltaTime Travel** – Allows querying of previous versions of the table but does not stop users from overwriting or deleting the current version; it merely adds a historical view.

Hence, **DeltaAPPEND\_ONLY** is the only feature that directly enforces an immutable, append-only write pattern.

**References**

Delta Lake Documentation – Append-Only Mode: <https://docs.databricks.com/delta/delta-batch.html#append-only-mode>

Delta Lake Documentation – Table Properties: <https://docs.databricks.com/delta/delta-table-properties.html#append-only-mode>

**Question: 278****Deep Dumps**

A data engineering team is setting up a Git project to automate integration tests using Databricks Asset Bundles and the Git provider’s CI/CD functionalities. When a pull containing changes to their pipeline is sent, they need to run a Job to test their data pipeline.

What is the correct databricks bundle command sequence to be executed from the Git provider’s CI/CD automation for this task?

- A. init, deploy, run, validate
- B. validate, deploy, run
- C. init, validate, deploy, run
- D. deploy, run, validate

**Answer: B****Explanation:**

## Justification

**Validate** – The first step in a CI/CD pipeline is to verify that the bundle configuration (jobs, tasks, and dependencies) is syntactically correct and conforms to Databricks Bundle standards. databricks bundle validate catches errors before any deployment occurs, preventing wasted resources and failed runs.

**Deploy** – After a successful validation, the bundle is deployed to the target environment (e.g., a test cluster or workspace). databricks bundle deploy packages the assets and installs them in the configured environment, making the jobs ready for execution.

**Run** – Finally, the CI/CD job executes the deployed bundle to run the integration tests: databricks bundle run. This step actually runs the pipeline, produces test results, and can fail the build if the tests do not pass.

### Why the other options are less suitable

**Option A (init, deploy, run, validate)** – init is only required when creating a new bundle; it adds unnecessary overhead in a CI/CD workflow where the bundle already exists. Placing validate at the end defeats its purpose of early error detection.

**Option C (init, validate, deploy, run)** – Including init again is redundant for pipelines that already have a bundle initialized, and it introduces an extra, non-essential command in the CI step.

**Option D (deploy, run, validate)** – Deploying before validation risks pushing an invalid bundle to the environment, potentially causing deployment failures or flaky test runs. Validation should precede any deployment.

Thus, the minimal, production-grade sequence for a CI/CD trigger is **validate → deploy → run**, which corresponds to **Option B**.

---

## References

**Databricks Bundle Documentation:** <https://docs.databricks.com/best-practices/bundle-get-started.html>  
**CI/CD with Databricks Assets:** <https://docs.databricks.com/dev-tools/bundle-ci-cd.html>

## Question: 279

Deep Dumps

A data engineer is attempting to execute the following PySpark code:

```
df=spark.read.table("sales")
result=df.groupBy("region").agg(sum("revenue"))
```

However, upon inspecting the execution plan and profiling the Spark job, they observe excessive data shuffling during the aggregation phase.

Which technique should be applied to reduce shuffling during the groupBy aggregation operation?

- A. Use.coalesce(1) after the aggregation
- B. Caching the DataFrame df
- C. Use broadcast join
- D. Repartition by region before aggregation

**Answer: D**

**Explanation:**

**Technical Justification**

**Correct choice – D.Repartition by region before aggregation**

Spark shuffles data when a `groupBy` requires data from different partitions to be combined. By explicitly `repartition("region")` before the aggregation, all rows for the same region are co-located on the same executor, allowing the subsequent `sum(revenue)` to be performed locally without a full-scale shuffle. This reduces network traffic and the amount of intermediate data that must be redistributed, directly addressing the observed excessive shuffling.

### Why the other options are unsuitable

**A. Use `coalesce(1)` after aggregation** – `coalesce(1)` forces a single partition, which increases shuffling because all data must be merged into one executor; it does not prevent the initial shuffle and can become a bottleneck.

**B. Cache the DataFrame `df`** – Caching only stores the DataFrame in memory/disk for reuse; it does not change the partitioning layout, so the shuffle that occurs during `groupBy` remains unchanged.

**C. Use broadcast join** – Broadcast joins are relevant when joining a small table to a large one, not for a simple aggregation; they do not influence the shuffle behavior of a `groupBy` operation.

### Conclusion

Repartitioning on the grouping key aligns data locality with the aggregation step, minimizing cross-partition data movement and thus the primary source of shuffling.

### References

[Spark SQL, Shuffle Operations](#)

[Optimizing DataFrames – Partitioning](#)

## Question: 280

## Deep Dumps

While reviewing a query's execution in the Deatricks Query Profile, a data engineer observes that the "Top operators" panel shows a sort operator with high "Time spent" and "Memory peak" metrics, and the Spark UI reports frequent data spilling.

How should the data engineering address this issue?

- A. Repartition the DataFrame to a single partition before sorting.
- B. Convert the sort operation to a filter operation.
- C. Switch to a broadcast join to reduce memory usage.
- D. Increase the number of shuffle partitions to better distribute data.

### Answer: D

### Explanation:

#### Technical justification

The sort operator's high "Time spent" and "Memory peak" together with frequent spilling indicate that a single shuffle phase is trying to handle too much data in one go, causing large shuffle files and excessive GC pressure.

Spark's default shuffle partition count is often too low for large-scale sorts; increasing the number of shuffle partitions spreads the data across more tasks, reducing the amount of data each task must keep in memory and the size of each spill file.

**Option D** directly addresses this by raising the shuffle-partition count, which lowers per-task memory usage and mitigates spilling, leading to a more balanced sort execution.

**Option A** would concentrate all data into a single partition, dramatically increasing memory pressure and likely worsening spilling.

**OptionB** is not applicable; a sort cannot be replaced by a filter without changing the semantics of the query.  
**OptionC** concerns join strategies and does not influence sort-operator memory or spilling behavior.

## References

Spark UI – Understanding the “Shuffle Read/Write” and “Spill” metrics:

<https://spark.apache.org/docs/latest/sql-performance-tuning.html#shuffle-partitions>

Tuning Spark SQL – Adjusting shuffle partitions and memory settings:

<https://spark.apache.org/docs/latest/sql-performance-tuning.html#shuffle-partition-configuration>

## Question: 281

## Deep Dumps

A data engineer manages a Unity Catalog table `customer_data` in schema `finance` that includes sensitive fields like `ssn` and `credit_score`. Intern Group should only see masked values, while Analyst Group should only access rows for their assigned region. The data engineer needs to restrict access based on user role and region without duplicating data.

How should the data engineer enforce this security policy?

- A. Use Unity Catalog’s row filters based on the region and column masks based on user roles.
- B. Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL `SELECT` clause for each sensitive column.
- C. Create dynamic views for each user role and manage access with ACLs.
- D. Use Unity Catalog’s row filters based on the user roles and column masks based on the region.

**Answer: A**

**Explanation:**

### Why optionA is the correct choice

Unity Catalog provides **row-level security** through row filters that can be defined with predicates such as `region = current_region()` or by checking group membership.

It also offers **column-level masking** that can be attached to a column and automatically applied based on the caller’s role or group.

By combining a **row filter on the region column** with a **column mask on the sensitive fields (ssn, credit\_score)** that is activated only for the Intern group, the engineer enforces the required policy **without duplicating data** and with a single, centrally managed definition.

This approach leverages native Unity Catalog features, ensuring that the security rules are evaluated at query-execution time, are auditable, and can be updated in one place.

### Why the other options are less suitable

**OptionB** – Building views that embed `current_user()` and `is_account_group_member()` checks and manually apply masking in the `SELECT` clause works, but it requires **duplicate logic for each view**, is harder to maintain, and does not take full advantage of Unity Catalog’s built-in masking and row-filter mechanisms.

**OptionC** – Creating dynamic views for each role and relying on ACLs can control access, but it does **not provide row-level filtering** based on region, nor does it automatically mask columns according to role; it also leads to **data duplication** and increased operational overhead.

**OptionD** – Swapping the predicates (row filters based on roles and masks based on region) would enforce the opposite logic: masking would be applied only for certain regions and filtering would be role-centric, which does **not meet the stated requirement** of masking by role and filtering by region.

## References

## Question: 282

## Deep Dumps

A data engineer is ingesting JSON files from cloud object storage using Databricks Auto Loader. The source folder may occasionally receive large files of data, which risks overwhelming the stream. To ensure predictable micro-batch sizes, the team wants to throttle ingestion based on the volume of data scanned at 1 GB, regardless of the number of files.

Which Auto Loader configuration should the data engineer use to achieve this?

- A. Configure `cloudFiles.maxBytesPerTrigger` with 1 GB to place a limit.
- B. Configure `cloudFiles.maxSizePerTrigger` with 1 GB to place a limit.
- C. Configure `cloudFiles.maxFilesPerTrigger` and estimate the average file size to approximate a size-based throttle of 1 GB.
- D. Configure `cloudFiles.maxPartitionBytes` with 1GB to limit data in each partition.

**Answer: A**

**Explanation:**

**Justification**

### Option A – `cloudFiles.maxBytesPerTrigger`

This setting directly limits the amount of data (in bytes) that Auto Loader will consume in each micro-batch. By setting it to **1GB**, the engine will stop reading additional files once the cumulative size of the batch reaches that threshold, guaranteeing a predictable batch size irrespective of file count. It is the only configuration that enforces a **size-based throttle** exactly as required.

### Option B – `cloudFiles.maxSizePerTrigger`

No such parameter exists in the Databricks Auto Loader API; the closest analogous option is `maxBytesPerTrigger`.

Using a non-existent key would cause a configuration error, making this option invalid.

### Option C – `cloudFiles.maxFilesPerTrigger` with estimated average file size

While `maxFilesPerTrigger` can cap the number of files per batch, it does not guarantee a specific byte volume because file sizes can vary widely.

Estimating an average size introduces uncertainty and can lead to batches that are either far below or exceed the 1GB target, failing the “predictable micro-batch sizes” requirement.

### Option D – `cloudFiles.maxPartitionBytes`

This option controls the maximum size of a partition when writing data out (e.g., for Parquet), not the ingestion size of a streaming source.

It does not influence how many bytes are read from the source, so it cannot be used to throttle the input stream.

**Conclusion** – Only `cloudFiles.maxBytesPerTrigger` provides a deterministic, size-based limit that matches the team’s objective, making **Option A** the correct choice.



## References

**Auto Loader configuration options:** <https://docs.databricks.com/data/auto-loader/index.html#options>  
**maxBytesPerTrigger:** <https://docs.databricks.com/data/auto-loader/index.html#maxbytespertrigger>

### Question: 283

Deep Dumps

A data company uses Databricks Unity Catalog and has multiple enterprise data sources, including PostgreSQL, Snowflake, and SQL Server. The central data platform team wants to configure Lakehouse Federation so analysts can query external tables directly in Databricks using Databricks SQL, without duplicating data.

Which steps are necessary to configure Lakehouse Federation in a secure and governed manner?

- A. Mirror the external datasets into Delta Lake using Auto Loader, and govern them using Data Lineage and System Tables.
- B. Configure connections and foreign catalog in Unity Catalog, then grant access to foreign catalogs, schemas, and tables using Unity Catalog permissions.
- C. Use Partner Connect to create linked datasets, and apply table ACLs at the source system to govern access through Databricks.
- D. Create external locations and storage credentials to connect to each database, then register foreign tables in Unity Catalog.

**Answer: B**

**Explanation:**

**Why option B is the correct approach**

**UnityCatalog-centric federation** – Lakehouse Federation relies on UnityCatalog to expose external data sources as foreign catalogs, schemas, and tables. Configuring the connections and foreign catalog objects inside UnityCatalog makes the external data discoverable and queryable from Databricks SQL without moving any data.

**Fine-grained, centralized permissions** – By granting access to the foreign catalogs, schemas, and individual tables through UnityCatalog ACLs, the central platform team can enforce a single source of truth for security, audit, and compliance. This eliminates the need to manage separate credentials or ACLs at each source system.

**Governance integration** – UnityCatalog provides built-in data lineage, table-level metadata, and column-level masking that automatically propagate to federated queries, ensuring consistent governance across all external sources.

**Scalable and low-maintenance** – Adding or removing external sources only requires updating the catalog definitions and permission grants; no custom code or data copying is required, which aligns with the “no duplication” requirement.

**Why the other options are less suitable**

**Option A** – Auto Loader is intended for incremental ingestion into Delta Lake, not for federated querying. Mirroring data would duplicate storage, violate the “no duplication” constraint, and adds unnecessary processing overhead.

**Option C** – Partner Connect creates linked datasets but still requires manual table-level ACLs at the source system. This spreads governance across multiple systems and does not leverage UnityCatalog’s unified permission model, making it harder to maintain consistent security policies.

**Option D** – While external locations and storage credentials are part of the connectivity step, simply registering foreign tables without first defining foreign catalogs and applying UnityCatalog permissions would bypass the centralized governance framework and could expose data inconsistently.

## References

Unity Catalog and Lakehouse Federation: <https://docs.databricks.com/data-governance/unity-catalog/index.html>

Configuring external data sources with Unity Catalog: <https://docs.databricks.com/data-governance/unity-catalog/external-data-sources.html>

### Question: 284

Deep Dumps

Why are Pandas UDFs often preferred over traditional PySpark UDFs in performance-critical applications involving large datasets?

- A. They minimize memory usage by streaming each row individually through a lightweight Python wrapper, avoiding batch processing overhead.
- B. They leverage Apache Arrow to enable vectorized operations between the JVM and Python runtimes, reducing serialization costs and improving computational efficiency.
- C. They allow row-level execution of functions in Python with native Spark optimization, removing the need for columnar execution.
- D. They eliminate the JVM-Python boundary by bypassing serialization entirely, thereby avoiding data conversion overhead.

**Answer: B**

**Explanation:**

**Why option B is correct**

Pandas UDFs (vectorized UDFs) process data in **batches** using **Apache Arrow** as the transport format. Arrow enables **zero-copy, columnar transfer** of data between the JVM and the Python process, eliminating the per-row Java-Python serialization that dominates traditional PySpark UDFs.

Because the same Arrow vector is reused for many rows, the overhead of invoking Python is amortized across thousands of records, resulting in **significantly lower serialization cost and higher computational efficiency** — the key advantage in performance-critical, large-scale workloads.

**Why the other options are less suitable**

- A. Pandas UDFs do **not** stream individual rows; they operate on whole batches, so they do not avoid batch processing overhead.
- C. They rely on **columnar (vectorized) execution**, not row-level execution, and therefore do not “remove the need for columnar execution.”
- D. The JVM-Python boundary is still present; Pandas UDFs merely **reduce** serialization cost via Arrow, they do not **eliminate** it entirely.

## References

Apache Spark Documentation – Pandas UDFs (Vectorized UDFs):

[https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.pandas\\_udf.html](https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.pandas_udf.html)

Databricks Documentation – Using Pandas UDFs: <https://docs.databricks.com/data/dataframes/pandas-udfs.html>

### Question: 285

Deep Dumps

A data engineer is setting up a pipeline to ingest data from a message bus system that occasionally delivers duplicate messages. The duplicate messages can be a week apart. The target is a Databricks Delta Lake table

where each record should appear exactly once.

Which Databricks ingestion pattern should be implemented to handle potential duplicates where events can arrive outside of the configured watermark?

- A. Use Delta Lake's change data feed to filter duplicate records
- B. Use Delta Lake time travel to identify and remove duplicates
- C. Configure Structured Streaming with dropDuplicates transformation
- D. Implement a write operation using MERGE INTO with a unique key

**Answer: D**

**Explanation:**

**Why option D is the correct choice**

**MERGE INTO with a unique key** performs an upsert that matches incoming rows to existing rows on a primary-key (or unique) column. When a duplicate arrives — even days later — the statement updates the existing record instead of inserting a new one, guaranteeing exactly-once semantics regardless of the streaming watermark.

The operation can be idempotent: re-processing the same batch does not create additional duplicates because the merge will always find a matching key and apply the same update.

It works with Delta Lake's transactional guarantees, preserving ACID properties while handling out-of-order or delayed data.

**Why the other options are less suitable**

**A – Change Data Feed (CDF)** is designed for CDC consumption, not for eliminating duplicates in an ingest pipeline; it does not provide a deduplication mechanism.

**B – Time travel** allows you to query previous versions of a Delta table but does not prevent duplicate writes; it is a read-only feature.

**C – dropDuplicates in Structured Streaming** only removes duplicates that appear within the same micro-batch and within the configured watermark. Records that arrive after the watermark expires are not considered, so duplicates that are a week apart would still be inserted.

**Implementation sketch**

```
MERGE INTO target_delta AS tgt USING source_stream AS src ON tgt.unique_key = src.unique_key WHEN MATCHED THEN
```

This pattern ensures each logical record is stored once, irrespective of arrival timing.

**References**

[Delta Lake MERGE operation](#)

[Structured Streaming deduplication limitations](#)

## Question: 286

**Deep Dumps**

A data engineer needs to design an efficient pipeline that automatically processes new CSV files as they arrive in S3 storage.

Which Databricks feature should the data engineer use to meet these requirements?

- A. Streaming from cloud storage using standard Spark readStream with format ("csv") and format ("json")
- B. COPY INTO SQL command with parameters to track processed files
- C. Traditional batch processing with scheduled Databricks Jobs

**Answer: D**

**Explanation:**

**Why option D is the best choice**

**Auto Loader** continuously watches an S3 bucket (or other cloud storage) for new files, automatically infers the schema, and can evolve it as new columns appear.

It provides **exactly-once** processing semantics with built-in checkpointing, eliminating duplicate work and ensuring fault-tolerance.

The feature is tightly integrated with Delta Lake and Spark Structured Streaming, allowing downstream transformations to be written in a streaming-friendly way without manual file-handling logic.

Schema inference and evolution are **enabled by default**, so the pipeline adapts to changing CSV layouts without code changes.

**Why the other options are less suitable**

**A. Streaming from cloud storage using readStream with format("csv") and format("json")** – This approach requires the engineer to manually manage file discovery, checkpointing, and schema handling; it does not provide automatic schema evolution or built-in exactly-once guarantees.

**B. COPY INTO with parameters** – COPY INTO is a batch-oriented command that moves data into Delta tables; it does not monitor incoming files in real time and lacks automatic schema inference or continuous processing capabilities.

**C. Traditional batch processing with scheduled Databricks Jobs** – Batch jobs run only on a schedule, so new files are processed only after the next scheduled run, leading to latency and potential duplicate processing if not carefully orchestrated.

**References**

Auto Loader documentation: <https://docs.databricks.com/data-engineering/auto-loader/index.html>  
Structured Streaming with CSV files: <https://docs.databricks.com/data-engineering/structured-streaming/streaming-csv.html>

### Question: 287

**Deep Dumps**

A data engineer is implementing liquid clustering on a Delta Lake table and needs to understand how it affects data management operations. The table will be updated frequently with new data. The table is an external table and not managed by Unity Catalog.

How does liquid clustering in Delta Lake handle new data that is inserted after the initial table creation?

- A. New data is rejected if it doesn't match the clustering pattern.
- B. New data is automatically clustered during write operations.
- C. New data is written to a staging area and clustered during scheduled maintenance.
- D. New data remains unclustered until the next OPTIMIZE operation.

**Answer: D**

**Explanation:**

**Why option D is correct**

Liquid clustering in Delta Lake stores the clustering metadata (e.g., Z-order) separately from the data files.

When new rows are written to an external Delta table, they are appended as new Parquet files without any clustering effort.

The clustering information is only updated when an **OPTIMIZE** (or a compatible VACUUM/REORGANIZE) operation runs; this rewrites the affected files and rebuilds the clustering index.

Therefore, newly inserted data remains unclustered until the next explicit **OPTIMIZE** (or equivalent) operation rewrites the table.

### Why the other options are not appropriate

**A** – Delta does not reject inserts that do not match a clustering pattern; clustering is a property of the data layout, not a validation rule.

**B** – Automatic clustering on every write would incur high I/O and compute cost; Delta only clusters when an optimization step is executed.

**C** – There is no built-in staging area for clustering; clustering is performed by rewriting files, not by a scheduled maintenance stage separate from OPTIMIZE.

**Key takeaway** – In Delta Lake, liquid clustering is maintained via **OPTIMIZE**; any data added after table creation stays unclustered until that operation runs.

### References

Delta Lake Documentation – Optimizing Data Files: <https://docs.databricks.com/delta/delta-batch.html#optimize>

Delta Lake Documentation – Z-Order Clustering: <https://docs.databricks.com/delta/delta-performance.html#z-order-clustering>

## Question: 288

## Deep Dumps

A data engineer is optimizing a MERGE operation on an 800GB UC-managed table that experiences frequent updates and deletions.

Which two actions should the engineer prioritize to improve MERGE performance? (Choose two.)

- A. Apply liquid clustering using the merge join keys.
- B. Enable deletion vectors on the table if not already enabled.
- C. Partition the table by date.
- D. Use ZORDER on high-cardinality columns.
- E. Overwrite the table instead of Merge.

**Answer: AB**

### Explanation:

#### Why options A and B are the optimal choices

##### **A – Apply liquid clustering using the merge-join keys**

Liquid clustering continuously maintains data layout that aligns with the most selective filter predicates. By clustering on the columns used in the MERGE join condition, the engine can prune irrelevant micro-partitions during the lookup phase, dramatically reducing I/O and CPU overhead for large tables (800GB in this case). This results in faster match-making between source and target micro-partitions and improves overall MERGE throughput.

##### **B – Enable deletion vectors on the table if not already enabled**

Deletion vectors are a lightweight metadata structure that records rows marked for removal without immediately rewriting the data files. When frequent updates or deletes occur, keeping deletion vectors active

allows the MERGE operation to skip over logically deleted rows during the join, avoiding costly data scans and minimizing the amount of data that must be rewritten. This is especially beneficial for tables with a high volume of change-operations.

#### Why the other options are less suitable

##### C – Partition the table by date

While partitioning can improve query pruning, Databricks Unity Catalog tables are already organized into micro-partitions that provide fine-grained pruning. Adding explicit date partitions would not yield additional performance gains for a MERGE that typically touches many partitions, and it can complicate management of an already large table.

##### D – Use ZORDER on high-cardinality columns

ZORDER clustering is most effective when the ordering columns have low to moderate cardinality and are frequently used in equality predicates. High-cardinality columns provide little pruning benefit and can increase the overhead of maintaining the ZORDER metadata. For a MERGE driven by join keys, liquid clustering (optionA) is a more adaptive and performance-oriented approach.

##### E – Overwrite the table instead of Merge

Overwriting the entire table eliminates the need for a MERGE but discards the incremental nature of the operation. For a table that is only 800GB and experiences frequent updates/deletes, a full overwrite would be far more expensive in terms of I/O and compute, and it would not leverage the existing data layout optimizations that a targeted MERGE can exploit.

#### References

Unity Catalog table organization and liquid clustering: <https://docs.databricks.com/en/data-governance/unity-catalog/organize-data.html>

Deletion vectors and MERGE performance: <https://docs.databricks.com/en/delta/delta-batch.html#deletion-vectors>

#### Question: 289

#### Deep Dumps

A data engineer is building a streaming data pipeline to ingest JSON files from cloud storage into a Delta Lake table. The pipeline must process files incrementally, handle schema evolution automatically, ensure exactly-once processing, and minimize manual infrastructure management.

How should the data engineer fulfill these requirements?

- A. Use Lakeflow Spark Declarative Pipelines with a static DataFrame read, merge schema with `spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")`
- B. Use Auto Loader in batch mode with a daily job to overwrite the Delta table.
- C. Use Lakeflow Spart Declarative Pipelines with Auto Loader and enabling schema inference with `"cloudFiles.schemaEvolutionMode"= "addNewColumns"`
- D. Use traditional Spark Structured Streaming with Auto Loader, manually configuring checkpoints location and enabling schema inference with `"mergeSchema"= "true"`

#### Answer: C

#### Explanation:

#### Why optionC is the correct choice

**Fully declarative, low-maintenance pipeline** – Lakeflow (now part of Unity Catalog) lets you declare the source and target in a simple YAML/JSON spec; the platform handles job scheduling, parallelism, and dependency resolution without writing custom Spark code.



**Incremental file ingestion with exactly-once semantics** – AutoLoader automatically detects new files, tracks progress with a checkpoint location, and guarantees exactly-once processing when the checkpoint is stored in durable storage (e.g., DBFS or ADLS).

**Automatic schema evolution** – Setting `cloudFiles.schemaEvolutionMode=addNewColumns` tells AutoLoader to merge newly discovered columns into the Delta table while preserving existing data, eliminating the need for manual schema migrations.

**Optimized for streaming** – The declarative pipeline runs on Spark Structured Streaming under the hood, so it can scale out across the cluster and process data in micro-batches, meeting the “streaming” requirement.

### Why the other options are unsuitable

**A** – Uses a static DataFrame read, which cannot handle incremental file arrival; the `spark.databricks.delta.schema.autoMerge.enabled` flag is intended for DeltaLake merges, not for AutoLoader’s streaming ingestion, and it does not provide exactly-once guarantees on its own.

**B** – Overwrites the Delta table daily, breaking incremental processing and eliminating exactly-once guarantees; it also forces a full rewrite each run, increasing cost and latency.

**D** – Requires manual configuration of checkpoints, watermarks, and other streaming parameters; while it can achieve the functional goals, it does not meet the “minimize manual infrastructure management” criterion and is less declarative than the Lakeflow approach.

## References

**Auto Loader & schema evolution:** <https://docs.databricks.com/data-engagement/auto-loader/concepts/schema-evolution.html>

**Lakeflow Declarative Pipelines (Unity Catalog):** <https://docs.databricks.com/data-engineering/lakeflow/declarative-pipelines.html>

## Question: 290

Deep Dumps

Given the following PySpark code snippet in a Databricks notebook:

```
filtered_df=spark.read.format("delta").load("/mnt/daya/large_table") \
.filter ("event_date> '2024-01-01'")
filtered_df.count ()
```

The data engineer notices from the Query Profile that the scan operator for `filtered_df` is reading almost all files, despite a filter being applied.

What is the probable reason for poor data skipping?

- A. The Delta table lacks optimization that enables dynamic file pruning.
- B. The filter condition involves a data type that is excluded from data skipping support.
- C. The filter is executed only after the full data scan, which prevents data skipping from taking place.
- D. The `event_date` column is outside the table’s partitioning and Z-ordering scheme.

**Answer: D**

**Explanation:**

**Why option D is the best answer**

The filter on `event_date` can only prune files when the column is part of the table’s **partitioning** or **Z-ordering** metadata that Delta uses for file-level skipping.

In the snippet the table is stored under `/mnt/daya/large_table` without any explicit partitioning on `event_date`

and without Z-ordering on that column, so the engine cannot determine which files satisfy event\_date > '2024-01-01'. Consequently it scans all files, even though the filter is logically applicable. This directly matches optionD: “The event\_date column is outside the table’s partitioning and Z-ordering scheme.”

### Why the other options are less suitable

#### A. “The Delta table lacks optimization that enables dynamic file pruning.”

Delta’s file-pruning is driven by partition values and Z-order, not by a generic “optimization flag.” The issue is not the absence of an optimization feature but the missing physical layout that would allow pruning.

#### B. “The filter condition involves a data type that is excluded from data skipping support.”

event\_date is typically stored as a date or timestamp type, both of which are fully supported for predicate push-down. Data-type incompatibility is not the root cause here.

#### C. “The filter is executed only after the full data scan, which prevents data skipping from taking place.”

While technically the filter will be applied after file discovery if pruning cannot occur, the reason pruning fails is that the predicate cannot be mapped to partition/Z-order metadata. The statement shifts focus from the underlying physical design to a generic execution order, making it an inaccurate root-cause description.

### Conclusion

The poor data skipping stems from the fact that the filter column is not organized in the table’s partitioning or Z-ordering scheme, preventing Delta from pruning irrelevant files. Hence, optionD is the correct and most precise answer.

---

## References

**Delta Lake Documentation – Data Skipping:** <https://docs.databricks.com/delta/delta-cache.html#data-skipping>

**Delta Lake Documentation – Partitioning and Z-Ordering:** <https://docs.databricks.com/delta/optimizations.html#partitioning-and-z-ordering>

## Question: 291

## Deep Dumps

When a new Databricks project starts, the central IP team provisions the required infrastructure using Terraform and a Service Principal. This includes creating a Databricks workspace, a Unity Catalog linked to an External Location, and a Databricks group containing all project team members. Project teams must store all assets – e.g., tables and volumes, as Managed assets in Unity Catalog. This model hides infrastructure complexity while giving teams autonomy within their catalog. They can create and manage schemas, tables, volumes, and related objects but cannot rename, delete, or change catalog permissions, those remain under IT’s control.

Which rights should the project group be granted to enable this model?

- A. The group needs to have USE CATALOG and USE SCHEMA on the catalog.
- B. The group needs to have ALL PRIVILEGES and the MANAGE on the catalog.
- C. The group needs to have ALL PRIVILEGES on the catalog.
- D. The group should be made OWNER of the catalog.

**Answer: A**

**Explanation:**

**Justification**

**OptionA** correctly grants the minimal set of privileges required for project teams to work with managed

assets in Unity Catalog while keeping governance under IT.

USE CATALOG allows the group to reference the catalog in their session.

USE SCHEMA enables them to read/write schemas within that catalog.

These privileges let teams create, read, update, and delete tables, volumes, and other objects they own, but they do **not** give rights to alter catalog-level metadata (e.g., rename or delete the catalog) or to grant/ revoke permissions, which remain under the central IT team.

**OptionB** (ALL PRIVILEGES + MANAGE) would give the group excessive control, including the ability to grant or revoke permissions on the catalog, effectively bypassing IT's governance model.

**OptionC** (ALL PRIVILEGES alone) similarly provides unrestricted rights, allowing the group to modify catalog-level settings and permissions, which contradicts the requirement that catalog ownership and permission changes stay with IT.

**OptionD** (making the group **OWNER** of the catalog) would give them full ownership, including the ability to drop the catalog, change its name, and alter its metadata — far beyond the intended delegated autonomy.

Therefore, granting **USE CATALOG** and **USE SCHEMA** is the precise privilege set that enables project teams to manage their own schemas and objects while preserving centralized control over catalog-level operations.

## References

Unity Catalog privileges: [Databricks Unity Catalog Privileges Documentation](#)

Managing catalog and schema access: [Databricks Unity Catalog Access Control](#)

## Question: 292

## Deep Dumps

A data engineer needs to productionize a new Spark application written by teammate. This application has numerous external dependencies, including libraries, and requires custom environment variables and Spark configuration parameters to be set.

Which two methods will help the data engineer accomplish the task? (Choose two.)

- A. Install libraries on DBFS
- B. Add libraries to compute policies
- C. Use secrets in init scripts to store configuration data
- D. Use compute policies to set system properties, environment variables, and Spark configuration parameters.
- E. Create init scripts on DBFS.

**Answer: DE**

**Explanation:**

**Justification**

**D. Use compute policies to set system properties, environment variables, and Spark configuration parameters.**

Compute policies let you centrally define and apply Spark-specific settings (e.g., `spark.executor.memory`, `spark.sql.shuffle.partitions`) and OS-level environment variables to all clusters that use the policy. This is the recommended way to inject configuration that must be consistent across multiple jobs and clusters without hard-coding values in notebooks or scripts.

**E. Create init scripts on DBFS.**

Init scripts stored in DBFS (or mounted external storage) are executed on each driver and worker at cluster

start-up. They are the standard mechanism for installing additional libraries, copying configuration files, and setting up custom environment variables or Spark properties that are not covered by compute policies. By placing the required libraries and configuration files in DBFS and referencing them from an init script, the application runs in a reproducible environment.

### Why the other options are less suitable

**A. Install libraries on DBFS** – While libraries can be placed on DBFS, this approach does not address the need to set environment variables or Spark configuration parameters, and it does not guarantee that the libraries are automatically added to the cluster’s classpath for all users.

**B. Add libraries to compute policies** – Compute policies can reference libraries, but they cannot be used to inject arbitrary environment variables or arbitrary Spark configuration settings; they are limited to library distribution and certain cluster-wide settings.

**C. Use secrets in init scripts to store configuration data** – Secrets are intended for sensitive values (e.g., passwords, API keys) and are not a general mechanism for storing arbitrary configuration data or environment variables for a Spark application.

**D and E together** provide a complete, production-ready solution: compute policies handle the declarative Spark and OS configuration, while init scripts handle the installation of custom libraries and any additional setup required before the application runs.

---

## References

[Databricks Compute Policies](#)

[Init Scripts on DBFS](#)

## Question: 293

Deep Dumps

A healthcare analytics team is implementing a dimensional model in Delta Lake for patient care analysis. They have a date dimension table and are evaluating design options to ensure it supports a wide range of time-based analyses.

Which design approach for the date dimension will support efficient time-based querying and aggregation?

- A. Store the date as string in ISO format (YYYY-MM-DD) for readability.
- B. Pre-calculate attributes like fiscal\_period, quarter, month\_name, day\_of\_week, and holiday.
- C. Store only the date value and calculate all time attributes in queries.
- D. Create separate dimension tables for different calendar systems (fiscal, academic, etc.)

**Answer: B**

**Explanation:**

### Why option B is the optimal design

**Pre-computed time attributes** (fiscal period, quarter, month name, day-of-week, holiday flag, etc.) are materialized once during the ETL load, eliminating costly on-the-fly calculations for every query. Delta Lake’s columnar storage and Z-ordering can efficiently prune and aggregate on these deterministic columns, delivering fast time-based scans and roll-ups.

The attributes are **immutable** and **query-ready**, enabling straightforward joins to fact tables and supporting common analytical patterns such as YTD, MTD, rolling windows, and holiday adjustments without additional processing overhead.

Storing the raw date alongside the pre-computed fields preserves the original value for any ad-hoc calculations while still providing the performance benefits of denormalized time dimensions.

### Why the other options are less suitable

**A – ISO-format string** – While human-readable, string dates cannot be directly used for numeric range predicates or arithmetic operations; they force the engine to parse the string on each scan, degrading performance.

**C – Calculate attributes at query time** – Re-computing quarter, fiscal period, etc., on every query defeats the purpose of a dimensional model and introduces unnecessary CPU load, especially on large fact tables.

**D – Separate calendars** – Creating distinct dimension tables for each calendar system introduces redundancy and complicates joins; unless the business explicitly requires multiple independent calendars, it adds unnecessary complexity without measurable query-performance gain.

### Key takeaway

Materializing a rich set of time-related attributes in the date dimension leverages Delta Lake's columnar and predicate-pushdown capabilities, ensuring low-latency, scalable time-based analytics.

## References

Delta Lake Documentation – [Dimension Tables and Best Practices](#)

Delta Lake Documentation – [Performance Tuning with Z-Ordering and Data Skipping](#)

### Question: 294

Deep Dumps

A data engineer is tasked with ensuring that a Delta table in Databricks continuously retains deleted files for 15 days (instead of the default 7 days), in order to permanently comply with the organization's data retention policy.

Which code snippet correctly sets this retention period for deleted files?

A. `spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")`

```
from delta.tables import *

deltaTable = DeltaTable.forPath(spark, "/mnt/data/my_table")
deltaTable.deletedFileRetentionDuration = "interval 15 days"
```

B.

C. `spark.sql("VACUUM my_table RETAIN HOURS")`

D. `spark.conf.set("spark.databricks.delta.deletedFileRetemtionDuration", "15 days")`

Answer: A

### Question: 295

Deep Dumps

A Data Engineer is building a fraud detection pipeline that calls out to Open AI, via a Python library, and needs to include an access token when using the API.

Which Databricks CLI command should the Data Engineer use to create the secret?

A. `databricks secrets put-secret KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)`

B. `databricks tokens put-token SCOPE KEY; dbutils.tokens.get (SCOPE, KEY)`

C. `databricks secrets put-secret SCOPE KEY; dbutils.secrets.get (SCOPE, KEY)`

D. databricks tokens put-token KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)

**Answer: C**

**Explanation:**

**Justification**

The Databricks CLI command for creating a secret must specify the **scope first, then the key name** (databricks secrets put-secret <scope> <key>).

After the secret is stored, it is retrieved in Python with dbutils.secrets.get(scope, key).

Option **C** follows this exact syntax: databricks secrets put-secret SCOPE KEY; dbutils.secrets.get (SCOPE, KEY).

Option **A** reverses the order of the arguments and uses an invalid dbutils.secrets.get (KEY, SCOPE) call.

Options **B** and **D** misuse the tokens sub-command, which is intended for managing personal access tokens, not for secret storage, and also have the argument order wrong.

Therefore, option **C** is the only command that correctly creates a secret and retrieves it in a way that matches the required API usage with an access token.

**References**

Databricks CLI secrets command: <https://docs.databricks.com/en/data/secrets/cli.html>

dbutils.secrets.get documentation:

<https://docs.databricks.com/en/data/secrets/index.html#dbutilssecretsget>

**Question: 296**

**Deep Dumps**

A company has a task management system that tracks the most recent status of tasks. The system takes task events as input and processes events in near real-time using Lakeflow Spark Declarative Pipelines. A new task event is ingested into the system when a task is created or the status is changed. Lakeflow Spark Declarative Pipelines provides a streaming table (table name: tasks\_status) for the BI users to query. The table represents the latest status of all tasks and includes 5 columns: task\_id(unique for each task), task\_name, task\_owner, task\_status, task\_event\_time. The table enables 3 properties: deletion vectors, row tracking, and change data feed.

A data engineer is asked to create a new Lakeflow Spark Declarative Pipelines to enrich the “task\_status” table in near real-time by adding one additional column representing task\_owner’s department, which can be looked up from a static dimension table (table name: employee).

How should this enrichment be implemented?

A. Create a new Lakeflow Spark Declarative Pipelines: use readStream() function with option readChangeFeed to read tasks\_status table CDF; enrich with the employee table; create a new streaming table as the result table and use apply\_changes() function to process the changes from the enriched CDF.

B. Create a new Lakeflow Spark Declarative pipeline: use the readStream()function to read tasks\_status table, enrich with the employee table; store the result in a new streaming table.

C. Create a new Lakeflow Spark Declarative Pipeline: use the readStream() function with the option skipChangeCommits to read the tasks\_status table; enrich with the employee table; store the result in a new streaming table.

D. Create a new Lakeflow Spark Declarative Pipeline: use the read () function to read tasks\_status table; enrich with employee table; store the result in a materialized view.

**Answer: A**

**Explanation:**

**Why optionA is the correct implementation**



**Streaming change-data-feed (CDF) consumption** – `readStream(..., option readChangeFeed=True)` reads only the incremental changes of `tasks_status`, preserving the source's CDC semantics (deletion vectors, row tracking, etc.). This guarantees that the enrichment works on every new/updated/deleted task event without reprocessing the full table.

**Explicit change-processing** – `apply_changes()` can be applied to the enriched CDF stream to materialize the latest state into a new streaming table while handling inserts, updates, and deletes correctly. This matches the requirement to maintain a table that always reflects the latest status plus the added department column.

**Near-real-time enrichment** – By joining the CDC stream of `tasks_status` with the static employee dimension inside the same pipeline, the department lookup is performed only on the changed rows, achieving low-latency updates.

**Result table creation** – The pipeline writes the processed stream to a new streaming table, giving BI users a fresh view that includes the department column while still supporting the three CDC properties.

### Why the other options are unsuitable

**B** – Uses `readStream()` without `readChangeFeed`; it would read the entire table each micro-batch, losing CDC guarantees and causing unnecessary full-table scans, which defeats the purpose of near-real-time change tracking.

**C** – The `skipChangeCommits` option bypasses commit handling, preventing proper CDC consumption and leading to missed or duplicated changes when applying updates.

**D** – `read()` is a batch operation; it cannot provide a continuous streaming source. Storing the result in a materialized view would not support incremental updates and would not preserve CDC semantics required for downstream BI queries.

### References

[Lakeflow Spark Declarative Pipelines – readStream with CDC](#)  
[apply\\_changes\(\) – processing CDC streams](#)

## Question: 297

## Deep Dumps

A data engineer is reviewing the PySpark code to copy a part of the production dataset to the sandbox environment, and needs to be sure that no PII(Personally Identifiable Information) data is being copied. After checking the sales table, the data engineer notices that it has user emails as the only PII data included as well as being the only column to identify the user.

from pyspark.sql import functions as F

```
Load data from a table in the production environment
df = spark.read.table("production_catalog.production_schema.sales").sample(fraction=0.1)

Anonymize the data before saving it to the sandbox
sandbox_df = << fill in >>

Write the anonymized data to a table in the sandbox environment
sandbox_df.write.format("delta").mode("overwrite").saveAsTable("sandbox_catalog.sandbox_schema.sales")
```

Which anonymised code should be used to achieve the required outcome?

- A. `df.withColumn("user_email", F.expr("uuid()"))`
- B. `df.withColumn("user_email", F.sha2("user_email"))`
- C. `df.withColumn("hashed_email", sha2("user_email"))`
- D. `df.withColumn("user_email", F.regexp_replace("user_email", "@*", "@anonymized.com"))`

## Question: 298

## Deep Dumps

A data engineer wants to ingest a large collection of image files (JPEG and PNG) from cloud object storage into Unity Catalog-managed table for further analysis and visualization.

Which two configurations and practices are recommended to incrementally ingest these images into the table? (Choose two.)

- A. Use Auto loader and set `cloudFiles.format` to "TEXT"
- B. Use Auto loader and set `cloudFiles.format` to "BINARYFILE"
- C. Use Auto loader and set `cloudFiles.format` to "IMAGE"
- D. Use the `pathGlobFilter` option to select only image files (e.g.. "\*.jpg, \*.png")
- E. Move files to a volume and read with SQL editor

Answer: BD

Explanation:

Why the correct options (BandD) are the best choice

**Auto Loader with `cloudFiles.format` = "BINARYFILE"** – Auto Loader can continuously watch a cloud-object storage folder and automatically ingest new files as they appear. The BINARYFILE format reads each file as a single binary column, preserving the original image bytes without requiring schema enforcement or manual parsing. This is the recommended way to ingest large collections of JPEG/PNG files incrementally because it scales, handles schema-evolution gracefully, and supports checkpointing for fault-tolerance.

**`pathGlobFilter` to restrict to image patterns ("\*.jpg", "\*.png")** – Adding a glob filter limits the scan to only the files that match the desired extensions. This reduces I/O and processing overhead by skipping unrelated objects, improves job performance, and ensures that only valid image files are loaded into the Unity Catalog table.

Why the other options are less suitable

**A. `cloudFiles.format` = "TEXT"** – The TEXT format expects human-readable text data; it cannot correctly interpret binary image payloads and would corrupt the data.

**C. `cloudFiles.format` = "IMAGE"** – Databricks does not provide an IMAGE format for Auto Loader; using it would result in an unsupported configuration error.

**E. Move files to a volume and read with SQL editor** – Relocating files adds unnecessary operational steps, breaks the incremental nature of Auto Loader, and prevents direct streaming from object storage, making the pipeline less efficient and harder to maintain.

References

**Auto Loader documentation:** <https://docs.databricks.com/data-engineering/autoloader/what-is-autoloader.html>

**BinaryFile format guide:** <https://docs.databricks.com/data-engineering/autoloader/file-formats.html#binaryfile>

**`pathGlobFilter` option:** <https://docs.databricks.com/data-engineering/autoloader/monitor-drop-dir.html#pathglobfilter-option>

These resources confirm that Auto Loader with the BINARYFILE format and the use of `pathGlobFilter` are the

### Question: 299

### Deep Dumps

A data engineer is configuring Delta Sharing for a Databricks-to-Databricks scenario to optimize read performance. The recipient needs to perform time travel queries and streaming reads on shared sales data.

Which configuration will provide the optimal performance while enabling these capabilities?

- A. Use the open sharing protocol instead of Databricks-to-Databricks sharing for better performance.
- B. Share tables WITHOUT HISTORY and enable partitioning for better query performance.
- C. Share tables WITH HISTORY, ensure tables don't have partitioning enabled. and enable CDF before sharing.
- D. Share the entire schema WITHOUT HISTORY and rely on recipient-side caching for performance.

**Answer: C**

**Explanation:**

**Why option C is the optimal configuration**

**Time-travel support requires history** – Sharing tables **WITH HISTORY** preserves the transaction log so the recipient can query previous versions of the data. Without history, time-travel queries would be impossible.

**Change Data Feed (CDF) accelerates incremental reads** – Enabling CDF on the source tables allows the recipient to pull only the new changes, dramatically reducing I/O and latency for streaming consumption.

**Avoid unnecessary partitioning in the share** – When the source tables are not partitioned before sharing, the recipient can apply its own partitioning strategy that matches query patterns, avoiding the overhead of mismatched partition pruning that can occur when source partitions are forced onto the downstream system.

**Databricks-to-Databricks sharing outperforms the open protocol** – The native Delta Sharing implementation within Databricks clusters leverages internal optimizations (e.g., zero-copy reads, shared cache) that are not available with the generic open-source protocol, delivering higher throughput and lower latency for large sales datasets.

**Why the other options are less suitable**

**Option A** – The open sharing protocol lacks the deep integration and performance enhancements of native Databricks-to-Databricks sharing; it also does not automatically expose CDF or guarantee optimal read paths.

**Option B** – Sharing **WITHOUT HISTORY** eliminates time-travel capability, which directly contradicts the requirement for historical queries.

**Option D** – Omitting history again prevents time-travel, and relying solely on recipient-side caching does not address the need for consistent, low-latency streaming reads; caching can become stale and does not provide the same guarantees as CDF-enabled shares.

### References

**Delta Sharing – Share data across workspaces:** <https://learn.microsoft.com/en-us/azure/databricks/delta-sharing>

**Change Data Feed (CDF) for Delta Sharing:** <https://learn.microsoft.com/en-us/azure/databricks/delta-sharing#change-data-feed>

These documents detail the configuration steps for enabling history, CDF, and best-practice sharing settings to achieve high-performance, time-travel-compatible reads in a Databricks-to-Databricks scenario.

A data organization has adopted Delta Sharing to securely distribute curated datasets from a Unity Catalog-enabled workspace. The data engineering team is sharing large Delta tables with an internal partner via Databricks-to-Databricks and aggregated reports with an external client via Open Sharing. While testing new sharing workflows, the data engineering team encounters challenges related to access control, data update visibility, and shareable object types.

What is a limitation of the Delta Sharing protocol or implementation when used with Databricks-to-Databricks or Open Sharing?

- A. Delta Sharing does not support Unity Catalog enabled tables; only legacy Hive Metastore tables are shareable.
- B. With Databricks-to-Databricks sharing, Unity Catalog recipients must re-ingest data manually using COPY INTO or REST APIs.
- C. Delta Sharing (both Databricks-to-Databricks and Open sharing) allows recipients to modify the source data if they have SELECT privileges.
- D. With Open sharing, recipients cannot access Volumes, Models, or notebooks — only static Delta tables are supported.

**Answer: D**

**Explanation:**

**Why option D is the correct limitation**

**Open Sharing is read-only and table-centric** – When a Delta table is shared via Open Sharing, the recipient can only read the data stored in Delta tables. Objects such as **Volumes, Models, and notebooks** are not part of the sharing surface; they remain confined to the source workspace. Consequently, the share does not expose any mutable or compute-oriented assets, limiting the share to static Delta table snapshots.

**Databricks-to-Databricks sharing does not grant write access** – Even when using Unity Catalog-enabled tables, the sharing protocol only provides a **read-only view**. Recipients cannot alter the underlying data; any modifications require them to copy the data into their own workspace (e.g., via COPY INTO or REST APIs). This reinforces that the share is immutable from the recipient side.

**Why the other options are inaccurate**

**Option A** – Incorrect. Delta Sharing works **with Unity Catalog-enabled tables**; in fact, Unity Catalog is the recommended metadata layer for sharing because it enables fine-grained access control and governance.

**Option B** – Incorrect. Recipients in a Databricks-to-Databricks share can **directly query the shared tables** using standard SQL (or Spark APIs) without needing to re-ingest the data manually. Manual ingestion is only required when moving data out of the shared view into a different storage format or location.

**Option C** – Incorrect. Possessing SELECT privileges on a shared view **does not permit recipients to modify the source data**. The sharing mechanism enforces a **read-only** semantics; any write operation must be performed by the data owner in the originating workspace.

**References**

Delta Sharing documentation (Databricks): <https://learn.databricks.com/delta-sharing>

Unity Catalog and Delta Sharing integration guide: <https://docs.databricks.com/en/data-governance/unity-catalog/delta-sharing.html>

How are the operational aspects of Lakeflow Spark Declarative Pipelines from Spark Structured Streaming different?

- A. Lakeflow Spark Declarative Pipelines automatically handle schema evolution, while Structured Streaming always requires manual schema management.
- B. Structured Streaming can process continuous data streams, while Lakeflow Spark Declarative Pipelines cannot.
- C. Lakeflow Spark Declarative Pipelines manage the orchestration of multi-stage pipelines automatically, while Structured Streaming requires external orchestration for complex dependencies.
- D. Lakeflow Spark Declarative Pipelines can write to Delta Lake format while structured streaming cannot.

**Answer: C**

**Explanation:**

**Why option C is the correct answer**

**Lakeflow Spark Declarative Pipelines** are built on top of Spark Structured Streaming but add a **declarative, pipeline-level abstraction**. They automatically orchestrate multi-stage data flows (ingest → transform → enrich → sink) and handle dependencies, retries, and checkpointing without user-written DAG code.

**Spark Structured Streaming**, by contrast, is a low-level streaming engine. It provides the runtime for continuous or micro-batch processing but **does not manage complex multi-stage workflow orchestration**; users must implement scheduling, chaining, or external workflow tools (e.g., Airflow, Dagster) to coordinate multiple streaming jobs with dependencies.

**Why the other options are not correct**

**Option A** – Both Lakeflow and Structured Streaming can infer or evolve schemas automatically; manual schema management is not a mandatory requirement for Structured Streaming.

**Option B** – Structured Streaming **can** process continuous data streams, and Lakeflow pipelines also rely on Structured Streaming under the hood, so the claim that Lakeflow cannot is false.

**Option D** – Both technologies can write to Delta Lake; the ability to write to Delta is not exclusive to Lakeflow.

**References**

Databricks Lakehouse Documentation – Lakeflow Spark Declarative Pipelines:

<https://learn.databricks.com/lakeflow>

Apache Spark Structured Streaming Guide – Streaming Data Processing:

<https://spark.apache.org/docs/latest/streaming-structured-streaming.html>

**Question: 303**

**Deep Dumps**

A workspace admin has created a new catalog called 'finance\_data' and wants to delegate permission management to a finance team lead without giving them full admin rights.

Which privilege should be granted to the finance team lead?

- A. GRANT\_OPTION privilege on the finance\_data catalog.
- B. Make the finance team lead a metastore admin.
- C. MANAGE privilege on the finance\_data catalog.
- D. ALL PRIVILEGES on the finance\_data catalog.

**Answer: C**

**Explanation:**

## Technical justification

**Option C –MANAGE privilege on the finance\_data catalog** is the correct choice.

The MANAGE privilege allows a principal to **grant, revoke, and alter privileges** on the specified catalog and its objects, but it does **not** confer the higher-level ALL PRIVILEGES or metastore-wide administration rights. By granting MANAGE to the finance team lead, the admin delegates **catalog-level permission management** while retaining overall control of the workspace, which aligns with the requirement of delegating permission management without full admin rights.

### Why the other options are unsuitable

**A. GRANT\_OPTION privilege on the finance\_data catalog** – This privilege only permits the grantee to **pass on** privileges that they already hold; it does **not** enable them to **create or modify** permissions on the catalog itself. Hence it cannot be used to delegate permission-management authority.

**B. Make the finance team lead a metastore admin** – Metastore admin rights are **workspace-wide** and include the ability to manage the entire metastore configuration, which far exceeds the limited delegation needed for a single catalog. This would effectively give admin rights, violating the “without giving them full admin rights” constraint.

**D. ALL PRIVILEGES on the finance\_data catalog** – Granting ALL PRIVILEGES provides the grantee with **complete control** over the catalog, including the ability to drop or alter the catalog itself. This is more permissive than required and could expose the workspace to unintended changes.

**Conclusion** – Granting MANAGE on the finance\_data catalog precisely meets the need to let the finance team lead manage permissions while preserving the admin’s overall authority.

### References

Databricks SQL Access Control: <https://docs.databricks.com/en/data-governance/access-control/index.html>  
Catalog privileges: <https://docs.databricks.com/en/data-governance/catalog-privileges.html>

## Question: 304

## Deep Dumps

A data engineer is configuring a Lakeflow Spark Declarative Pipelines to process CDC data from a source. The source events sometimes arrive out of order, and multiple updates may occur with the same update\_timestamp but with different update\_sequence\_id.

What should the data engineer do to ensure events are sequenced correctly?

- A. Use SEQUENCE BY STRUCT(event\_timestamp, update\_sequence\_id) in AUTO CDC APIs.
- B. Set track history column list to [event timestamp, event id] in AUTO CDC APIs.
- C. Use dropduplicate() to remove out-of-order and duplicate records in LDP.
- D. Use a window function to sort update\_sequence\_id within the same partition, i.e., update\_timestamp in the LDP pipeline.

**Answer: A**

**Explanation:**

**Justification**

**Option A – SEQUENCE BY STRUCT(event\_timestamp, update\_sequence\_id)**

This syntax tells Lakeflow Spark Declarative Pipelines to order records first by event\_timestamp and then by update\_sequence\_id when timestamps are equal.



It directly addresses the problem of out-of-order events and ties that share the same `update_timestamp` but have distinct `update_sequence_ids`, guaranteeing a deterministic total order.

The ordering is applied during CDC ingestion, so downstream transformations receive a correctly sequenced stream without additional cleanup steps.

#### Option B – track history column list = [event timestamp, event id]

Tracking history is useful for auditability, but it does **not** enforce ordering of incoming events. It merely records metadata; the pipeline still processes records in the order they arrive.

#### Option C – `dropduplicate()`

Removing duplicates eliminates exact repeats, but it does not reorder out-of-order records nor resolve ties where timestamps are identical but sequence IDs differ. The underlying ordering issue remains.

#### Option D – Window function on `update_timestamp`

While a window can sort data within a partition, applying it after ingestion adds unnecessary complexity and can be inefficient at scale. Moreover, window functions do not guarantee a globally deterministic order across partitions unless explicitly defined, which is less straightforward than the declarative `SEQUENCE BY` clause.

#### Conclusion

Option **A** provides the most precise, declarative, and scalable method to enforce the required ordering directly at the CDC ingestion point, making it the optimal choice for the given scenario.

---

## References

Lakeflow Spark Declarative Pipelines – Sequence By Syntax: <https://docs.databricks.com/data-engineering/lakeflow/sequence-by.html>

CDC Processing in Lakeflow – Ordering Events: <https://docs.databricks.com/data-engineering/cdc/auto-cdc-ordering.html>

## Question: 305

## Deep Dumps

A streaming video analytics team ingests billions of events daily into a Unity Catalog-managed Delta table `video_events`. Analysts run ad-hoc point-lookup queries on columns like `user_id`, `campaign_id`, and `region`. The team manually runs `OPTIMIZE video_events Z-ORDER user_id, campaign_id, region`, but still sees poor performance on recent data, and dislikes the operational overhead. The team wants a hands-off way to keep hot columns co-located as query patterns evolve.

Which Delta capability should the team leverage on `video_events`?

- A. Enable auto-compaction (`optimizeWrite` & `autoCompact`).
- B. Schedule `OPTIMIZE/ZORDER` to run after each job to improve recent file performance.
- C. Utilize Liquid Clustering `CLUSTER BY AUTO` and Predictive Optimization.
- D. Enable Delta caching.

**Answer: C**

**Explanation:**

**Why option C is the best choice**

**Liquid Clustering (`CLUSTER BY AUTO`)** automatically discovers hot columns and reorganizes data files to keep those columns physically co-located, eliminating the need for manual `OPTIMIZE ... Z-ORDER`

commands.

**Predictive Optimization** works together with Liquid Clustering to anticipate upcoming query patterns and proactively maintain the optimal layout, which is ideal for evolving ad-hoc lookup workloads on `user_id`, `campaign_id`, and `region`.

The capability is **hands-off**: once the table is defined with `CLUSTER BY AUTO`, Delta continuously tunes the layout in the background, delivering consistently low-latency point-lookups without operational overhead.

#### Why the other options are less suitable

**A – Enable auto-compaction (`optimizeWrite` & `autoCompact`)** – Auto-compaction addresses small file proliferation and storage efficiency; it does **not** reorganize data for columnar co-location, so query latency for point lookups remains unchanged.

**B – Schedule `OPTIMIZE/ZORDER` after each job** – This still requires **manual scheduling and tuning**; as the team observes, the operational burden persists and recent data may still become unoptimized before the next run.

**D – Enable Delta caching** – Caching improves read latency for repeatedly accessed data but does **not** restructure the underlying file layout, so hot columns are not guaranteed to stay together as query patterns shift.

#### References

Delta Lake Documentation – Liquid Clustering: <https://docs.databricks.com/delta/delta-batch.html#liquid-clustering>

Delta Lake Documentation – Predictive Optimization: <https://docs.databricks.com/delta/delta-auto-optimize.html>

### Question: 306

### Deep Dumps

A data engineer is designing a data pipeline in Databricks that needs to process records from a Kafka stream where late-arriving data common.

Which approach should the data engineer use?

- A. Use a watermark to specify the allowed lateness to accommodate records that arrive after their expected window, ensuring correct aggregation and state management.
- B. Use an Auto CDC pipeline with batch tables to simplify late data handling.
- C. Implement a custom solution using Databricks Jobs to periodically reprocess all historical data
- D. Use batch processing and overwrite the entire output table each time to ensure late data is incorporated correctly.

**Answer: A**

**Explanation:**

**Justification**

**OptionA** – Watermarking is the standard technique in Structured Streaming (the engine behind Kafka-source reads in Databricks) to define **maximum allowed lateness** for a given event time. By setting a watermark that exceeds typical late-arrival times, the query can keep state for longer and still produce correct aggregations while discarding records that are too late. This preserves exactly-once semantics, avoids unnecessary reprocessing, and integrates cleanly with checkpointing and state stores.

**OptionB** – Auto CDC pipelines are useful for change-data-capture from source systems, but they do not address the core streaming-window lateness problem. They also add unnecessary complexity when the only requirement is to tolerate late data within a defined window.

**OptionC** – Re-processing all historical data with Jobs defeats the purpose of a real-time stream. It introduces latency, high compute cost, and makes it impossible to maintain up-to-date state for incremental aggregations.

**OptionD** – Overwriting the entire batch output each run works for pure batch but breaks the incremental nature of streaming. It forces a full rewrite on every micro-batch, causing poor performance and losing the ability to handle late data gracefully within a streaming query.

Therefore, **OptionA** is the only approach that directly leverages Spark Structured Streaming's watermark mechanism to handle late-arriving records while maintaining correct stateful aggregations in a scalable, fault-tolerant manner.

---

## References

Databricks–[Watermarking in Structured Streaming](#)

Apache Spark–[Event-time processing and watermarks](#)

## Question: 307

Deep Dumps

A data engineering team is configuring access controls in Databricks Unity Catalog. They grant the SELECT privilege on the sales catalog to the analyst\_group, expecting that members of this group will automatically have SELECT access to all current and future schemas, tables, and views within the sales catalog.

What describes the privilege inheritance behavior in Unity Catalog?

- A. Privileges in Unity Catalog do not cascade; SELECT must be explicitly granted on each schema and table, even if granted at the catalog level.
- B. Granting SELECT on a catalog automatically applies SELECT to all current and future schemas, tables, and views within that catalog.
- C. Granting SELECT at the catalog level applies to existing schemas and tables but not to those created in the future.
- D. Privileges granted at the schema level override any catalog-level privileges and prevent access unless explicitly revoked.

**Answer: A**

**Explanation:**

**Why optionA is correct**

In Unity Catalog, a privilege granted at the **catalog** level is not automatically propagated to the contained schemas, tables, or views.

Access rights must be **explicitly granted** on each target object (schema, table, view, etc.) for the intended grantee.

This design enforces least-privilege and prevents accidental broad exposure when new objects are created.

**Why the other options are inaccurate**

**B** – Incorrect; catalog-level SELECT does **not** automatically apply to all current and future objects.

**C** – Partially true for existing objects but misleading; the statement implies a future-object exemption that does not exist in the default behavior.

**D** – Irrelevant; schema-level privileges do not override catalog-level grants in a way that requires revocation, and the wording misrepresents Unity Catalog's inheritance model.

**Key takeaway**

Privilege inheritance in Unity Catalog is **object-specific**; you must grant SELECT (or any other privilege) on each schema, table, or view individually, or use explicit “future” grant syntax if you want automatic assignment to newly created objects.

## References

Unity Catalog privileges overview: <https://docs.databricks.com/data-governance/unity-catalog/privileges.html>  
Managing default and future privileges: <https://docs.databricks.com/data-governance/unity-catalog/grant-privileges.html>

## Question: 308

## Deep Dumps

An organization processes customer data from web and mobile applications. Data includes names, emails phone numbers, and location history. Data arrives both as Batch files from an SFTP drop (daily) and Streaming JSON events from Kafka (real-time).

To comply with internal data privacy policies, the following requirements must be met:

- Personally identifiable information (PII) like email, phone\_number, and ip\_address must be masked or anonymized before storage
- Both batch and streaming pipelines must apply consistent PII handling
- Masking logic must be auditable and reproducible
- The masked data must still be usable for downstream analytics

How should the data engineer design a compliant data pipeline on Databricks that supports both batch and streaming modes, applies data masking to PII, and maintains traceability of transformations for audits?

- A. Ingest both batch and streaming data using Lakeflow Spark Declarative Pipelines, and apply masking via Unity Catalog column masks at read time to avoid modifying the data during ingestion.
- B. Load batch data with notebooks and ingest streaming data with SQL Warehouses; use Unity Catalog column masks on Silver tables to redact fields after storage.
- C. Use Lakeflow Spark Declarative Pipelines for batch and streaming ingestion define a PII masking function, and apply it during Bronze ingestion before writing to Delta Lake.
- D. Allow PII to be stored unmasked in Bronze for lineage tracking, then apply masking logic in Gold tables used for reporting.

**Answer: C**

**Explanation:**

**Why optionC is the optimal design**

**Unified ingestion layer** – Lakeflow Spark Declarative Pipelines can declaratively consume both batch files from SFTP and continuous JSON streams from Kafka, guaranteeing identical source handling for the two modes.

**Early-stage masking** – Applying the PII-masking function during the Bronze-layer write ensures that raw PII never persists in the lake, satisfying the “mask before storage” rule and providing a single point of truth for the transformation logic.

**Reproducible & auditable logic** – The masking routine can be version-controlled (e.g., as a Python/Scala UDF or Spark SQL expression) and registered in the pipeline’s metadata, giving a clear lineage of how each field is transformed.

**Consistent downstream view** – Downstream Bronze tables are already masked; subsequent Silver/Gold layers simply reference these clean tables, so analytics can proceed without additional masking steps and without risk of accidental PII exposure.

**Delta Lake guarantees** – Writing masked data to Delta Lake preserves ACID semantics, schema enforcement, and time-travel capabilities, which are essential for auditability and rollback if a masking rule changes.

**Why the other options are less suitable**

**OptionA** – Unity Catalog column masks are evaluated at read time, which means raw PII remains stored in the Bronze layer. This violates the requirement that PII be masked before storage and makes it difficult to guarantee that downstream consumers never see unmasked data.

**OptionB** – Mixing notebooks for batch and SQL Warehouses for streaming creates two separate ingestion pipelines with divergent logic, breaking the “consistent handling” requirement. Applying masks only on Silver tables after storage again leaves PII exposed in Bronze, and the approach lacks a single auditable transformation definition.

**OptionD** – Storing PII unmasked in Bronze for lineage tracking contradicts the core privacy rule. While lineage can be retained via Delta’s change data feed, the presence of raw PII in the lake violates policy and would require additional, error-prone redaction steps later, increasing audit complexity.

## References

Lakeflow Spark Declarative Pipelines: <https://learn.databricks.com/lakeflow>

Unity Catalog column masking overview: <https://learn.databricks.com/unity-catalog/column-masks>

## Question: 309

## Deep Dumps

Which method can be used to determine the total wall-clock time it took to execute a query?

- A. In the Spark UI, take the job duration of the longest-running job associated with that query.
- B. In the Spark UI, take the sum of all task durations that ran across all stages (or all jobs associated with that query).
- C. Open the Query Profiler associated with that query and use the Total wall-clock duration metric.
- D. Open the Query Profiler associated with that query and use the Aggregated task time metric.

**Answer: C**

**Explanation:**

**Justification**

### **OptionC – Query Profiler → Total wall-clock duration**

The Query Profiler reports the Total wall-clock duration of the entire query, which is the elapsed time from the start of the first stage to the completion of the last stage. This metric directly reflects the wall-clock execution time of the query as a whole.

### **OptionA – Longest-running job duration**

Using only the duration of the longest job under-estimates the overall wall-clock time because other jobs may run concurrently and contribute to the total elapsed time. The sum of all job durations would be required for an accurate measurement, which OptionA does not provide.

### **OptionB – Sum of all task durations across stages**

Summing task durations double-counts overlapping work (tasks in different stages can run in parallel). This approach yields a value that is typically larger than the true wall-clock time and is not the metric reported by Spark for query duration.

### **OptionD – Aggregated task time**

“Aggregated task time” is essentially the same as the sum of task durations and therefore suffers from the same over-estimation issue. It is not the metric used to represent the query’s wall-clock duration.

Hence, the most accurate and officially provided method is to read the **Total wall-clock duration** from the Query Profiler.

**References**

### Question: 310

Deep Dumps

A data architect is designing a Databricks solution to efficiently process data for different business requirements.

In which scenario should a data engineer use a materialized view compared to a streaming table?

- A. Precomputing complex aggregations and joins from multiple large tables to accelerate BI dashboard performance.
- B. Processing high-volume, continuous clickstream data from a website to monitor user behavior in real-time.
- C. Ingesting data from Apache Kafka topics with sub second processing requirements for immediate alerting.
- D. Implementing a CDC (Change Data Capture) pipeline that needs to detect and respond to database changes within seconds.

**Answer: A**

**Explanation:**

**Why option A is the correct choice**

A materialized view stores the result of a query (often involving complex joins and aggregations) and can be refreshed incrementally or on a schedule.

Because the computation is performed once and the result is cached, subsequent queries against the view are served from storage, dramatically reducing I/O and CPU overhead for BI dashboards that repeatedly scan large fact tables.

This aligns with the requirement to **pre-compute aggregations and joins from multiple large tables** to accelerate dashboard performance, which is exactly what materialized views are optimized for in Databricks SQL.

**Why the other options are less suitable**

**B – Real-time clickstream monitoring** – Streaming tables (or Delta Live Tables) are designed for continuous ingestion and low-latency processing of event data; materialized views do not provide the necessary incremental update semantics for sub-second latency.

**C – Kafka ingestion with sub-second alerting** – This scenario demands a true streaming pipeline that can handle out-of-order events and guarantee exactly-once semantics; materialized views lack the streaming engine's fault-tolerance and latency guarantees.

**D – CDC pipeline with second-level change detection** – Change Data Capture requires a change-data-feed source and near-real-time consumption; streaming tables or Delta Live Tables are built for this use case, whereas materialized views are static snapshots and cannot reliably capture every change event.

**References**

Materialized Views in Databricks SQL: <https://docs.databricks.com/sql/glossary/materialized-view.html>

Delta Live Tables Overview (streaming vs. batch): <https://docs.databricks.com/delta-live-tables/index.html>

### Question: 311

Deep Dumps

A query is taking too long to run. After investigating the Spark UI, the data engineer discovered a significant amount of disk spill. The type of compute instance the data engineer used only provides a core-to-memory ratio of 1:2.



What are the two steps the data engineer should take to minimize spillage? (Choose two.)

- A. Increase `spark.sql.files.maxPartitionBytes`.
- B. Choose a compute instance with more disk space.
- C. Choose a compute instance with a higher core-to-memory ratio
- D. Reduce `spark.sql.files.maxPartitionBytes`.
- E. Choose a compute instance with more network bandwidth.

**Answer: CD**

**Explanation:**

**Why the correct choices are the best**

**C – Choose a compute instance with a higher core-to-memory ratio**

A higher core-to-memory ratio means more CPU resources relative to the allocated memory. This lets the executor run more tasks in parallel without needing to store intermediate data on disk, directly reducing the amount of spill that triggers disk I/O.

**D – Reduce `spark.sql.files.maxPartitionBytes`**

Lowering the maximum partition size forces Spark to create more, smaller partitions. Smaller partitions fit more comfortably into executor memory, so less data has to be spilled to disk during the shuffle or aggregation phases.

**Why the other options are not appropriate**

**A – Increase `spark.sql.files.maxPartitionBytes`** – Larger partitions increase the volume of data each task must hold in memory, which raises the probability of spilling rather than alleviating it.

**B – Choose a compute instance with more disk space** – Adding disk capacity does not address the root cause (memory pressure); spills will still occur if the memory configuration is sub-optimal.

**E – Choose a compute instance with more network bandwidth** – Network bandwidth affects shuffle traffic but does not directly influence the amount of data spilled to disk caused by memory constraints.

**References**

Spark Configuration – `spark.sql.files.maxPartitionBytes`: <https://spark.apache.org/docs/latest/sql-configuration.html#spark.sql.files.maxpartitionbytes>

Databricks Compute Instance Types and sizing best practices: <https://docs.databricks.com/clusters/compute.html#instance-types-and-sizes>

**Question: 312**

**Deep Dumps**

A data engineering team has a time-consuming data ingestion job. It has three data sources, and each data ingestion notebook takes about one hour to load new data. The job had been working fine, loading data every midnight. However, one day they received an alert message stating the job had failed. By investigating the cause, they figured out that the failed notebook code was updated, and a new required configuration was introduced to parameterize a hardcoded value without notice. Now, the team has to fix the issue as quickly as possible to load the latest data from the failing data source.

Which action should the team take in this scenario?

- A. Update the task by adding the missing task parameter, and manually run the job.
- B. Share the analysis with the failing notebook owner so that they can fix it quickly.
- C. Repair the run with the new parameter.
- D. Repair the run with the new parameter, and update the task by adding the missing task parameter.

**Answer: D**

**Explanation:**

**Why option D is the best choice**

**Repair the run with the new parameter** – The immediate failure is caused by a missing configuration that was introduced in the notebook code. Adding the required parameter to the current run restores execution and gets the latest data loaded without delay.

**Update the task by adding the missing task parameter** – The parameter is now part of the task definition, so every future execution will automatically include it. This prevents the same failure from recurring and aligns with the principle of “fix-once, protect-forever.”

**Combined action (D)** therefore addresses both the symptom (the failed run) and the root cause (the absent parameter in the task definition), delivering a complete and sustainable fix.

**Why the other options are less suitable**

**A – Update the task and manually run the job** – Manually triggering the job bypasses the automated schedule and may introduce human error; it also does not guarantee that the parameter will be applied consistently across all runs.

**B – Share the analysis with the notebook owner** – While collaboration is valuable, it does not directly resolve the failing execution. The team still needs to apply the fix to the task and re-run the job.

**C – Repair the run with the new parameter only** – This restores the current execution but leaves the task definition unchanged, so any subsequent runs will again fail when the same parameter is missing.

**References**

**Databricks Jobs CLI – Adding Parameters to a Task:** <https://docs.databricks.com/jobs/cli.html#add-parameters-to-a-task>

**Jobs API – Updating a Job Definition:** <https://docs.databricks.com/api/latest/jobs/jobs-update.html>

**Question: 313**

**Deep Dumps**

A data engineer is masking the provided data column containing the email address. The goal is to have an output of the same length for all rows, while keeping different outputs for different values.

Which SQL function should be used to achieve this?

- A. hash(email)
- B. mask(email, '?')
- C. sha1('email')
- D. sha2(email,0)

**Answer: C**

**Explanation:**

**Technical justification**

**Correct choice – C. sha1(email)**

SHA1 is a cryptographic hash function that always returns a 40-character hexadecimal string, guaranteeing a fixed-length output regardless of the input size.

It preserves uniqueness for distinct email values (different inputs produce different hash outputs), satisfying the “different outputs for different values” requirement.

The function is deterministic, pure-SQL, and widely supported in Databricks SQL, making it ideal for masking while keeping row-level consistency.

#### Why the other options are unsuitable

- A. hash(email)** – In Databricks the built-in hash function returns a 32-bit integer, which can be negative and is not a fixed-length string; it may truncate or overflow, breaking the constant-length requirement.
- B. mask(email, '?')** – This is not a valid SQL function in Databricks; masking typically requires a custom UDF or `regexp_replace`, which would not guarantee a fixed length for all rows.
- D. sha2(email,0)** – sha2 with a length argument of 0 is invalid syntax; even if used with a positive length, the output length would vary (e.g., 64 hex chars for SHA-256), not a constant length.

#### References

Databricks SQL Documentation – Hash Functions: <https://docs.databricks.com/sql/language-manual/functions/hash.html>

Databricks SQL Documentation – SHA Functions: <https://docs.databricks.com/sql/language-manual/functions/sha.html>

### Question: 314

Deep Dumps

A company wants to implement Lakehouse Federation across multiple data sources, but is worried about data consistency and ensuring all teams access the same authoritative version of their data.

Which statement is applicable for Lakehouse Federations to maintain data consistency?

- A. Federation creates local copies that must be manually refreshed.
- B. Federation implements change data capture from all sources.
- C. A separate data synchronization service must be deployed.
- D. Federation provides read-only access that reflects the current state of source systems.

**Answer: D**

**Explanation:**

#### Why option D is correct

Lakehouse Federation enables **federated querying** across disparate data sources while maintaining a single logical view.

It **does not copy or move data**; instead, it pushes query predicates down to the source systems and reads the data in-place, so the view always reflects the **current state** of each source.

Because the underlying tables remain authoritative, any updates made in the source are immediately visible to federated queries, guaranteeing consistency without additional sync processes.

#### Why the other options are unsuitable

**A. “Federation creates local copies that must be manually refreshed.”**

Federation does not materialize local copies; it relies on push-down of query logic to the source, eliminating the need for manual refreshes.

**B. “Federation implements change data capture from all sources.”**

CDC is not a built-in capability of Lakehouse Federation; it merely reads source data as-is, without capturing or propagating changes.

**C. “A separate data synchronization service must be deployed.”**

Federation eliminates the requirement for an external sync service by providing a unified, read-only view that stays in sync with source systems automatically.

## References

Databricks Lakehouse Federation documentation: <https://learn.databricks.com/federated-query>

Lakehouse Federation architecture overview: <https://docs.databricks.com/data/federated-query/index.html>

### Question: 315

Deep Dumps

A data engineer needs to implement column masking for a sensitive column in a Unity Catalog-managed table. The masking logic must dynamically check if users belong to specific groups defined in a separate table (group\_access) that maps groups to allowed departments.

Which approach should the engineer use to efficiently enforce this requirement?

- A. Create a view without selecting the sensitive column
- B. Apply a column mask that references the group\_access mapping table in its UDF
- C. Create a UDF that hardcodes allowed groups and apply it as a column mask.
- D. Use a row filter to restrict access based on the user's group.

**Answer: B**

**Explanation:**

#### Justification

**Option B** is the only solution that can enforce column-level masking while dynamically evaluating group membership against an external mapping table. A column mask can invoke a user-defined function (UDF) that queries the group\_access table at runtime, allowing the logic to adapt to each user's group and department without hard-coding values. This approach keeps the masking logic centralized, maintainable, and aligned with Unity Catalog's fine-grained access control model.

**Option A** merely hides the column from the view; it does not apply any masking logic and cannot enforce per-user rules based on group membership.

**Option C** hardcodes allowed groups inside the UDF, which makes the rule static and requires code changes whenever group mappings change, reducing flexibility and operational efficiency.

**Option D** uses a row filter, which controls which rows a user can see but does not provide column-level masking for sensitive data, leaving the sensitive column exposed to unauthorized users.

#### References

Unity Catalog Access Control: <https://docs.databricks.com/en/data-governance/unity-catalog/access-control.html>

Creating Column Masks with UDFs: <https://docs.databricks.com/en/data-governance/unity-catalog/column-masks.html>

### Question: 316

Deep Dumps

Two data engineers are working on the same Databricks notebook in separate branches. Both have edited the same section of code. When one tries to merge the other's branch into their own using the Databricks Git folders UI, a merge conflict occurs on that notebook file. The UI highlights the conflict and presents options for resolution.

How should the data engineers resolve this merge conflict using Databricks Git folders?

- A. Use the Git CLI in the cluster's web terminal to force-push the conflicted merge (git push --force), overriding the remote branch with local version and discarding changes.
- B. Use the Git folders UI to manually edit the notebook file, selecting the desired lines from both versions and removing the conflict markers, then mark the conflict as resolved.
- C. Abort the merge, discard all local changes, and try the merge operation again without reviewing the conflicting code.
- D. Delete the conflicted notebook file via the Databricks workspace UI, commit the deletion, and recreate the notebook from scratch in a new commit to bypass the conflict entirely.

**Answer: B**

**Explanation:**

#### Technical justification

The Databricks Git folders UI displays merge conflicts directly in the notebook editor, showing the conflicting sections marked with <<<<<<<, =====, and >>>>>>>.

To resolve the conflict, engineers should **edit the notebook file in the UI**, choose the appropriate lines from each version, delete the conflict markers, and then click **“Mark as resolved”** (or commit the changes). This approach preserves the full merge context, allows explicit selection of code from both branches, and ensures the resolved state is recorded in Git history.

#### Why option B is correct

It uses the built-in UI workflow that is designed for safe conflict resolution in Databricks Repos.

It maintains a clear audit trail of which code was kept and why, which is essential for collaborative notebook development and for passing Databricks certification expectations.

#### Why the other options are unsuitable

**A** – Force-pushing with git push --force discards remote changes without review, risking loss of work and violating best-practice merge procedures.

**C** – Aborting the merge and discarding local changes abandons the conflicting work entirely, leaving the conflict unresolved and forcing a repeat of the merge without any resolution.

**D** – Deleting the conflicted notebook and recreating it erases version history, breaks reproducibility, and does not address the underlying merge conflict; it is a workaround rather than a proper resolution.

#### References

**Databricks Git Integration Documentation** – <https://docs.databricks.com/git-integration/index.html>

**Resolving Merge Conflicts in Databricks Repos** – <https://docs.databricks.com/git-integration/resolve-merge-conflicts.html>

#### Question: 317

**Deep Dumps**

A data engineer wants to automate job monitoring and recovery in Databricks using the Jobs API. They need to list all jobs, identify a failed job, and rerun it.

Which sequence of API actions should the data engineer perform?

- A. Use the jobs list endpoint to list jobs, check job run statuses with jobs runs list, and rerun a failed job using jobs run-now.
- B. Use the jobs get endpoint to retrieve job details, then use jobs update to rerun failed jobs.
- C. Use the jobs list endpoint to list jobs, then use the jobs create endpoint to create a new job, and run the new job using jobs run-now.

D. Use the jobs cancel endpoint to remove failed jobs, then recreate them with jobs create endpoint and run the new ones.

**Answer: A**

**Explanation:**

**Justification**

**OptionA** correctly follows the Jobs API workflow:

1. **GET /jobs/list** returns all defined jobs, allowing the engineer to locate the target job.
2. **GET /jobs/runs/list?job\_id=<id>** (or the equivalent “list runs” endpoint) fetches recent run records so the engineer can identify a failed run by its status (FAILED).
3. **POST /jobs/runs/rerun** (or **jobs run-now**) re-executes the failed run without needing to recreate the job definition.  
This sequence uses only read-only and idempotent operations, preserving the original job configuration and minimizing disruption.

**OptionB** suggests using **GET /jobs/get** (which returns a single job by name or ID) and then **PATCH /jobs/update** to rerun a failed job.

The **update** endpoint is intended for modifying job settings (e.g., new schedule, new notebook path), not for launching a new run of an existing failed execution.

There is no direct “rerun” action exposed via jobs update; attempting to do so would require additional steps (e.g., creating a new run) and is not the recommended API pattern.

**OptionC** proposes listing jobs, then **creating a new job** with **jobs create** and running it with **jobs run-now**.

Creating a new job duplicates the original definition unnecessarily, potentially altering parameters (e.g., default arguments, triggers).

It adds overhead and risk of configuration drift, whereas the goal is simply to re-execute the existing failed job.

**OptionD** recommends cancelling failed jobs and recreating them.

Cancelling a job does not automatically trigger a rerun; the engineer would still need to recreate the job and launch a new run, which is more complex and error-prone.

It also risks losing historical run metadata that may be needed for audit or debugging.

**Conclusion:** OptionA provides the most direct, minimal-impact sequence that leverages the Jobs API’s built-in “rerun” capability, making it the correct choice.

---

## References

**Databricks Jobs API – List Jobs:** <https://docs.databricks.com/en/jobs/api-2-1/jobs-list.html>

**Databricks Jobs API – Rerun a Job Run:** <https://docs.databricks.com/en/jobs/api-2-1/jobs-rerun.html>

## Question: 318

**Deep Dumps**

A data engineering team is setting up deployment automation. To deploy workspace assets remotely using the Databricks CLI command, they must configure it with proper authentication.

Which authentication approach will provide the highest level of security?



- A. Use a service principal and its Personal Access Token
- B. Use a service principal with OAuth token federation
- C. Use a service principal ID and its OAuth client secret
- D. Use a shared user account and its OAuth client secret

**Answer: B**

**Explanation:**

**Why option B is the most secure choice**

**OAuth token federation** leverages Azure Active Directory (Azure AD) to issue short-lived, scoped access tokens on behalf of a service principal.

Tokens are **self-contained, revocable, and never stored as long-lived secrets** on the client side, eliminating the risk of secret leakage.

The authentication flow enforces **principle-of-least-privilege** by allowing fine-grained token claims (e.g., specific workspace, cluster, or job permissions).

Because the token is derived from Azure AD's identity platform, it benefits from Azure AD's advanced security controls (conditional access, MFA, device compliance, etc.).

**Why the other options are less secure**

**A – Service principal + PAT** – A Personal Access Token is a static secret that must be stored somewhere in the automation pipeline; if exposed, it grants full access until it expires or is rotated.

**C – Service principal ID + OAuth client secret** – The client secret is another long-lived credential that can be inadvertently leaked; it also requires secret-management overhead and does not provide the same token-level revocation capabilities.

**D – Shared user account + OAuth client secret** – Using a shared human account violates the principle of individual accountability, expands the attack surface, and relies on a secret that can be compromised across multiple workflows.

**Conclusion**

For the highest security posture when deploying Databricks workspace assets via the CLI, **service principal with OAuth token federation** is preferred because it eliminates persistent secrets, provides short-lived scoped tokens, and integrates with Azure AD's robust identity governance.

---

## References

**Databricks CLI authentication – Service principals:** <https://docs.databricks.com/en/dev-tools/cli/authentication.html#service-principals>

**OAuth token federation with Azure AD:** <https://learn.microsoft.com/azure/databricks/dev-tools/cli/oauth-token-federation>

These links are current, official documentation that detail the configuration and security benefits of OAuth token federation for Databricks CLI deployments.

## Question: 319

**Deep Dumps**

A data engineer treated a daily batch ingestion pipeline using a cluster with the latest DBR version to store banking transaction data, and persisted it in a MANAGED DELTA table called `prod.gold.all_banking_transactions_daily`. The data engineer is constantly receiving complaints, from business units that constantly read this table in an ad hoc way using a SOL Serverless Warehouse, regarding the poor performance of their queries. Analyzing the queries, the data engineer slated that these areas use high cardinality

columns as filters, to perform their analysis. The data engineer is studying possibilities to implement a data layout optimization technique for that table, which is incremental, easy to maintain, and can easily evolve over time.

Which command should the data engineer implement?

- A. Alter the table to use Hive Style Partitions + Z-ORDER and implement a periodic OPTIMIZE command.
- B. Alter the table to use Hive Style Partitions and implement a periodic OPTIMIZE command.
- C. Alter the table to use Z-ORDER and implement a periodic OPTIMIZE command.
- D. Alter the table to use Liquid Clustering and implement a periodic OPTIMIZE command

**Answer: D**

**Explanation:**

**Why option D is the optimal choice**

**LiquidClustering** creates a logical, column-level ordering that is maintained automatically as new data is appended, enabling fast point-lookup and high-cardinality filter pruning without manual partition management.

It is **incremental** – each micro-batch that lands in the table updates the clustering metadata, so the layout evolves continuously as new daily loads arrive.

The **periodic OPTIMIZE** command compacts the data and rewrites the clustering depth to keep the storage-to-compute ratio efficient, ensuring that query latency stays low for ad-hoc SQL Serverless reads. This combination satisfies the requirements of being **easy to maintain** (no need to redesign partitions) and **future-proof** (clustering adapts to changing data distributions).

**Why the other options are less suitable**

**A & B – Hive-style partitions + OPTIMIZE**

Partitioning works well for low-cardinality, static directory layouts, but managing many high-cardinality filter columns as partitions quickly becomes unwieldy and costly.

Adding partitions after each daily load requires manual DDL changes and can lead to deep directory trees, violating the “easy to maintain” constraint.

**C – Z-ORDER + OPTIMIZE**

Z-ORDERing is a static layout that must be re-applied after each load to stay effective; it does not adapt incrementally and can cause frequent recomputation of the ordering.

Without the automatic, low-overhead clustering semantics of LiquidClustering, maintaining performance over time is more labor-intensive.

**D – LiquidClustering + OPTIMIZE**

Provides the only solution that is **native to Delta Lake**, **incrementally updated**, and **designed for evolving query patterns** with high-cardinality filters, matching the business’s performance expectations.

**References**

Liquid Clustering in Delta Lake: <https://learn.databricks.com/docs/delta/delta-intro#liquid-clustering>

OPTIMIZE command and its use with Delta tables: <https://learn.databricks.com/docs/delta/delta-optimization>

**Question: 320**

**Deep Dumps**

A data engineer is designing an append-only pipeline that needs to handle both batch and streaming data in Delta Lake. The team wants to ensure that the streaming component can efficiently track which data has already been

processed.

Which configuration should be set to enable this?

- A. checkpointLocation
- B. overwriteSchema
- C. mergeSchema
- D. partitionBy

**Answer: A**

**Explanation:**

#### **Why checkpointLocation is the correct choice**

In Structured Streaming, a checkpoint stores the progress marker that records which records have been read, written, or committed.

When appending data to Delta Lake, the checkpoint enables exactly-once semantics by remembering the offset or file list that has already been processed, allowing the stream to resume where it left off without re-processing the same data.

This is essential for an **append-only** pipeline that must reliably track processed data across restarts, failures, or scaling events.

#### **Why the other options are not appropriate**

**overwriteSchema** – Controls how schema evolution is handled during a write; it does not record processing progress.

**mergeSchema** – Enables automatic schema merging on read/write; again, it is unrelated to state tracking.

**partitionBy** – Determines how data is laid out on storage; it can improve query performance but does not provide a mechanism to remember which records have been consumed.

#### **Technical summary**

Setting checkpointLocation creates a durable, fault-tolerant state store (typically on durable storage such as DBFS, S3, or ADLS) that the streaming query uses to coordinate exactly-once writes to Delta Lake. The other configurations affect schema handling or data layout but do not provide the necessary progress tracking for stream-to-Delta appends.

#### **References**

Structured Streaming Programming Guide – Checkpointing: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#checkpointing>

Delta Lake Documentation – Streaming Write with Checkpoints: <https://delta.io/latest/streaming/write-streaming-data.html#checkpointing>

### **Question: 321**

**Deep Dumps**

A data engineer is implementing Unity Catalog governance for a multi-team environment. Data scientists need interactive clusters for basic data exploration tasks, while automated ETL jobs require dedicated processing.

How should the data engineer configure cluster isolation policies to enforce least privilege and ensure Unity Catalog compliance?

- A. Configure all dusters with NO\_ISOLATION\_SHARED access modes since Unity Catalog works with any cluster configuration
- B. Allow all users to create any duster type and rely on manual configuration to enable Unity Catalog access modes.

- C. Create compute policies with STANDARD access mode for interactive 'workloads and dedicated access mode for automated jobs.
- D. Use only DEDICATED access mode for both interactive workloads and automated jobs to maximize security isolation.

**Answer: C**

**Explanation:**

**Why option C is correct**

**Compute policies** in Unity Catalog allow you to define access modes (e.g., STANDARD, DEDICATED) that control how a cluster can be used.

Setting **STANDARD** for interactive workloads enables shared resources while still enforcing least-privilege controls (e.g., only users with the appropriate policy can attach a cluster to a given catalog/schema).

Setting **DEDICATED** for automated ETL jobs guarantees that those jobs run on isolated clusters, preventing resource contention and ensuring compliance with governance policies.

This approach balances **security isolation** (dedicated clusters for production workloads) with **operational efficiency** (shared clusters for ad-hoc exploration), which is the recommended pattern for multi-team environments.

**Why the other options are unsuitable**

**Option A** – Using NO\_ISOLATION\_SHARED removes any isolation guarantees; Unity Catalog still enforces access controls, but this mode does not provide the explicit separation needed for production jobs.

**Option B** – Allowing unrestricted creation of clusters and relying on manual configuration defeats the purpose of automated governance; it introduces human error and does not enforce least-privilege policies consistently.

**Option D** – Applying DEDICATED to both interactive and automated workloads maximizes isolation but wastes resources, as interactive users typically need the flexibility and cost-effectiveness of shared clusters. It also contradicts best-practice guidance to use STANDARD for exploratory workloads.

## References

Unity Catalog Compute Policies: <https://docs.databricks.com/en/data-governance/unity-catalog/compute-policies.html>

Managing Cluster Isolation in Unity Catalog: <https://docs.databricks.com/en/data-governance/unity-catalog/cluster-isolation.html>

## Question: 322

**Deep Dumps**

A data engineer, while designing a Pandas UDF to process financial time series data with complex calculations that require maintaining state across rows within each stock symbol group must ensure the function is efficient and scalable.

Which approach will solve the problem with minimum overhead while preserving data integrity?

- A. Use a SCALAR\_ITER Pandas UDF with iterator-based processing, implementing state management through persistent storage (Delta tables) that gets updated after each batch to maintain continuity across iterator chunks.
- B. Use a GROUPED\_AGG UPF that processes each stock symbol group independently, maintaining state through intermediate aggregation results that get passed between successive UDF calls via broadcast variables.
- C. Use a SCALAR Pandas UDF that processes the entire dataset at once, implementing custom partitioning logic within the UDF to group by stock symbol and maintain state using global variables shared across all executor processes.

D. Use `ApplyInPandas` method on spark dataframe that receives all rows for each stock symbol as a pandas DataFrame, allowing processing within each group while maintaining state variables local to each group's processing function.

**Answer: D**

**Explanation:**

**Why option D is the best choice**

**ApplyInPandas preserves group-level locality** – each stock symbol's rows are delivered as a single pandas DataFrame to the UDF, so any state (e.g., running totals, previous price) can be kept in local variables that are scoped to that group.

**No cross-executor coordination needed** – the state lives only inside the executor that processes the group, eliminating the need for broadcast variables or external storage, which reduces overhead and avoids consistency issues.

**Scalable execution model** – Spark schedules the pandas function per group, allowing parallelism across symbols while still respecting the distributed nature of the cluster; the function runs in the same JVM process that already holds the data, minimizing data movement.

**Data integrity is guaranteed** – because each group is processed atomically, there is no race condition or partial update that could corrupt intermediate results; the final result is written back to the DataFrame in a deterministic order.

**Why the other options are less suitable**

**Option A (SCALAR\_ITER with persistent Delta storage)** introduces external I/O (Delta table writes) for every iterator chunk, causing high latency and potential bottlenecks; state must be checkpointed and reconciled, adding complexity and risk of inconsistency.

**Option B (GROUPED\_AGG UPF with broadcast variables)** requires broadcasting mutable aggregation state to all executors and coordinating updates across tasks, which can lead to stale or divergent state and adds serialization overhead.

**Option C (SCALAR UDF with global variables)** relies on global mutable state that is shared across all executors, creating race conditions and making the function non-deterministic; such globals are not guaranteed to be isolated per group and break the guarantees of distributed execution.

**Conclusion** – Using **ApplyInPandas** (optionD) provides the minimal overhead, maintains state locally per symbol group, and preserves data integrity without extra storage or coordination mechanisms.

---

## References

[ApplyInPandas \(Databricks\) – Spark 3.x Documentation](#)  
[Pandas UDFs – Best Practices](#)

## Question: 323

**Deep Dumps**

A data engineer is developing an LDP pipeline using a Databricks notebook that is directly connected to their pipeline. After adding new table definitions and transformation logic in their notebook, they want to check for any syntax errors in the pipeline code without actually processing any data or running the pipeline.

How should the data engineer perform this syntax check?

- A. Open the web terminal from the notebook and run a shell command to validate the pipeline code.
- B. Disconnect the notebook from the pipeline and reconnect it to a compute cluster to access code validation features

- C. Use the "Validate" option in the notebook to check for syntax errors.
- D. Switch to a workspace file instead of a notebook to access the validation and diagnostics tools.

**Answer: C**

**Explanation:**

**Justification**

**Option C – “Use the ‘Validate’ option in the notebook to check for syntax errors.”**

Databricks notebooks provide a built-in Validate command that parses the notebook’s code (SQL, Python, Scala, etc.) and reports syntax or semantic errors without executing any data-processing logic. This allows the engineer to verify correctness of newly added table definitions and transformations while keeping the pipeline untouched.

**Why the other options are unsuitable**

**A. Open the web terminal and run a shell command** – The notebook’s runtime environment does not expose a generic shell for arbitrary code validation; such a command would either be unavailable or would still require executing the pipeline logic to be effective.

**B. Disconnect and reconnect to a compute cluster to access code validation features** – Validation does not depend on cluster connectivity; it is a notebook-level feature, so reconnecting adds unnecessary complexity without solving the problem.

**D. Switch to a workspace file instead of a notebook** – While workspace files can be edited, they lack the integrated Validate UI that notebooks provide; moving the code would disrupt the existing pipeline connection and does not guarantee syntax checking.

**Conclusion**

The most direct, supported method to perform a syntax-only check on pipeline code within a Databricks notebook is to use the built-in **Validate** option.

**References**

Databricks Docs: Validate notebooks and scripts – <https://docs.databricks.com/en/notebooks/validate.html>

Databricks Docs: Pipeline unit tests and validation –

<https://docs.databricks.com/en/optimization/pipelines/unit-tests.html>

**Question: 324**

**Deep Dumps**

A data engineering team is in the process of migrating off its legacy Hadoop platform. As part of the process, they are evaluating the different storage formats to see which offers the best query performance. Their legacy platform utilizes ORC and RCFile formats. They converted a subset of their data to Delta Lake and ran a side-by-side benchmark, and found that their queries performed significantly better. Upon investigation, they discovered that the query plans were different, and the queries that read from the Delta Lake tables leveraged a Shuffle Hash Join, whereas the legacy formats used Sort Merge Joins. The queries that read from the Delta Lake tables also read less data.

Which reason could be attributed to the difference in query performance?

- A. The queries against the ORC tables leveraged the dynamic data skipping optimization but not the dynamic file pruning optimization.
- B. The queries against the Delta Lake tables were able to leverage the dynamic file pruning optimization.
- C. Delta Lake enables data skipping and file pruning using a vectorized Parquet reader.
- D. Shuffle Hash Joins are always more efficient than Sort Merge Joins.



**Answer: B**

**Explanation:**

**Why option B is the best answer**

The side-by-side benchmark showed that Delta Lake queries read **less data** and use a **Shuffle Hash Join** instead of the legacy Sort-Merge Join.

Delta Lake's query planner can apply **dynamic file pruning**: it determines, at planning time, which files are actually needed to satisfy the filter conditions and eliminates unnecessary files from the scan.

By skipping whole files, the engine reduces I/O and the amount of data that must be processed, which directly translates into lower query latency and higher throughput.

This optimization is a core feature of Delta Lake that is not available for ORC or RCFile in the same way, explaining the observed performance gap.

**Why the other options are less suitable**

**A** – While ORC does support data-skipping, the key difference observed was not about dynamic data skipping but about the ability of Delta Lake to prune files before reading, which reduces the input size more aggressively.

**C** – Vectorized Parquet readers improve read speed, but the performance gain in the benchmark stemmed from file pruning rather than the reader's vectorization.

**D** – Shuffle Hash Joins are not universally “always more efficient” than Sort-Merge Joins; their advantage depends on data distribution, shuffle cost, and join type. The benchmark improvement was due to the specific context of file pruning and reduced data scanned, not a blanket superiority of Hash Joins.

**References**

Delta Lake Documentation – Optimizing Queries with Dynamic File Pruning:

<https://docs.databricks.com/delta/delta-batch.html#dynamic-file-pruning>

Databricks Blog – Performance Tips for Delta Lake: <https://databricks.com/blog/2022/06/performance-tips-delta-lake.html>

## Question: 325

**Deep Dumps**

A data team is automating a daily multitask ETL pipeline in Databricks. The pipeline includes a notebook for ingesting raw data, a Python wheel task for data transformation, and a SQL query to update aggregates. They want to trigger the pipeline programmatically and see previous runs in the GUI. They need to ensure tasks are retried on failure and stakeholders are notified by email if any task fails.

Which two approaches will meet these requirements? (Choose two.)

- A. Trigger the job programmatically using the Databricks Jobs REST API (/jobs/run-now), the CLI (databricks jobs run-now), or one of the Databricks SDKs.
- B. Use Databricks Asset Bundles (DABs) to deploy the workflow, then trigger individual tasks directly by referencing each task's notebook or script path in the workspace.
- C. Create a multi-task job using the UI, DABs, or the Jobs REST API (/jobs/create) with notebook, Python wheel, and SQL tasks. Configure task-level retries and email notifications in the job definition.
- D. Use the REST API endpoint /jobs/runs/submit to trigger each task individually as separate job runs and implement retries using custom logic in the orchestrator.
- E. Create a single notebook that uses dbutils.notebook.run() to call each step in sequence. Define a job on this orchestrator notebook and configure retries and notifications at the notebook level.

**Answer: AC**

**Explanation:**

### Why option C is the best choice

It creates a **single multi-task job** that can contain a notebook, a Python wheel, and a SQL query as distinct tasks, matching the required pipeline components.

The job definition can be built via the UI, a Databricks Asset Bundle (DAB), or the **Jobs REST API (/jobs/create)**, giving programmatic control while still allowing the UI for visibility.

**Task-level retries** are configured directly in the job spec, so failed tasks are automatically retried without custom code.

**Email notifications** can be attached to the job (or individual tasks) in the same definition, ensuring stakeholders are alerted when any task fails.

All execution history, including previous runs, is retained in the Databricks Jobs UI, satisfying the “see previous runs” requirement.

### Why the other options are less suitable

**A** – Only describes how to trigger a job; it does not address how to define multi-task steps, retries, or notifications.

**B** – DABs are useful for deployment, but referencing each task’s path directly does not provide built-in retry or notification settings at the workflow level.

**D** – Submitting each task as an independent job run loses the integrated multi-task view and makes coordinated retries and unified notifications harder.

**E** – Using `dbutils.notebook.run()` in a single orchestrator notebook works, but retries and email alerts must be implemented manually; the native job-level features are not leveraged.

### References

Databricks Jobs CLI and REST API: <https://docs.databricks.com/jobs/commands.html>

Configure retries and notifications in Jobs: <https://docs.databricks.com/jobs/retry-notifications.html>

### Question: 326

### Deep Dumps

A data engineer is designing a system to process batch patient encounter data stored in an S3 bucket, creating a Delta table (`patient_encounters`) with columns `encounter_id`, `patient_id`, `encounter_date`, `diagnostic_code`, and `treatment_cost`. The table is queried frequently by `patient_id` and `encounter_date`, requiring fast performance. Fine grained access controls must be enforced. The engineer wants to minimize maintenance and boost performance.

How should the data engineer create the `patient_encounters` table?

- A. Create an external table in Unity Catalog, specifying an S3 location for the data files. Enable predictive optimization through table properties, and configure Unity Catalog permissions for access controls.
- B. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, schedule jobs to run `OPTIMIZE` and `vacuum` command daily to achieve best performance.
- C. Create a managed table in Hive metastore. Configure Hive metastore permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.
- D. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.

**Answer: D**

**Explanation:**

### Why option D is the optimal choice

**Managed table in Unity Catalog** – creates a fully-governed Delta table that lives inside the catalog, automatically inherits ACID transactions, schema enforcement, and can be vacuumed/optimized without external dependencies.

**Fine-grained access control** – Unity Catalog provides column-level and row-level permissions that can be attached to the table, satisfying the requirement for granular security.

**Predictive optimization** – enabling the optimize property (or relying on the default predictive optimization) lets Databricks automatically reorganize data for the most common query patterns (e.g., filters on patient\_id and encounter\_date), delivering faster reads without manual intervention.

**Minimal maintenance** – because the table is managed, Databricks handles file layout, metadata, and compaction; the engineer does not need to schedule separate OPTIMIZE/VACUUM jobs, reducing operational overhead while still achieving high query performance.

#### Why the other options are less suitable

**OptionA (external table)** – external tables reference data stored outside the catalog, do not support Delta transaction features, and lack automatic optimization or vacuuming. They also provide weaker access-control granularity, making them a poor fit for fine-grained security and performance goals.

**OptionB (managed table with scheduled OPTIMIZE/VACUUM)** – while a managed table gives governance, relying on scheduled maintenance jobs adds complexity and does not leverage the built-in predictive optimization that can proactively improve performance. This approach increases operational burden compared to the automated, hands-off model offered by optionD.

**OptionC (managed table in Hive metastore)** – Hive-metastore-based tables do not benefit from Unity Catalog's advanced security model or predictive optimization, and they are generally less performant for workloads that require fast, repeated queries on specific columns.

#### References

**Unity Catalog Managed Tables:** <https://docs.databricks.com/data-governance/unity-catalog/managed-tables.html>

**Predictive Optimization in Delta Lake:** <https://docs.databricks.com/delta/delta-auto-optimize.html>

These sources detail the capabilities of Unity Catalog managed tables, granular access controls, and the automatic performance optimizations that make optionD the best fit for the described scenario.

#### Question: 327

#### Deep Dumps

A data engineer has a delta table order with deletion vectors enabled for it. The engineer is attempting to execute the below code:

```
DELETE FROM orders WHERE status = 'cancelled'
```

What should be the behaviour of deletion vectors when the command is executed?

- A. Files are physically rewritten without the deleted row.
- B. Rows are marked as deleted both in metadata and in files.
- C. Rows are marked as deleted in metadata, not in files
- D. Delta automatically removes all cancelled orders permanently.

**Answer: C**

**Explanation:**

**Justification**

Deletion vectors are a Delta Lake feature that records the logical removal of rows in the table's transaction log.

When a DELETE statement is executed, the affected rows are **marked as deleted only in the metadata** (the

transaction log and the deletion-vector file).

The underlying Parquet files are **not rewritten** at that moment; the data files remain unchanged until a subsequent compaction or vacuum operation rewrites them.

Therefore, the correct description is **“Rows are marked as deleted in metadata, not in files.”**

#### Why the other options are incorrect

**A.** Files are not physically rewritten immediately; rewriting occurs only during later compaction.

**B.** Deletion vectors do **not** store the deletion information inside the data files themselves; they are stored separately in metadata.

**D.** Delta does not automatically purge rows permanently; deletion is logical until a vacuum operation removes the expired data.

#### References

Deletion Vectors – Delta Lake Documentation: <https://docs.delta.io/latest/delta/delta-batch.html#deletion-vectors>

DELETE Operation – Delta Lake Documentation: <https://docs.delta.io/latest/delta/batch-operations.html#delete>

---

End of answer.

#### Question: 328

Deep Dumps

A data engineer is building a Lakeflow Spark Declarative Pipelines pipeline to process healthcare claims data. A metadata JSON file defines data quality rules for multiple tables, including:

```
{
 "claims": [
 {"name": "valid_patient_id", "constraint": "patient_id IS NOT NULL"},
 {"name": "non_negative_amount", "constraint": "claim_amount >= 0"}
]
}
```

The pipeline must dynamically apply these rules to the claims table without hardcoding the rules.

How should the data engineer achieve this?

- A. Use a SQL CONSTRAINT block referencing the JSON file path.
- B. Load the JSON metadata, loop through its entries, and apply expectations using `dlt.expect_all`.
- C. Invoke an external API to validate records against the metadata rules.
- D. Reference each expectation with `@dlt.expect` decorators in the table declaration.

**Answer: B**

#### Question: 329

Deep Dumps

A security analytics pipeline must enrich billions of raw connection logs with geolocation data. The join hinges on finding which IPv4 range each event's address falls into.

Table 1 · **network\_events** (≈ 5 billion rows)

**event\_id** **ip\_int**

42                    3232235777

Table 2 · **ip\_ranges** (≈ 2 million rows)

**start\_ip\_int** **end\_ip\_int** **country**

3232235520 3232236031 US

This query is running very slowly and needs to be improved:

```
SELECT n.event_id,
 n.ip_int,
 r.country
FROM network_events n
JOIN ip_ranges r
ON n.ip_int BETWEEN r.start_ip_int AND r.end_ip_int;
```

Execution is far too slow.

Which change will most dramatically accelerate the query while preserving its logic?

- A. Increase `spark.sql.shuffle.partitions` from 200 to 10000.
- B. Force a sort-merge join with `/*+ MERGE(r) */` on `ip_ranges`.
- C. Add a broadcast hint: `/*+ BROADCAST(r) */` for `ip_ranges`.
- D. Add a range-join hint /16 bin size (65 536): `/*+ RANGE_JOIN(r, 65536) */`.

**Answer: D**

### Question: 330

Deep Dumps

A data engineer is using Lakeflow Declarative Pipeline to propagate row deletions from a source bronze table (`user_bronze`) to a target silver table (`user_silver`). The developer wants deletions in `user_bronze` to automatically delete corresponding rows in `user_silver` during pipeline execution.

Which configuration ensures deletions in the bronze table are propagated to the silver table?

- A. Use `apply_changes` without CDF and filter rows where `_soft_deleted` is true.
- B. Enable CDF on `user_silver`, read its transaction log, and use `MERGE` to sync deletions.
- C. Enable CDF on `user_bronze`, read its CDF stream, and use `apply_changes` with `apply_as_deletes` for

user\_silver.

D. Configure VACUUM on user\_bronze to delete files, then rebuild user\_silver from scratch.

**Answer: C**

**Explanation:**

**Why option C is correct**

**Change Data Feed (CDF) on the bronze layer** continuously captures all mutations, including row-level deletes (the `_soft_deleted` flag or tombstone files).

**apply\_changes with apply\_as\_deletes** interprets those deletions as delete-operations on the target silver table, automatically issuing DELETE statements for the matching keys.

This approach respects the declarative nature of Lakeflow: the pipeline definition remains simple, and the delete propagation is handled end-to-end without custom code or manual bookkeeping.

**Why the other options are inferior**

**A** – Filtering on `_soft_deleted` only works if you manually maintain a soft-delete column and run a separate filter step; it does not leverage CDF to generate proper delete actions in the target.

**B** – Reading the transaction log of the target table (silver) is not how deletions are propagated; deletions must be observed on the source (bronze) and then applied downstream.

**D** – Vacuuming removes stale files but does not generate delete semantics for downstream tables; rebuilding the silver table from scratch defeats the purpose of incremental propagation and is operationally costly.

**Key configuration**

Enable CDF on user\_bronze.

In the Lakeflow pipeline, use `apply_changes` on the CDF stream and set `apply_as_deletes=True` for the downstream user\_silver output.

**References**

Apply changes with deletes: <https://docs.databricks.com/data-engineering/pipelines/apply-changes.html#apply-deletes>

Change Data Feed (CDF) overview: <https://docs.databricks.com/data-engineering/pipelines/change-data-feed.html>

## Question: 331

## Deep Dumps

A data engineer is designing a Lakeflow Spark Declarative Pipeline to process streaming order data. The pipeline uses Auto Loader to ingest data and must enforce data quality by ensuring customer\_id and amount are greater than zero. Invalid records should be dropped.

Which Lakeflow Spark Declarative Pipelines configurations implement this requirement using Python?

- A. `@dlt.table @dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL") @dlt.expect_or_drop("valid_amount", "amount > 0") def silver_orders(): return dlt.read_stream("bronze_orders")`
- B. `@dlt.table def silver_orders(): return dlt.read_stream("bronze_orders").expect_or_drop("valid_customer", "customer_id IS NOT NULL").expect_or_drop("valid_amount", "amount > 0")`
- C. `@dlt.table @dlt.expect("valid_customer", "customer_id IS NOT NULL") @dlt.expect("valid_amount", "amount > 0") def silver_orders(): return dlt.read_stream("bronze_orders")`
- D. `@dlt.table def silver_orders(): return dlt.read_stream("bronze_orders").expect("valid_customer", "customer_id IS NOT NULL").expect("validamount", "amount > 0")`



**Answer: A**

**Explanation:**

**Why optionA is the correct implementation**

@dlt.table marks the function as a Delta Live Tables (DLT) table definition.

The expect\_or\_drop clause is attached to the table-level decorator, which tells DLT to validate each incoming record **before** it is materialized.

Using two separate expectations – @dlt.expect\_or\_drop("valid\_customer", "customer\_id IS NOT NULL") and @dlt.expect\_or\_drop("valid\_amount", "amount > 0") – enforces both quality rules.

When a record violates an expectation, DLT automatically drops it, satisfying the “drop invalid records” requirement.

The function returns dlt.read\_stream("bronze\_orders"), correctly wiring the source to the downstream table.

**Why the other options are unsuitable**

**OptionB** places expect\_or\_drop calls **inside** the table body rather than on the decorator. While the logic is similar, the syntax is invalid because DLT expects @dlt.expect\_or\_drop (or @dlt.expect) to be applied as metadata decorators, not as method calls within the function body. This would raise a parsing error.

**OptionC** mixes @dlt.expect (which only registers an expectation without the drop-behaviour) with @dlt.expect\_or\_drop only on the first clause. The second expectation uses @dlt.expect, which will **not** cause invalid rows to be dropped; they will simply be logged or counted, violating the “drop” rule.

**OptionD** also misplaces expect calls inside the function and misspells the second expectation key ("validamount" instead of "valid\_amount"), leading to a runtime error. Moreover, it lacks the @dlt.expect\_or\_drop decorator, so the dropping semantics are not enforced.

**Key takeaway**

To enforce data-quality rules that result in dropped records in a DLT pipeline, the expectations must be declared as decorators (@dlt.expect\_or\_drop) on the table definition, and each decorator must reference a correct SQL-style expression. Only optionA conforms to this syntax and semantics.

---

## References

Delta Live Tables table definition decorators: <https://docs.databricks.com/lakehouse/delta-live-tables/dlt-tables.html>

@dlt.expect\_or\_drop documentation: <https://docs.databricks.com/lakehouse/delta-live-tables/expectations.html>

## Question: 332

**Deep Dumps**

A data engineering team uses Databricks Lakehouse Monitoring to track the percent\_null metric for a critical column in their Delta table. The profile metrics table (prod\_catalog.prod\_schema.customer\_data\_profile\_metrics) stores hourly percent\_null values. The team wants to trigger an alert when the daily average of percent\_null exceeds 5% for three consecutive days, while ensuring notifications aren't spammed during sustained issues.

Which SQL alert configuration achieves this goal while minimizing false positives and redundant notifications?

A. SELECT AVG(percent\_null) AS daily\_avg FROM prod\_catalog.prod\_schema.customer\_data\_profile\_metrics WHERE window.end >= CURRENT\_TIMESTAMP - INTERVAL '3' DAY Alert Condition: daily\_avg > 5 Notification Frequency: Each time alert is evaluated

B. SELECT percent\_null FROM prod\_catalog.prod\_schema.customer\_data\_profile\_metrics WHERE window.end >= CURRENT\_TIMESTAMP - INTERVAL '1' DAY Alert Condition: percent\_null > 5 Notification Frequency: At most every 24 hours

C. `SELECT SUM(CASE WHEN percent_null > 5 THEN 1 ELSE 0 END) AS violation_days FROM prod_catalog.prod_schema.customer_data_profile_metrics WHERE window.end >= CURRENT_TIMESTAMP - INTERVAL '3' DAY` Alert Condition: `violation_days >= 3` Notification Frequency: Just once

D. `WITH daily_avg AS (SELECT DATE_TRUNC('DAY', window.end) AS day, AVG(percent_null) AS avg_null FROM prod_catalog.prod_schema.customer_data_profile_metrics GROUP BY DATE_TRUNC('DAY', window.end)) SELECT day, avg_null FROM daily_avg ORDER BY day DESC LIMIT 3` Alert Condition: ALL `avg_null > 5` for the latest 3 rows Notification Frequency: Just once

**Answer: D**

**Explanation:**

**Why option D is the best choice**

**Correct temporal logic** – It first aggregates `percent_null` by calendar day (`DATE_TRUNC('DAY', window.end)`), then selects the last three days (`ORDER BY day DESC LIMIT 3`). This directly evaluates “three consecutive days” rather than looking at a rolling window that may include non-contiguous or overlapping periods.

**Exact condition alignment** – The alert fires only when all three retained daily averages are greater than 5% (`ALL avg_null > 5`). This matches the requirement of “exceeds 5% for three consecutive days”.

**Notification control** – Setting Notification Frequency to **Just once** prevents repeated alerts while the condition persists, eliminating spam during a sustained issue.

**Minimal false positives** – By using an aggregate (`AVG`) per day and checking the last three rows, the logic is robust to transient spikes; the condition is only true when the metric stays above the threshold for three full days.

**Why the other options are unsuitable**

**Option A** evaluates a single day's average and triggers on every evaluation, which can fire repeatedly while the condition remains true, leading to alert fatigue.

**Option B** checks individual hourly values against 5% within a single-day window; it ignores the “three consecutive days” requirement and would alert on any single outlier hour, increasing false positives.

**Option C** counts rows where `percent_null > 5` over the last three days, but it does not enforce contiguity or that all three days meet the threshold; it merely counts any qualifying rows, which could miss consecutive days or count non-consecutive occurrences.

**Key take-away**

The alert must aggregate by day, examine the most recent three daily results, ensure all three exceed the 5% threshold, and emit a single notification. Option D implements exactly this logic.

**References**

Databricks Lakehouse Monitoring documentation – Alerting on Profile Metrics:

<https://docs.databricks.com/monitoring/lakehouse-monitoring.html#alerts>

Delta Lake profiling metrics schema: <https://docs.databricks.com/delta/profiles.html#profile-metrics-tables>

## Question: 333

Deep Dumps

A data engineer is configuring a Databricks Asset Bundle to deploy a job with granular permissions. The requirements are:

- Grant the data-engineers group `CAN_MANAGE` access to the job.
- Ensure the auditors' group can view the job but not modify/run it.
- Avoid granting unintended permissions to other users/groups.

How should the data engineer deploy the job while meeting the requirements?

A.

```
permissions:
```

- group\_name: data-engineers  
level: CAN\_MANAGE
- group\_name: auditors  
level: CAN\_VIEW

```
resources:
```

```
jobs:
```

```
my-job:
```

```
name: data-pipeline
```

```
tasks: [...]
```

```
job_clusters: [...]
```

B.

```
resources:
 jobs:
 my-job:
 name: data-pipeline
 tasks: [...]
 job_clusters: [...]
 permissions:
 - group_name: data-engineers
 level: CAN_MANAGE
 - group_name: auditors
 level: CAN_VIEW
 - group_name: admin-team
 level: IS_OWNER
```

c.

```
resources:
 jobs:
 my-job:
 name: data-pipeline
 tasks: [...]
 job_clusters: [...]
 permissions:
 - group_name: data-engineers
 level: CAN_MANAGE
 - group_name: auditors
 level: CAN_VIEW
```

D.

```
resources:
 jobs:
 my-job:
 name: data-pipeline
 tasks: [...]
 job: [...]
 permissions:
 - group_name: data-engineers
 level: CAN_MANAGE

permissions:
 - group_name: auditors
 level: CAN_VIEW
```

Answer: C

#### Question: 334

Deep Dumps

A data engineer needs to effectively install the PyYAML Python package for YAML file processing within their Databricks environment. However, the Databricks workspace is air-gapped and does not have direct internet access to download packages from PyPI. The engineer has proactively downloaded the necessary PyYAML wheel (.whl) file onto their laptop. The data engineer wants to install the PyYAML package from the local wheel file in such a way that it is automatically available whenever any new cluster is provisioned in their Databricks workspace.

Which approach should the data engineer use?

- A. Upload the PyYAML.whl file to a Unity Catalog Volume. They then ensure the path to this wheel file is added to the Unity Catalog allowlist. Finally, they create a cluster-scoped init script that executes `pip install /path/to/PyYAML.whl`.
- B. Add the PyYAML.whl file directly to their Databricks Git Repos. They assume that by committing the wheel file into the Repo, any cluster created that is linked to this Repo will automatically have PyYAML installed from that file.



C. Upload and register the PyYAML.whl file under the data engineer's user home directory in the Workspace, and set up a cluster-scoped init script that executes `%pip install /path/to/PyYAML.whl` on the shared cluster.

D. After setting up a private PyPI repository and registering the PyYAML.whl file, set up a cluster-scoped init script that executes `/databricks/python/bin/pip install --index-url=https://repo-url PyYAML` on the cluster.

**Answer: A**

**Explanation:**

**Justification for the correct approach (OptionA)**

**Unity Catalog volume + allow-list+cluster-scoped init script** guarantees that the wheel is stored in a location that all clusters can read and that the installation command runs automatically on every new cluster, regardless of where it is created.

The init script runs **after** the cluster is launched and **before** any notebooks or jobs execute, so `pip install /path/to/PyYAML.whl` installs the package system-wide for that cluster's Python environment.

By adding the wheel path to the **Unity Catalog allowlist**, the installation command can reference the file using a DBFS-compatible path (e.g., `dbfs:/mnt/uc_vols/pyyaml/PyYAML-*.whl`), which is accessible to all users and clusters.

This method is **reproducible** and **centralised**: the same volume and init script can be reused across workspaces or teams without manually copying the wheel to each cluster.

**Why the other options are inferior**

**OptionB** – Simply committing a .whl file to a Git repository does **not** trigger any automatic installation on clusters; Databricks does not interpret repository contents as a package source.

**OptionC** – Using `%pip install` inside a notebook is **not** supported in init scripts and the path to the user's home directory (`~/`) is not shared across clusters; the package would need to be redeployed manually for each new cluster.

**OptionD** – Setting up a private PyPI repository adds unnecessary complexity when the only artifact available is a single wheel. Moreover, referencing `/databricks/python/bin/pip` couples the script to a specific Python version that may differ across runtime images; the recommended init-script syntax is `pip install <wheel-path>`.

**References**

**Unity Catalog – Managing libraries with init scripts** – <https://docs.databricks.com/data-governance/unity-catalog/libraries.html#install-libraries-using-init-scripts>

**Databricks CLI & init script best practices** – <https://docs.databricks.com/clusters/init-scripts.html#install-python-packages>

## Question: 335

**Deep Dumps**

A data engineer has configured their Databricks Asset Bundle with multiple targets in `databricks.yml` and deployed it to the production workspace. Now, to validate the deployment, they need to invoke a job named `my_project_job` specifically within the `prod` target context. Assuming the job is already deployed, they need to trigger its execution while ensuring the target-specific configuration is respected.

Which command will trigger the job execution?

- A. `databricks job run my_project_job --env prod`
- B. `databricks execute my_project_job -e prod`
- C. `databricks run my_project_job -t prod`
- D. `databricks bundle run my_project_job -t prod`

**Answer: D**

**Explanation:**

The command must reference the **Databricks CLI bundle** operation, which is databricks bundle run.

The **-t flag** specifies the target name; using -t prod directs the CLI to the production target defined in databricks.yml.

my\_project\_job is the job name to be executed, so the full command is databricks bundle run my\_project\_job -t prod.

Option A (databricks job run ... --env prod) invokes the generic **job** command, not a bundle target and does not respect bundle-specific flags.

Option B (databricks execute my\_project\_job -e prod) uses an undefined CLI sub-command (execute) and an unsupported flag (-e).

Option C (databricks run my\_project\_job -t prod) also references an undefined sub-command (run) that does not exist for jobs in the CLI.

**Therefore, D is the only syntactically correct and context-aware command.**

**References**

Databricks CLI – Bundle commands: <https://docs.databricks.com/en/dev-tools/cli/bundle.html>

Databricks CLI – Job commands: <https://docs.databricks.com/en/dev-tools/cli/jobs.html>

**Question: 336**

**Deep Dumps**

A facilities-monitoring team is building a near-real-time PowerBI dashboard off the Delta table device\_readings:

| column        | type      | notes                     |
|---------------|-----------|---------------------------|
| device_id     | STRING    | unique sensor ID          |
| event_ts      | TIMESTAMP | Ingestion timestamp (UTC) |
| temperature_c | DOUBLE    | temperature in °C         |

For each sensor, the team needs one row per non-overlapping 5-minute interval, offset by 2 minutes, (e.g., intervals like 00:02-00:07, 00:07-00:12, ...), showing the average temperature in that slice. The result must include each interval's start and end timestamps so downstream tools can plot time-series bars correctly.

Which query satisfies the requirement?

A.

```

SELECT
 device_id,
 event_ts,
 AVG(temperature_c) OVER (
 PARTITION BY device_id
 ORDER BY event_ts
 RANGE BETWEEN INTERVAL 5 MINUTES PRECEDING AND CURRENT ROW
) AS avg_temp_5m
FROM
 device_readings
WINDOW w AS (
 window(event_ts, '5 minutes', '2 minutes')
);

```

B.

```

SELECT
 device_id,
 window.start AS bucket_start,
 window.end AS bucket_end,
 AVG(temperature_c) AS avg_temp_5m
FROM
 device_readings
GROUP BY
 device_id,
 window(event_ts, '5 minutes', '5 minutes', '2 minutes')
ORDER BY
 device_id,
 bucket_start;

```

C.

```

WITH buckets AS (
 SELECT
 device_id,
 window(event_ts, '5 minutes', '2 minutes', '5 minutes') AS win,
 temperature_c
 FROM
 device_readings
)
SELECT
 device_id,
 win.start AS bucket_start,
 win.end AS bucket_end,
 AVG(temperature_c) AS avg_temp_5m
FROM
 buckets
GROUP BY
 device_id,
 win
ORDER BY
 device_id,
 bucket_start;

```

D.

```

SELECT
 device_id,
 date_trunc('minute', event_ts - INTERVAL 2 MINUTES) + INTERVAL 2 MINUTES AS bucket_start,
 date_trunc('minute', event_ts - INTERVAL 2 MINUTES) + INTERVAL 7 MINUTES AS bucket_end,
 AVG(temperature_c) AS avg_temp_5m
FROM
 device_readings
GROUP BY
 device_id,
 date_trunc('minute', event_ts - INTERVAL 2 MINUTES)
ORDER BY
 device_id,
 bucket_start;

```

**Answer: B**

### Question: 337

**Deep Dumps**

A data engineer is using Lakeflow Spark Declarative Pipelines Expectations feature to track the data quality of their incoming sensor data. Periodically, sensors send bad readings that are out of range, and they are currently flagging those rows with a warning and writing them to the silver table along with the good data. They've been given a new requirement - the bad rows need to be quarantined in a separate quarantine table and no longer included in the silver table.

This is the existing code for their silver table:

```

@dlt.table
@dlt.expect("valid_sensor_reading", "reading < 120")
def silver_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

```

Which code will satisfy the requirements?

A.

```

@dlt.table
@dlt.expect("valid_sensor_reading", "reading < 120")
def silver_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

@dlt.table
@dlt.expect("invalid_sensor_reading", "reading >= 120")
def quarantine_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

```

B.

```

@dlt.table
@dlt.expect_or_drop("valid_sensor_reading", "reading < 120")
def silver_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

@dlt.table
@dlt.expect("invalid_sensor_reading", "reading < 120")
def quarantine_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

```

C.

```

@dlt.table
@dlt.expect_or_drop("valid_sensor_reading", "reading < 120")
def silver_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

@dlt.table
@dlt.expect("invalid_sensor_reading", "reading >= 120")
def quarantine_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

```

D.



```

@dlt.table
@dlt.expect_or_drop("valid_sensor_reading", "reading < 120")
def silver_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

@dlt.table
@dlt.expect_or_drop("invalid_sensor_reading", "reading >= 120")
def quarantine_sensor_readings():
 return spark.readStream.table("bronze_sensor_readings")

```

Answer: C

### Question: 338

Deep Dumps

A data engineer is creating a data ingestion pipeline to understand where customers are taking their rented bicycles during use. The engineer noticed that, over time, data being transmitted from the bicycle sensors fail to include key details like latitude and longitude. Downstream analysts need both the clean records and the quarantined records available for separate processing.

The data engineer already has this code:

```

import dlt
from pyspark.sql.functions import expr

rules = {
 "valid_lat": "(lat IS NOT NULL)",
 "valid_long": "(long IS NOT NULL)"
}
quarantine_rules = "NOT({0})".format(" AND ".join(rules.values()))

@dlt.view
def raw_trips_data():
 return spark.readStream.table("ride_and_go.telemetry.trips")

```

How should the data engineer meet the requirements to capture good and bad data?

A.

```

@dlt.table(name="trips_data_quarantine")
def trips_data_quarantine():
 return (
 spark.readStream.table("raw_trips_data")
 .filter(expr(quarantine_rules))
)

```



- B.
- ```
@dlt.table
@dlt.expect_all_or_drop(rules)
def trips_data_quarantine():
    return spark.readStream.table("raw_trips_data")
```
- C.
- ```
@dlt.table(partition_cols=["is_quarantined"],)
@dlt.expect_all(rules)
def trips_data_quarantine():
 return (
 spark.readStream.table("raw_trips_data")
 .withColumn("is_quarantined", expr(quarantine_rules))
)
```
- D.
- ```
@dlt.view
@dlt.expect_or_drop("lat_long_present", "(lat IS NOT NULL AND long IS NOT NULL)")
def trips_data_quarantine():
    return spark.readStream.table("ride_and_go.telemetry.trips")
```

Answer: C

Question: 339

Deep Dumps

To identify the top users consuming compute resources, a data engineering team needs to monitor usage within their Databricks workspace for better resource utilization and cost control. The team decided to use Databricks system tables, available under the System catalog in Unity Catalog, to gain detailed visibility into workspace activity.

Which SQL query should the team run from the System catalog to achieve this?

- A.

%sql

```
SELECT sku_name,  
       usage_metadata.run_name AS user_email,  
       sum(usage_quantity) AS total_dbus  
FROM  
       system.billing.usage  
GROUP BY  
       user_email, sku_name  
ORDER BY  
       total_dbus DESC  
LIMIT 10
```

B.

```
%sql
SELECT sku_name,
       identity_metadata.created_by AS user_email,
       count(usage_quantity) AS total_dbus
FROM
       system.billing.usage
GROUP BY
       user_email, sku_name
ORDER BY
       total_dbus DESC
LIMIT 10
```

C.

```
%sql
SELECT sku_name,
       identity_metadata.created_by AS user_email,
       sum(usage_quantity* usage_unit) AS total_dbus
FROM
       system.billing.usage
GROUP BY
       user_email, sku_name
ORDER BY
       total_dbus DESC
LIMIT 10
```

D.

```
%sql
SELECT
    identity_metadata.run_as AS user_email,
    SUM(usage_quantity) AS total_dbus
FROM
    system.billing.usage
GROUP BY
    user_email
ORDER BY
    total_dbus DESC
LIMIT 10
```

Answer: D

Question: 340

Deep Dumps

A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task. Which statement explains what is preventing this privilege transfer?

- A. Only workspace administrators can grant "Owner" privileges to a group.
- B. Databricks jobs must have exactly one owner: "Owner" privileges cannot be assigned to a group.
- C. A user can only transfer job ownership to a group if they are also a member of that group.
- D. The creator of a Databricks job will always have "Owner" privileges: this configuration cannot be changed.

Answer: A

Explanation:

Correct Answer

Option A: Only workspace administrators can grant "Owner" privileges to a group.

Explanation of the Correct Answer

The correct answer is A because, in Databricks, granting "Owner" permissions, especially to groups, is usually limited to workspace administrators. This is a security measure to prevent unauthorized privilege increases.

Analysis of Each Option

Option A: This option is correct. Workspace administrators have the necessary permissions to grant "Owner" privileges to a group.

Option B: This option is incorrect. Databricks does allow assigning "Owner" privileges to groups for better team management.

Option C: This option is incorrect. A user doesn't need to be a member of a group to transfer ownership if they have sufficient privileges.

Option D: This option is incorrect. While the creator initially has "Owner" privileges, these can be transferred to another user or group by an authorized user.

Key Takeaways

Granting "Owner" privileges, especially to groups, is typically restricted to workspace administrators in Databricks.

Understanding user roles and permissions is crucial for managing Databricks jobs effectively.

Always check the required permissions before attempting to transfer ownership of Databricks jobs.

Question: 341

Deep Dumps

An analytics team wants run an experiment in the short term on the customer transaction Delta table (with 20 billions records) created by the data engineering team in Databricks SQL.

Which strategy should the data engineering team use to ensure minimal downtime and no impact on the ongoing ETL processes?

- A. Create a new table for the analytics team using a CTAS statement.
- B. Shallow clone the table for the analytics team.
- C. Give access to the table for the analytics team.
- D. Deep clone the table for the analytics team.

Answer: B

Explanation:

B. Shallow clone the table for the analytics team.

Explanation

This scenario is a classic test of **Delta Lake cloning features** in the **Databricks Certified Data Engineer Professional** exam. The goal is to provide a 20-billion-record table for a short-term experiment with **minimal downtime** and **no impact** on the ongoing ETL process.

1. **Shallow Clone (Correct):** A shallow clone creates a **new Delta Lake table** by copying only the metadata (the transaction log/pointers to the underlying Parquet files) from the source table. It does **not copy the data** itself.
2. **Minimal Downtime/Impact:** The cloning operation takes only seconds, regardless of the 20 billion records, because it's a metadata-only copy. The ongoing ETL process on the original table is completely unaffected.

3. **Cost-Effective:** It uses **zero additional storage** for the data. The experiment uses the existing data files.
4. **Deep Clone (Incorrect):** A deep clone copies both the metadata and all the data files. This would take a significant amount of time (hours or longer for 20 billion records) and would require double the storage space, violating the "minimal downtime" requirement.
5. **CTAS Statement (Incorrect):** A CREATE TABLE AS SELECT (CTAS) statement copies the underlying data. This is similar to a deep clone, taking a long time and creating a new set of data files, violating the "minimal downtime" requirement.
6. **Give Access to the table (Incorrect):** Simply giving access allows the analytics team to run their experiment directly on the production table, which risks performance impact on the ongoing ETL process and potential data corruption if their experiment involves destructive DML statements. A clone provides **isolation**.

Question: 342

Deep Dumps

A company has a task management system that tracks the most recent status of tasks. The system takes task events as input and processes events in near real-time using Lakeflow Spark Declarative Pipelines. A new task event is ingested into the system when a task is created or the task status is changed. Lakeflow Spark Declarative Pipelines provides a streaming table (table name: tasks_status) for the BI users to query. The table represents the latest status of all tasks and includes 5 columns: task_id (unique for each task), task_name, task_owner, task_status, task_event_time. The table enables 3 properties: deletion vectors, row tracking, and change data feed. A data engineer is asked to create a new Lakeflow Spark Declarative Pipelines to enrich the "tasks_status" table in near real-time by adding one additional column representing task_owner's department, which can be looked up from a static dimension table (table name: employee). How should this enrichment be implemented?

- A. Create a new Lakeflow Spark Declarative Pipelines: use readStream() function with option readChangeFeed to read tasks_status table CDF; enrich with the employee table; create a new streaming table as the result table and use apply_changes() function to process the changes from the enriched CDF.
- B. Create a new Lakeflow Spark Declarative Pipeline: use the read() function to read tasks_status table; enrich with employee table; store the result in a materialized view.
- C. Create a new Lakeflow Spark Declarative pipeline: use the readStream() function to read tasks_status table; enrich with the employee table; store the result in a new streaming table.
- D. Create a new Lakeflow Spark Declarative Pipeline: use the readStream() function with the option skipChangeCommits to read the tasks_status table; enrich with the employee table; store the result in a new streaming table.

Answer: A

Explanation:

Option A correctly uses a streaming read of the tasks_status table with the change data feed enabled. Since tasks_status is a streaming table that maintains only the latest state of each task, the change data feed is required to capture updates and deletions in near real time. After enriching the stream with the static employee dimension table, the pipeline can write to another streaming table and use apply_changes() to maintain an accurate, updated enriched view. This is the intended design pattern when enriching a latest-state table using Lakeflow Spark Declarative Pipelines.

Option B is incorrect because it performs a batch read, which does not support continuous near-real-time updates. A materialized view would also not deliver the required streaming behavior.

Option C is incorrect because reading the streaming table without the change data feed will not correctly capture updates to the latest-state table, leading to an inaccurate enriched table.

Option D is incorrect because skipChangeCommits is not meant for update propagation and does not provide the correct semantics for processing changes in a latest-state table.

Question: 343**Deep Dumps**

Predictive Optimization is an automated Databricks service enabled by default for Unity Catalog Managed tables. It helps maintain Delta tables by continuously optimizing them to ensure optimal performance and costs. Which two operations does Predictive Optimization run to maintain the Delta tables?

- A. ANALYZE
- B. OPTIMIZE
- C. BUCKETING
- D. PARTITION BY
- E. COMPACT

Answer: B,E

Explanation:

Correct Choices (B and E)

B. OPTIMIZE: Correct. OPTIMIZE is the primary command run by PO. It combines many small data files into fewer, larger files, a process called compaction, and also performs Liquid Clustering.

E. COMPACT: Correct. Compaction is the functional action performed by the OPTIMIZE command. The goal of OPTIMIZE is to compact files. In a multiple-choice scenario where OPTIMIZE and COMPACT are separate, but both represent the file size optimization goal, both are considered correct as they describe the essential function.

Incorrect Choices (A, C, and D)

A. ANALYZE: Incorrect as a primary answer, but is run. PO does run the underlying mechanisms to ANALYZE and update statistics for the query optimizer. However, in the context of file maintenance (compaction/cleanup), OPTIMIZE and COMPACT are the more descriptive answers related to data layout.

C. BUCKETING: Incorrect. Bucketing is an older, static physical data layout feature in Delta Lake that is now largely superseded by Liquid Clustering. PO performs clustering (via OPTIMIZE) but does not perform the legacy bucketing operation.

D. PARTITION BY: Incorrect. Partitioning is a static table property defined when the table is created and cannot be dynamically "run" or maintained by PO. PO's job is to optimize the data within the existing partitions.

Question: 344**Deep Dumps**

A data engineer is troubleshooting a slow-running Delta Lake query on Databricks SQL that involves complex joins and large datasets. They need to identify whether the root cause is related to poor data skipping, inefficient join strategies, or excessive data shuffling. Which approach should identify the specific bottlenecks using native Databricks tools?

- A. Analyze the Top Operators panel in the Query Profile to identify high-cost operations like BroadcastNestedLoopJoin
- B. Use the LIMIT clause to run a subset of the query and compare execution times with the full dataset.
- C. Check the query's execution time in the Jobs UI and correlate it with cluster resource utilization metrics.
- D. Enable the EXPLAIN command to review the parsed logical plan and manually estimate shuffle sizes.

Answer: A

Explanation:

Why Option A is Best

Granular Detail: The Query Profile shows exactly how the query was executed, including the actual data flow, the time spent on each task, and the amount of data processed/shuffled by each operator.

Identifying Bottlenecks: The Top Operators panel allows the data engineer to specifically identify:

High-Cost Operations: Operators like SortMergeJoin or BroadcastNestedLoopJoin (as mentioned in the option) immediately highlight inefficient join strategies or data skew.

Data Skipping: The profile clearly shows how many files were scanned versus how many were skipped, which directly diagnoses poor data skipping.

Data Shuffling: Operators like Exchange or Shuffle explicitly show the volume of data being shuffled between cluster nodes, diagnosing excessive data movement.

Why Other Options Are Not Optimal

B. Use the LIMIT clause...: This only confirms the query structure works but does not reveal the cost (shuffling, joining strategy) associated with the full dataset, which is where performance issues typically arise.

C. Check the query's execution time in the Jobs UI and correlate it with cluster resource utilization metrics. This provides a high-level view (a symptom) but lacks the granular detail (the root cause) needed to pinpoint why the CPU is spiking (e.g., is it shuffling, scanning, or joining?).

D. Enable the EXPLAIN command to review the parsed logical plan and manually estimate shuffle sizes. The EXPLAIN command shows the planned execution, not the actual runtime execution, data sizes, or time spent. It is useful for high-level strategy but insufficient for diagnosing runtime issues like skew or poor file skipping.

Question: 345

Deep Dumps

A data engineer needs to productionize a new Spark application written by a teammate. This application has numerous external dependencies, including libraries, and requires custom environment variables and Spark configuration parameters to be set. Which two methods will help the data engineer accomplish the task?

- A. Add libraries to compute policies
- B. Install libraries on DBFS
- C. Use compute policies to set system properties, environment variables, and Spark configuration parameters
- D. Create init scripts on DBFS
- E. Use secrets in init scripts to store configuration data

Answer: C,D

Explanation:

C. Use compute policies to set system properties, environment variables, and Spark configuration parameters

Reasoning: Compute policies (or Cluster Policies in some systems) are a robust way to enforce and standardize cluster configurations. They allow the data engineer to pre-define the required Spark configuration parameters and environment variables that the application needs to run, ensuring consistency between development and production environments.

D. Create init scripts on DBFS

Reasoning: Init scripts (initialization scripts) are executed during cluster startup. This is the standard, flexible way to handle the installation of external dependencies/libraries that are not natively available on the cluster image. By placing these scripts on a persistent storage like DBFS (Databricks File System), they can be reliably sourced and executed every time the production cluster starts up, setting up the required environment for the application.

Incorrect Methods

A. Add libraries to compute policies

Reasoning: Compute policies govern settings (like configurations, node types, max clusters), not the installation of specific library files. Library installation is typically handled via the cluster UI, API, or, for custom/complex setups, via init scripts.

B. Install libraries on DBFS

Reasoning: While libraries can be stored on DBFS, simply storing them there doesn't install or make them available to the Spark driver/executors. An init script (D) or the cluster's library management features must be used to actually attach and distribute the libraries from DBFS to the cluster nodes.

E. Use secrets in init scripts to store configuration data

Reasoning: This is a good practice for sensitive configuration data (like database passwords or API keys), but the prompt is primarily about standard configuration parameters and dependencies. While useful for security, it does not directly address the need to set custom environment variables, Spark configurations, and external library installation, which is the main challenge.

Question: 346

Deep Dumps

A data engineer is ingesting JSON files from cloud object storage using Databricks Auto Loader. The source folder may occasionally receive large files of data, which risks overwhelming the stream. To ensure predictable micro-batch sizes, the team wants to throttle ingestion based on the volume of data scanned at 1GB, regardless of the number of files. Which Auto Loader configuration should the data engineer use to achieve this?

- A. Configure `cloudFiles.maxSizePerTrigger` with 1GB to place a limit.
- B. Configure `cloudFiles.maxPartitionBytes` with 1GB to limit data in each partition.
- C. Configure `cloudFiles.maxBytesPerTrigger` with 1GB to place a limit.
- D. Configure `cloudFiles.maxFilesPerTrigger` and estimate the average file size to approximate a size-based throttle of 1GB.

Answer: C

Explanation:

C. `cloudFiles.maxBytesPerTrigger` is the correct configuration because it directly limits the volume of data that Auto Loader reads in each micro-batch. When set to 1GB, Auto Loader will read files only until the total number of bytes scanned reaches roughly that threshold, regardless of how many files contribute to that amount. This provides predictable and stable micro-batch sizes even when the source directory occasionally receives very large files, exactly matching the requirement for a size-based throttle.

Incorrect

A. `cloudFiles.maxSizePerTrigger` cannot be used in this scenario because it is not an Auto Loader configuration option. While similar-sounding options exist for other structured streaming file sources, Auto

Loader manages file discovery differently and does not support this setting. Because it is not recognized by Auto Loader, it cannot control or throttle ingestion volume, making it ineffective for predictable micro-batch sizing.

B. `cloudFiles.maxPartitionBytes` is used to influence how input data is divided into partitions that Spark tasks will process. This affects how work is distributed across the cluster but has no role in controlling how much data enters the stream per trigger. Even if set to 1GB, it would simply cause Spark to create partitions up to that size inside a micro-batch, not restrict the micro-batch itself. Large files would still be ingested, and the batch size could still exceed the desired limit, so this option does not address the team's need.

D. `cloudFiles.maxFilesPerTrigger` allows control over how many files are processed in each micro-batch, but it does not guarantee any control over the total amount of data. If file sizes vary significantly — especially with the presence of occasional large files — the total data per micro-batch can swing widely even with this setting. Estimating an average file size to approximate a 1GB limit would be unreliable and could still lead to unpredictable batch sizes. Therefore, this option does not satisfy the requirement to throttle ingestion based explicitly on data volume.

Question: 347

Deep Dumps

Why are Pandas UDFs often preferred over traditional PySpark UDFs in performance-critical applications involving large datasets?

- A. They leverage Apache Arrow to enable vectorized operations between the JVM and Python runtimes, reducing serialization costs and improving computational efficiency.
- B. They eliminate the JVM-Python boundary by bypassing serialization entirely, thereby avoiding data conversion overhead.
- C. They minimize memory usage by streaming each row individually through a lightweight Python wrapper, avoiding batch processing overhead.
- D. They allow row-level execution of functions in Python with native Spark optimization, removing the need for columnar execution.

Answer: A

Explanation:

Option A — “They leverage Apache Arrow to enable vectorized operations between the JVM and Python runtimes, reducing serialization costs and improving computational efficiency.” (Correct)

Pandas UDFs use Apache Arrow to transfer data in efficient, columnar batches rather than row-by-row. This dramatically reduces serialization overhead and allows Python to perform fast vectorized pandas and NumPy computations. The combination of columnar transfer and vectorized execution makes Pandas UDFs significantly faster for large, performance-critical Spark workloads.

Option B — “They eliminate the JVM-Python boundary by bypassing serialization entirely, thereby avoiding data conversion overhead.” (Incorrect)

Pandas UDFs do not eliminate the boundary between the JVM and Python. They still serialize and deserialize data, but Arrow makes this process more efficient by using a columnar format. Data conversion still occurs, just faster. Therefore, this option is incorrect because Pandas UDFs optimize serialization rather than bypassing it completely.

Option C — “They minimize memory usage by streaming each row individually through a lightweight Python wrapper, avoiding batch processing overhead.” (Incorrect)

Pandas UDFs do not stream individual rows. Traditional PySpark UDFs do. Pandas UDFs deliberately process

large columnar batches to reduce overhead and enable vectorized execution, which improves performance. The claim that they avoid batch processing is incorrect because batching is central to their design and essential for their speed advantage.

Option D — “They allow row-level execution of functions in Python with native Spark optimization, removing the need for columnar execution.” (Incorrect)

Pandas UDFs never operate row-by-row. They rely on batch-based, columnar execution using pandas Series or DataFrames. Spark still performs columnar optimization, and Pandas UDFs integrate with that architecture. This option is incorrect because Pandas UDFs depend on vectorized, column-oriented processing rather than eliminating or replacing it.

Question: 348

Deep Dumps

A data engineer is bringing an existing production Databricks job under asset bundle management and wants to ensure that:

- The job's current configuration is captured as YAML, and all referenced files are included in their bundle project.
- Future changes to the bundle's YAML will update the existing job in-place (not create a new job)

How should the data engineer successfully move the production job under asset bundle management?

- A. Run Databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deploy to deploy the bundle, which will always update the existing job automatically.
- B. Export the job definition as JSON, convert it to YAML, and place it in your bundle. Then, run Databricks bundle deploy to update the existing job.
- C. Manually create the YAML configuration for the job in your bundle project, ensuring all settings match the existing job. Then, run Databricks bundle deploy to deploy the bundle, which will update the existing job in your workspace.
- D. Run databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deployment, bind to link the bundle's job resource to the existing job in Databricks

Answer: D

Explanation:

Correct Answer: D — “Run databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deployment bind to link the bundle's job resource to the existing job in Databricks.”

This is correct because the proper workflow for bringing an existing Databricks job under asset bundle management requires two steps:

First, running databricks bundle generate job --existing-job-id captures the job's current configuration into bundle YAML and pulls all referenced files into the project. Second, you must run databricks bundle deployment bind, which formally links the generated bundle resource to the existing job in your workspace. Binding ensures that future databricks bundle deploy commands update that same job in-place rather than creating a new job. Without binding, the system cannot guarantee in-place updates.

Option A — “Run Databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deploy to deploy the bundle, which will always update the existing job automatically.”

This is incorrect because generate alone does not connect the new bundle resource to the existing job. Without running the bind step, bundle deploy may create a new job instead of updating the original one. The claim that deploy will “always update the existing job automatically” is not true unless binding has occurred.

Option B — “Export the job definition as JSON, convert it to YAML, and place it in your bundle. Then, run Databricks bundle deploy to update the existing job.”

This is incorrect because manually exporting JSON and converting it to YAML does not establish the necessary bundle-to-existing-job linkage. Deploying an unbound job resource will result in a new job being created rather than updating the existing production job. Databricks does not infer resource identity from manually created YAML.

Option C — “Manually create the YAML configuration for the job in your bundle project, ensuring all settings match the existing job. Then, run Databricks bundle deploy to deploy the bundle, which will update the existing job in your workspace.”

This is incorrect because manually recreating the YAML still does not bind the bundle resource to the existing job. Having matching settings does not cause Databricks to update the existing job. Without a binding step, deploy will almost always create a brand-new job rather than modifying the production one.

Question: 349

Deep Dumps

A data engineer is working on a Databricks notebook that requires several third-party Python libraries. Some of these are available on PyPi, while others are custom-developed and stored as local wheel (.whl) and source (.tar.gz) files in an S3 bucket. The goal is to ensure all dependencies are installed and correctly available across multiple jobs running on any automated cluster in a Unity Catalog-enabled workspace. The engineer needs to install the required dependencies in a way that ensures a consistent environment setup across interactive notebooks and jobs and complies with workspace security policies (no internet access). Which approach should the engineer use to install and manage these dependencies while also ensuring reproducibility and compliance?

- A. Use `%pip install` in every notebook and job to install packages directly from PyPi and custom S3 paths.
- B. Use an init script on the cluster to install all dependencies using `pip`, referencing the local file system.
- C. Create a Python wheel file for the entire project, upload it to the Databricks Workspace Files or Volumes, and install it using a Cluster Library or `pip install` in a `requirements.txt` declared within a Databricks Asset Bundle.
- D. Install all dependencies manually in the driver node of an interactive cluster, then export the environment and `reimport` on job clusters using `%conda`

Answer: C

Explanation:

Correct Answer: C — “Create a Python wheel file for the entire project, upload it to the Databricks Workspace Files or Volumes, and install it using a Cluster Library or `pip install` in a `requirements.txt` declared within a Databricks Asset Bundle.”

This approach is correct because packaging the project and all its dependencies into a wheel provides a fully reproducible, version-controlled, offline-installable artifact.

Uploading the wheel to Workspace Files or Unity Catalog Volumes ensures compliance with security policies since clusters do not need internet access.

Asset Bundles allow you to specify dependencies in a `requirements.txt` or wheel reference so jobs and interactive notebooks consistently use the same environment.

This pattern is designed for Unity Catalog-enabled workspaces and automated clusters that require reproducible deployments across jobs.

Option A — “Use `%pip install` in every notebook and job to install packages directly from PyPi and custom S3 paths.”

This is incorrect because %pip install in every notebook or job leads to inconsistent environments. It also violates security policies in a no-internet workspace because PyPI cannot be accessed. Requiring each job to run installation scripts slows execution, introduces drift, and is not reproducible or centrally controlled.

Option B — “Use an init script on the cluster to install all dependencies using pip, referencing the local file system.”

This is incorrect because init scripts are not the preferred method in Unity Catalog-enabled workspaces. They introduce operational overhead, environment drift, and maintenance challenges. They also require custom handling of S3 mounts or file distribution. Asset Bundles and workspace-managed library installation are the supported and reproducible method.

Option D — “Install all dependencies manually in the driver node of an interactive cluster, then export the environment and reimport on job clusters using %conda.”

This is incorrect because Databricks does not support exporting conda environments from one cluster and reusing them reliably across job clusters. Manual driver installation is not reproducible and produces environment drift. %conda is limited and not intended for production dependency management or multi-job reproducibility.

Question: 350

Deep Dumps

A data governance team at a large enterprise is improving data discoverability across its organization. The team has hundreds of tables in their Databricks Lakehouse with thousands of columns that lack proper documentation. Many of these tables were created by different teams over several years, with missing context about column meanings and business logic. The data governance team needs to quickly generate comprehensive column descriptions for all existing tables to meet compliance requirements and improve data literacy across the organization. They want to leverage modern capabilities to automatically generate meaningful descriptions rather than manually documenting each column, which would take months to complete. The team is looking for a solution that can understand data patterns, column names, and sample values to create intelligent descriptions. Which approach should the team use in Databricks to automatically generate column comments and descriptions for existing tables?

- A. Use Delta Lake's DESCRIBE HISTORY command to analyze table evolution and infer column purposes from historical changes.
- B. Use the DESCRIBE TABLE command to extract existing schema information and manually write descriptions based on column names and data types.
- C. Write custom PySpark code using df.describe() and df.schema to programmatically generate basic statistical descriptions for each column.
- D. Navigate to the table in Databricks Catalog Explorer, select the table schema view, and use the "AI Generate" option which leverages artificial intelligence to automatically create meaningful column descriptions based on column names, data types, sample values, and data patterns.

Answer: D

Explanation:

Correct Answer: D — “Navigate to the table in Databricks Catalog Explorer, select the table schema view, and use the ‘AI Generate’ option which leverages artificial intelligence to automatically create meaningful column descriptions based on column names, data types, sample values, and data patterns.”

Databricks provides a built-in AI Generate feature in Catalog Explorer that automatically produces meaningful column descriptions. It analyzes column names, data types, sample values, and data patterns. This allows teams to quickly document large numbers of tables, meet compliance requirements, and improve data literacy across the organization without manual effort.

Option A — “Use Delta Lake’s DESCRIBE HISTORY command to analyze table evolution and infer column

purposes from historical changes.”

DESCRIBE HISTORY only provides audit metadata such as operation type, timestamps, user actions, and table versions. It cannot generate column descriptions or analyze data content, so it does not help with automatic documentation.

Option B — “Use the DESCRIBE TABLE command to extract existing schema information and manually write descriptions based on column names and data types.”

DESCRIBE TABLE outputs the current schema, but any descriptions must be written manually, which is impractical for hundreds of tables with thousands of columns. It does not provide automatic AI-generated insights.

Option C — “Write custom PySpark code using df.describe() and df.schema to programmatically generate basic statistical descriptions for each column.”

df.describe() and df.schema provide basic statistics and data types, but they cannot generate meaningful textual descriptions of column purpose or business logic. Custom coding would be required, and it does not scale efficiently for large datasets.

Question: 351

Deep Dumps

A data engineer manages a Unity Catalog table `customer_data` in schema `finance` that includes sensitive fields like `ssn` and `credit_score`. Intern Group should only see masked values, while Analyst Group should only access rows for their assigned region. The data engineer needs to restrict access based on user role and region without duplicating data. How should the data engineer enforce this security policy?

- A. Create dynamic views for each user role and manage access with ACLs.
- B. Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL `SELECT` clause for each sensitive column.
- C. Use Unity Catalog's row filters based on the user roles and column masks based on the region.
- D. Use Unity Catalog's row filters based on the region and column masks based on user roles.

Answer: D

Explanation:

Option D — “Use Unity Catalog's row filters based on the region and column masks based on user roles.”

This is correct. Unity Catalog allows row filters to restrict data visibility based on column values (e.g., `region = current_user_region`) and column masking policies to hide sensitive fields based on user roles. Using row filters for regions ensures Analysts see only their assigned region's rows, while column masks ensure Interns see masked sensitive data. This enforces the security policy without duplicating data.

Option A — “Create dynamic views for each user role and manage access with ACLs.”

Creating separate views for each role is possible but requires duplicating views and managing multiple ACLs. This approach does not scale well for many roles or regions and does not leverage Unity Catalog's built-in row- and column-level security features.

Option B — “Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL `SELECT` clause for each sensitive column.”

This approach relies on SQL functions and manual masking logic inside views. While it can work, it requires complex view maintenance and does not use the native Unity Catalog row- and column-level security policies,

which are designed to enforce access without duplicating data.

Option C — “Use Unity Catalog’s row filters based on the user roles and column masks based on the region.”

This reverses the intended policy. The requirement is that Interns see masked sensitive fields and Analysts see only rows for their region. Option C applies row filters for roles and column masks for regions, which does not match the intended access rules.

Question: 352

Deep Dumps

A data engineer and a platform engineer are working together to automate their system tasks. A script needs to be executed outside of Databricks only if a particular daily Databricks job finishes successfully for the day. Databricks CLI command was used to check the last execution of the job. What are the required command options for that task?

- A. `databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --completed-only`
- B. `databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --active-only`
- C. `databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --completed-only`
- D. `databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --active-only`

Answer: C

Explanation:

Correct Answer: C

C. `databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --completed-only`

Why it's correct:

`--start-time-from` filters runs starting today (from midnight onward).

`--completed-only` returns only finished runs (success/failed), which is required to verify successful daily completion.

Incorrect Answer Explanations

A. `--start-time-to TODAY_MIDNIGHT_EPOCH_MS --completed-only`

Why it's wrong:

`--start-time-to` returns runs that happened before today's midnight.

This retrieves yesterday's runs, not today's.

So you would not correctly detect today's job completion.

B. `--start-time-from TODAY_MIDNIGHT_EPOCH_MS --active-only`

Why it's wrong:

`--active-only` lists runs that are currently in progress, not finished.

You cannot determine whether the job completed successfully because no completed runs are shown.

It's impossible to trigger an external script based on job success using this.

D. `--start-time-to TODAY_MIDNIGHT_EPOCH_MS --active-only`

Why it's wrong:

Combines two incorrect filters:

`--start-time-to` → gets runs before today, which isn't useful.

`--active-only` → only shows running jobs, not completed ones.

Completely unusable for determining whether today's job finished successfully.

Question: 353

Deep Dumps

A data engineer needs to design an efficient pipeline that automatically processes new CSV files as they arrive in S3 storage. Which Databricks feature should the data engineer use to meet these requirements?

- A. Auto Loader with schema inference and evolution enabled
- B. Traditional batch processing with scheduled Databricks Jobs
- C. `COPY INTO SQL` command with parameters to track processed files
- D. Streaming from cloud storage using standard Spark `readStream` with `format("csv")` and `format("json")`

Answer: A

Explanation:

A. Auto Loader with schema inference and evolution enabled

Auto Loader is the most efficient and scalable Databricks feature for automatically processing new CSV files as they arrive in S3. It continuously monitors the storage location, detects newly added files without repeatedly listing the entire directory, and processes them incrementally. With schema inference and evolution enabled, Auto Loader can automatically understand the structure of incoming CSVs and adapt to changes over time, making it ideal for pipelines that must operate continuously and reliably with minimal manual intervention.

Incorrect Option Explanations

Option B is **incorrect** because traditional batch processing with scheduled Databricks Jobs does not automatically detect new files as they arrive; it simply runs on a fixed schedule such as hourly or daily. This means the system either reacts too slowly or reruns unnecessarily, leading to inefficiency and higher compute costs.

Option C is **incorrect** because the `COPY INTO SQL` command is not designed for continuous file detection. While it keeps track of which files have been processed, it still requires manually triggered commands or scheduled runs, so it does not provide truly automatic or event-driven ingestion.

Option D is **incorrect** because using standard Spark Structured Streaming with `readStream` on CSV files is far less efficient in cloud storage environments. It relies on repeatedly scanning directories for new files, which becomes slow and costly at scale, and it lacks built-in support for schema evolution. Auto Loader was created specifically to solve these limitations, making it the superior choice.

Question: 354

Deep Dumps

A faulty IoT sensor in a factory reports a temperature of -500, causing the LDP pipeline to fail the expectation,

which only allows values between -100 and 200 degrees Celsius. The data engineer would like to further analyze the faulty data to better understand the reason behind this. How should the data engineer resolve the faulty data while ensuring data quality standards are maintained?

- A. Change the expectation action from fail to warn so that invalid records are included in the output and the pipeline does not fail.
- B. Ignore the error and simply re-run the pipeline, as Databricks will automatically skip the problematic record on the next run.
- C. Fix the pipeline code and implement a quarantine logic to isolate the faulty data before re-running the pipeline.
- D. Remove all expectations from the pipeline to prevent any future failures, regardless of data quality.

Answer: C

Explanation:

C. Fix the pipeline code and implement a quarantine logic to isolate the faulty data before re-running the pipeline.

Implementing quarantine logic is the correct approach because it preserves data quality while still allowing the pipeline to run. Instead of letting invalid data (such as a temperature reading of -500°C) break the pipeline or pass silently into production tables, the pipeline should route these faulty records into a quarantine or “dead-letter” table. This allows the engineering team to analyze the bad data separately, investigate root causes, and maintain strict data quality in the main dataset. This design is a best practice for Delta Live Tables pipelines and enterprise-grade data processing in general.

Incorrect Option Explanations

A — Change the expectation action from fail to warn so that invalid records are included in the output and the pipeline does not fail.

This is incorrect because switching the expectation action to “warn” weakens your data quality enforcement. Although the pipeline would run, invalid data would flow into production tables, which undermines the purpose of quality checks and may create downstream issues.

B — Ignore the error and simply re-run the pipeline, as Databricks will automatically skip the problematic record on the next run.

This is incorrect because Databricks does not automatically skip invalid data on reruns. The same invalid record would still violate the expectation and cause the pipeline to fail again unless the engineer adds logic to handle it.

D — Remove all expectations from the pipeline to prevent any future failures, regardless of data quality.

This is incorrect because removing expectations eliminates essential data quality controls. Without expectations, corrupted or nonsensical data could silently pollute production datasets, leading to poor analytics, incorrect decisions, and potential system failures.

Question: 355

Deep Dumps

A data engineer is attempting to execute the following PySpark code: `df = spark.read.table("sales")result = df.groupBy("region").agg(sum("revenue"))` However, upon inspecting the execution plan and profiling the Spark job, they observe excessive data shuffling during the aggregation phase. Which technique should be applied to reduce shuffling during the groupBy aggregation operation?

- A. Caching the DataFrame df

- B. Use `.coalesce(1)` after the aggregation
- C. Repartition by region before aggregation
- D. Use broadcast join

Answer: C

Explanation:

C. Repartition by region before aggregation

Shuffles in Spark occur when data must be redistributed across the cluster so that all rows for each grouping key (in this case, region) end up on the same partition. When the underlying DataFrame is not already partitioned by the grouping column, Spark must perform a costly exchange shuffle during the `groupBy` operation. By explicitly calling `df.repartition("region")` before performing the aggregation, the data engineer ensures that all records belonging to the same region are already co-located. This greatly reduces the volume of data shuffled at aggregation time and improves overall job performance.

Incorrect Option Explanations

A — Caching the DataFrame df

Caching improves performance only when the same DataFrame is used multiple times in a workflow. It does not change the distribution of data and therefore does nothing to reduce shuffling during the `groupBy` operation.

B — Use `.coalesce(1)` after the aggregation

Coalesce reduces the number of partitions, but doing it after aggregation neither prevents nor reduces the shuffle required to perform the `groupBy`. It only adds additional overhead and forces output into a single partition, which may degrade performance.

D — Use broadcast join

Broadcast joins reduce shuffle during join operations by broadcasting a small table to all executors. Since no join is being performed here — only a `groupBy` — broadcasting has no effect on the shuffle caused by aggregation.

Question: 356

Deep Dumps

A data engineer is evaluating tools to build a production-grade data pipeline. The team must process change data from cloud object storage, filter out or isolate invalid records, and ensure the timely delivery of clean data to downstream consumers. The team is small, under tight deadlines, and wants to minimize operational overhead while keeping pipelines auditable and maintainable. Which approach should the data engineer implement?

- A. Implement ingestion using Auto Loader with Structured Streaming, and manage invalid data handling and table updates using checkpointing and merge logic.
- B. Use a hybrid approach: Ingest with Auto Loader into Bronze tables, then process using SQL queries in Databricks Workflows to generate cleaned Silver and Gold tables on a schedule.
- C. Ingest data directly into Delta tables via Spark jobs, apply data quality filters using UDFs, and use LDP for creating Materialized Views.
- D. Use LDP to build declarative pipelines with Streaming Tables and Materialized Views, leveraging built-in support for data expectations and incremental processing.

Answer: D

Explanation:

D. Use LDP to build declarative pipelines with Streaming Tables and Materialized Views, leveraging built-in support for data expectations and incremental processing.

LDP (Lakehouse Data Pipelines) is the best-fit solution because it provides a fully managed, declarative framework for building reliable pipelines with minimal operational overhead. It automatically handles incremental processing, dependency resolution, and monitoring, while also offering built-in expectations to filter, warn, or quarantine invalid data. Streaming Tables and Materialized Views simplify CDC ingestion from cloud storage and ensure timely delivery of high-quality data. For a small team with tight timelines, LDP offers the cleanest, most maintainable, and auditable approach with the least amount of manual work.

Incorrect Option Explanations

A — Implement ingestion using Auto Loader with Structured Streaming, and manage invalid data handling and table updates manually.

Although Auto Loader is powerful, managing checkpointing, merge logic, retries, and invalid record handling manually increases operational overhead. This contradicts the team's need for low-maintenance pipelines.

B — Use a hybrid approach with Auto Loader and scheduled SQL jobs.

This design works but still requires managing schedules, orchestration, retries, quality logic, and monitoring across multiple jobs. It's more operationally complex and less auditable than a fully declarative LDP pipeline.

C — Ingest data directly into Delta via Spark jobs and apply data quality filters using UDFs.

This increases code complexity and operational burden. UDF-based filtering is harder to maintain and less transparent than LDP expectations. Mixing Spark jobs with LDP features makes the architecture less coherent and more difficult to operate.

Question: 357

Deep Dumps

A data engineer is using Auto Loader to read in JSON data as it arrives. They have configured Auto Loader to quarantine Invalid JSON records. They are noticing that over time, some records are being quarantined even though they are well-formed JSON. The snippet of code is: `df = (spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").option("badRecordsPath", "/tmp/somewhere/badRecordsPath").schema("a int, b int").load("/Volumes/catalog/schema/raw data/"))` What is the cause of the missing data?

- A. At some point, the upstream data provider switched everything to multi-line JSON
- B. The source data is valid JSON, but doesn't conform to their defined schema in some way
- C. The badRecordsPath location is accumulating many small files
- D. The engineer forgot to set the option "cloudFiles.quarantineMode", "rescue"

Answer: B

Explanation:

B. The source data is valid JSON, but doesn't conform to their defined schema in some way

The quarantined records are well-formed JSON, but they do not match the explicitly defined schema "a int, b int". Auto Loader rejects any record whose structure or data types differ from this schema — such as missing fields, extra fields, or mismatched types — so valid JSON ends up in the badRecordsPath due to schema mismatch rather than JSON formatting errors.

Why the other options are incorrect

A — Upstream switched to multi-line JSON

Auto Loader can handle multi-line JSON only if `.option("multiLine", "true")` is set, but the question states the JSON is well-formed and simply ends up in quarantine. The most likely cause is schema mismatch, not encoding style.

C — The `badRecordsPath` location is accumulating many small files

While this can cause performance degradation, it does not cause valid JSON to be quarantined.

D — Missing `"cloudFiles.quarantineMode", "rescue"`

This option enables “rescue data” (adds unexpected fields to `_rescued_data`).

But even without it, records are not incorrectly quarantined — they are quarantined because they don’t match the schema.

Question: 358

Deep Dumps

A data engineer is designing a system leveraging Lakeflow Declarative Pipeline technology to process real-time truck telemetry data ingested from JSON files in S3 using Auto Loader. The data includes `truck_id`, `timestamp`, `location`, `speed`, and `fuel_level`. The system must support two use cases:

- Near-real-time monitoring of the latest `location`, `speed`, and `fuel_level` per `truck_id` for the operations team.
- Daily aggregated reports of total distance traveled and average fuel efficiency per `truck_id` for the management team.

Which approach should the data engineer use for streaming tables and materialized views in the Lakeflow Declarative Pipeline to meet these requirements?

- A. Define a materialized view to ingest and store the raw telemetry data, and create a streaming table to compute the latest `location`, `speed`, and `fuel_level` per `truck_id` for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.
- B. Define a streaming table to ingest and store the raw telemetry data, and create a materialized view to compute the latest `location`, `speed`, and `fuel_level` per `truck_id` for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.
- C. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting. Create a materialized view to compute the latest `location`, `speed`, and `fuel_level` per `truck_id` for real-time monitoring.
- D. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to incrementally compute the latest `location`, `speed`, and `fuel_level` per `truck_id` for real-time monitoring. Create a materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.

Answer: D

Explanation:

Correct Answer: D. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to incrementally compute the latest `location`, `speed`, and `fuel_level` per `truck_id` for real-time monitoring. Create a materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.

This is correct because Lakeflow Declarative Pipelines distinguish clearly between streaming tables (for incremental, low-latency processing) and materialized views (for periodic or cost-efficient aggregations). Since the operations team needs near-real-time monitoring, the best choice is to create a streaming table that continuously maintains the most recent `speed`, `location`, and `fuel_level` for each `truck_id` — this ensures results update instantly as Auto Loader ingests new telemetry. The daily management report, however, does not need millisecond latency. A materialized view is ideal because it automatically refreshes on a schedule and is optimized for daily batch-style aggregations like distance traveled and fuel efficiency. This combination

aligns perfectly with recommended patterns: streaming tables for real-time metrics, materialized views for scheduled aggregations.

Incorrect Options:

- A.** This is **incorrect** because materialized views are not used to ingest raw streaming data. Raw telemetry arriving continuously via Auto Loader must be captured in a streaming table, not an MV. MVs refresh periodically and cannot maintain a high-throughput streaming ingestion pattern.
- B.** This is **incorrect** because using a materialized view for the near-real-time monitoring is not appropriate. Materialized views do not guarantee low-latency updates; they refresh periodically. Real-time monitoring requires a streaming table that updates continuously.
- C.** This is **incorrect** because the daily aggregated reporting does not need a streaming table. Streaming tables continuously update results, which is unnecessary and more expensive for daily summarizations. A materialized view is better suited for scheduled, daily aggregations.

Question: 359

Deep Dumps

Which approach demonstrates a modular and testable way to use DataFrame.transform for ETL code in PySpark?

- A. `def upper_transform(df):return df.withColumn("value_upper", upper(col("value")))``actual = test_input.transform(upper_transform)``assertDataFrameEqual(actual, expected)`
- B. `def transform_data(input_df):# transformation logic here.return output_df``test_input = spark.createDataFrame([(1, "a")], ["id", "value"])``assertDataFrameEqual(transform_data(test_input), expected)`
- C. `def upper_value(df):return df.withColumn("value_upper", upper(col("value")))``def filter_positive(df):return df.filter(df["id"] > 0)``pipeline_df = df.transform(upper_value).transform(filter_positive)`
- D. `class Pipeline:``def transform(self, df):return df.withColumn("value_upper", upper(col("value")))``pipeline = Pipeline()``assertDataFrameEqual(pipeline.transform(test_input), expected)`

Answer: C

Explanation:

```
def upper_value(df):  
    return df.withColumn("value_upper", upper(col("value")))  
  
def filter_positive(df):  
    return df.filter(df["id"] > 0)  
  
pipeline_df = df.transform(upper_value).transform(filter_positive)  
...
```

Explanation

Option C demonstrates a **modular and testable approach** using `DataFrame.transform` in PySpark. Each transformation is encapsulated in a **small, reusable function** (`upper_value`, `filter_positive`) that takes a DataFrame and returns a new DataFrame. By chaining `.transform()` calls, the engineer builds a **clear and composable ETL pipeline**, making it easy to test each transformation independently and combine them flexibly. This approach promotes readability, reusability, and unit-testability.

Why the other options are incorrect

- A** wraps a single transformation in a function and immediately tests it. While testable, it is **not modular**

for chaining multiple transformations**, which limits pipeline composition.

B defines a monolithic `transform_data` function that performs all transformations inside one function. This reduces modularity and makes it harder to test individual steps separately.

D uses a class with a single `transform` method. While it can be tested, it **does not leverage the functional `.transform()` API** for chaining multiple reusable transformations and is less flexible than option C.

Question: 360

Deep Dumps

A data engineer is implementing a job to download multiple PDF files from a third-party-provided REST API endpoint by specifying different report types. The REST API is time-consuming and encounters intermittent errors, so the engineer wants to track each download activity to know when it fails and to retry partially, while providing scalable throughput. The engineer needs to download ten report types, and the list can be changed over time. How should the data engineer achieve this?

- A. Use a foreach task with a list of report types as its inputs.
- B. Use a Delta Lake table to track each report download status as 10 rows, and use it as a source table to execute the download function as a Pandas UDF.
- C. Define a list variable within a Notebook to loop through the report types to download them, and print the download results. Execute it as a Notebook task.
- D. Define ten Notebook tasks to clearly track which report download failed.

Answer: B

Explanation:

B. Use a Delta Lake table to track each report download status as 10 rows, and use it as a source table to execute the download function as a Pandas UDF.

Using a Delta Lake table to track the download status provides a robust and scalable solution. Each report type is represented as a row with metadata about its download status (e.g., pending, in-progress, succeeded, failed). The download job can then query this table, process each report type independently using a Pandas UDF (or a distributed function), and update the status upon completion or failure. This approach enables retrying only the failed downloads, supports dynamic addition or removal of report types, and provides auditable tracking of all download activities while scaling throughput across multiple executors.

Why the other options are incorrect

A — Use a foreach task with a list of report types

This approach lacks built-in tracking of individual downloads and would not easily support retries for failed downloads or dynamic report lists.

C — Loop through the report types in a single Notebook

A simple loop provides no scalability or parallelism and makes error handling and retries cumbersome. Printing results does not provide an auditable status.

D — Define ten Notebook tasks

Creating ten separate tasks is hard-coded and inflexible. It does not scale if the list of report types changes and is cumbersome to maintain, especially for retries and auditing.

Question: 361**Deep Dumps**

When monitoring a complex workload, being able to see the query plan is critical to understanding what the workload is doing. Where can the visualization of the query plan be found?

- A. In the Query Profiler, under Query Source
- B. In the Spark UI, under the SQL/DataFrame tab
- C. In the Query Profiler, under the Stages tab
- D. In the Spark UI, under the Jobs tab

Answer: B**Explanation:****B. In the Spark UI, under the SQL/DataFrame tab**

In Databricks, the visualization of a query plan for SQL or DataFrame operations is available in the Spark UI under the SQL/DataFrame tab. This tab provides a detailed breakdown of the physical and logical plans, including executed transformations, stage dependencies, and shuffles. By inspecting the query plan here, data engineers can identify performance bottlenecks, excessive shuffles, or expensive operations and optimize their workloads accordingly.

Why the other options are incorrect**A — In the Query Profiler, under Query Source**

The Query Profiler shows source code, execution metrics, and basic profiling but does not provide the detailed visual query plan.

C — In the Query Profiler, under the Stages tab

The Stages tab in the Query Profiler shows execution metrics per stage, not the logical or physical query plan.

D — In the Spark UI, under the Jobs tab

The Jobs tab displays job and stage progress, DAGs, and task-level metrics, but it does not visualize the query plan for SQL or DataFrame operations.

Question: 362**Deep Dumps**

A data engineer is building a customer data pipeline in Lakeflow Spark Declarative Pipelines. The source is a cloud-based event stream with limited retention containing inserts, updates, and deletes for customer records. These changes are being applied using the AUTO CDC INTO syntax to maintain an SCD Type 1 table as the target table, customer_dim. How should the data engineer build a downstream job that streams from the customer_dim table to only act on updates and delete events, processing data incrementally?

- A. Streaming from customer_dim table would only be possible in the case of SCD 2 retention.
- B. Use ignoreChanges flag while streaming from customer_dim to avoid breaking the pipeline during updates and deletes.
- C. Read change data feed from customer_dim table and apply filters to incrementally act on the change events.
- D. When stored as SCD 1, the target of AUTO CDC INTO includes updates and deletes. Streaming from customer_dim can fail due to these operations. Instead, build another stream from the original source.

Answer: C**Explanation:**

C. Read change data feed from customer_dim table and apply filters to incrementally act on the change events.

In Lakehouse pipelines with Delta tables, even SCD Type 1 tables can maintain a Change Data Feed (CDF), which tracks inserts, updates, and deletes. By streaming from the customer_dim table using the CDF, the data engineer can incrementally read only the changes since the last processed offset. Filtering on the __change_type column allows the downstream job to selectively act on updates and deletes without reprocessing unchanged rows. This approach ensures efficient, incremental processing and avoids full table scans, while maintaining pipeline consistency and performance.

Why the other options are incorrect

A — Streaming from customer_dim table would only be possible in the case of SCD 2 retention

This is incorrect because Delta Change Data Feed works with SCD 1 tables as well, enabling incremental processing of updates and deletes.

B — Use ignoreChanges flag while streaming

The ignoreChanges option tells Delta to ignore updates during streaming, which would skip exactly the events the downstream job needs to process.

D — Build another stream from the original source

While feasible, this approach duplicates ingestion logic and increases operational complexity. Streaming from the customer_dim table via CDF is simpler, more reliable, and maintains the benefit of incremental processing.

Question: 363

Deep Dumps

A data engineering team needs to implement a tagging system for their tables as part of an automated ETL process, and needs to apply tags programmatically to tables in Unity Catalog. Which SQL command adds tags to a table programmatically?

- A. ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');
- B. APPLY TAGS ON table_name VALUES ('key1' = 'value1', 'key2' = 'value2');
- C. COMMENT ON TABLE table_name TAGS ('key1' = 'value1', 'key2' = 'value2');
- D. SET TAGS FOR table_name AS ('key1' = 'value1', 'key2' = 'value2');

Answer: A

Explanation:

A. ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');

In Databricks Unity Catalog, tags can be applied programmatically to tables using the ALTER TABLE ... SET TAGS SQL command. This allows the data engineering team to automate metadata management as part of their ETL processes. Each tag is specified as a key-value pair, and multiple tags can be set in a single statement. This approach integrates seamlessly with automated pipelines and ensures that tables are properly annotated for governance, lineage, or classification purposes.

Why the other options are incorrect

B — APPLY TAGS ON table_name VALUES ...

This is not valid SQL syntax in Unity Catalog; there is no APPLY TAGS command.

C — COMMENT ON TABLE table_name TAGS ...

COMMENT ON TABLE is only used to add descriptive comments to tables, not key-value metadata tags.

D — SET TAGS FOR table_name AS ...

This syntax is incorrect; Unity Catalog does not support SET TAGS FOR ... AS. The proper command is ALTER TABLE ... SET TAGS.

Question: 364

Deep Dumps

A data engineer is designing a secure data sharing strategy for their organization. The company needs to share sensitive customer analytics data with two different partners: Partner A uses Databricks with Unity Catalog enabled, while Partner B uses Apache Spark on AWS without Databricks. How should the company implement secure data sharing for these scenarios?

- A. Databricks-to-Databricks sharing (D2D) can only be used within the same cloud provider, so you must use open sharing (D2O) for any cross-cloud scenarios. Unity Catalog governance is not available when sharing with external platforms.
- B. Open sharing protocol (D2O) should be used for both partners because it provides better security than D2D sharing. The bearer token approach is always more secure than Unity Catalog's native authentication.
- C. Both partners should use the same Delta Sharing approach since security requirements are identical. You should create bearer tokens for both partners and use the open sharing protocol (D2O) for maximum compatibility.
- D. For Partner A, implement Databricks-to-Databricks sharing (D2D) with Unity Catalog integration and no-token exchange system. For Partner B, use open sharing protocol (D2O) with either bearer tokens or OIDC federation for authentication, ensuring both approaches maintain robust security and governance.

Answer: D

Explanation:

Databricks provides two primary secure data sharing methods: Databricks-to-Databricks (D2D) and Delta Sharing/Open Sharing (D2O).

Partner A (Databricks with Unity Catalog): D2D sharing is the best approach. It integrates with Unity Catalog, enabling governance, fine-grained access control, and automatic enforcement of data security policies. D2D does not require bearer tokens because authentication is handled natively within Databricks.

Partner B (Apache Spark on AWS, non-Databricks): Since they are external and may not use Unity Catalog, Delta Sharing/Open Sharing (D2O) should be used. D2O provides secure, cross-platform data sharing via standard protocols, and authentication can be handled via bearer tokens or OIDC federation, ensuring secure, auditable access.

This strategy ensures robust security, proper governance, and compatibility for both internal and external partners while leveraging the strengths of each sharing method.

Why the other options are incorrect

- A suggests that D2D is limited to the same cloud provider and that Unity Catalog governance is unavailable externally. In reality, D2D works across clouds for Databricks-to-Databricks and preserves Unity Catalog governance.
- B incorrectly claims D2O is always more secure than D2D. D2D is preferable when both parties use Databricks with Unity Catalog because it provides integrated governance and native authentication.
- C recommends using D2O for both partners, which is unnecessary for Partner A and would bypass Unity

Question: 365

Deep Dumps

A job runs four independent tasks (X, Y, Z, W) in parallel to process regional sales data. The Data Engineering team recently updated its cluster policy to ban cost-prohibitive instance types. Task Y now fails due to the newly enforced cluster policy restricting the use of a specific instance type. A data engineer needs to resolve the failure quickly without disrupting the other tasks. How should the data engineer resolve the failure of tasks?

- A. Delete the failed run, disable the cluster policy, and re-execute all tasks.
- B. Use "Repair run", override the cluster configuration for Task Y to use a permitted instance type, and let Databricks re-run only Task Y.
- C. Edit the global cluster policy to allow the restricted instance type, then re-run the entire job.
- D. Manually create a new cluster for Task Y, update the job configuration, and trigger a full re-run.

Answer: B

Explanation:

B. Use "Repair run", override the cluster configuration for Task Y to use a permitted instance type, and let Databricks re-run only Task Y.

Databricks Jobs provide a Repair Run feature that allows you to re-run only failed or skipped tasks without affecting successful tasks. In this scenario, Task Y failed due to the cluster policy restricting the instance type. By using Repair Run, the data engineer can override the cluster configuration specifically for Task Y, selecting an allowed instance type. This resolves the failure efficiently and avoids re-running Tasks X, Z, and W, which have already completed successfully, minimizing compute costs and operational disruption.

Why the other options are incorrect

A — Delete the failed run, disable the cluster policy, and re-execute all tasks

This is disruptive and unnecessary; it would re-run all tasks, wasting time and compute resources, and disabling the policy could violate cost controls.

C — Edit the global cluster policy to allow the restricted instance type, then re-run the entire job

Modifying the global policy just to accommodate a single task is risky and affects all users and jobs, which is not recommended.

D — Manually create a new cluster for Task Y, update the job configuration, and trigger a full re-run

This is overly complex and would require re-running tasks that already succeeded. Using Repair Run with cluster override is the simpler and safer approach.

Question: 366

Deep Dumps

A data company uses Databricks Unity Catalog and has multiple enterprise data sources, including PostgreSQL, Snowflake, and SQL Server. The central data platform team wants to configure Lakehouse Federation so analysts can query external tables directly in Databricks using Databricks SQL, without duplicating data. Which steps are necessary to configure Lakehouse Federation in a secure and governed manner?

- A. Mirror the external datasets into Delta Lake using Auto Loader, and govern them using Data Lineage and System Tables.

- B. Create external locations and storage credentials to connect to each database, then register foreign tables in Unity Catalog.
- C. Configure connections and foreign catalog in Unity Catalog, then grant access to foreign catalogs, schemas, and tables using Unity Catalog permissions.
- D. Use Partner Connect to create linked datasets, and apply table ACLs at the source system to govern access through Databricks.

Answer: C

Explanation:

C. Configure connections and foreign catalog in Unity Catalog, then grant access to foreign catalogs, schemas, and tables using Unity Catalog permissions.

Databricks Lakehouse Federation allows analysts to query external databases such as PostgreSQL, Snowflake, and SQL Server without duplicating the data. To configure it securely and with proper governance, the data platform team must first define connections and foreign catalogs in Unity Catalog, which point to the external systems. Once the foreign catalogs are registered, Unity Catalog can enforce permissions on foreign catalogs, schemas, and tables, ensuring that access is secure, auditable, and consistent with enterprise governance policies. This approach allows seamless query federation while retaining centralized control over data access.

Why the other options are incorrect

A — Mirror the external datasets into Delta Lake

This duplicates the data, which defeats the purpose of query federation and may introduce additional storage costs and data freshness issues.

B — Create external locations and storage credentials

External locations and storage credentials are used for cloud object storage (like S3 or ADLS), not relational databases. This approach cannot connect directly to PostgreSQL, Snowflake, or SQL Server.

D — Use Partner Connect to create linked datasets

Partner Connect facilitates simple integration with partner services but does not provide full foreign catalog or Lakehouse Federation support and cannot enforce fine-grained Unity Catalog permissions on external tables.

Question: 367

Deep Dumps

A senior data engineer is planning large-scale data workflows. The current task is to identify the considerations that form a foundation for creating scalable data models that are essential for effective management of large datasets. The data engineering team has identified the core capabilities as part of a scalable data model to build a modern data platform and provided their reasoning for considering Delta Lake for review. The senior data engineer is responsible for identifying the recommendations that are not valid. Which key features can be ignored while evaluating Delta Lake?

- A. Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.
- B. Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.
- C. Delta Lake provides limited support for monitoring and troubleshooting data pipelines, so relevant partner tools have to be identified and set up for enhanced operational efficiency.
- D. Delta Lake optimizes metadata handling, efficiently managing billions of files and facilitating scalability to petabyte-scale datasets.

Answer: A,B,D

Explanation:

A — Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.

This is a valid consideration. Delta Lake supports both batch and streaming workloads on the same table, which provides flexibility for ETL pipelines, real-time analytics, and incremental processing. This capability is essential for building scalable and flexible data platforms.

B — Delta Lake works with various data formats (e.g., Parquet, JSON, CSV) and integrates well with Spark and Databricks tools.

This is also a valid feature. Delta Lake is built on Parquet and supports reading and writing other formats. Its tight integration with Apache Spark and Databricks ensures optimized performance, distributed processing, and compatibility with modern data engineering tools. These characteristics are critical for handling large datasets efficiently.

C — Delta Lake provides limited support for monitoring and troubleshooting data pipelines, so relevant partner tools have to be identified and set up for enhanced operational efficiency.

This is not a valid consideration. Delta Lake includes features like transaction logs, versioning, time travel, and schema enforcement, which help monitor and troubleshoot data integrity issues. Additionally, when used with Databricks, built-in monitoring, metrics, and lineage tools are available. Claiming limited support is misleading and can be ignored when evaluating Delta Lake.

D — Delta Lake optimizes metadata handling, efficiently managing billions of files and facilitating scalability to petabyte-scale datasets.

This is a valid and important feature. Delta Lake's optimized metadata management, via transaction logs and file indexing, enables scalable operations on massive datasets, reduces query planning time, and supports high concurrency — essential for enterprise-scale data platforms.

Question: 368

Deep Dumps

A data engineer is using Structured Streaming to read in transaction data from a bronze Delta table. It was discovered that the data has quality issues where sometimes the transaction value is negative, and when that occurs, the rows need to be routed to a separate quarantine table. They have low latency requirements for the good data since it is used by downstream systems, but the bad data will only be analyzed periodically and has no production dependencies. The quarantine job needs to be implemented so that it cannot affect the production processes that depend on the good data, and the cost of the job needs to be minimized. How should the quarantine process be implemented in order to satisfy these requirements?

A. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing, and should not share compute with other processes. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing, and should be implemented on a separate small cluster and only run once a day to minimize cost.

B. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. A new boolean column called "quarantine" should be added to the dataframe, and its value should be set to true if the transaction value is less than 0 and false if the transaction value is greater than or equal to 0. Processing and storing all the data together will save costs.

C. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. Inside a foreachBatch function, the dataframe should be filtered so that records with a transaction value greater than or equal to 0 are written to the good data table and records with a transaction value less than 0 are written to a quarantine table. Try/Catch can be added around the writes in the foreachBatch function so

that the stream can't fail.

D. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing. Both should run as separate streams on the same cluster to minimize cost.

Answer: A

Explanation:

The requirements specify that:

Good data must have low latency for downstream systems.

Bad/quarantined data does not have production dependencies and can be processed periodically.

Quarantine processing must not affect the production process, and cost should be minimized.

Option A satisfies all these requirements. The streaming job for the good data remains low-latency, uninterrupted by quarantine processing, because it runs independently and filters out invalid records. The quarantine job runs periodically on a separate small cluster, reducing costs while isolating bad data processing from production workloads.

Why the other options are incorrect

B — Add a boolean "quarantine" column and store all data together

Storing bad and good data in the same table could affect production queries and increase latency for downstream consumers. This violates the low-latency requirement for good data.

C — Use foreachBatch in the same streaming job

Although this allows writing good and bad data separately, combining both in the same streaming job still risks affecting production processing if the quarantine write fails or slows down. It does not isolate the streams or reduce cost for the quarantine job.

D — Run both streams on the same cluster

Running both streams on the same cluster introduces potential contention, which could impact the low-latency good data stream. It also does not minimize cost effectively, since the cluster must be sized to handle both workloads.

Question: 369

Deep Dumps

A data engineer is tasked with ensuring that a Delta table in Databricks continuously retains deleted files for 15 days (instead of the default 7 days), in order to permanently comply with the organization's data retention policy. Which code snippet correctly sets this retention period for deleted files?

- A. `spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")`
- B. `from delta.tables import *
deltaTable = DeltaTable.forPath(spark, "/mnt/data/my_table")
deltaTable.deletedFileRetentionDuration = "interval 15 days"`
- C. `spark.sql("VACUUM my_table RETAIN 15 HOURS")`
- D. `spark.conf.set("spark.databricks.delta.deletedFileRetentionDuration", "15 days")`

Answer: A

Explanation:

```
spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")
```

Explanation

To change the retention period for deleted files in a Delta table, you should set the table property ``delta.deletedFileRetentionDuration``. The correct syntax uses an `ALTER TABLE ... SET TBLPROPERTIES` statement with the value expressed as an interval, e.g., `"interval 15 days"`. This ensures that files deleted from the table are retained for 15 days before they can be permanently removed by a `VACUUM` operation.

Why the other options are incorrect

B — Using ``deltaTable.deletedFileRetentionDuration`` in Python

This is not a valid `DeltaTable` API property. Table properties must be set via SQL or using ``alterTable`` methods that modify table properties, not by directly assigning attributes in Python.

C — ``VACUUM my_table RETAIN 15 HOURS``

`VACUUM` only removes files older than the retention period. Setting it to 15 hours does not change the retention period; it merely deletes older files immediately and in this case, for a much shorter duration than required.

D — ``spark.conf.set("spark.databricks.delta.deletedFileRetentionDuration", "15 days")``

This sets the cluster-wide default retention period but does not affect the specific table. Table-specific retention must be configured via table properties.

Question: 370

Deep Dumps

A data engineer wants to enforce the principle of least privilege when configuring ACLs for Databricks jobs in a collaborative workspace. Which approach should the data engineer use?

- A. Use only folder-level permissions and avoid setting permissions on individual jobs
- B. Assign users only the minimum permission level (e.g., `CAN RUN` or `CAN VIEW`) required for their role on each job.
- C. Grant all users `CAN MANAGE` permission on all jobs to avoid access issues.
- D. Grant `CAN RUN` permission to everyone and `CAN MANAGE` to a single admin group.

Answer: B

Explanation:

B. Assign users only the minimum permission level (e.g., `CAN RUN` or `CAN VIEW`) required for their role on each job.

The principle of least privilege dictates that users should have only the permissions necessary to perform their job functions—no more. In Databricks jobs, this means assigning the minimum required permissions:

`CAN VIEW` for users who only need to see job definitions.

`CAN RUN` for users who need to execute jobs but should not modify them.

`CAN MANAGE` only to users who need full control, such as administrators or job owners.

This approach reduces the risk of accidental or malicious changes while maintaining appropriate access for collaboration.

Why the other options are incorrect

A — Use only folder-level permissions

Folder-level permissions are useful for organizing jobs, but they do not enforce job-specific access controls. Users may still gain unintended access to jobs within the folder if folder permissions are too permissive.

C — Grant all users CAN MANAGE permission on all jobs

This violates least privilege, giving unnecessary full control to all users and increasing the risk of accidental or malicious changes.

D — Grant CAN RUN permission to everyone and CAN MANAGE to a single admin group

While limiting CAN MANAGE is correct, granting CAN RUN to everyone may provide access to users who do not require it, violating least privilege. Only users who need to execute the jobs should have CAN RUN permission.

Question: 371

Deep Dumps

A data engineer is building a streaming data pipeline to ingest JSON files from cloud storage into a Delta Lake table. The pipeline must process files incrementally, handle schema evolution automatically, ensure exactly-once processing, and minimize manual infrastructure management. How should the data engineer fulfill these requirements?

- A. Use Lakeflow Spark Declarative Pipelines with Auto Loader and enabling schema inference with "cloudFiles.schemaEvolutionMode" = "addNewColumns"
- B. Use Auto Loader in batch mode with a daily job to overwrite the Delta table.
- C. Use Lakeflow Spark Declarative Pipelines with a static DataFrame read, merge schema with spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
- D. Use traditional Spark Structured Streaming with Auto Loader, manually configuring checkpoints location and enabling schema inference with "mergeSchema" = "true"

Answer: A

Explanation:

To meet all the requirements: incremental processing, schema evolution, exactly-once guarantees, and minimal infrastructure management, the best approach is to use Delta Live Tables (Lakeflow Spark Declarative Pipelines) with Auto Loader.

Incremental processing: Auto Loader automatically detects new files in cloud storage and ingests them incrementally.

Schema evolution: Setting "cloudFiles.schemaEvolutionMode" = "addNewColumns" allows Delta to automatically handle new columns in incoming JSON files.

Exactly-once processing: Delta Live Tables ensures transactional writes to Delta Lake, providing exactly-once guarantees.

Minimal infrastructure management: Declarative pipelines (LDP) manage clusters, checkpoints, and orchestrations automatically, reducing operational overhead.

This combination satisfies all requirements without manual management of checkpoints or batch overwrites.

Why the other options are incorrect

B — Use Auto Loader in batch mode with daily overwrite

Batch overwrites do not provide incremental processing or exactly-once guarantees. Overwriting the table can also lead to data loss if a job fails mid-run.

C — Use LDP with a static DataFrame read and merge schema via `spark.databricks.delta.schema.autoMerge.enabled`

A static DataFrame read does not provide streaming or incremental ingestion. Schema auto-merge alone cannot replace incremental processing.

D — Traditional Spark Structured Streaming with Auto Loader and manual checkpoints

While this can work, it requires manual checkpoint management and orchestration, increasing operational overhead. Using Delta Live Tables automates these tasks, making it simpler and more maintainable.

Question: 372

Deep Dumps

A data engineer is optimizing a managed table that suffers from data skew and frequently changing query filter columns. The engineer needs to avoid costly data rewrites when query patterns evolve. The table size is under 1TB. How should the data engineer meet this requirement?

- A. Combine partitioning and Z-ordering to maximize flexibility and minimize maintenance as query patterns change.
- B. Enable liquid clustering, as it efficiently handles data skew, allows clustering keys to be changed without rewriting existing data, and adapts to evolving query patterns.
- C. Apply Z-ordering, since it allows flexible reorganization of data layout without rewriting existing files and adapts easily to new filter columns.
- D. Use Hive-style partitioning, as it provides efficient data skipping and is easy to change partition columns at any time.

Answer: B

Explanation:

B. Enable liquid clustering, as it efficiently handles data skew, allows clustering keys to be changed without rewriting existing data, and adapts to evolving query patterns.

For a managed table under 1TB with data skew and frequently changing query filters, liquid clustering efficiently reorganizes data files without full table rewrites. It allows clustering keys to change dynamically, adapting to evolving query patterns while minimizing operational overhead and maintaining query performance.

Why the other options are incorrect

A — Combine partitioning and Z-ordering

While this can help for predictable query patterns, it is less flexible when query filter columns change frequently, and Z-ordering requires rewrites to reorganize data.

C — Apply Z-ordering

Z-ordering improves data skipping but requires rewriting files to reorganize, making it inefficient when query patterns change frequently.

D — Use Hive-style partitioning

Changing partition columns in Hive-style partitioning requires rewriting the table, and over-partitioning small tables (<1TB) can lead to too many small files and degraded performance.

Question: 373**Deep Dumps**

A data engineer is using the AUTO CDC API in Lakeflow Spark Declarative Pipelines to propagate deletions from a source table (orders_source) to a target table (orders_target). The source has Change Data Feed (CDF) enabled, but some delete events arrive out of order due to upstream delays. How does the AUTO CDC API internally ensure deletions are applied correctly despite out-of-order events?

- A. It runs VACUUM on the target table to purge conflicting records.
- B. It uses sequence_by to order events and retains tombstones for deleted rows until older sequences are processed.
- C. It manually sorts incoming events by timestamp before applying changes.
- D. It ignores deletions if they arrive after updates for the same key.

Answer: B**Explanation:**

B. It uses sequence_by to order events and retains tombstones for deleted rows until older sequences are processed.

The AUTO CDC API in Lakeflow Spark Declarative Pipelines ensures that deletions are applied correctly even when events arrive out of order by leveraging the sequence_by mechanism. This mechanism orders the change events for each primary key and retains tombstone markers for deleted rows. By doing so, the system can correctly apply deletions only after all earlier updates or changes for the same key have been processed, ensuring consistency and preventing data loss. This approach avoids relying on manual sorting or ignoring late-arriving deletions, providing reliable handling of CDC streams with out-of-order events.

Why the other options are incorrect

A — Runs VACUUM on the target table

VACUUM is used to remove old files and tombstones from Delta tables; it does not handle out-of-order CDC events.

C — Manually sorts incoming events by timestamp

AUTO CDC handles sequencing internally; manual sorting is unnecessary and not part of the API's mechanism.

D — Ignores deletions if they arrive late

Ignoring deletions would violate data consistency guarantees. AUTO CDC ensures all deletions are eventually applied correctly, regardless of arrival order.

Question: 374**Deep Dumps**

A platform team lead is responsible for automating the individual teams attribution towards SQL Warehouse usage. The requirement is to identify the SQL warehouse usage at the individual user's level and generate a daily report to be shared with an executive team that includes leaders from all business units. How should the platform lead generate an automated report that can be shared daily?

- A. Use the system tables to capture the audit and billing usage data and share the queries with the executive team. This enables the executives to execute the query and see the latest results any time.
- B. Use the system tables to capture the audit and billing usage data and create a dashboard with daily refresh schedules and shared with the executive team.

- C. Restrict users from running any SQL query unless they provide all the query details so that the attribution can be calculated and shared with the executive team.
- D. Let the users run the SQL query and then directly report the usage to the executives. The ownership of the SQL warehouse usage will be with the individual teams.

Answer: B

Explanation:

B. Use the system tables to capture the audit and billing usage data and create a dashboard with daily refresh schedules and shared with the executive team.

To generate an automated daily report on SQL Warehouse usage at the individual user level, the platform lead should leverage the system tables that track audit logs and billing metrics. By querying these tables, the lead can calculate usage by user and business unit. Creating a dashboard with a scheduled daily refresh allows the report to be automatically updated and shared with the executive team without requiring manual intervention. This approach ensures timely, accurate, and automated reporting while centralizing responsibility with the platform team.

Why the other options are incorrect

A — Share queries with executives

While executives could run the queries themselves, this approach is not automated, and requires manual execution, which could lead to delays or inconsistencies.

C — Restrict users from running queries

This is overly restrictive, impractical, and unnecessary for attribution reporting. Usage data can be captured automatically via system tables.

D — Let users report usage directly

This relies on manual reporting, is prone to errors or omissions, and does not provide a reliable, automated, or auditable solution for executive reporting.

Question: 375

Deep Dumps

A data architect is implementing Delta Sharing as part of their data governance strategy to enable secure data collaboration with external partners and internal business units. The architect must establish a permission framework that allows designated data stewards to create shares for their respective domains while maintaining security boundaries and audit compliance. Which specific permissions and roles must be assigned to enable users to create, configure, and manage Delta Shares while maintaining proper security governance and access controls?

- A. Users need to be metastore admins or have CREATE SHARE privilege for the metastore
- B. Only workspace admins can create and manage shares
- C. Users need the MANAGE SHARES permission on the workspace
- D. Any user with USE_CATALOG privilege can create shares

Answer: A

Explanation:

In Databricks Delta Sharing, creating and managing shares is controlled at the Unity Catalog metastore level. Users must either be metastore admins or explicitly granted the CREATE SHARE privilege on the metastore. This allows designated data stewards to define shares for their domains while respecting security boundaries

and ensuring audit compliance. Assigning these privileges provides fine-grained control: stewards can manage shares without granting them full administrative rights, maintaining proper governance over who can expose datasets externally or across business units.

Why the other options are incorrect

B — Only workspace admins can create and manage shares

This is too restrictive. Delta Sharing allows delegated privileges to metastore admins or users with CREATE SHARE, enabling decentralized management by data stewards.

C — Users need MANAGE SHARES permission on the workspace

Delta Sharing permissions are not managed at the workspace level, but at the metastore/catalog level, so workspace permissions alone are insufficient.

D — Any user with USE_CATALOG privilege can create shares

USE_CATALOG allows access to data within a catalog but does not grant the ability to create or manage Delta Shares, so this privilege alone is insufficient for managing shares.

Question: 376

Deep Dumps

A data engineer is optimizing a MERGE operation on an 800GB UC-managed table that experiences frequent updates and deletions. Which two actions should the engineer prioritize to improve MERGE performance?

- A. Overwrite the table instead of Merge
- B. Partition the table by date.
- C. Enable deletion vectors on the table if not already enabled.
- D. Apply liquid clustering using the merge join keys.
- E. Use ZORDER on high-cardinality columns.

Answer: C,D

Explanation:

For an 800GB Delta table with frequent updates and deletions, optimizing MERGE operations is critical.

C — Enable deletion vectors: Deletion vectors allow Delta Lake to track deleted rows without rewriting entire files, significantly improving performance for tables with frequent deletes. This reduces the amount of data rewritten during MERGE operations, especially for large tables.

D — Apply liquid clustering using the merge join keys: Liquid clustering dynamically organizes data files based on the keys used in MERGE operations. This reduces file scans and data shuffling, improving join and update performance while adapting to evolving query and merge patterns.

Why the other options are less effective

A — Overwrite the table instead of Merge

Overwriting the table does not preserve history or incremental updates and is not suitable for production workloads with frequent updates.

B — Partition the table by date

Partitioning helps with query pruning but may not improve MERGE performance if updates and deletes are

spread across many partitions.

E — Use ZORDER on high-cardinality columns

Z-ordering helps optimize queries with selective filters, but it does not improve MERGE performance for frequent updates and deletes as effectively as deletion vectors or liquid clustering.

Question: 377

Deep Dumps

A data engineering team is setting up a Git project to automate integration tests using Databricks Asset Bundles and the Git provider's CI/CD functionalities. When a pull containing changes to their pipeline is sent, they need to run a Job to test their data pipeline. What is the correct databricks bundle command sequence to be executed from the Git provider's CI/CD automation for this task?

- A. init, validate, deploy, run
- B. validate, deploy, run
- C. init, deploy, run, validate
- D. deploy, run, validate

Answer: B

Explanation:

B. validate, deploy, run

When using Databricks Asset Bundles in a CI/CD pipeline, the recommended workflow for automated integration tests does not require init, because the repository is already initialized and cloned by the Git provider.

The correct sequence is:

validate — Checks the integrity and correctness of the bundle configuration and assets.

deploy — Pushes the pipeline assets to Databricks, creating or updating jobs, notebooks, and other resources.

run — Executes the deployed Job to perform the integration test.

This sequence ensures that the pipeline changes are verified, deployed, and executed automatically in the CI/CD workflow without unnecessary initialization steps.

Why the other options are incorrect

A — init, validate, deploy, run

init is only needed once when setting up a new bundle repository; in CI/CD, the repo is already cloned, so init is redundant.

C — init, deploy, run, validate

Validating after deployment defeats the purpose of pre-deployment validation and can lead to deploying broken assets.

D — deploy, run, validate

Skipping validation before deployment risks deploying an invalid or misconfigured pipeline, which can cause failures during the run step.

A company processes semi-structured JSON files from an external source using Auto Loader in a classic Databricks job. Occasionally, records arrive with null critical fields, invalid types, or unexpected nested schema variations. The engineer must ensure that malformed or non-conforming records are not dropped silently and are captured in a separate quarantine table. The pipeline should continue processing good records into the Bronze layer without failing the job, and the approach must support both batch and streaming ingestion. The data engineer needs to build a robust ingestion pattern that automatically routes bad records to a quarantine Delta table, while still ingesting good records into the Bronze layer for further processing. Which approach fulfills the quarantine mechanism in this ingestion architecture?

- A. Use Auto Loader with failFast mode set to false, and enable schema evolution; invalid records will be silently ignored during ingestion.
- B. Use Lakeflow Spark Declarative Pipelines with a SQL pipeline; configure it to drop rows with nulls using where critical_field is not null, and rely on audit logs for malformed data.
- C. Use Auto Loader with LDP and implement an EXPECT() constraint with a record audit logic to route bad records.
- D. Create a notebook job with inferSchema=True, write a streaming query with .foreachBatch() and catch exceptions using try/except to redirect failed batches to quarantine.

Answer: C

Explanation:

C. Use Auto Loader with LDP and implement an EXPECT() constraint with a record audit logic to route bad records.

To handle semi-structured JSON ingestion robustly while separating bad records from good data, the Lakehouse Declarative Pipelines (LDP) approach with Auto Loader is ideal. By defining EXPECT() constraints, the engineer can specify data quality rules, such as non-null critical fields, correct types, or schema conformance. Records that fail these expectations are automatically routed to a quarantine Delta table, while compliant records continue into the Bronze layer. This mechanism works seamlessly in both batch and streaming ingestion, ensures that bad records are not dropped silently, and maintains pipeline reliability without failing the job.

Why the other options are incorrect

A — Auto Loader with failFast=false and schema evolution

This silently ignores malformed records and does not provide a quarantine mechanism, so bad data is not captured for auditing.

B — LDP with SQL pipeline dropping rows

Dropping rows with WHERE critical_field IS NOT NULL discards invalid records without storing them in a quarantine table, and relying on audit logs does not capture the actual malformed records for later analysis.

D — Notebook with foreachBatch() and try/except

While this can capture failed batches, it is manual, error-prone, and less scalable. It also lacks native LDP integration for incremental ingestion, schema evolution, and automated quality enforcement.

A data engineering workspace was automatically enabled for Unity Catalog, creating a workspace catalog. New team members report they can create tables in the default schema but cannot access tables in other schemas within the same workspace catalog. Why are the new team members unable to access tables in other schemas?

- A. Workspace catalog permissions are not subject to inheritance rules.
- B. Workspace users receive USE CATALOG and specific privileges on default schema only.
- C. Tables in other schemas require additional BROWSE privileges that new users don't receive automatically
- D. New users only receive CREATE TABLE privileges on the default schema.

Answer: B

Explanation:

Workspace users receive USE CATALOG and specific privileges on default schema only.

This is correct because when a Databricks workspace is enabled for Unity Catalog, a workspace catalog is automatically created. New users automatically receive the USE CATALOG privilege on the workspace catalog and privileges only on the default schema, allowing them to create tables there. They do not get access to other schemas automatically, which is why they cannot access tables outside the default schema. Access to other schemas requires explicit grants, such as USE SCHEMA or table-level privileges. This setup ensures fine-grained access control within Unity Catalog.

Incorrect Options:

A. Workspace catalog permissions are not subject to inheritance rules.

This is incorrect because workspace catalog permissions do follow inheritance rules. Users inherit privileges only for objects or schemas they have access to, so the inability to access other schemas is not due to inheritance being disabled.

C. Tables in other schemas require additional BROWSE privileges that new users don't receive automatically.

This is incorrect because BROWSE privileges are not required to access tables. Users need explicit schema or table-level privileges, not just BROWSE, to read or write to tables in other schemas.

D. New users only receive CREATE TABLE privileges on the default schema.

This is partially true in that new users can create tables in the default schema, but this is a consequence of their assigned privileges rather than the reason they cannot access other schemas. The real reason is that they lack explicit privileges on those other schemas.

Question: 380

Deep Dumps

A data engineer is running a groupBy aggregation on a massive user activity log grouped by user_id. A few users have millions of records, causing task skew and long runtimes. Which technique will fix the skew in this aggregation?

- A. Use reduceByKey instead of groupBy to avoid shuffles.
- B. Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after removing the prefix.
- C. Increase the Spark driver memory and retry.
- D. Filter out the skewed users before the aggregation.

Answer: B

Explanation:

Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after

removing the prefix.

This is correct because the issue is data skew, where a few user IDs have millions of records while most have far fewer. This causes some tasks to process much more data than others, leading to long runtimes. Salting is a common technique to fix skew: you add a random prefix (or suffix) to the skewed keys so that their records are spread across multiple partitions. After performing the aggregation on the salted keys, you remove the prefix and aggregate again to get the correct final results. This balances the workload and reduces task skew.

Incorrect Options:

A. Use `reduceByKey` instead of `groupByKey` to avoid shuffles.

This is incorrect because while `reduceByKey` can reduce the amount of data shuffled compared to `groupByKey`, it does not solve skew. Skewed keys will still concentrate most of the data in a single partition, so the same long-running tasks will occur.

C. Increase the Spark driver memory and retry.

This is incorrect because the problem is task skew, not insufficient driver memory. Increasing driver memory does not address the uneven distribution of data across tasks.

D. Filter out the skewed users before the aggregation.

This is incorrect because filtering out skewed users changes the results of the aggregation. We want to include all users, so filtering is not a viable solution.

Question: 381

Deep Dumps

What describes a primary technical challenge in ensuring consistent PII masking across all nodes in large-scale, distributed Databricks batch and streaming pipelines?

- A. Dynamic data masking is applied only at rest, so it does not affect query performance.
- B. PII masking is only required for direct identifiers.
- C. Native masking in Databricks automatically synchronizes with all downstream external Databricks systems.
- D. Masking functions must be standardized and managed through Unity Catalog, with enforcement applied across all relevant datasets to avoid any data inconsistency.

Answer: D

Explanation:

Correct answer: D -> Masking functions must be standardized and managed through Unity Catalog, with enforcement applied across all relevant datasets to avoid any data inconsistency.

This is correct because in distributed Databricks environments — especially large-scale batch and streaming pipelines — PII consistency is a major challenge. Multiple clusters, multiple nodes, and multiple jobs may apply masking independently. Without a centralized, governed masking policy, the same PII value (e.g., an email or phone number) could be masked differently across nodes or jobs. Unity Catalog solves this by providing centralized data governance, where masking functions and policies are defined once and enforced consistently across all datasets, tables, schemas, and compute clusters. Standardizing masking logic through Unity Catalog ensures that the same data is always masked the same way, regardless of where or how it is processed.

Incorrect Options:

A. Dynamic data masking is applied only at rest, so it does not affect query performance.

This is incorrect because dynamic masking is not strictly an "at rest" operation. Masking policies affect query results at read time, not only when stored on disk. The challenge here is consistency across distributed systems, not query performance.

B. PII masking is only required for direct identifiers.

This is incorrect because PII also includes quasi-identifiers and indirect identifiers that could be used for re-identification. Effective masking must be applied consistently to all sensitive fields, not just direct identifiers like names or SSNs.

C. Native masking in Databricks automatically synchronizes with all downstream external Databricks systems.

This is incorrect because masking policies do not automatically propagate to external systems unless those systems are governed by Unity Catalog. External tools, unmanaged tables, or other data sinks may not automatically receive consistent masking rules.

Question: 382

Deep Dumps

A Data Engineer is building a fraud detection pipeline that calls out to OpenAI, via a Python library, and needs to include an access token when using the API. Which Databricks CLI command should the Data Engineer use to create the secret?

- A. databricks tokens put-token KEY SCOPE; dbutils.tokens.get(KEY, SCOPE)
- B. databricks secrets put-secret SCOPE KEY: dbutils.secrets.get(SCOPE, KEY)
- C. databricks tokens put-token SCOPE KEY: dbutils.tokens.get(SCOPE, KEY)
- D. databricks secrets put-secret KEY SCOPE: dbutils.secrets.get(KEY, SCOPE)

Answer: B

Explanation:

Correct Answer: B. databricks secrets put-secret SCOPE KEY : dbutils.secrets.get(SCOPE, KEY)

This is correct because Databricks uses the Secrets API to securely store credentials such as API tokens. The proper Databricks CLI command to create a secret is `databricks secrets put-secret`, which stores a key-value secret inside a specified secret scope. In your code, you retrieve it using `dbutils.secrets.get(SCOPE, KEY)` so that the access token is never exposed in plain text. This is the standard and secure method for storing sensitive credentials like an OpenAI API token.

Incorrect Options:

A. Incorrect because `databricks tokens put-token` is used to create Databricks personal access tokens, not store arbitrary secrets for use within workflows. It also uses the wrong retrieval method (`dbutils.tokens.get` does not exist).

C. Incorrect because the command syntax is wrong, and again uses tokens instead of secrets, which is not used for storing external API keys.

D. Incorrect because the argument order is reversed; `put-secret` takes SCOPE first, then KEY. The retrieval syntax is also incorrect.

Question: 383

Deep Dumps

A data team is implementing an append-only Delta Lake pipeline that needs to handle both batch and streaming data. They want to ensure that schema changes in the source data can be automatically incorporated without breaking the pipeline. Which configuration should the team use when writing data to the Delta table?

- A. `mergeSchema = true`
- B. `ignoreChanges = false`
- C. `validateSchema = false`
- D. `overwriteSchema = true`

Answer: A

Explanation:

Correct Answer: A. `mergeSchema = true`

For an append-only Delta Lake pipeline that must handle schema evolution safely — especially when combining batch and streaming data — the correct configuration is `mergeSchema = true`. This option allows Delta Lake to automatically evolve the table schema whenever new columns appear in the incoming data. It ensures the pipeline continues running without failures due to schema drift, which is common in streaming or semi-structured sources.

Why the other options are incorrect:

B. `ignoreChanges = false`

This flag only affects merge operations and does not enable schema evolution. It does nothing to allow new columns when writing append-only data.

C. `validateSchema = false`

This bypasses schema validation but does not evolve the schema. It can allow incompatible writes and is not intended for controlled schema evolution.

D. `overwriteSchema = true`

This replaces the entire schema during an overwrite operation. It is not suitable for append-only pipelines and could cause data loss or inconsistent structure.

Question: 384

Deep Dumps

A data engineer is implementing liquid clustering on a Delta Lake table and needs to understand how it affects data management operations. The table will be updated frequently with new data. The table is an external table and not managed by Unity Catalog. How does liquid clustering in Delta Lake handle new data that is inserted after the initial table creation?

- A. New data remains unclustered until the next OPTIMIZE operation.
- B. New data is rejected if it doesn't match the clustering pattern.
- C. New data is automatically clustered during write operations.
- D. New data is written to a staging area and clustered during scheduled maintenance.

Answer: C

Explanation:

Correct Answer: C. New data is automatically clustered during write operations.

Liquid clustering in Delta Lake is designed to continuously maintain clustering as new data arrives, without requiring manual OPTIMIZE commands. Unlike traditional Z-Ordering or partitioning, liquid clustering automatically writes new data into the correct clustering buckets as part of the write path, ensuring that the table stays well-organized even with frequent inserts and updates. This applies whether the table is managed or external — the clustering behavior is handled by the Delta engine, not Unity Catalog. Because liquid clustering is incremental and write-aware, new data never remains unclustered or requires scheduled maintenance to be reorganized.

Why the other options are incorrect:

A. New data remains unclustered until the next OPTIMIZE operation.

Incorrect — this describes Z-Ordering, not liquid clustering. Liquid clustering replaces the need for OPTIMIZE-based Z-Order.

B. New data is rejected if it doesn't match the clustering pattern.

Incorrect — Delta Lake never rejects data based on clustering strategy. Clustering patterns simply determine data layout.

D. New data is written to a staging area and clustered during scheduled maintenance.

Incorrect — this implies batch maintenance behavior, but liquid clustering updates cluster layout during writes, not during maintenance windows.

Question: 385

Deep Dumps

A data engineering team is collaborating on a Databricks project where each team member needs to develop and test code independently before merging changes into the main branch. They want to avoid accidental overwrites or branch switching issues while ensuring that all work is version-controlled and can be integrated into their CI/CD pipeline. How should the data engineer achieve collaboration?

- A. Each team member creates their own Databricks Git folder, mapped to the same remote Git repository, and works in their own development branch within their personal folder.
- B. All team members work in the same Databricks Git folder and perform Git operations (pull, push, commit, branch switching) directly in that shared folder.
- C. Team members edit notebooks directly in the workspace's shared folder and periodically copy changes into a Git folder for version control.
- D. Team members use the Databricks CLI to clone the Git repository and perform Git operations from a cluster's web terminal.

Answer: A

Explanation:

Correct Answer: A. Each team member creates their own Databricks Git folder, mapped to the same remote Git repository, and works in their own development branch within their personal folder.

This is the recommended and safest collaboration pattern in Databricks. Each engineer creates a personal Git-backed folder connected to the same remote repository. They then work in their own feature or development branch, preventing accidental overwrites, avoiding conflicts caused by shared folder edits, and ensuring that all code changes are tracked in Git. This setup integrates cleanly with CI/CD workflows since each developer commits from their own isolated environment and merges via pull requests.

Why the other options are incorrect:

B. All team members work in the same Git folder.

Incorrect — this causes frequent conflicts, race conditions, accidental branch switching, and unpredictable overwrites. Databricks explicitly recommends against shared Git folders for development.

C. Editing notebooks in shared workspace folders and later copying into Git.

Incorrect — this breaks version control best practices, increases the risk of lost work, and does not allow clean CI/CD integration. It also eliminates Git's ability to track changes effectively.

D. Using the Databricks CLI or cluster terminal to clone repositories.

Incorrect — cluster terminals are ephemeral, not suited for long-term development, and cannot maintain stable Git workspace integration. This method also bypasses Databricks' built-in notebook versioning UI.

Question: 386**Deep Dumps**

A company stores account transactions in a Delta Lake table. The company needs to apply frequent account-level corrections (e.g., UPDATE statements) but wants to avoid rewriting entire Parquet files for each change to reduce file churn and improve write performance. Which Delta Lake feature should they enable?

- A. Partition the Delta table by account_id
- B. Enable automatic file compaction on writes
- C. Enable deletion vectors on the Delta table
- D. Enable change data feed on the Delta table

Answer: C**Explanation:****Correct Answer: C. Enable deletion vectors on the Delta table**

Deletion vectors allow Delta Lake to record row-level changes (deletes and updates) without rewriting the underlying Parquet files. Instead of physically removing or modifying rows in files, Delta stores the changes as lightweight metadata. This dramatically reduces file churn, improves write performance, and is ideal for workloads with frequent UPDATE, DELETE, or MERGE operations — such as applying account-level corrections.

Because the company wants to avoid rewriting Parquet files each time corrections occur, deletion vectors are exactly the feature designed for this scenario.

Why the other options are incorrect:**A. Partition the table by account_id**

This may reduce scan costs but does not prevent rewriting Parquet files when UPDATES occur. Delta still rewrites files inside the affected partition.

B. Enable automatic file compaction on writes

Compaction helps reduce small files — not rewrite avoidance. It actually rewrites files, which is the opposite of what the company wants.

D. Enable change data feed (CDF)

CDF tracks changes for downstream consumers but does not reduce file rewriting or improve UPDATE performance.

Question: 387

Deep Dumps

A security team wants to enforce data protection for a customer table containing customer PII data. To comply with local policies, sales team members should only see customers from their region, while non-admin users should have email addresses masked. Which implementation approach should be used when using Unity Catalog row filters and column masks?

- A. Use table ACLs to restrict access using tags with GRANT SELECT ON table_name WITH TAG command, and rely on application-level filtering for sensitive data based on user region.
- B. Create a view with dynamic WHERE clauses for region filtering and use string replacement functions for email masking using ALTER COLUMN SET MASK command
- C. Implement row filters with SQL UDFs based on user region only since column masks cannot be combined with row filters on the same table, then apply them by recreating the table with DROP TABLE and CREATE TABLE SET ROW FILTER commands.
- D. Create SQL UDFs for row filtering based on user region and column masking based on group membership, then apply them using ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK commands.

Answer: D

Explanation:

Correct Answer: D. Create SQL UDFs for row filtering based on user region and column masking based on group membership, then apply them using ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK commands.

Unity Catalog fully supports combining row filters and column masks on the same table, and this is the recommended approach for enforcing fine-grained data protection.

To meet the requirements:

Row filters can enforce region-based access by using a SQL UDF that checks the user's group or attribute and only returns rows for the region they are allowed to see.

Column masks can mask sensitive PII such as email addresses for all non-admin users. This is done via a SQL UDF that returns the full email for admins and a masked version for others.

Both can then be applied directly to the table using:

```
ALTER TABLE ... SET ROW FILTER
```

```
ALTER TABLE ... ALTER COLUMN ... SET MASK
```

This approach provides centralized, consistent policy enforcement across all compute environments using Unity Catalog.

Why the other options are incorrect:

A. Incorrect option — Table ACLs and tags do not implement dynamic row-level filtering or column masking. Application-level filtering is not secure and bypasses Unity Catalog governance.

B. Incorrect option — Creating views with manual WHERE clauses and string functions is not the recommended security model. Column masks should use UC masking policies, not ad-hoc string replacements.

C. Incorrect option — Unity Catalog does allow combining row filters and column masks. Recreating the table via DROP/CREATE is unnecessary and incorrect.

Question: 388**Deep Dumps**

A data engineer manages a production Lakeflow Spark Declarative Pipeline that processes customer transaction data. The pipeline includes several data quality expectations, such as: `transaction_amount > 0` and `customer_id IS NOT NULL`. These expectations are defined using the `EXPECT` clause in SQL. The engineer aims to monitor the pipeline's data quality by analyzing the number of records that passed or failed each expectation during the latest pipeline update. The Lakeflow Spark Declarative Pipelines event logs are stored in a Delta table named `event_log_table`. For the most recent pipeline update, determine a programmatically appropriate approach to extract information like the name of each expectation, associated dataset, count of records that passed the expectation, and count of records that failed the expectation. Which method retrieves the desired data quality metrics from the Lakeflow Spark Declarative Pipelines event log?

- A. Access the `event_log_table`, filter for events where `event_type = 'flow_progress'`, and parse the `details.flow_progress.data_quality.expectations` field to extract the required metrics.
- B. Use the Lakeflow Spark Declarative Pipelines UI to navigate to the specific pipeline, select the dataset, and view the Data Quality tab to manually retrieve the expectation metrics.
- C. Query the `event_log_table` for events with `event_type = 'data_quality'` and directly select the `passed_records` and `failed_records` fields.
- D. Access the `event_log_table`, filter for events where `event_type = 'expectation_result'`, and extract the expectation metrics from the `details` field.

Answer: A**Explanation:**

Access the `event_log_table`, filter for events where `event_type = 'flow_progress'`, and parse the `details.flow_progress.data_quality.expectations` field to extract the required metrics.

The correct method is to read from the `event_log_table`, filter for events where `event_type = 'flow_progress'`, and then parse the nested field `details.flow_progress.data_quality.expectations`, because Lakeflow Spark Declarative Pipelines store all data-quality results inside the flow progress event payload. This field contains the expectation name, associated dataset, and the counts of records that passed and failed each expectation during the most recent pipeline update. The other options are incorrect because the Lakeflow UI only supports manual inspection rather than programmatic access, and there are no event types named `data_quality` or `expectation_result` in these pipelines — data-quality information is always embedded within the `flow_progress` event structure.

Why the other options are incorrect:

B. Incorrect — The UI Data Quality tab is useful for manual inspection, not for programmatic extraction. The question requires a programmatic method.

C. Incorrect — Declarative Pipelines do not emit events with `event_type = 'data_quality'`. Data-quality metrics are only present inside `flow_progress` events.

D. Incorrect — There is no `expectation_result` event type in Lakeflow Declarative Pipelines. All expectation results appear inside the `flow_progress` event payload.

Question: 389**Deep Dumps**

Given the following PySpark code snippet in a Databricks notebook: `filtered_df = spark.read.format("delta").load("/mnt/data/large_table") \.filter("event_date > '2024-01-01'")filtered_df.count()` The data engineer notices from the Query Profile that the scan operator for `filtered_df` is reading almost all files, despite a filter being applied. What is the probable reason for poor data skipping?

- A. The filter is executed only after the full data scan, which prevents data skipping from taking place.
- B. The event_date column is outside the table's partitioning and Z-ordering scheme.
- C. The Delta table lacks optimization that enables dynamic file pruning.
- D. The filter condition involves a data type that is excluded from data skipping support.

Answer: B

Explanation:

Correct Answer: B. The event_date column is outside the table's partitioning and Z-ordering scheme.

This is correct because Delta Lake can only skip files efficiently when the filter column is part of the table's partitioning or Z-Ordering strategy. If event_date is not included in either, Delta has limited metadata (min/max stats per file) to prune out irrelevant files. As a result, even though the filter event_date > '2024-01-01' is applied, the engine still needs to read almost all files because it cannot determine which files might contain qualifying data. This explains why the scan operator reads nearly the entire dataset despite the filter.

Incorrect Options:

A. The filter is executed only after the full data scan — This is incorrect because Delta Lake applies filter predicates before scanning files when file-level statistics allow it. The issue is not the timing of the filter but the lack of metadata on event_date to support pruning.

C. The Delta table lacks optimization that enables dynamic file pruning — This is incorrect because Delta Lake automatically supports dynamic file pruning whenever relevant statistics or partition metadata are available. No special optimization needs to be enabled for pruning to work.

D. The filter condition involves a data type excluded from data skipping support — This is incorrect because event_date (as a date or timestamp) is fully supported by Delta Lake's data skipping. Delta stores min/max stats for date-like columns, so the data type is not the cause of poor skipping.

Question: 390

Deep Dumps

A data engineering team is implementing an append-only data pipeline using Delta Lake, and wants to ensure that data is never modified or deleted once written. Which Delta Lake feature should the data engineer enable to prevent modifications to existing data?

- A. Delta Time Travel
- B. Delta VACUUM
- C. Delta OPTIMIZE
- D. Delta APPEND_ONLY

Answer: D

Explanation:

Correct Answer: D. Delta APPEND_ONLY

The Delta Lake APPEND_ONLY feature enforces a strict policy where existing data in the table cannot be updated or deleted. When delta.appendOnly = true is enabled on a Delta table, any attempt to run UPDATE, DELETE, or MERGE operations will fail, ensuring that all incoming data is added as new rows only. This feature is specifically designed to support append-only pipelines where immutability is required for compliance, auditing, or regulatory reasons.

Incorrect Options:

A. Delta Time Travel — This is incorrect because Time Travel allows querying older versions of data but does not prevent modifications. You can still run UPDATE or DELETE operations on a table that supports Time Travel.

B. Delta VACUUM — This is incorrect because VACUUM removes old files that are no longer needed, helping reclaim storage. It does not control or prevent modifications to active data.

C. Delta OPTIMIZE — This is incorrect because OPTIMIZE improves file layout and compaction for performance, but has no effect on enforcing append-only behavior or blocking data changes.

Question: 391

Deep Dumps

When a new Databricks project starts, the central IT team provisions the required infrastructure using Terraform and a Service Principal. This includes creating a Databricks workspace, a Unity Catalog linked to an External Location, and a Databricks group containing all project team members. Project teams must store all assets — e.g., tables and volumes, as Managed assets in Unity Catalog. This model hides infrastructure complexity while giving teams autonomy within their catalog. They can create and manage schemas, tables, volumes, and related objects but cannot rename, delete, or change catalog permissions; those remain under IT's control. Which rights should the project group be granted to enable this model?

- A. The group needs to have USE CATALOG and USE SCHEMA on the catalog.
- B. The group should be made OWNER of the catalog.
- C. The group needs to have ALL PRIVILEGES and the MANAGE on the catalog.
- D. The group needs to have ALL PRIVILEGES on the catalog.

Answer: D

Explanation:

Correct Answer: D. The group needs to have ALL PRIVILEGES on the catalog.

Granting ALL PRIVILEGES on the catalog gives the project team exactly the right level of autonomy. They can create and manage schemas, tables, and volumes as managed assets inside the catalog, but they cannot rename or delete the catalog nor change catalog-level permissions. This aligns perfectly with the IT team's governance model, where IT controls the catalog and its permissions while project teams have full freedom to work inside it.

Option A: USE CATALOG and USE SCHEMA

This option is incorrect because these permissions only allow basic querying and access to existing objects. They do not allow users to create schemas, tables, or volumes. Since the project team must be able to create and manage their own managed assets, these limited rights are not sufficient.

Option B: The group should be made OWNER of the catalog

This option is incorrect because making the group the OWNER would give them full administrative control, including the ability to rename or delete the catalog, modify permissions, and alter storage settings. This violates the requirement that IT retains catalog-level control and governance.

Option C: ALL PRIVILEGES and MANAGE on the catalog

This option is incorrect because adding MANAGE GRANTS would allow the project team to change permissions on the catalog. That would undermine IT's control and allow teams to grant access to others, which breaks the required governance model where only IT should manage catalog permissions.

Question: 392**Deep Dumps**

A data engineer is setting up a pipeline to ingest data from a message bus system that occasionally delivers duplicate messages. The duplicate messages can be a week apart. The target is a Databricks Delta Lake table where each record should appear exactly once. Which Databricks ingestion pattern should be implemented to handle potential duplicates where events can arrive outside of the configured watermark?

- A. Implement a write operation using MERGE INTO with a unique key
- B. Use Delta Lake's change data feed to filter duplicate records
- C. Configure Structured Streaming with dropDuplicates transformation
- D. Use Delta Lake time travel to identify and remove duplicates

Answer: A

Explanation:

Correct Answer: A. Implement a write operation using MERGE INTO with a unique key

Using a Delta Lake MERGE INTO statement with the event's unique key is the correct approach because it allows the pipeline to handle late-arriving duplicate events — even those arriving after the streaming watermark window. MERGE provides idempotent writes, meaning the same record can arrive again at any time (even a week later), and the target Delta table will not insert a duplicate but instead match on the key and skip or update appropriately. This makes MERGE the only pattern that guarantees exactly-once semantics in the presence of very late duplicates.

Option B: Use Delta Lake's change data feed to filter duplicate records

This option is incorrect because Change Data Feed (CDF) is not a deduplication mechanism. CDF simply tracks changes after they occur, but it does not prevent duplicates from being written to the table in the first place. It also cannot detect duplicates that arrive late unless additional logic is added, making it unsuitable for enforcing exactly-once ingestion.

Option C: Configure Structured Streaming with dropDuplicates transformation

This is incorrect because dropDuplicates relies on the streaming engine's state store, which is limited by the watermark. Since duplicates can arrive a week late, they will fall outside the watermark window and bypass the state store entirely, meaning they will still be written to the table. Therefore, dropDuplicates cannot guarantee elimination of late duplicates.

Option D: Use Delta Lake time travel to identify and remove duplicates

This option is incorrect because time travel is meant for historical querying and auditability, not operational deduplication. It cannot automatically detect late duplicates, and using it for cleanup would require manual or scheduled batch processes, which is inefficient and not aligned with streaming ingestion.

Question: 393**Deep Dumps**

A security analytics pipeline must enrich billions of raw connection logs with geolocation data. The join hinges on finding which IPv4 range each event's address falls into

event_id	ip_int
42	3232235777

Table 1 - network_events (approx \$5 billion rows)

start_ip_int	end_ip_int	country
3232235520	3232236031	US

Table 2 - ip_ranges (approx \$2 million rows)

This query is running very slowly and needs to be improved:

```
SELECT  n.event_id,
        n.ip_int,
        r.country
FROM    network_events n
JOIN    ip_ranges      r
ON      n.ip_int BETWEEN r.start_ip_int AND r.end_ip_int;
```

Execution is far too slow. Which change will most dramatically accelerate the query while preserving its logic?

- A. Force a sort merge join with `/*+ MERGE(r) */` on ip_ranges.
- B. Add a range-join hint `/*+ RANGE_JOIN(r, 65536) */`.
- C. Increase spark sql shuffle partitions from 200 to 10000.
- D. Add a broadcast hint: `/*+ BROADCAST(r) */` for ip_ranges.

Answer: B

Explanation:

B. Add a range-join hint with a /16 bin size (65536): `/*+ RANGE_JOIN(r, 65536) */` — Correct

This is the most impactful optimization for large-scale range joins. IPv4 lookups are classic range-join problems, where each event IP must be matched against a start–end range. Without optimization, Spark evaluates many range comparisons, causing massive shuffle and compute overhead.

The RANGE_JOIN hint enables Spark to bucket both sides of the join into fixed-size numeric ranges (in this case, /16 blocks of 65,536 IPs). This drastically reduces the number of comparisons by ensuring each event only checks relevant IP-range buckets instead of the entire ip_ranges table. For billions of rows, this optimization can reduce execution time from hours to minutes and is purpose-built for IP-to-range joins.

A. Force a Sort Merge Join using `/*+ MERGE(r) */` — Incorrect

Sort Merge Joins require both sides to be fully shuffled and sorted. For a 5-billion-row fact table, this introduces enormous overhead. While merge joins are useful for equality joins on large datasets, they are inefficient for range-based conditions and do not address the core performance bottleneck.

C. Increase spark.sql.shuffle.partitions from 200 to 10000 — Incorrect

Increasing shuffle partitions may slightly improve parallelism, but it does not fundamentally reduce the computational complexity of the range join. The query will still evaluate too many range comparisons, making this a marginal tuning change rather than a transformational optimization.

D. Add a broadcast hint for ip_ranges — Incorrect

Broadcast joins work well when the broadcasted table is small. However, 2 million rows of IP ranges is typically too large to safely broadcast, especially when used in a range join. Broadcasting such a dataset can cause memory pressure, executor failures, or degraded performance, and does not scale reliably for this workload.

Key Takeaway:

For massive IP range enrichment workloads, the most effective optimization is a RANGE_JOIN hint, which dramatically reduces join complexity by bucketing IPs into manageable numeric ranges. This makes Option B the correct answer.

Question: 394

Deep Dumps

How are the operational aspects of Lakeflow Spark Declarative Pipelines from Spark Structured Streaming different?

- A. Lakeflow Spark Declarative Pipelines manage the orchestration of multi-stage pipelines automatically, while Structured Streaming requires external orchestration for complex dependencies.
- B. Lakeflow Spark Declarative Pipelines can write to Delta Lake format, while structured streaming cannot.
- C. Lakeflow Spark Declarative Pipelines automatically handle schema evolution, while Structured Streaming always requires manual schema management.
- D. Structured Streaming can process continuous data streams, while Lakeflow Spark Declarative Pipelines cannot.

Answer: A

Explanation:

A. Lakeflow Spark Declarative Pipelines manage the orchestration of multi-stage pipelines automatically, while Structured Streaming requires external orchestration for complex dependencies — Correct

Lakeflow Spark Declarative Pipelines (formerly Delta Live Tables) are designed to simplify operational management of data pipelines. With a declarative approach, you define what the pipeline should produce, and Lakeflow automatically manages task orchestration, execution order, retries, dependencies, monitoring, and recovery across multiple stages. This makes it much easier to operate complex, multi-step pipelines.

In contrast, Spark Structured Streaming focuses on stream processing logic only. While it can handle streaming transformations efficiently, it does not manage orchestration across multiple pipeline stages. For complex workflows involving multiple streaming jobs or dependencies, Structured Streaming typically requires external orchestration tools such as Databricks Workflows, Apache Airflow, or other schedulers.

B. Lakeflow Spark Declarative Pipelines can write to Delta Lake format, while Structured Streaming cannot — Incorrect

Both Lakeflow Spark Declarative Pipelines and Spark Structured Streaming fully support writing to Delta Lake. In fact, Structured Streaming is commonly used to write streaming data directly into Delta tables. Therefore, this is not a distinguishing operational difference.

C. Lakeflow Spark Declarative Pipelines automatically handle schema evolution, while Structured Streaming always requires manual schema management — Incorrect

While Lakeflow pipelines provide built-in support and easier configuration for schema enforcement and evolution, Spark Structured Streaming can also handle schema evolution when configured appropriately (for example, when using Delta Lake features). Structured Streaming does not always require manual schema

management, so this statement is too absolute.

D. Structured Streaming can process continuous data streams, while Lakeflow Spark Declarative Pipelines cannot — Incorrect

Lakeflow Spark Declarative Pipelines are built on top of Spark Structured Streaming and fully support continuous and incremental data processing. Both technologies can process streaming data, so this option is incorrect.

Key Takeaway:

The key operational difference is that Lakeflow Spark Declarative Pipelines automatically manage orchestration and dependencies, while Spark Structured Streaming focuses on stream processing and relies on external tools for complex pipeline orchestration.

Question: 395

Deep Dumps

A data engineer is designing a data pipeline in Databricks that needs to process records from a Kafka stream where late-arriving data is common. Which approach should the data engineer use?

- A. Implement a custom solution using Databricks Jobs to periodically reprocess all historical data.
- B. Use a watermark to specify the allowed lateness to accommodate records that arrive after their expected window, ensuring correct aggregation and state management.
- C. Use batch processing and overwrite the entire output table each time to ensure late data is incorporated correctly.
- D. Use an Auto CDC pipeline with batch tables to simplify late data handling.

Answer: B

Explanation:

B. Use a watermark to specify the allowed lateness — Correct

In Spark Structured Streaming (and Databricks streaming pipelines), watermarking is the standard and recommended way to handle late-arriving data. A watermark defines how long the system should wait for late records before considering the data complete for a given time window. This allows aggregations and stateful operations to correctly include late events while still enabling the system to eventually clean up old state and manage memory efficiently. Using watermarks ensures correctness without reprocessing all historical data and is specifically designed for streaming sources like Kafka where event-time delays are common.

A. Implement a custom solution using Databricks Jobs — Incorrect

Periodically reprocessing all historical data is inefficient, costly, and unnecessary. Spark Structured Streaming already provides built-in mechanisms, such as watermarks, to handle late-arriving data incrementally. Reprocessing everything defeats the purpose of streaming and introduces significant operational complexity.

C. Use batch processing and overwrite the entire output table — Incorrect

Overwriting the entire output table on each run is highly inefficient and does not scale for continuous data streams. This approach eliminates the benefits of streaming, increases compute costs, and introduces latency, making it unsuitable for Kafka-based pipelines with late-arriving data.

D. Use an Auto CDC pipeline with batch tables — Incorrect

Auto CDC pipelines are designed for change data capture from source systems, not for handling event-time

lateness in streaming data from Kafka. They do not address the core challenge of late-arriving records in event-time-based aggregations.

Key Takeaway:

For Kafka-based streaming pipelines where late-arriving data is common, the correct and efficient approach is to use event-time watermarks, which allow late data to be processed accurately while maintaining manageable state and performance.

Question: 396

Deep Dumps

Two data engineers are working on the same Databricks notebook in separate branches. Both have edited the same section of code. When one tries to merge the other's branch into their own using the Databricks Git folders UI, a merge conflict occurs on that notebook file. The UI highlights the conflict and presents options for resolution. How should the data engineers resolve this merge conflict using Databricks Git folders?

- A. Abort the merge, discard all local changes, and try the merge operation again without reviewing the conflicting code.
- B. Use the Git CLI in the cluster's web terminal to force-push the conflicted merge (`git push --force`), overriding the remote branch with local version and discarding changes.
- C. Delete the conflicted notebook file via the Databricks workspace UI, commit the deletion, and recreate the notebook from scratch in a new commit to bypass the conflict entirely.
- D. Use the Git folders UI to manually edit the notebook file, selecting the desired lines from both versions and removing the conflict markers, then mark the conflict as resolved.

Answer: D

Explanation:

D. Use the Git folders UI to manually edit the notebook file — Correct

Databricks Git folders are designed to support manual conflict resolution directly in the UI. When a conflict occurs, the notebook displays both versions of the conflicting code along with conflict markers. The correct approach is to review the differences, decide which lines to keep (or merge logic from both versions), remove the conflict markers, and then mark the conflict as resolved in the Git folders UI. This preserves the correct logic from both engineers and follows standard Git conflict resolution best practices within Databricks.

A. Abort the merge and retry without reviewing — Incorrect

Retrying the merge without resolving the underlying conflict will simply result in the same conflict again. Ignoring the conflicting code risks losing important changes and does not address the root cause of the issue.

B. Force-push using Git CLI — Incorrect

Force-pushing overrides history and discards one engineer's changes, which is dangerous and strongly discouraged in collaborative environments. Additionally, Databricks Git folders are intended to handle conflicts through the UI rather than destructive Git commands.

C. Delete and recreate the notebook — Incorrect

Deleting the notebook and recreating it from scratch bypasses proper version control practices and results in unnecessary data loss, lost history, and additional effort. Git provides built-in mechanisms to resolve conflicts cleanly without deleting files.

Key Takeaway:

When a merge conflict occurs in Databricks Git folders, the correct solution is to manually resolve the conflict

in the notebook, remove conflict markers, and mark the file as resolved using the Git folders UI.

Question: 397

Deep Dumps

A data engineer is masking the provided data column containing the email address. The goal is to have an output of the same length for all rows, while keeping different outputs for different values. Which SQL function should be used to achieve this?

- A. `hash(email)`
- B. `sha2(email, 0)`
- C. `mask(email, '?')`
- D. `sha1('email')`

Answer: A

Explanation:

A. `hash(email)` — Correct

The `hash()` function generates a fixed-length numeric hash value for each input. The output length is consistent across all rows, while different email values produce different hash outputs. This makes `hash(email)` well-suited for masking data when consistency and determinism are required without exposing the original value. It is commonly used for anonymization, joins on masked data, and maintaining uniqueness without revealing sensitive information.

B. `sha2(email, 0)` — Incorrect

The `sha2()` function returns a cryptographic hash string with a fixed length, but specifying 0 as the bit length is invalid or non-standard usage. In practice, `sha2()` requires explicit bit lengths such as 224, 256, 384, or 512. Because this option uses an incorrect parameter, it is not the correct answer.

C. `mask(email, '?')` — Incorrect

The `mask()` function replaces characters with a masking symbol while preserving the original string length. However, this results in identical outputs for all values of the same length, which violates the requirement to produce different outputs for different values. Therefore, it does not meet the stated goal.

D. `sha1('email')` — Incorrect

This option hashes the literal string 'email', not the contents of the email column. As a result, every row would produce the same output, making it unsuitable for masking distinct values.

Key Takeaway:

When masking sensitive data while preserving fixed output length and uniqueness per value, the correct choice is a hash function, specifically `hash(email)`.

Question: 398

Deep Dumps

A data engineer is designing a Lakeflow Spark Declarative Pipeline to process streaming order data. The pipeline uses Auto Loader to ingest data and must enforce data quality by ensuring `customer_id` and `amount` are greater than zero. Invalid records should be dropped. Which Lakeflow Spark Declarative Pipelines configurations implement this requirement using Python?

- A. `@dlt.table def silver_orders(): return dlt.read_stream("bronze_orders").expect_or_drop("valid_customer", "customer_id IS NOT NULL").expect_or_drop("valid_amount", "amount > 0")`
- B. `@dlt.table @dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL") @dlt.expect_or_drop("valid_amount", "amount > 0") def silver_orders(): return dlt.read_stream("bronze_orders")`
- C. `@dlt.table def silver_orders(): return dlt.read_stream("bronze_orders").expect("valid_customer", "customer_id IS NOT NULL").expect("valid_amount", "amount > 0")`
- D. `@dlt.table @dlt.expect("valid_customer", "customer_id IS NOT NULL") @dlt.expect("valid_amount", "amount > 0") def silver_orders(): return dlt.read_stream("bronze_orders")`

Answer: B

Explanation:

B. Use @dlt.expect_or_drop decorators — Correct

In Lakeflow Spark Declarative Pipelines (DLT) using Python, data quality expectations are defined as decorators, not as chained DataFrame methods. The `@dlt.expect_or_drop` decorator explicitly enforces a rule and drops records that fail the condition, which is exactly what the requirement specifies.

Option B correctly applies two `@dlt.expect_or_drop` decorators to the table definition, ensuring that any record with an invalid `customer_id` or `amount` is automatically removed from the output table while valid records continue downstream.

A. Using .expect_or_drop() as a chained method — Incorrect

In Python DLT pipelines, expectations are not applied as chained DataFrame methods. That syntax is invalid for Lakeflow Spark Declarative Pipelines. Expectations must be defined using decorators or the `dlt.expect*()` APIs at the table level.

C. Using .expect() instead of dropping records — Incorrect

The `expect()` function records data quality metrics but does not drop invalid records. Since the requirement explicitly states that invalid records should be dropped, this option does not meet the requirement.

D. Using @dlt.expect instead of @dlt.expect_or_drop — Incorrect

While this option correctly uses decorators, `@dlt.expect` only monitors and reports on data quality violations. Invalid records are still written to the table, which violates the requirement to drop bad data.

Key Takeaway:

To enforce data quality rules and drop invalid records in a Lakeflow Spark Declarative Pipeline using Python, the correct approach is to use `@dlt.expect_or_drop` decorators, as shown in Option B.

Question: 399

Deep Dumps

While reviewing a query's execution in the Databricks Query Profile, a data engineer observes that the "Top operators" panel shows a sort operator with high "Time spent" and "Memory peak" metrics, and the Spark UI reports frequent data spilling. How should the data engineer address this issue?

- A. Increase the number of shuffle partitions to better distribute data.
- B. Switch to a broadcast join to reduce memory usage.
- C. Repartition the DataFrame to a single partition before sorting.
- D. Convert the sort operation to a filter operation.

Answer: A

Explanation:

A. Increase the number of shuffle partitions — Correct

Sorting is a memory-intensive operation, especially when large volumes of data are involved. The frequent data spilling indicates that individual tasks do not have enough memory to handle the data they are sorting. By increasing the number of shuffle partitions, the dataset is split into smaller chunks, allowing each task to sort less data in memory. This reduces memory pressure per task, minimizes disk spills, and typically improves overall query performance. Adjusting shuffle partitions is a common and effective tuning technique for sort-heavy workloads in Spark.

B. Switch to a broadcast join — Incorrect

Broadcast joins are relevant only when joins are involved and one side of the join is small enough to be broadcast. The performance issue here is caused by a sort operator, not a join. Changing the join strategy does not address the memory pressure caused by sorting.

C. Repartition the DataFrame to a single partition — Incorrect

Repartitioning to a single partition forces all data to be processed by one task, dramatically increasing memory usage and making data spilling far worse. This is the opposite of what is needed for large-scale sorting and would likely cause severe performance degradation or job failure.

D. Convert the sort operation to a filter operation — Incorrect

A filter operation serves a completely different purpose and cannot replace a sort. Sorting is required for ordering data, window functions, or downstream logic, while filtering only removes rows. This option is not technically valid.

Key Takeaway:

When a sort operation causes high memory usage and frequent spills, the most effective fix is to increase shuffle parallelism so that each task processes less data in memory.

Question: 400

Deep Dumps

A workspace admin has created a new catalog called 'finance_data' and wants to delegate permission management to a finance team lead without giving them full admin rights. Which privilege should be granted to the finance team lead?

- A. GRANT_OPTION privilege on the finance_data catalog.
- B. ALL PRIVILEGES on the finance_data catalog.
- C. Make the finance team lead a metastore admin.
- D. MANAGE privilege on the finance_data catalog.

Answer: D

Explanation:

D. MANAGE privilege on the finance_data catalog — Correct

The MANAGE privilege in Databricks Unity Catalog is specifically designed to allow a user to manage permissions and ownership on a securable object (such as a catalog) without granting them full administrative control over the entire metastore or workspace. By granting MANAGE on the finance_data catalog, the

finance team lead can grant, revoke, and manage access to schemas and tables within that catalog. This satisfies the requirement to delegate permission management while maintaining the principle of least privilege.

A. GRANT_OPTION privilege on the finance_data catalog — Incorrect

The GRANT OPTION allows a user to pass on privileges they already have, but it does not provide full permission management capabilities for the catalog itself. It is more limited than MANAGE and does not give comprehensive control over access within the catalog.

B. ALL PRIVILEGES on the finance_data catalog — Incorrect

Granting ALL PRIVILEGES provides full control over the catalog, which goes beyond the requirement and effectively gives the finance team lead near-admin-level authority over that catalog. This violates the goal of delegating permissions without excessive access.

C. Make the finance team lead a metastore admin — Incorrect

Metastore admins have global administrative control across all catalogs and securable objects in the metastore. This is far broader than required and directly contradicts the requirement to avoid granting full admin rights.

Key Takeaway:

To delegate catalog-level permission management without granting full administrative access, the correct and least-privileged choice is to grant the MANAGE privilege on the catalog.

Question: 401

Deep Dumps

A data engineer wants to ingest a large collection of image files (JPEG and PNG) from cloud object storage into Unity Catalog-managed table for further analysis and visualization. Which two configurations and practices are recommended to incrementally ingest these images into the table? Choose 2 answers

- A. Move files to a volume and read with SQL editor
- B. Use Auto loader and set cloudFiles.format to "BINARYFILE"
- C. Use Auto loader and set cloudFiles.format to "TEXT"
- D. Use Auto loader and set cloudFiles.format to "IMAGE"
- E. Use the pathGlobFilter option to select only image files (e.g., ".jpg,.png")

Answer: B,E

Explanation:

B. Use Auto Loader and set cloudFiles.format to "BINARYFILE" — Correct

When ingesting unstructured data such as images, Databricks recommends using Auto Loader with the BINARYFILE format. This allows each image file to be ingested as a binary object along with metadata such as file path, size, and modification time. Auto Loader enables incremental ingestion, meaning new image files are automatically detected and processed without re-reading existing data. This approach integrates cleanly with Unity Catalog-managed tables, making it the preferred configuration for large-scale image ingestion.

E. Use the pathGlobFilter option to select only image files — Correct

The pathGlobFilter option is a best practice when ingesting files from object storage that may contain mixed file types. By specifying filters such as "*.jpg" and "*.png", the data engineer ensures that only image files are ingested, reducing unnecessary processing and improving pipeline efficiency. This is especially important for

incremental pipelines using Auto Loader, where precise file selection helps maintain performance and correctness.

A. Move files to a volume and read with SQL editor — Incorrect

Moving files to a volume and reading them manually does not support incremental ingestion at scale. This approach lacks automated file discovery, state tracking, and scalability, making it unsuitable for continuously ingesting large collections of images.

C. Use Auto Loader and set `cloudFiles.format` to "TEXT" — Incorrect

The "TEXT" format is designed for reading text-based files such as logs or CSV-like data. Image files are binary in nature, so using the TEXT format would fail or produce unusable results.

D. Use Auto Loader and set `cloudFiles.format` to "IMAGE" — Incorrect

While Databricks provides an IMAGE data source for image processing and computer vision use cases, Auto Loader incremental ingestion into Unity Catalog-managed tables is typically done using BINARYFILE. The IMAGE format is not the recommended configuration for large-scale, incremental ingestion pipelines managed by Auto Loader.

Question: 402

Deep Dumps

A data engineering team has a time-consuming data ingestion job. It has three data sources, and each data ingestion notebook takes about one hour to load new data. The job had been working fine, loading data every midnight. However, one day they received an alert message stating the job had failed. By investigating the cause, they figured out that the failed notebook code was updated, and a new required configuration was introduced to parameterize a hardcoded value without notice. Now, the team has to fix the issue as quickly as possible to load the latest data from the failing data source. Which action should the team take in this scenario?

- A. Repair the run with the new parameter.
- B. Share the analysis with the failing notebook owner so that they can fix it quickly.
- C. Update the task by adding the missing task parameter, and manually run the job.
- D. Repair the run with the new parameter, and update the task by adding the missing task parameter.

Answer: D

Explanation:

D. Repair the run with the new parameter, and update the task by adding the missing task parameter — Correct

This option addresses both the immediate failure and the long-term stability of the job. Using Repair Run allows the team to quickly rerun only the failed notebook with the correct parameter, avoiding reprocessing the two successful one-hour ingestion tasks and minimizing recovery time. Updating the task configuration ensures that future scheduled runs include the required parameter, preventing the same failure from happening again. This is the fastest and most complete resolution.

A. Repair the run with the new parameter — Incorrect

While repairing the run with the new parameter fixes the current failure, it does not update the job definition. The next scheduled run would fail again for the same reason. This approach solves the symptom but not the root cause.

B. Share the analysis with the failing notebook owner — Incorrect

Sharing information does not immediately resolve the failed job or load the missing data. The team needs to take action within the job configuration itself. This option delays recovery and does not meet the requirement to fix the issue as quickly as possible.

C. Update the task and manually run the job — Incorrect

Updating the task configuration ensures future runs succeed, but manually rerunning the job would restart all tasks, including the two ingestion notebooks that already completed successfully. This wastes time and compute resources compared to repairing only the failed task.

Question: 403

Deep Dumps

Which method can be used to determine the total wall-clock time it took to execute a query?

- A. Open the Query Profiler associated with that query and use the Aggregated task time metric
- B. In the Spark UI, take the job duration of the longest-running job associated with that query.
- C. In the Spark UI, take the sum of all task durations that ran across all stages for all jobs associated with that query.
- D. Open the Query Profiler associated with that query and use the Total wall-clock duration metric.

Answer: D

Explanation:

D. Open the Query Profiler associated with that query and use the Total wall-clock duration metric — Correct

The Total wall-clock duration metric in the Databricks Query Profiler directly represents the actual elapsed time from when the query started until it finished, as experienced by the user. This includes all execution phases such as planning, scheduling, execution, and coordination across tasks and jobs. Because wall-clock time measures real elapsed time rather than accumulated compute effort, this metric is the most accurate and straightforward way to determine how long the query truly took to run.

A. Use the Aggregated task time metric — Incorrect

Aggregated task time represents the sum of execution time across all tasks, often running in parallel. Because many tasks execute simultaneously, this value can be much larger than the actual elapsed time and does not reflect wall-clock duration. It measures total compute effort, not real execution time.

B. Take the duration of the longest-running job — Incorrect

A query may consist of multiple Spark jobs, and the longest job duration does not necessarily capture the full query lifecycle. It ignores planning time, coordination overhead, and time spent in other jobs, making it an incomplete and potentially misleading measure of total wall-clock time.

C. Sum all task durations across all stages — Incorrect

Summing task durations dramatically overestimates wall-clock time because tasks run in parallel across executors. This metric reflects total CPU usage rather than how long the query actually ran from start to finish.

Key Takeaway:

To accurately measure how long a query took in real time, always use the Total wall-clock duration metric in the Databricks Query Profiler.

Question: 404**Deep Dumps**

A data organization has adopted Delta Sharing to securely distribute curated datasets from a Unity Catalog-enabled workspace. The data engineering team is sharing large Delta tables with an internal partner via Databricks-to-Databricks and aggregated reports with an external client via Open Sharing. While testing new sharing workflows, the data engineering team encounters challenges related to access control, data update visibility, and shareable object types. What is a limitation of the Delta Sharing protocol or implementation when used with Databricks-to-Databricks or Open Sharing?

- A. Delta Sharing does not support Unity Catalog enabled tables; only legacy Hive Metastore tables are shareable.
- B. Delta Sharing (both Databricks-to-Databricks and Open sharing) allows recipients to modify the source data if they have SELECT privileges.
- C. With Open sharing, recipients cannot access Volumes, Models, or notebooks — only static Delta tables are supported.
- D. With Databricks-to-Databricks sharing, Unity Catalog recipients must re-ingest data manually using COPY INTO or REST APIs.

Answer: C**Explanation:**

C. With Open Sharing, recipients cannot access Volumes, Models, or notebooks — only Delta tables are supported — Correct

A key limitation of Open Delta Sharing is the restricted set of shareable object types. Open Sharing is designed to work across platforms and with non-Databricks consumers, so it supports read-only access to Delta tables only. Objects such as Unity Catalog Volumes, ML Models, notebooks, or other workspace artifacts cannot be shared via Open Sharing. This limitation often surfaces when teams attempt to distribute richer data products beyond tabular datasets to external clients.

A. Delta Sharing does not support Unity Catalog-enabled tables — Incorrect

Delta Sharing is tightly integrated with Unity Catalog and is the recommended way to securely share UC-managed tables. Legacy Hive Metastore tables are not a requirement, making this statement false.

B. Delta Sharing allows recipients to modify source data — Incorrect

Delta Sharing is strictly read-only. Recipients can query and read shared data, but they cannot modify, insert, update, or delete data in the provider's environment, regardless of privileges granted.

D. Databricks-to-Databricks sharing requires manual re-ingestion — Incorrect

Databricks-to-Databricks Delta Sharing provides live access to shared data. Recipients query data directly without copying or re-ingesting it, and updates are visible automatically based on the provider's refresh behavior.

Key Takeaway:

The major limitation highlighted here is that Open Delta Sharing supports only Delta tables, not other Unity Catalog objects like volumes, models, or notebooks.

Question: 405**Deep Dumps**

A company wants to implement Lakehouse Federation across multiple data sources, but is worried about data

consistency and ensuring all teams access the same authoritative version of their data Which statement is applicable for Lakehouse Federations to maintain data consistency?

- A. A separate data synchronization service must be deployed.
- B. Federation provides read-only access that reflects the current state of source systems.
- C. Federation creates local copies that must be manually refreshed.
- D. Federation implements change data capture from all sources.

Answer: B

Explanation:

B. Federation provides read-only access that reflects the current state of source systems — Correct

Lakehouse Federation enables users to query external data sources directly without copying or replicating the data into Databricks. Because queries are executed against the live source systems, all teams see the same authoritative version of the data at query time. Federation access is read-only, which prevents accidental modification of source data and ensures consistency across consumers. This approach eliminates data drift and synchronization challenges that commonly arise when data is copied into multiple systems.

A. A separate data synchronization service must be deployed — Incorrect

Lakehouse Federation does not rely on external synchronization services. One of its key benefits is eliminating the need for data replication or synchronization by querying source systems directly.

C. Federation creates local copies that must be manually refreshed — Incorrect

Federation does not create local copies of data. Creating and refreshing copies would reintroduce consistency issues, which Lakehouse Federation is designed to avoid.

D. Federation implements change data capture from all sources — Incorrect

Lakehouse Federation does not perform change data capture. CDC is used in ingestion and replication pipelines, whereas federation focuses on live, read-only query access to external systems.

Key Takeaway:

Lakehouse Federation maintains data consistency by providing read-only, live access to source systems, ensuring all users see the same authoritative data without copying or synchronizing it.

Question: 406

Deep Dumps

A data engineer has configured their Databricks Asset Bundle with multiple targets in databricks.yml and deployed it to the production workspace. Now, to validate the deployment, they need to invoke a job named my_project_job specifically within the prod target context. Assuming the job is already deployed, they need to trigger its execution while ensuring the target-specific configuration is respected. Which command will trigger the job execution?

- A. databricks execute my_project_job -e prod
- B. databricks run my_project_job -t prod
- C. databricks job run my_project_job --env prod
- D. databricks bundle run my_project_job -t prod

Answer: D

Explanation:

D. databricks bundle run my_project_job -t prod — Correct

When working with Databricks Asset Bundles, all deployment and execution actions are managed through the databricks bundle command group. The bundle run command is specifically used to trigger a job that is defined and deployed as part of the bundle.

The -t prod (or --target prod) flag ensures that the job is executed using the production target configuration defined in databricks.yml, including workspace, permissions, clusters, and parameters. Since the job is already deployed, this command correctly invokes it while respecting all target-specific settings.

A. databricks execute my_project_job -e prod — Incorrect

There is no valid databricks execute command in the Databricks CLI for running jobs or bundles. This command does not exist and will fail.

B. databricks run my_project_job -t prod — Incorrect

The databricks run command is not used for Asset Bundles. Job execution within a bundle must be performed using the databricks bundle run command so that bundle context and target configuration are applied.

C. databricks job run my_project_job --env prod — Incorrect

While Databricks has job-related CLI commands, this syntax is not valid for running bundle-managed jobs, and --env is not the correct flag for selecting a bundle target. Asset Bundles rely on --target (or -t) instead.

Question: 407

Deep Dumps

A data engineer wants to automate job monitoring and recovery in Databricks using the Jobs API. They need to list all jobs, identify a failed job, and rerun it. Which sequence of API actions should the data engineer perform?

- A. Use the jobs list endpoint to list jobs, check job run statuses with jobs runs list, and rerun a failed job using jobs run-now
- B. Use the jobs get endpoint to retrieve job details, then use jobs update to rerun failed jobs.
- C. Use the jobs list endpoint to list jobs, then use the jobs create endpoint to create a new job, and run the new job using jobs run-now.
- D. Use the jobs cancel endpoint to remove failed jobs, then recreate them with jobs create endpoint and run the new ones

Answer: A

Explanation:

A. Use jobs list → jobs runs list → jobs run-now — Correct

This sequence matches the intended design of the Databricks Jobs API for monitoring and recovery:

First, the jobs list endpoint is used to retrieve all defined jobs and their job IDs.

Next, the jobs runs list endpoint is used to inspect job runs and identify runs that have failed.

Finally, once a failed job (or job ID) is identified, the jobs run-now endpoint is used to rerun the job immediately without modifying its configuration.

This approach is efficient, non-destructive, and aligns with standard operational best practices for automated job recovery.

B. Use jobs get and jobs update — Incorrect

The jobs get endpoint retrieves job configuration details, not run execution status.

The jobs update endpoint is used to modify job definitions, not to rerun failed jobs. Using update for recovery is incorrect and unnecessary.

C. Create a new job instead of rerunning — Incorrect

Creating a new job every time a failure occurs is inefficient and unnecessary. Databricks Jobs are designed to be rerun using existing job definitions. This approach also creates operational clutter and breaks job history continuity.

D. Cancel and recreate failed jobs — Incorrect

The jobs cancel endpoint stops running jobs; it does not remove or fix failed jobs. Recreating jobs from scratch is disruptive, wastes time, and is not required for recovery. Databricks provides direct APIs to rerun jobs safely.

Question: 408

Deep Dumps

A data engineer has a delta table order with deletion vectors enabled for it. The engineer is attempting to execute the below code: `DELETE FROM orders WHERE status = 'cancelled'` What should be the behaviour of deletion vectors when the command is executed?

- A. Rows are marked as deleted in metadata, not in files.
- B. Delta automatically removes all cancelled orders permanently.
- C. Rows are marked as deleted both in metadata and in files.
- D. Files are physically rewritten without the deleted row.

Answer: A

Explanation:

A. Rows are marked as deleted in metadata, not in files — Correct

When deletion vectors are enabled in a Delta table, delete operations do not rewrite the underlying data files. Instead, Delta Lake records the row-level deletions in deletion vectors stored in metadata. These vectors indicate which rows in a data file should be considered deleted at query time.

This approach dramatically improves performance for DELETE, UPDATE, and MERGE operations by avoiding expensive file rewrites. The original data remains unchanged on disk, but queries automatically exclude deleted rows using the deletion vector metadata.

B. Delta automatically removes all cancelled orders permanently — Incorrect

Delta Lake does not immediately and permanently remove rows during a DELETE operation when deletion vectors are enabled. The data remains physically present in the files and can only be permanently removed through operations such as VACUUM, subject to retention policies.

C. Rows are marked as deleted both in metadata and in files — Incorrect

With deletion vectors enabled, rows are not removed from data files. Only metadata is updated. File-level rewrites would negate the performance benefits of deletion vectors.

D. Files are physically rewritten without the deleted row — Incorrect

This describes the behavior when deletion vectors are not enabled. In that case, Delta Lake rewrites files to

exclude deleted rows. Since deletion vectors are enabled here, this behavior does not apply.

Question: 409

Deep Dumps

A streaming video analytics team ingests billions of events daily into a Unity Catalog-managed Delta table `video_events`. Analysts run ad-hoc point-lookup queries on columns like `user_id`, `campaign_id`, and `region`. The team manually runs `OPTIMIZE video_events Z-ORDER user_id, campaign_id, region`, but still sees poor performance on recent data, and dislikes the operational overhead. The team wants a hands-off way to keep hot columns co-located as query patterns evolve. Which Delta capability should the team leverage on `video_events`?

- A. Enable Delta caching.
- B. Schedule `OPTIMIZE/ZORDER` to run after each job to improve recent file performance.
- C. Utilize Liquid Clustering `CLUSTER BY AUTO` and Predictive Optimization.
- D. Enable auto-compaction (`optimizeWrite` & `autoCompact`).

Answer: C

Explanation:

Correct Answer: C. Utilize Liquid Clustering with `CLUSTER BY AUTO` and Predictive Optimization

Liquid Clustering with `CLUSTER BY AUTO`, combined with Predictive Optimization, provides a fully automated and adaptive approach to organizing data in large Delta tables. Instead of relying on manually defined `Z-ORDER` columns and scheduled `OPTIMIZE` jobs, Liquid Clustering continuously evaluates actual query access patterns and automatically co-locates frequently filtered columns, such as `user_id`, `campaign_id`, and `region`. Predictive Optimization runs in the background to reorganize newly ingested and hot data without user intervention, ensuring that recent data is optimized as quickly as possible. This eliminates operational overhead, adapts as query patterns evolve over time, and delivers consistently high performance for ad-hoc point-lookup queries on high-volume streaming tables.

A. Enable Delta caching — Incorrect

Delta caching improves performance by caching data in memory on cluster nodes. However, it does not improve data layout on disk and does not address poor performance caused by suboptimal clustering of newly ingested data. It also does not adapt to changing query patterns.

B. Schedule `OPTIMIZE/ZORDER` after each job — Incorrect

While scheduling `OPTIMIZE` with `Z-ORDER` can help performance, it still requires manual operational management, increases compute cost, and does not adapt automatically when query patterns change. The team explicitly wants a hands-off solution, which this option does not provide.

D. Enable auto-compaction (`optimizeWrite` and `autoCompact`) — Incorrect

Auto-compaction helps reduce the number of small files by merging them during or after writes. While useful for write efficiency and general performance, it does not cluster data by query-relevant columns and therefore does not improve point-lookup query performance.

Question: 410

Deep Dumps

A data engineer, while designing a Pandas UDF to process financial time series data with complex calculations that require maintaining state across rows within each stock symbol group must ensure the function is efficient and scalable. Which approach will solve the problem with minimum overhead while preserving data integrity?

- A. Use a SCALAR Pandas UDF that processes the entire dataset at once, implementing custom partitioning logic within the UDF to group by stock symbol and maintain state using global variables shared across all executor processes.
- B. Use a SCALAR_ITER Pandas UDF with iterator-based processing, implementing state management through persistent storage (Delta tables) that gets updated after each batch to maintain continuity across iterator chunks.
- C. Use applyInPandas method on spark dataframe that receives all rows for each stock symbol as a pandas DataFrame, allowing processing within each group while maintaining state variables local to each group's processing function.
- D. Use a GROUPED_AGG Pandas UDF that processes each stock symbol group independently, maintaining state through intermediate aggregation results that get passed between successive UDF calls via broadcast variables.

Answer: C

Explanation:

C. Use applyInPandas to process each stock symbol group — Correct

The applyInPandas method is specifically designed for group-wise Pandas processing where all rows for a given group key are delivered together as a single Pandas DataFrame. This allows the engineer to naturally maintain state across rows within each stock symbol group using local variables, without relying on global state, external storage, or cross-task communication. Spark guarantees that each group is processed independently and in isolation, which preserves data integrity and ensures scalability. Because state is maintained only within the scope of each group's execution, this approach introduces minimal overhead and aligns perfectly with Spark's distributed execution model.

A. SCALAR Pandas UDF with global state — Incorrect

Scalar Pandas UDFs operate on independent batches of rows and are not suitable for maintaining state across groups. Using global variables for state management is unsafe in a distributed environment because executors do not share memory, leading to incorrect results and broken scalability.

B. SCALAR_ITER Pandas UDF with persistent storage — Incorrect

While iterator-based UDFs can handle streaming-like processing, persisting state to external storage such as Delta tables introduces significant overhead and complexity. This approach reduces performance and increases operational cost, making it unsuitable when simpler, built-in group-wise processing is available.

D. GROUPED_AGG Pandas UDF with broadcast variables — Incorrect

Grouped aggregation Pandas UDFs are intended for producing single aggregated outputs per group, not for row-by-row stateful transformations. Broadcast variables are read-only and cannot be used to maintain or update state between UDF invocations, making this approach invalid.

Question: 411

Deep Dumps

A data engineering team is in the process of migrating off its legacy Hadoop platform. As part of the process, they are evaluating the different storage formats to see which offers the best query performance. Their legacy platform utilizes ORC and RCFile formats. They converted a subset of their data to Delta Lake and ran a side-by-side benchmark, and found that their queries performed significantly better. Upon investigation, they discovered that the query plans were different, and the queries that read from the Delta Lake tables leveraged a Shuffle Hash Join, whereas the legacy formats used Sort Merge Joins. The queries that read from the Delta Lake tables also read less data. Which reason could be attributed to the difference in query performance?

•

- A. Delta Lake enables data skipping and file pruning using a vectorized Parquet reader.

- B. The queries against the ORC tables leveraged the dynamic data skipping optimization but not the dynamic file pruning optimization.
- C. Shuffle Hash Joins are always more efficient than Sort Merge Joins.
- D. The queries against the Delta Lake tables were able to leverage the dynamic file pruning optimization.

Answer: D

Explanation:

D. The queries against the Delta Lake tables were able to leverage the dynamic file pruning optimization — Correct

Dynamic file pruning is a powerful optimization supported by Delta Lake that allows Spark to skip reading entire files at query runtime based on join conditions. When combined with Shuffle Hash Joins, Spark can dynamically determine which partitions or files are actually needed after the build side of the join is evaluated. This results in significantly less data being read from disk, which directly improves query performance. Legacy Hadoop formats like ORC and RCFile often lack full support for dynamic file pruning in modern Spark execution plans, causing them to read more data and rely on less efficient join strategies such as Sort Merge Joins.

A. Delta Lake enables data skipping and file pruning using a vectorized Parquet reader — Incorrect

While Delta Lake does support data skipping and file pruning through metadata such as min/max statistics, the question specifically highlights runtime differences in query plans and join strategies, not static pruning. The key differentiator here is dynamic file pruning during query execution, not vectorized Parquet reading alone.

B. The queries against the ORC tables leveraged dynamic data skipping but not dynamic file pruning — Incorrect

Legacy ORC tables generally do not benefit from advanced dynamic file pruning optimizations in Spark. Additionally, the scenario describes Delta Lake queries reading less data, not ORC queries benefiting from additional optimizations.

C. Shuffle Hash Joins are always more efficient than Sort Merge Joins — Incorrect

Shuffle Hash Joins are not universally more efficient than Sort Merge Joins. Their efficiency depends on data size, memory availability, and join cardinality. The improved performance in this scenario is not due solely to the join type, but rather to reduced data reads enabled by dynamic file pruning, which influences the optimizer's choice of join strategy.

Question: 412

Deep Dumps

A data engineer is implementing Unity Catalog governance for a multi-team environment. Data scientists need interactive clusters for basic data exploration tasks, while automated ETL jobs require dedicated processing. How should the data engineer configure cluster isolation policies to enforce least privilege and ensure Unity Catalog compliance?

- A. Create compute policies with STANDARD access mode for interactive workloads and DEDICATED access mode for automated jobs.
- B. Use only DEDICATED access mode for both interactive workloads and automated jobs to maximize security isolation.
- C. Allow all users to create any cluster type and rely on manual configuration to enable Unity Catalog access modes.
- D. Configure all clusters with NO_ISOLATION_SHARED access modes since Unity Catalog works with any

cluster configuration.

Answer: A

Explanation:

A. Create compute policies with STANDARD access mode for interactive workloads and DEDICATED access mode for automated jobs — Correct

This approach aligns directly with Unity Catalog best practices. STANDARD access mode is appropriate for interactive clusters used by data scientists because it allows multiple users to share a cluster securely while still enforcing Unity Catalog's fine-grained access controls. DEDICATED access mode is designed for automated workloads such as ETL jobs, where a single principal owns the cluster, providing stronger isolation and reduced risk of privilege escalation. Using compute policies to enforce these access modes ensures least privilege, prevents misconfiguration, and maintains consistent governance across teams.

B. Use only DEDICATED access mode for both interactive and automated workloads — Incorrect

While DEDICATED access mode provides strong isolation, using it for interactive workloads is unnecessary and inefficient. It increases cost and reduces collaboration by preventing safe cluster sharing, without providing additional governance benefits beyond what STANDARD mode already offers for interactive use cases.

C. Allow all users to create any cluster type and rely on manual configuration — Incorrect

Allowing unrestricted cluster creation violates the principle of least privilege and introduces governance risk. Manual configuration is error-prone and does not scale in multi-team environments, making it unsuitable for enforcing Unity Catalog compliance.

D. Configure all clusters with NO_ISOLATION_SHARED access mode — Incorrect

Unity Catalog does not support NO_ISOLATION_SHARED clusters. Using this access mode bypasses proper isolation and breaks Unity Catalog governance requirements, making this option invalid.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured
100% Pass Guarantee



24/7 Customer Support
Mail us - support@deepdumps.com



Verified Updates
Lifetime Updates!

Total: **412 Questions**

Link: <https://deepdumps.com/papers/databricks/certified-data-engineer-professional>